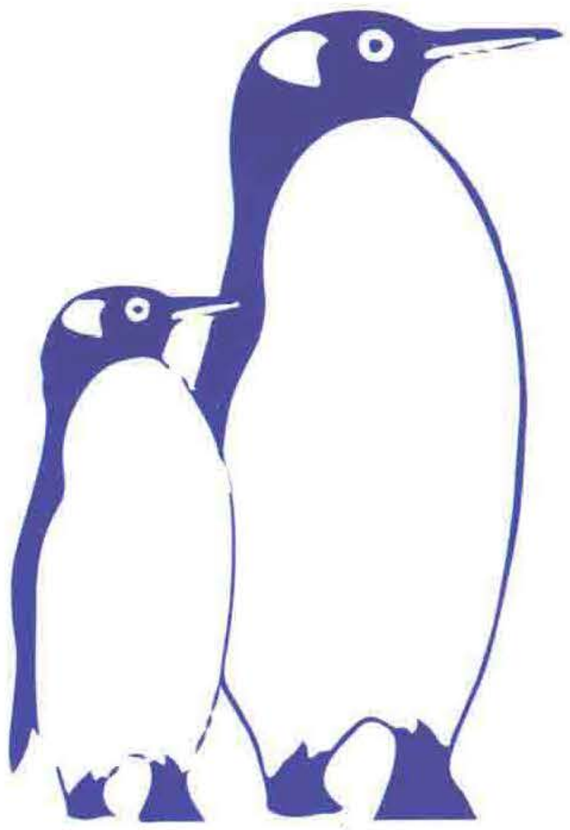
华清远见系列图书

华清远见 FARSIGHT 始于2004

Broadview® www.broadview.com.cn



超大容量多媒体，总时长超过35小时

Linux Shell编程从初学到精通 第2版

华清远见嵌入式培训中心 伍之昂◎等编著

提供全 :视频849分钟

提供VMware工具、Linux命令工具、编辑器工具、GCC工具、GDB工具、Shell工具、make工具、Eclipse开发工具、KDevelop开发工具，

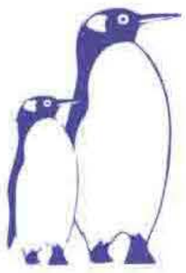
以及项目管理Subversion工具等Linux常用工具教学视频431分钟

提供209个常用Linux命令教学视频583分钟

提供Linux入门教学视频347分钟

電子工業出版社
PUBLISHING HOUSE OF ELECTRONICS INDUSTRY
http://www.phei.com.cn

Broadview®
www.broadview.com.cn
博文视点·IT出版旗舰品牌
技术凝聚实力·专业创新出版



Linux Shell编程从初学到精通 第2版

·学员评价·

我现在从事的工作正是我梦寐以求的嵌入式开发。偶尔回味自己当初从Linux Shell编程学起的那段时光，感到窃喜，因为工作中正用上了所学的知识，感谢华清远见的专业图书给我的帮助……

网友 刻骨铭心

作为一名初学者，我非常感谢华清远见的专业图书给我的帮助。他们的图书内容通俗易懂，书中的示例非常有用。

江苏大学 李东亮

- Shell脚本编程概述
- Linux文件系统和文本编辑器
- 正则表达式
- sed命令和awk编程
- 文件的排序、合并和分割
- 变量和引用
- 退出、测试、判断及操作符
- 循环与结构化命令
- 变量的高级用法
- I/O重定向
- Linux/UNIX Shell类型与区别
- 子Shell与进程处理
- 函数
- 别名、列表及数组
- 一些混杂的主题
- Shell脚本调试技术
- bash Shell编程范例

博文视点Broadview
新浪微博 weibo.com
@博文视点Broadview
策划编辑：孙学瑛
责任编辑：李利健
封面设计：侯士卿
上架建议：操作系统
ISBN 978-7-121-24822-1
定价：79.00元（含DVD光盘1张）

华清远见系列图书

Linux Shell编程从初学到精通 第2版

华清远见嵌入式培训中心 伍之昂◎等编著

電子工業出版社
Publishing House of Electronics Industry
北京•BEIJING

混混藏书阁 http://book-life.blog.163.com

内 容 简 介

Shell 是用户与内核进行交互操作的一种接口，是 Linux 最重要的软件之一，目前最流行的 Shell 称为 bash Shell。bash Shell 脚本编程以其简洁、高效而著称，多年来成为 Linux 程序员和系统管理员解决实际问题的利器。

本书结合大量的示例，系统、全面地介绍了 bash Shell 脚本编程的语法、命令、技巧、调试等内容，在书中还有很多练习可以引导读者思考，力求使读者掌握 Linux bash Shell 编程的所有特性。本书结构清晰、易教易学、实例丰富、可操作性强、学以致用，对易混淆和实用性强的内容进行了重点提示和讲解，并配有光盘，光盘中提供书中出现的所有脚本文件、各章的讲解 PPT，以及各章的讲解录像。

本书面向广大工程技术工作者，既可作为高等学校的教师和相关专业学生的教材，又可作为各类培训班的培训教程。

未经许可，不得以任何方式复制或抄袭本书之部分或全部内容。
版权所有，侵权必究。

图书在版编目（CIP）数据

Linux Shell 编程从初学到精通 / 伍之昂等编著. —2 版. —北京：电子工业出版社，2015.1
（华清远见系列图书）
ISBN 978-7-121-24822-1

Ⅰ. ①L… Ⅱ. ①伍… Ⅲ. ①Linux 操作系统—程序设计 Ⅳ. ①TP316.89

中国版本图书馆 CIP 数据核字（2014）第 273419 号

策划编辑：孙学瑛
责任编辑：李利健
印　　刷：三河市双峰印刷装订有限公司
装　　订：三河市双峰印刷装订有限公司
出版发行：电子工业出版社
　　北京市海淀区万寿路 173 信箱　邮编 100036
开　　本：787×1092　1/16　印张：31　字数：773 千字
版　　次：2011 年 3 月第 1 版
　　2015 年 1 月第 2 版
印　　次：2015 年 1 月第 1 次印刷
定　　价：79.00 元（含 DVD 光盘 1 张）

凡所购买电子工业出版社图书有缺损问题，请向购买书店调换。若书店售缺，请与本社发行部联系，联系及邮购电话：（010）88254888。

质量投诉请发邮件至 zlts@phei.com.cn，盗版侵权举报请发邮件至 dbqq@phei.com.cn。

服务热线：（010）88258888。

前　言

随着 Linux 逐步成为主流的服务器操作系统，Shell 脚本编程成为一名优秀的 Linux 开发者和系统管理员必须掌握的技术之一。bash Shell 为当前大部分 Linux 版本所使用，本书旨在系统地介绍 bash 4.2.25 版本下的 Shell 脚本编程。

本书共分为 17 章：第 1 章介绍 Shell 的概念、Shell 脚本编程的优势和结构等入门知识；第 2 章讲述 Shell 脚本编程不可或缺的 Linux 系统的基础知识；第 3 章介绍正则表达式和 grep 命令族；第 4 章阐述 sed 命令和 awk 命令这两种在 Shell 编程中常用的工具；第 5 章介绍 Shell 编程在文件排序、合并和分割上的一些命令；第 6 章探讨变量和引用；第 7 章介绍退出、测试及操作符；第 8 章介绍循环与结构化命令；第 9 章深入讨论了变量的高级用法；第 10 章详述 I/O 重定向，包含管道、exec 命令等重要内容；第 11 章简述了 UNIX/Linux 发展过程中出现的其他类型的 Shell；第 12 章介绍子 Shell、限制性 Shell 和进程等内容；第 13 章介绍函数的用法；第 14 章介绍别名、列表及数组；第 15 章罗列了无法归入其他章节的混杂主题，包含脚本编写风格、脚本优化、/dev 和/proc 文件系统等；第 16 章介绍了 Shell 脚本的调试技术；第 17 章给出六个 Shell 编程的实例，读者需要综合使用前面章节所述的 Shell 命令和编程技巧，涉及系统管理、文本处理和数据库等多个方面。

本书内容丰富，覆盖了 Shell 编程的大部分技术，并结合典型示例透彻地介绍了 Shell 命令、选项、结构中的重点和难点。各章最后还配有一定数量的练习题供读者学习。为了帮助读者更加直观地学习本书，我们将书中出现的所有脚本文件、各章的讲解 PPT，以及各章的讲解录像都收录到本书的配套光盘中。

本书面向广大工程技术工作者，既可作为高等学校的教师和相关专业学生的教材，又可作为各类培训班的培训教程。

本书由南京财经大学江苏省电子商务重点实验室伍之昂博士和王有权组织编写。在本书编写的过程中，实验室主任曹杰教授在全书的体系结构、理论阐释和实例选取等方面提出了许多精辟的见解，研究生陈志杰同学精心润色了本书的文字。参加编写工作的还有吕平、高克臻、张云霞、张璐、许小荣、王冬、王龙、张银芳、周新国、陈可汤、陈作聪、苏静、周艳丽、祁招娣、张秀梅、张玉兰、李爽、卿前华、王文婷、肖岳平、肖斌、蔡娜等，在此，对他们表示诚挚的谢意！

限于笔者水平，本书一定有不少错误和不妥之处，希望得到计算机专家、同行和读者的批评与指正。您可以通过 E-mail 的方式与笔者联系，地址是 zawu@seu.edu.cn。

编　者

混混藏书阁 http://book-life.blog.163.com

目　录

第 1 章　Shell 脚本编程概述 …… 1

1.1　Linux 和 Shell 概述 …… 2
1.1.1　Linux 简介 …… 2
1.1.2　Shell 简介 …… 5
1.1.3　Shell 的种类和版本 …… 7
1.2　Shell 脚本编程的优势 …… 7
1.3　第一个 Shell 脚本例子 …… 8
1.3.1　Shell 脚本的基本元素 …… 8
1.3.2　执行 Shell 脚本 …… 10
1.4　本章小结 …… 11

第 2 章　Linux 文件系统和文本编辑器 …… 12

2.1　用户和用户组管理 …… 13
2.1.1　用户管理常用命令 …… 13
2.1.2　用户组管理常用命令 …… 17
2.2　文件和目录操作 …… 19
2.2.1　文件操作常用命令 …… 20
2.2.2　目录操作常用命令 …… 25
2.2.3　文件和目录权限管理 …… 28
2.2.4　查找文件命令——find …… 32
2.3　文本编辑器 …… 34
2.3.1　vim 编辑器 …… 34
2.3.2　Gedit 编辑器 …… 40
2.4　本章小结 …… 41
2.5　上机提议 …… 41

第 3 章　正则表达式 …… 43

3.1　正则表达式基础 …… 44
3.2　正则表达式的扩展 …… 47
3.3　通配 …… 48
3.4　grep 命令 …… 50
3.4.1　grep 命令基本用法 …… 51
3.4.2　grep 和正则表达式结合使用的一组例子 …… 57
3.4.3　grep 命令族简介 …… 61
3.5　本章小结 …… 62
3.6　上机提议 …… 63

第 4 章　sed 命令和 awk 编程 …… 64

4.1　sed 命令基本用法 …… 65
4.2　sed 命令的一组示例 …… 67
4.2.1　sed 命令选项的一组例子 …… 67
4.2.2　sed 文本定位的一组例子 …… 70
4.2.3　sed 基本编辑命令的一组例子 …… 72
4.2.4　sed 高级编辑命令的一组例子 …… 80
4.3　awk 编程 …… 83
4.3.1　awk 编程模型 …… 84
4.3.2　awk 调用方法 …… 85
4.4　awk 编程的一组例子 …… 85
4.4.1　awk 模式匹配 …… 85
4.4.2　记录和域 …… 86
4.4.3　关系和布尔运算符 …… 89
4.4.4　表达式 …… 91
4.4.5　系统变量 …… 92
4.4.6　格式化输出 …… 94
4.4.7　内置字符串函数 …… 95
4.4.8　向 awk 脚本传递参数 …… 97
4.4.9　条件语句和循环语句 …… 99
4.4.10　数组 …… 99

8.8 上机提议……215
第 9 章 变量的高级用法……217
9.1 内部变量……218
9.2 字符串处理……226
9.3 有类型变量……232
9.4 间接变量引用……235
9.5 bash 数学运算……237
9.5.1 expr 命令……238
9.5.2 bc 运算器……239
9.6 本章小结……241
9.7 上机提议……241
第 10 章 I/O 重定向……243
10.1 管道……244
10.1.1 管道简介……244
10.1.2 cat 和 more 命令……245
10.1.3 sed 命令与管道……249
10.1.4 awk 命令与管道……250
10.2 I/O 重定向……253
10.2.1 文件标识符……253
10.2.2 I/O 重定向符号及其用法……255
10.2.3 exec 命令的用法……259
10.2.4 代码块重定向……262
10.3 命令行处理……265
10.3.1 命令行处理流程……266
10.3.2 eval 命令……268
10.4 本章小结……271
10.5 上机提议……271
第 11 章 Linux/UNIX Shell 类型与区别……273
11.1 Linux/UNIX Shell 起源与分类……274
11.2 dash 简介……275
11.3 tcsh 简介……277
11.4 Korn Shell 简介……282
11.5 本章小结……287
第 12 章 子 Shell 与进程处理……288
12.1 子 Shell……289
12.1.1 内建命令……289
12.1.2 圆括号结构……292
12.2 Shell 的限制模式……297
12.3 进程处理……299
12.3.1 进程和作业……301
12.3.2 作业控制……302
12.3.3 信号……307
12.3.4 trap 命令……309
12.4 本章小结……312
12.5 上机提议……312
第 13 章 函数……314
13.1 函数的定义和基本知识……315
13.2 向函数传递参数……318
13.3 函数返回值……321
13.4 函数调用……322
13.4.1 脚本放置多个函数……322
13.4.2 函数相互调用……324
13.4.3 一个函数调用多个函数……325
13.5 局部变量和全局变量……327
13.6 函数递归……328
13.6.1 使用局部变量的递归…329
13.6.2 不使用局部变量的递归……330
13.7 本章小结……332
13.8 上机提议……333
第 14 章 别名、列表及数组……335
14.1 别名……336
14.2 列表……339
14.3 数组……342
14.3.1 数组的基本用法……342
14.3.2 数组的特殊用法……346

混混藏书阁 http://book-life.blog.163.com

14.3.3 用数组实现简单的数据结构 ……350
14.4 本章小结 ……356
14.5 上机提议 ……356
第 15 章 一些混杂的主题 ……359
15.1 脚本编写风格 ……360
15.1.1 缩进 ……360
15.1.2 {}的格式 ……362
15.1.3 空格和空行的用法 ……362
15.1.4 判断和循环的编程风格 ……363
15.1.5 命名规范 ……364
15.1.6 注释风格 ……365
15.2 脚本优化 ……366
15.2.1 简化脚本 ……366
15.2.2 保持脚本的灵活性 ……368
15.2.3 给用户足够的提示 ……369
15.3 Linux 中的特殊命令 ……371
15.3.1 shift 命令 ……371
15.3.2 getopts 命令 ……374
15.4 交互式和非交互式 Shell 脚本 ……376
15.4.1 非交互式 Shell 脚本 ……376
15.4.2 交互式 Shell 脚本 ……378
15.5 /dev 文件系统 ……379
15.5.1 /dev 文件系统基础知识 ……380
15.5.2 /dev/zero 伪设备 ……381
15.5.3 /dev/null 伪设备 ……383
15.6 /proc 文件系统 ……384
15.6.1 使用/proc/sys 优化系统参数 ……386
15.6.2 查看运行中的进程信息 ……387
15.6.3 查看文件系统信息 ……388
15.6.4 查看网络信息 ……389
15.7 Shell 包装 ……390
15.8 带颜色的脚本 ……392
15.9 Linux 脚本安全 ……398
15.9.1 使用 shc 工具加密 Shell 脚本 ……398
15.9.2 Linux Shell 脚本编写的病毒 ……399
15.9.3 Linux Shell 中的木马 ……400
15.10 本章小结 ……401
15.1 上机提议 ……401
第 16 章 Shell 脚本调试技术 ……403
16.1 Shell 脚本调试概述 ……404
16.2 Shell 脚本调试技术 ……406
16.2.1 使用 trap 命令 ……406
16.2.2 使用 tee 命令 ……409
16.2.3 调试钩子 ……411
16.2.4 使用 Shell 选项 ……412
16.3 本章小结 ……417
16.4 上机提议 ……417
第 17 章 bash Shell 编程范例 ……420
17.1 将文本文件转化为 HTML 文件 ……421
17.2 查找文本中 *n* 个出现频率最高的单词 ……425
17.3 伪随机数的产生和应用 ……427
17.4 crontab 的设置和应用 ……431
17.5 使用 MySQL 数据库 ……434
17.5.1 MySQL 基础 ……434
17.5.2 Shell 脚本使用 MySQL ……437
17.6 Linux 服务器性能监控系统 ……442
17.6.1 Ganglia 简介及安装 ……443
17.6.2 提取服务器性能参数名称及数据 ……448
17.6.3 动态更新服务器监控数据 ……455
17.7 本章小结 ……457

17.8 上机提议 457

附录 459

附录 A POSIX 标准简介 460
附录 B 常用 ASCII 码对照表 461
附录 C Linux 信号及其意义 466
附录 D bash 内建变量索引 467
附录 E bash 内建命令索引 469

参考文献 472

混混藏书阁 http://book-life.blog.163.com

代码示例目录

第 1 章　Shell 脚本编程概述 …… 1

#例 1-1：介绍 Linux 命令的组成部分 …… 9
#例 1-2：分号的用法 …… 10
#例 1-3：执行 whologged.sh 脚本 …… 10

第 2 章　Linux 文件系统和文本编辑器 …… 12

#例 2-1：创建用户 wang 并查看其相关信息，该操作需要 root 权限 …… 14
#例 2-2：使用 usermod 修改密码，同样需要 root 权限 …… 15
#例 2-3：删除正在登录用户的用户账号所产生的错误 …… 16
#例 2-4：userdel 命令使用-r 选项和不使用-r 选项的区别，该操作也需要 root 权限 …… 16
#例 2-5：为 wang 用户创建账号密码，需要 root 权限 …… 17
#例 2-6：使用 groupadd 命令添加用户组 wangyq，需要 root 权限 …… 18
#例 2-7：使用 groupmod 修改用户组号，需要 root 权限 …… 19
#例 2-8：用 groupdel 删除用户组，需要 root 权限 …… 19
#例 2-9：列出/home/alloy/linuxshell/CH04 目录下的所有文件和目录的详细信息 …… 21
#例 2-10：将文件 CH04/argv.awk 复制到目录 testdir 中 …… 22
#例 2-11：将文件 CH04/argv.awk 复制到目录 testdir 中，且不改变权限和修改日期 …… 22
#例 2-12：将目录中的所有内容复制到另一个目录中 …… 23
#例 2-13：将文件 file1.txt 重命名为 file2.txt …… 24
#例 2-14：将目录 testdir 命名为 chapter2 …… 24
#例 2-15：使用 rm 删除一个目录及该目录下的文件和子目录 …… 25
#例 2-16：通过 mkdir 创建一个名为 tsk 的目录，所有的用户都可读和执行 …… 25
#例 2-17：演示 mkdir 命令-p 选项的用法 …… 26
#例 2-18：使用 rmdir 删除存在子目录的目录 …… 26
#例 2-19：一个使用 cd 的例子 …… 28
#例 2-20：使用“~”快速到达用户主目录 …… 28
#例 2-21：查看文件和目录的权限 …… 28
#例 2-22：修改文件 testvim 的属性 …… 29
#例 2-23：使用 chmod 数字设定法对文件 testvim1 进行权限控制 …… 30
#例 2-24：使用 chown 改变用户属主 …… 30
#例 2-25：使用 chmod 命令实现 SUID/GUID …… 31

#例 2-26：演示 find 命令的 print 操作 ····· 33
#例 2-27：演示 find 命令的 exec 操作 ····· 33
#例 2-28：演示 find 命令的 ok 操作 ····· 34
第 3 章　正则表达式 ····· 43
#例 3-1：*符号的意义 ····· 45
#例 3-2：.符号的意义 ····· 45
#例 3-3：^符号的意义 ····· 45
#例 3-4：^符号、.符号和*符号结合使用 ····· 45
#例 3-5：$符号的意义 ····· 45
#例 3-6：空行的表示方法 ····· 45
#例 3-7：包含一个字符的行 ····· 46
#例 3-8：匹配任意一个数字 ····· 46
#例 3-9：匹配字母 ····· 46
#例 3-10：^表示取反 ····· 46
#例 3-11：匹配所有的英文单词 ····· 46
#例 3-12：转义符 ····· 46
#例 3-13：精确匹配 ····· 46
#例 3-14：\{\}系列符号的用法 ····· 47
#例 3-15：精确匹配 5 个小写字母 ····· 47
#例 3-16：?符号的意义 ····· 47
#例 3-17：+符号的意义 ····· 48
#例 3-18：()符号和|符号的意义 ····· 48
#例 3-19：列出以.awk 结尾文件的详细信息 ····· 49
#例 3-20：列出以 0 开头、后面跟 1 个字符且以.pem 为后缀的文件 ····· 49
#例 3-21：列出以 a～h 范围内的字母开头，以.awk 结尾的文件 ····· 49
#例 3-22：列出 a～h 范围内的字母开头，不以.awk 结尾的文件 ····· 50
#例 3-23：列出匹配[a-h]*.awk 或 0?.pem 的所有文件 ····· 50
#例 3-24：模式包含空格时，是否使用双引号的区别 ····· 51
#例 3-25：演示 grep 的多文件查询 ····· 52
#例 3-26：用通配符表示多文件 ····· 52
#例 3-27：grep -c 的用法 ····· 53
#例 3-28：grep -n 的用法 ····· 53
#例 3-29：grep -v 的用法 ····· 53
#例 3-30：grep -i 的用法 ····· 53
#例 3-31：grep -h 的用法 ····· 54
#例 3-32：grep -l 的用法 ····· 54
#例 3-33：grep -s 的用法 ····· 54

混混藏书阁 http://book-life.blog.163.com

#例 3-34：grep -r 的用法 …… 55
#例 3-35：grep -w 的用法 …… 55
#例 3-36：说明 grep 命令的-w 和-x 选项的区别 …… 55
#例 3-37：演示 grep -q 选项 …… 56
#例 3-38：演示 grep 命令的-b 和-o 选项 …… 57
#例 3-39：grep 查找以-符号开头的行 …… 57
#例 3-40：查找空白行 …… 57
#例 3-41：用[]符号设置大小写 …… 58
#例 3-42：grep 和.符号 …… 58
#例 3-43：grep 和*符号 …… 58
#例 3-44：grep 和转义符 …… 59
#例 3-45：-字符在 grep 命令中的特殊性 …… 59
#例 3-46：利用 POSIX 字符类搜索以大写字母开头的行 …… 60
#例 3-47：搜索以空格开头文本行 …… 60
#例 3-48：精确匹配 …… 60
#例 3-49：grep 命令与|字符 …… 61
#例 3-50：egrep 命令的用法 …… 62
#例 3-51：fgrep 命令的用法 …… 62

第 4 章　sed 命令和 awk 编程 …… 64

#例 4-1：sed -n 的用法 …… 67
#例 4-2：sed 命令打印范围行 …… 68
#例 4-3：sed 打印匹配模式行 …… 68
#例 4-4：sed -e 的用法 …… 68
#例 4-5：sed 追加文本 …… 69
#例 4-6：append.sed 演示 sed 追加文本的用法 …… 69
#例 4-7：sed 匹配元字符 …… 70
#例 4-8：sed 结合$符号匹配最后一行 …… 71
#例 4-9：用.和*符号匹配任意字符 …… 71
#例 4-10：匹配不在指定行号范围内的行 …… 71
#例 4-11：sed 命令使用行号与关键字匹配限定行范围 …… 71
#例 4-12：insert.sed 脚本演示 sed 插入文本的用法 …… 72
#例 4-13：modify.sed 脚本演示修改文本的用法 …… 73
#例 4-14：sed 命令删除第一行和最后一行 …… 73
#例 4-15：删除指定范围内所有的行 …… 74
#例 4-16：delete.sed 脚本演示删除不区分大小写与 certificate 匹配的行的用法 …… 74
#例 4-17：sed 命令替换文本时 p 选项的用法 …… 75
#例 4-18：sed 替换选项 g 的用法 …… 76

#例 4-19：sed 替换第 n 次匹配……76
#例 4-20：sed 命令 w 选项的用法……77
#例 4-21：ed 替换文本中&符号的用法……77
#例 4-22：将 1～5 行写入 output 文件……77
#例 4-23：将匹配关键字的行写入文件……77
#例 4-24：sed 利用 r 选项读文件……78
#例 4-25：sed 退出命令……79
#例 4-26：演示 sed 变换命令的用法……79
#例 4-27：sed 显示控制字符……80
#例 4-28：sed {}符号的用法……80
#例 4-29：sed n 编辑命令的用法……81
#例 4-30：sed h、x 和 G 命令的用法……81
#例 4-31：sed H 命令的用法……82
#例 4-32：利用分号分隔多个编辑命令……82
#例 4-33：在二级命令提示符下输入多个编辑命令……83
#例 4-34：第一种方法调用 awk 打印空白行……85
#例 4-35：第二种方法调用 awk 打印空白行……86
#例 4-36：第三种方法调用 awk 打印空白行……86
#例 4-37：awk 命令打印域……87
#例 4-38：$后跟变量指定域号……87
#例 4-39：-F 选项改变分隔符……88
#例 4-40：使用 FS 变量改变分隔符……88
#例 4-41：不同的域分隔符对同一条记录解析出不同结果……88
#例 4-42：使用匹配正则表达式~符号……89
#例 4-43：awk 中的 if 语句……90
#例 4-44：if 语句的多条件匹配……90
#例 4-45：统计 input 文件中的空白行……91
#例 4-46：++x 和 x++的区别……92
#例 4-47：scr2.awk 脚本演示如何计算 sturecord 文件中的平均值……92
#例 4-48：FS、NF、NR 和 FILENAME 的用法……93
#例 4-49：printf 的基本用法……94
#例 4-50：printf 修饰符-和 width 的用法……95
#例 4-51：printf 修饰符.prec 的用法……95
#例 4-52：gsub 函数的用法……96
#例 4-53：index 和 length 函数的用法……96
#例 4-54：match 函数的用法……96
#例 4-55：sub 函数的用法……97
#例 4-56：substr 函数的用法……97

混混藏书阁 http://book-life.blog.163.com

#例 4-57：pass.awk 脚本判断 NF 是否等于 MAX 变量 …… 98
#例 4-58：重定义输出分隔符 …… 98
#例 4-59：命令行参数对 BEGIN 字段无效 …… 98
#例 4-60：论证字符串和数字的区别 …… 100
#例 4-61：判断数组元素是否存在 …… 100
#例 4-62：split 函数的用法 …… 101
#例 4-63：array.awk 脚本演示 split 函数所生成的数组内容 …… 101
#例 4-64：argv.awk 演示 ARGV 中存储了哪些命令行参数 …… 101
#例 4-65：findphone.awk 脚本演示如何应用 ARGV 查找姓名所对应的电话号码 …… 102
#例 4-66：打印 ENVIRON 数组 …… 103

第 5 章　文件的排序、合并和分割 …… 105

#例 5-1：sort 命令用-t 选项设置分隔符 …… 106
#例 5-2：sort 命令用-k 选项指定排序的域号 …… 107
#例 5-3：sort 命令用-n 选项设置根据数值大小排序 …… 108
#例 5-4：sort 命令用-r 选项将排序结果逆向 …… 108
#例 5-5：sort 命令-u 选项的用法 …… 108
#例 5-6：sort 命令-o 选项的用法 …… 109
#例 5-7：sort 命令-c 选项的用法 …… 109
#例 5-8：sort 命令-m 选项的用法 …… 110
#例 5-9：利用 sort 和 awk 实现文件块的排序 …… 111
#例 5-10：uniq 命令的基本用法 …… 112
#例 5-11：uniq 命令-c 选项的用法 …… 113
#例 5-12：uniq 命令-m 选项的用法 …… 113
#例 5-13：join 命令的基本用法 …… 115
#例 5-14：join 命令-a 选项的用法 …… 116
#例 5-15：join 命令-v 选项的用法 …… 117
#例 5-16：join 命令-o 选项的用法 …… 117
#例 5-17：join 命令-1 和-2 选项的用法 …… 117
#例 5-18：TEACHER.db 未排序，join 命令的出错信息 …… 118
#例 5-19：cut 命令-c 选项的用法 …… 119
#例 5-20：cut 命令-d 和-f 选项的用法 …… 119
#例 5-21：paste 命令的基本用法 …… 120
#例 5-22：paste 命令-d 选项的用法 …… 120
#例 5-23：paste 命令-s 选项的用法 …… 121
#例 5-24：paste 命令-选项的用法 …… 121
#例 5-25：split 命令按行切割文件 …… 122
#例 5-26：split 命令-b 选项的用法 …… 123

#例 5-27：split 命令-C 选项的用法 …… 123
#例 5-28：tr 命令-d 选项的用法 …… 124
#例 5-29：tr 命令-s 选项删除空白行 …… 125
#例 5-30：tr 命令-s 选项压缩重复字符 …… 126
#例 5-31：tr 命令实现大小写字母的互换 …… 126
#例 5-32：用 POSIX 字符类表示大小写 …… 126
#例 5-33：tr 命令-c 选项的用法 …… 126
#例 5-34：[character*n]在 tr 命令中的用法 …… 127
#例 5-35：tar 命令-c 和-f 选项的用法 …… 127
#例 5-36：tar 命令-r 选项的用法 …… 128
#例 5-37：tar 命令解压压缩包 …… 129
#例 5-38：gzip 命令基本用法 …… 129
#例 5-39：gzip 命令-d 选项的用法 …… 130

第 6 章　变量和引用 …… 133

#例 6-1：变量的赋值和替换 …… 134
#例 6-2：变量赋值中使用另一个变量 …… 135
#例 6-3：unset 命令的用法 …… 135
#例 6-4：“:=” 和 “:-” 的相似之处 …… 135
#例 6-5：“:=” 和 “:-” 的区别 …… 136
#例 6-6：“:?” 和 “?” 的用法 …… 136
#例 6-7：设置变量只读 …… 136
#例 6-8：显示所有的只读变量 …… 136
#例 6-9：integer.sh 脚本分析 Shell 处理无类型变量的方式 …… 137
#例 6-10：null-undeclare.sh 脚本处理空字符串和未定义变量的方式 …… 138
#例 6-11：演示定义环境变量 …… 139
#例 6-12：列出系统中的环境变量 …… 139
#例 6-13：清除环境变量 …… 140
#例 6-14：打印 PWD 和 OLDPWD 变量值 …… 140
#例 6-15：打印 PATH 变量值 …… 140
#例 6-16：显示 root 和 editor 两个用户的 HOME 变量值 …… 141
#例 6-17：打印 USER 和 UID 变量的值 …… 141
#例 6-18：打印 PS1 变量值 …… 141
#例 6-19：改变 PS1 变量值 …… 142
#例 6-20：打印 PS2 变量值 …… 142
#例 6-21：改变 PS2 变量值 …… 142
#例 6-22：改变 IFS 变量值 …… 142
#例 6-23：.profile 文件示例 …… 143

混混藏书阁 http://book-life.blog.163.com

#例 6-24：father.sh 脚本输出其进程号，分别定义本地变量和环境变量，并创建子进程……144
#例 6-25：position 脚本说明$0,$1,$2,…,${10},$*……145
#例 6-26：无引用和引用的区别……147
#例 6-27：double.sh 脚本演示$variable 和"$variable"的区别……148
#例 6-28：双引号和单引号的区别……148
#例 6-29：stephane.sh 脚本说明了单引号不屏蔽单引号本身……148
#例 6-30：反引号的基本用法……149
#例 6-31：反引号和引号的综合使用……149
#例 6-32：命令替换删除换行符……150
#例 6-33：命令替换删除换行符……150
#例 6-34：反引号与$()在处理双反斜线符号的区别……150
#例 6-35：bash Shell 和 C 程序结合……151
#例 6-36：将双引号和美元符号转义……152
#例 6-37：将转义符本身转义……152
#例 6-38：将转义符赋给变量……152
#例 6-39：weirdvars.sh 脚本演示怪异的变量赋值……153
#例 6-40：escape.sh 脚本演示 echo -e 和\符号……153
#例 6-41：$"符号与-e 选项等价……154
#例 6-42：将特殊字符赋给变量的方法……154
#例 6-43：echo 命令的-n 选项……155

第 7 章　退出、测试、判断及操作符……157

#例 7-1：用退出状态显示文件是否创建成功……158
#例 7-2：退出状态为 2 和 127 的例子……158
#例 7-3：一个整数变量与一个整数常量比较的例子……160
#例 7-4：两个整数变量比较的例子……160
#例 7-5：将整数运算符用于浮点数运算……161
#例 7-6：字符串比较运算的例子……161
#例 7-7：比较两字符串是否相等……161
#例 7-8：空格造成测试结果不相等的例子……162
#例 7-9：大小写区分的例子……162
#例 7-10：变量弱化造成的赋值结果的不同……162
#例 7-11：判断输入的文件是文件还是目录……163
#例 7-12：测试文件是否存在……163
#例 7-13：用于测试文件权限的例子……163
#例 7-14：显示通过 chmod 命令增减文件权限的测试结果……164
#例 7-15：一个逻辑非的例子……165
#例 7-16：一个逻辑与的例子……165

#例 7-17：一个逻辑或的例子 …… 166
#例 7-18：if_exam1.sh 脚本判断输入的数是否小于 15 …… 167
#例 7-19：if_exam2.sh 脚本用于创建一个文件，然后测试是否创建成功，并测试文件权限 …… 168
#例 7-20：exit_exam.sh 脚本用于判断用户输入的字符串是否为空 …… 169
#例 7-21：ifelse_exam1 脚本用于判断输入的文件名是否有对应的文件存在 …… 170
#例 7-22：ifelse_exam2.sh 脚本用于删除一个文件，并判断是否执行了该操作 …… 170
#例 7-23：ifelse_exam3.sh 脚本用于检查输入的字符串是否是一个当前目录的文件名 …… 172
#例 7-24：ifelse_exam4 脚本用于对学生成绩进行分类 …… 173
#例 7-25：ifelifelse_exam1.sh 脚本使用 if/elif/else 结构对脚本 ifelse_exam4 进行改写 …… 174
#例 7-26：ifelifelse_exam2.sh 脚本用于判断输入的年份是否是闰年 …… 175
#例 7-27：case_exam1.sh 脚本提示输入一个数字（1～12），然后显示其对应月份的英文 …… 177
#例 7-28：case_exam2.sh 脚本用于对输入一个分数类别（A～E）显示其对应的分数段 …… 179
#例 7-29：一个关于整数运算符的例子 …… 180
#例 7-30：一个取余和幂运算的例子 …… 180
#例 7-31：除 0 错误 …… 180
#例 7-32：复合运算符的例子 …… 181
#例 7-33：位运算的例子 …… 182
#例 7-34：复合位运算的例子 …… 182
#例 7-35：increment_and_decrement_exam1.sh 脚本实现变量的自增自减 …… 183
#例 7-36：constant_exam1.sh 脚本实现数字常量 …… 184
#例 7-37：constant_exam1.sh 脚本使用 num#实现进制表示 …… 184

第 8 章　循环与结构化命令 …… 187

#例 8-1：for_exam1.sh 脚本演示利用 for 循环显示 5 次欢迎操作 …… 188
#例 8-2：for_exam2.sh 脚本利用 for 循环显示 5 次欢迎操作 …… 188
#例 8-3：for_exam3.sh 脚本使用列表 for 循环计算 1～100 内所有的奇数之和 …… 189
#例 8-4：for_exam4.sh 脚本使用列表 for 循环和 seq 命令计算 1～100 内所有的奇数之和 …… 189
#例 8-5：for_exam5.sh 脚本显示周一到周日对应的英文翻译 …… 190
#例 8-6：for_exam6.sh 脚本通过 ls 命令和列表 for 循环显示当前目录下的文件 …… 190
#例 8-7：for_exam7.sh 脚本通过通配符（*）和列表 for 循环显示当前目录下的文件 …… 191
#例 8-8：for_exam8.sh 脚本演示列表 for 循环实现通过命令行来传递脚本中 for 循环列表参数 …… 191
#例 8-9：for_exam9.sh 脚本演示一个不带参数列表 for 循环的例子 …… 192
#例 8-10：for_exam10.sh 脚本使用类 C 风格的 for 循环实现输出前 5 个正整数 …… 193
#例 8-11：for_exam11.sh 脚本使用类 C 风格的 for 循环计算 1～100 内所有的奇数之和 …… 194
#例 8-12：for_exam12.sh 脚本使用逗号运算符对两个变量进行操作 …… 195
#例 8-13：for_exam13.sh 脚本演示产生死循环的例子 …… 195
#例 8-14：while_exam1.sh 脚本演示一个 while 的例子，用于显示 1～5 …… 197
#例 8-15：while_exam2.sh 脚本使用计数器控制的 while 循环计算 1～100 内所有的奇数之和 …… 197

混混藏书阁 http://book-life.blog.163.com

#例 8-16：while_exam3.sh 脚本演示使用结束标记控制的 while 循环实现猜 1～10 内的数……198
#例 8-17：while_exam4.sh 演示使用结束标记控制的 while 循环实现阶乘的操作……199
#例 8-18：while_exam5.sh 脚本演示使用标志控制的 while 循环实现猜 1～10 内的数……200
#例 8-19：while_exam6.sh 脚本演示标志控制的 while 循环求累加和（1+2+…+n）……201
#例 8-20：while_exam7.sh 脚本命令行控制的 while 循环……202
#例 8-21：until_exam1.sh 脚本通过 until 循环计算 1～5 的平方……203
#例 8-22：until_exam2.sh 脚本演示使用 until 循环实现猜 1～10 内的数……203
#例 8-23：nested_loop_exam1.sh 脚本演示使用 for 循环嵌套实现九九乘法表……204
#例 8-24：nested_loop_exam2.sh 脚本演示 for 循环嵌套实现一个 8×8 格的棋盘……205
#例 8-25：nested_loop_exam3.sh 脚本演示 for 循环嵌套实现“*”图案排列……206
#例 8-26：nested_loop_exam4.sh 脚本演示 for 和 while 循环结合的循环嵌套反序的“*”图案排列…207
#例 8-27：no _break_exam.sh 脚本演示不使用 break 的循环计算 1～100 内整数的和……208
#例 8-28：break_exam1.sh 演示使用 break 的循环计算 1～100 内整数的和……208
#例 8-29：break_exam2.sh 演示使用 break 的循环跳出内层循环……209
#例 8-30：break_exam3.s 脚本演示使用 break 控制符跳出整个循环……210
#例 8-31：continue_exam1.sh 脚本演示使用 continue 循环显示 100 以内能被 7 整除的数……211
#例 8-32：continue_exam2.sh 脚本演示使用 continue 循环跳出内层循环的九九乘法表……212
#例 8-33：continue_exam3.sh 脚本演示使用 continue 控制符跳出外层循环……212
#例 8-34：select_exam1.sh 脚本演示交互式显示用户喜欢的颜色，并让用户选择……213
#例 8-35：select_exam2.sh 脚本演示命令行输入形式的 select 结构……214

第 9 章　变量的高级用法……217

#例 9-1：BASH 和 SHELL 变量的值……218
#例 9-2：bashver.sh 输出 BASH_VERSINFO 数组的值……218
#例 9-3：BASH_VERSION 变量值……219
#例 9-4：四条 pushd 命令后的目录栈结果……219
#例 9-5：演示目录栈和$DIRSTACK 变量……219
#例 9-6：演示$GLOBIGNORE 变量的用法……220
#例 9-7：root 用户的 GROUPS 变量值……221
#例 9-8：显示 HOSTTYPE 和 MACHTYPE 变量的值……221
#例 9-9：readreply.sh 演示 REPLY 变量在 read 命令中的用法……221
#例 9-10：selectreply.sh 脚本演示 REPLY 变量在 select 命令中的用法……222
#例 9-11：修改 PS3 变量，影响 select 命令的提示符……223
#例 9-12：runsec.sh 脚本演示 SECONDS 变量的用法……223
#例 9-13：timedread.s 脚本等待用户输入 3 秒……225
#例 9-14：演示计算字符串长度的两种方法……226
#例 9-15：字符串是否用引号引起的区别……226
#例 9-16：expr index 命令的用法……227

#例 9-17：expr match 命令的用法 …… 227
#例 9-18：演示使用${string:position}命令抽取子串 …… 228
#例 9-19：演示使用${string:position:length}命令抽取子串 …… 228
#例 9-20：演示使用从 string 的右边抽取子串 …… 228
#例 9-21：缺少$length 参数的 expr substr 命令 …… 229
#例 9-22：${...}和 expr substr 命令的比较结果 …… 229
#例 9-23：演示如何抽取 another_string 开头处的子串 …… 229
#例 9-24：演示如何抽取 another_string 结尾处的子串 …… 230
#例 9-25：删除 another_string 开头处与 substring 匹配的最短子串和最长子串 …… 230
#例 9-26：删除 another_string 结尾处与 substring 匹配的最短子串和最长子串 …… 231
#例 9-27：替换与 substring 匹配的子串 …… 231
#例 9-28：*字符在替换子串命令中的用法 …… 232
#例 9-29：替换开头处和结尾处子串的命令 …… 232
#例 9-30：vartype.sh 脚本说明 declare -i 命令和 let 命令 …… 233
#例 9-31：doubleparenthese.sh 脚本演示双圆括号的用法 …… 233
#例 9-32：cstyle.sh 脚本利用双圆括号方法实现 C 语言风格的变量操作 …… 234
#例 9-33：indirect.sh 脚本演示间接变量引用 …… 235
#例 9-34：indirectapp1.sh 脚本说明间接变量引用的应用 …… 236
#例 9-35：演示 expr 操作符是元字符 …… 238
#例 9-36：expr 操作符两端必须有空格 …… 238
#例 9-37：演示 bc 在命令行中的用法 …… 239
#例 9-38：演示 bc 运算器 scale 变量的作用 …… 240
#例 9-39：calare.sh 脚本用 bc 运算器计算圆面积 …… 240

第 10 章　I/O 重定向 …… 243

#例 10-1：演示管道的基本用法 …… 244
#例 10-2：三个命令用管道进行通信 …… 244
#例 10-3：ls -l | more 命令的执行结果 …… 247
#例 10-4：演示 cat 命令-v 选项的用法 …… 248
#例 10-5：用 sed 处理 ls -l 命令的结果 …… 249
#例 10-6：sed 和 cat 结合对文件进行处理 …… 249
#例 10-7：sed、管道和变量赋值相结合的示例 …… 250
#例 10-8：awk 和管道结合处理字符串 …… 250
#例 10-9：awk 直接引用变量名的结果 …… 251
#例 10-10：将/etc/passwd 的第 1 域按字母排序后打印 …… 251
#例 10-11：用 awk 打印 ls /usr 命令的结果 …… 252
#例 10-12：用 awk 查看文件系统的可用空间 …… 252
#例 10-13：演示 tee 命令的基本用法 …… 254

混混藏书阁 http://book-life.blog.163.com

#例 10-14：tee -a 命令的用法……254
#例 10-15：cat 和>符号结合成为简易文本编辑器……255
#例 10-16：利用>>符号向 newfile 追加文本……255
#例 10-17：说明>符号与>>符号的区别……256
#例 10-18：演示>|符号的基本用法……256
#例 10-19：标准错误输出的用法……257
#例 10-20：说明<符号的用法……258
#例 10-21：演示<<delimiter 符号的用法……258
#例 10-22：-<<delimiter 符号的用法……258
#例 10-23：execin.sh 脚本使用 exec 将 stdin 重定向到文件……259
#例 10-24：execout.sh 脚本将 stdout 重定向到文件……260
#例 10-25：execerr.sh 脚本将 stdout 和 stderr 重定向到文件……261
#例 10-26：rewhile.sh 脚本演示 while 循环的重定向……263
#例 10-27：refor.sh 脚本演示 for 循环的重定向……264
#例 10-28：reif.sh 脚本演示 if/then 结构的输出重定向……265
#例 10-29：命令的执行结果……268
#例 10-30：eval 执行简单命令……269
#例 10-31：eval 执行复杂命令……269
#例 10-32：evalpos.sh 脚本显示最后一个输入参数……269
#例 10-33：显示 evalsource 文件内容……270
#例 10-34：evalre.sh 脚本将 evalsource 文件中每一行的第 1 列作为变量名，第 2 列作为变量值……270

第 12 章　子 Shell 与进程处理……288

#例 12-1：conlon.sh 脚本演示冒号表示永真的用法……291
#例 12-2：利用冒号清空文件……292
#例 12-3：subsvar.sh 脚本演示圆括号结构用法和 BASH_SUBSHELL 变量……292
#例 12-4：subsep.sh 脚本测试子 Shell 定义环境变量是否对父 Shell 有效……293
#例 12-5：subsenv.sh 脚本演示子 Shell 设定 bash Shell 选项的用法……294
#例 12-6：subscol.sh 脚本演示利用子 Shell 测试变量是否定义过……295
#例 12-7：subsparallel.sh 脚本演示子 Shell 用于并行计算的用法……296
#例 12-8：resshell.sh 脚本演示 Shell 的正常模式和限制模式的区别……297
#例 12-9：anotherres.sh 脚本演示以/bin/bash -r 方式进入限制模式……298
#例 12-10：后台运行一个作业，[1]是作业号，4693 是进程号……302
#例 12-11：演示 norify 选项的功能……302
#例 12-12：演示利用 fg 命令将后台作业放到前台运行……302
#例 12-13：演示 fg 命令利用作业号指定作业……303
#例 12-14：内建命令 jobs 的用法……304
#例 12-15：将正在运行的作业阻塞的用法……304

#例 12-16：演示在 Ctrl+Z 组合键之后利用 bg 命令将阻塞态作业转入后台运行 …… 305
#例 12-17：disown 命令的用法 …… 305
#例 12-18：backls.sh 脚本演示 wait 命令的用法 …… 306
#例 12-19：Ctrl+C 组合键的用法 …… 307
#例 12-20：Ctrl+\的用法 …… 307
#例 12-21：kill 命令的用法 …… 308
#例 12-22：selfkill.sh 脚本演示 kill 命令通过用进程号杀掉自己的进程的用法 …… 308
#例 12-23：traploop.sh 脚本演示 trap 命令捕捉 INT 信号的用法 …… 309
#例 12-24：nokillme.sh 脚本演示 trap 命令忽略信号的用法 …… 310
#例 12-25：forever.sh 脚本演示无限循环等待 …… 310
#例 12-26：subsig.sh 脚本演示子 Shell 继承父 Shell 所忽略的信号 …… 311

第 13 章　函数 …… 314

#例 13-1：function1.sh 演示了一个简单的函数调用过程 …… 315
#例 13-2：function2.sh 脚本实现在循环中调用函数 …… 316
#例 13-3：function3.sh 用函数来显示当前目录中存在多少个文件和多少个子目录 …… 317
#例 13-4：脚本 function4.sh 用于说明函数如何传递参数和传值前后的变量值如何变化 …… 318
#例 13-5：function5.sh 用于实现两数加、减、乘和除四则运算 …… 319
#例 13-6：function6.sh 演示了函数如何传递间接参数 …… 320
#例 13-7：function7.sh 用于根据用户输入显示星期 …… 321
#例 13-8：function8.sh 实现了在脚本中放置多个函数 …… 322
#例 13-9：function9.sh 实现函数相互调用 …… 324
#例 13-10：function10.sh 用于显示一个不多于 5 位的正整数的位数，并按顺序显示各个数位的值 … 326
#例 13-11：function11.sh 用于调用局部变量和全局变量 …… 327
#例 13-12：function12.sh 演示了一个函数递归调用 …… 328
#例 13-13：function13.sh 演示了一个使用局部变量的函数递归调用的例子 …… 329
#例 13-14：function14.sh 通过不使用局部变量的函数递归实现汉诺塔 …… 331

第 14 章　别名、列表及数组 …… 335

#例 14-1：alias 命令的用法 …… 336
#例 14-2：unalias 命令 …… 337
#例 14-3：alias.sh 脚本演示在脚本中使用 alias 命令 …… 337
#例 14-4：loopalias.sh 脚本演示测试别名能否在 if/then、while 循环内使用 …… 338
#例 14-5：andlist1.sh 脚本演示与列表的基本用法 …… 340
#例 14-6：andlist2.sh 脚本演示与列表的基本用法 …… 340
#例 14-7：orlist.sh 脚本演示利用或列表实现对输入参数的个数判断 …… 341
#例 14-8：array_eval1.sh 脚本演示数组赋值的方法 …… 342
#例 14-9：array_eval2.sh 脚本演示用圆括号对数组赋值的方法 …… 343

混混藏书阁 http://book-life.blog.163.com

#例 14-10：array_eval3.sh 脚本演示用圆括号对数组赋值的用法……343
#例 14-11：array_eval4.sh 脚本演示用圆括号对数组赋值的方法……344
#例 14-12：array_print1.sh 脚本演示利用@或*符号打印数组的所有元素……345
#例 14-13：array_ print2.sh 脚本演示当数组不连续赋值时，@和*符号的输出结果……346
#例 14-14：string_array.sh 脚本演示数组的字符串操作……347
#例 14-15：arrivedcity.sh 脚本演示数组与 read 命令、unset 命令的用法……348
#例 14-16：combine_array.sh 脚本演示数组的连接操作……349
#例 14-17：stack.sh 脚本演示利用数组实现堆栈的方法……350
第 15 章　一些混杂的主题……359
#例 15-1：一种坏风格缩进形式的脚本……360
#例 15-2：一种好风格缩进形式的脚本，是对例 15-1 中的脚本进行的改写……361
#例 15-3：在例 15-2 的基础上加上必要的空行使代码更清晰……362
#例 15-4：一个编程风格良好的 Linux Shell 脚本……365
#例 15-5：一个复杂但可简化的程序……366
#例 15-6：对例 15-5 中的代码进行简化……367
#例 15-7：一个不灵活的脚本例子……368
#例 15-8：一个灵活的脚本例子……369
#例 15-9：一个有参数但无提示说明的脚本……370
#例 15-10：一个有参数且有提示说明的脚本……370
#例 15-11：一个未使用 shift 命令的例子……371
#例 15-12：一个使用 shift 命令对例 15-11 中的脚本进行改动……372
#例 15-13：回显命令行参数，并形成一个至多 3 行参数命令组成的倒三角形……373
#例 15-14：回显命令行参数，并形成一个不限制行参数命令组成的倒三角形……373
#例 15-15：一个关于 getopts 的例子……374
#例 15-16：一个复杂点的 getopts 的例子……375
#例 15-17：一个非交互性脚本的例子……377
#例 15-18：判断执行脚本的用户是否是 root，若不是，则不执行脚本的其他内容……377
#例 15-19：一个交互式 Shell 脚本的例子……378
#例 15-20：使用 df 命令查看当前已经加载的设备的主设备号……380
#例 15-21：/dev 目录中的目录和文件……380
#例 15-22：dev1.sh 脚本通过/dev/zero 建立一个交换文件……381
#例 15-23：dev2.sh 通过/dev/null 将文件 file1.sh 隐藏……383
#例 15-24：通过 cat 命令查看中断……385
#例 15-25：通过 echo 命令对文件大小的最大上限进行修改……386
#例 15-26：通过 ps 命令查看正在运行的进程……387
#例 15-27：查看/proc/6206 目录下的信息……387
#例 15-28：proc1.sh 用于判断进程是否运行……387

#例 15-29：proc2.sh 用于查看文件系统支持的类型……388
#例 15-30：通过 cat 命令获得网络套接字的使用统计……389
#例 15-31：通过 cat 命令显示 TCP 的使用情况……389
#例 15-32：一个包装了 sed 的脚本……390
#例 15-33：包装 awd 脚本的 Shell 包装……391
#例 15-34：color_script1.sh 脚本输出带颜色的脚本……393
#例 15-35：一个体现错误、警告、完成和普通信息区别的例子……395
#例 15-36：用 shc 工具对 array_eval2.sh 脚本进行加密……399
#例 15-37：virus1.sh 脚本实现一个简单的病毒……399
#例 15-38：对例 15-37 中的病毒脚本进行的改进……400
#例 15-39：一个特洛伊木马的例子……400

第 16 章　Shell 脚本调试技术……403

#例 16-1：misskey.sh 脚本演示漏写关键字错误……404
#例 16-2：runsec.sh 脚本演示逻辑错误……405
#例 16-3：computer.sh 脚本演示逻辑错误……406
#例 16-4：trapdebug.sh 脚本利用 trap 命令捕捉 DEBUG 信号跟踪变量值……407
#例 16-5：trapexit.sh 脚本利用 trap 命令捕捉 EXIT 信号跟踪函数结束……408
#例 16-6：traperr.sh 脚本演示 trap 命令捕捉 ERR 信号跟踪函数或命令异常的用法……409
#例 16-7：obtainIP.sh 脚本获取本地 IP 地址，将其存储到 localIP 变量中……409
#例 16-8：debugblock.sh 脚本演示调试钩子的用法……411
#例 16-9：misskey.sh 脚本演示利用-n 选项来测试脚本的语法错误……412
#例 16-10：演示用 sh -n 命令对脚本进行语法检查……413
#例 16-11：将-x 选项与 trap 捕捉 DEBUG 信号相结合对 trapdebug.sh 脚本进行调试……414
#例 16-12：nestfun.sh 脚本演示-x 选项提示符的定制功能……416

第 17 章　bash Shell 编程范例……420

#例 17-1：htmlconver.sh 脚本将文本文件转换为 HTML 文件……422
#例 17-2：topn.sh 脚本查找文本中 *n* 个出现频率最高的单词……425
#例 17-3：random.sh 脚本产生 5 个随机数……427
#例 17-4：seqrand.sh 脚本产生定长的随机字符串……428
#例 17-5：dice.sh 脚本模拟投骰子游戏……429
#例 17-6：显示了 alloy 用户的 crontab……431
#例 17-7：重复工作的解决方案……432
#例 17-8：fileback.sh 脚本用于将/etc 目录的所有内容进行备份……433
#例 17-9：log.sh 脚本登录 MySQL 服务器……437
#例 17-10：查看数据库和进入数据库的命令……438
#例 17-11：create.sh 脚本创建 people 表……439

混混藏书阁 http://book-life.blog.163.com

#例 17-12：insert.sh 脚本用于向 people 表中插入数据……440
#例 17-13：insert2.sh 脚本演示按照 name sex birth birthaddr 的顺序插入记录……440
#例 17-14：female.sh 脚本打印出 newpeople 表中所有女性的姓名和出生地……441
#例 17-15：Ganglia 的安装……444
#例 17-16：启动和查看 gmond……448
#例 17-17：获取 Ganglia 所监控的服务器性能参数……449
#例 17-18：initialization.sh 脚本实现监控数据表的初始化……452
#例 17-19：monitor.sh 脚本提取监控数据并存储到数据库……453
#例 17-20：run.sh 脚本封装监控系统……455

第1章

Shell 脚本编程概述

Shell 脚本语言是 Linux/UNIX 系统中一种重要的脚本语言，在 Linux/UNIX 领域应用极为广泛，熟练掌握 Shell 脚本编程是一个优秀的 Linux/UNIX 开发者和系统管理员的必经之路。作为本书的开篇，本章从 Linux 和 Shell 的概念开始介绍，分析 Shell 脚本语言和高级程序设计语言的区别，总结 Shell 脚本编程的优势，再给出第一个 Shell 脚本的例子，结合例子介绍 Shell 脚本的基本元素及其执行脚本的方法。

混混藏书阁 http://book-life.blog.163.com

1.1 Linux 和 Shell 概述

1.1.1 Linux 简介

Linux 是一套可免费使用和自由传播的类 UNIX 操作系统。1991 年，芬兰赫尔辛基大学学生 Linus 开发了 Linux 内核。此后一大批程序爱好者、软件技术专家对 Linux 进行修改和完善。Linux 操作系统从诞生到现在，其开放、安全、稳定的特性得到越来越多用户的认可，又由于其低成本、自由开发以及安全可靠等优势，促使各国政府和企业纷纷对 Linux 提供强有力的支持。Linux 的应用和发展前景变得越来越广阔。

自 1991 年 10 月 5 日 Linus Torvalds 在新闻组 comp.os.minix 发表了 Linux V0.01，Linux 开启了其迅猛发展的步伐。经过近 20 年的发展，Linux 成为一个支持多用户、多进程、多线程、实时性较好、功能强大而稳定的操作系统。它可以运行在 x86、Sun Sparc、Digital Aopha、680x0、PowerPC、MIPS、ARM 等平台上，是目前支持硬件平台最多的操作系统。由于用户操作习惯等因素的制约，Linux 在桌面领域发展不是很好，但是在其他领域都取得了巨大的进步和成功。在企业应用领域方面，Linux 得到了除微软公司之外几乎所有知名软件和硬件公司的支持。这包括 IBM、HP、Sun、Intel、AMD、Sony 等，软件公司有 CA、Veritas、BEA、Oracle、SAP、Borland 等，使得 Linux 操作系统在企业运算领域具有强大的发展潜力。

1. Linux 发展大事记

1991 年 9 月，Linux 0.01 诞生。

1991 年 10 月，Linux 0.02 诞生，并且正式被取名为 Linux。

1991 年 11 月，Linux 0.10 推出。

1991 年 12 月，Linux 0.11 随后推出，当 Linux 非常接近于一种稳定可靠的系统时，Linus 决定将 0.13 版本改为 0.95 版本。

1992 年 4 月，第一个 Linux 新闻组 comp.os.linux 诞生，同时 Linux 0.95 首次可以运行 X-Window。

1992 年 10 月，第一个可以安装的 Linux 版本 SLS（Softlanding Linux System）诞生，它由 Peter MacDonald 推出，其安装界面如图 1-1 所示。

1992 年至 1994 年期间，在随后的岁月中给 Linux 带来了深厚影响力的三个发行版 Slackware、Red Hat（红帽）和 Debian（蝶变）都出现了。

1994 年 3 月，终于出现了带有独立宣言意味的 Linux 1.0.0 版本，共 176.250 行代码，此时 Linux 已经成为一个功能完备的操作系统，其内核写得紧凑高效，可以充分发挥硬件的性能，在 4MB 内存的 80386 机器上也表现得非常好，同时其还支持 TCP/IP 栈和 X 窗口系统，图 1-2 是一个当时的 Linux 运行界面截图。此时 Linux 的三大著名发行版包括 Debian、SUSE（由 Slackware 发展而来）、Red Hat。

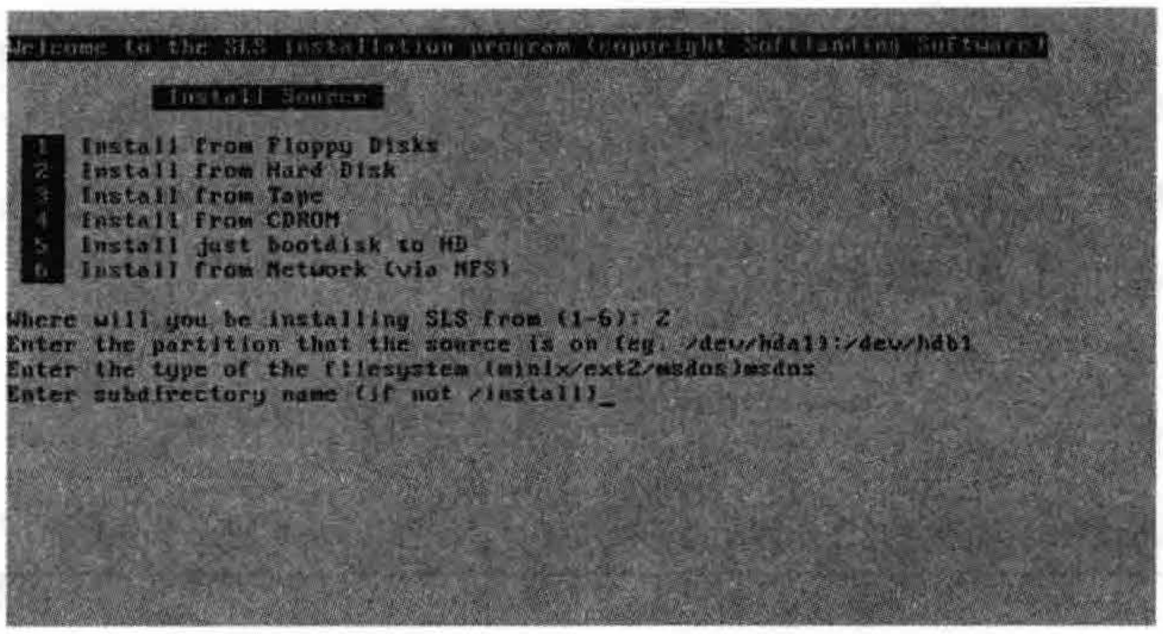


图 1-1　第一个可安装的 Linux 版本 SLS 安装界面

图 1-2　Linux 1.0.0 运行截图

1995 年至 2000 年，进入 1995 年之后，Linux 得到了飞速的发展，从以上三个大的发行版中分裂出了许多其他优秀的发行版，其中就有基于 Red Hat 发展而来的中科红旗（Red Flag）。在这几年中最重要的事件莫过于 KDE 和 GNOME 的发布，前者于 1998 年发布 1.0 版本，被 Mandrake 发行版第一个采用，其早期界面如图 1-3 所示；而后者于 1999 年发布了第一版，随后被 Red Hat 采用，图 1-4 是 Red Hat 上运行的 GNOME。

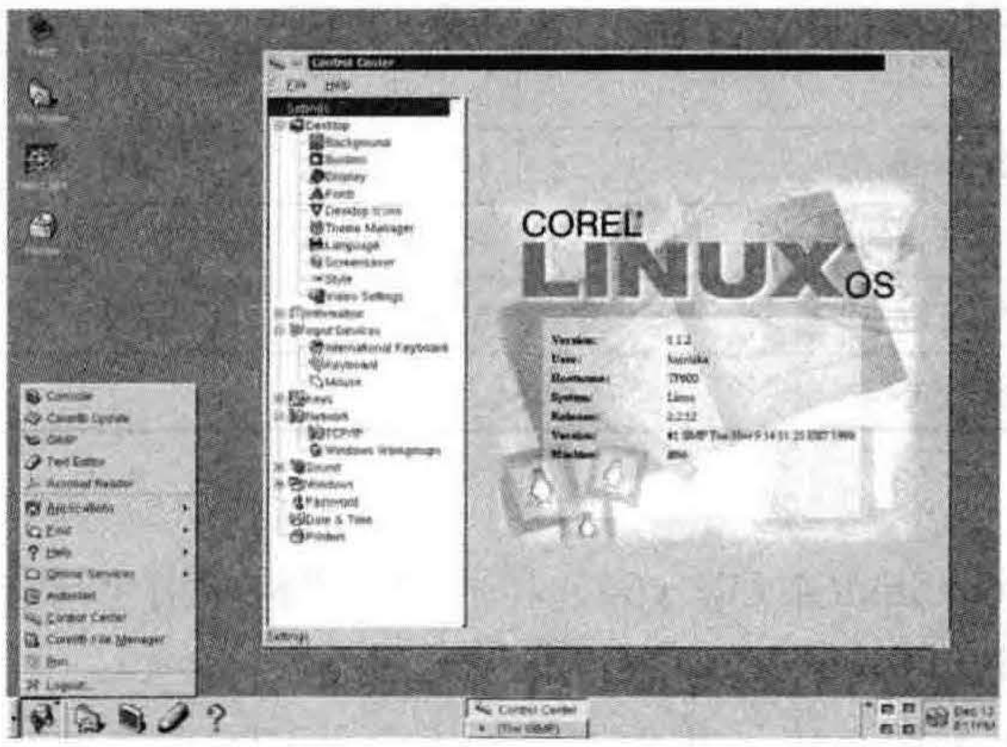


图 1-3　早期 KDE 的运行界面

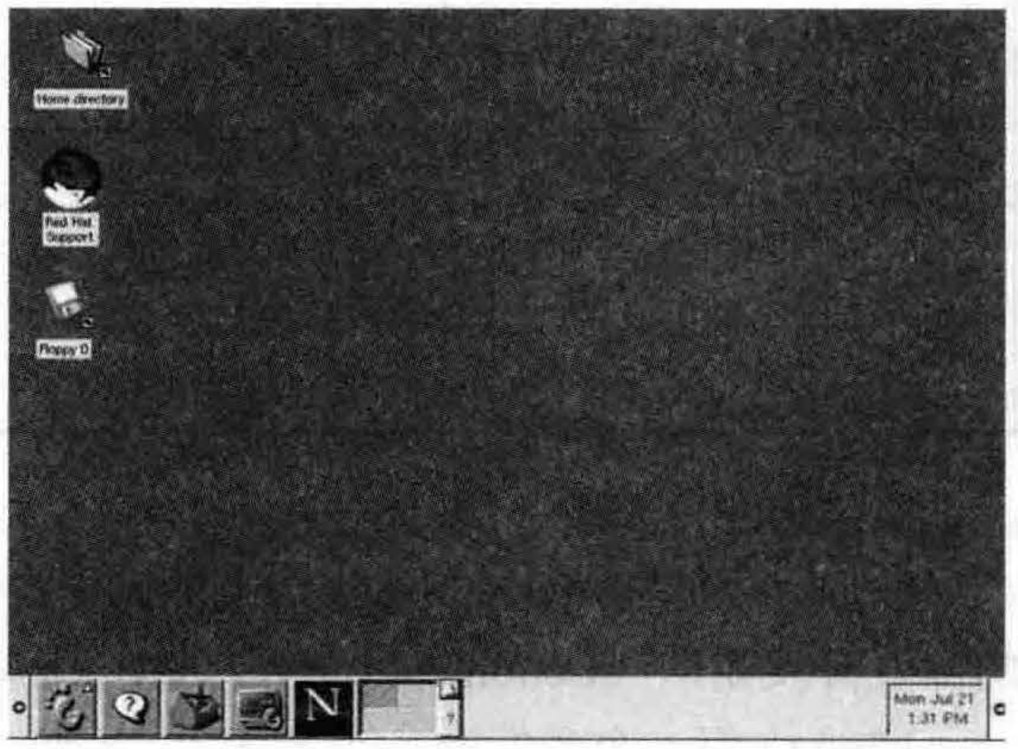


图 1-4　早期 GNOME 的运行界面

2000 年至 2005 年，此时的 Linux 在众多开发者的共同努力下得以完善，2.4 版本的 Linux 内核提供了对 USB、PC 卡、ISA、蓝牙、RAID 和 EXT3 文件系统等的支持；2.6 版本的 Linux 内核则进一步提供了对 PAE、64 位处理器、16TB 大容量存储器以及 EXT4 文件系统等的支持。2000 年，Linux 基金会成立，开始赞助 Linux 系统的相关开发工作，并且致力于维护 Linux 内在的自由、合作的核心价值观。在这些年中，Linux 操作系统还出现了一种新的发行版形式——Live（使用），这种发行版支持从光盘直接运行 Linux 操作系统，可以给不熟悉 Linux 的用户最快捷的体验。同样在这些年中包括 Red Hat 在内的大量商业 Linux 公司开始架构和发布不同的商业 Linux 版本，大大促进了 Linux 操作系统的发展。在 2004 年 10 月 20 日，截止到目前（2014 年），最流行的桌面 Linux 发行版 Ubuntu 发布了第一个版本 4.10。

2006 年至 2014 年，在这段时间内，Linux 的发行版呈现了爆炸式的增强（参考表 1-1），

混混藏书阁 http://book-life.blog.163.com

KDE 发布了 KDE 4.2，GNOME 则发布了 GNOME 3，一个全新的桌面环境 Unity 在 Ubuntu 11.04 发行版上出现，而 Linux 内核也发布到了 3.13.3 版（2014 年 2 月 13 日）。在这几年中，另外一个突破是 Linux 被大量地移植到基于 ARM 等处理器的嵌入式系统中，而基于 Linux 内核的移动端商用操作系统 Android（安卓）也在 2009 年 9 月发布。

2. Linux 的常见发行版

Linux 的常见发行版说明如表 1-1 所示。

表 1-1　Linux 操作系统的发行版

原始版本	最新发行版		
Debian	Debian	Ubuntu	Linux Mint
	Knoppix	MEPIS	sidux
	CrunchBang Linux	Chromium OS	Google chrome OS
Red Hat	Red Hat Enterprise Linux	Fedora	CentOS
	Scientific Linux	Oracle Linux	
Mandriva	Mandriva Linux	PCLinux OS	Unity Linux
	Mageia		
Gentoo	Gentoo Linux	Sabayon Linux	Calculate Linux
	Funtoo Linux		
Slackware	Slackware	Zenwalk	VectorLinux
其他	SUSE	Arch Linux	Puppy Linux
	Damn Small Linux	MeeGo	Slitaz
	Tizen	StartOS	

3. Ubuntu 简介

Ubuntu 是基于 Debian GNU/Linux，支持 x86、amd64（即 x64）和 ARM 架构，由全球化的专业开发团队（Canonical Ltd）打造的开源 GNU/Linux 操作系统。

Ubuntu 每 6 个月发布一个新版本，而每个版本都有代号和版本号，其中有 LTS 是长期支持版。版本号基于发布日期，例如，最新的版本 13.10 代表是在 2013 年 10 月发行的。

在 Ubuntu 的发展历史上有以下两个很重要的版本。

- 8.04 LTS 版，在该版本中提供了 Wubi（Windows Ubuntu-Based Installer）安装方式，其支持用户在 Windows 中以安装普通应用软件的方法安装和卸载 Ubuntu，其在不需要改变分区设置，不需要修改启动文件，不会给 Windows 带来任何改变的同时提供了完整的硬件接入。
- 11.04 版，该版本采用 Unity 作为默认的桌面环境，Unity 是一个由 Canonical 公司开发的基于 GNOME 的桌面环境，其目的是更有效地利用显示器的屏幕尺寸，并且消耗更少的系统资源，例如，将一些快捷方式放在左侧，如图 1-5 所示是 Unity 的运行界面。

Ubuntu 作为 Linux 发行版的最大特点是比其他发行版本的界面更加友好，同时有更好、更加稳定的技术支持和较快的更新速度，方便对计算机不熟悉的用户使用。

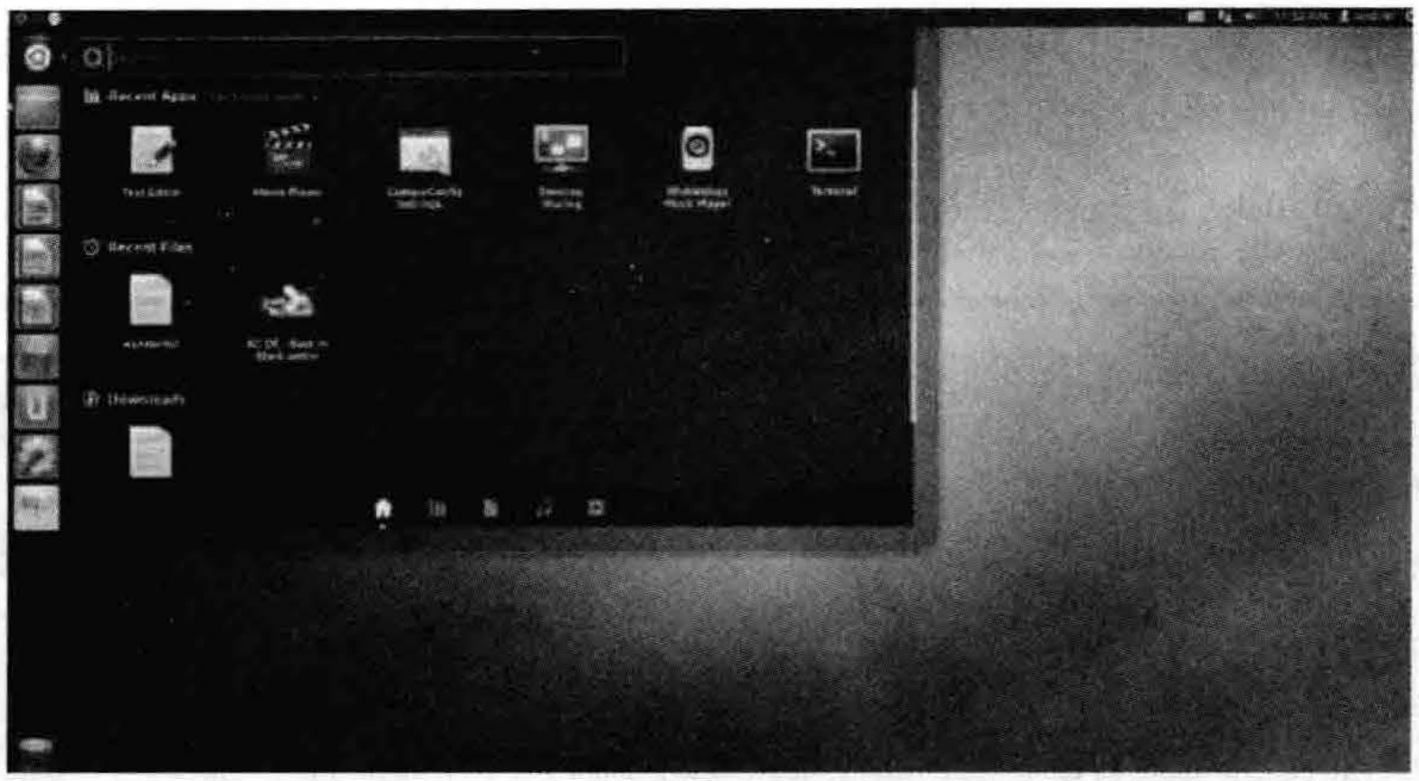


图 1-5　Unity 桌面环境

Ubuntu 具有以下一些特点。

- 桌面环境集成了一些常用社交网站、音乐站点，并且支持大量的邮件和新闻服务。
- 支持从远程主机登录（需要设置远程登录账号）。
- Unity Dash 可以提供 Amazon 网络搜索结果。
- 附加驱动整合到了软件源。
- 提供了对普通桌面、手机、平板电脑、电视、服务器等不同应用的多种版本。

注意：本书使用的 Linux 发行版是 Ubuntu 12.04.4 LTS，其内核版本是 3.8.0-35。

1.1.2　Shell 简介

Shell 是一种具备特殊功能的程序，它提供了用户与内核进行交互操作的一种接口。它接收用户输入的命令，并把它送入内核去执行。内核是 Linux 系统的心脏，从开机自检时就驻留在计算机的内存中，直到计算机关闭为止，而用户的应用程序存储在计算机的硬盘上，仅当需要时才被调入内存。Shell 是一种应用程序，当用户登录 Linux 系统时，Shell 就会被调入内存执行。Shell 独立于内核，它是连接内核和应用程序的桥梁，并由输入设备读取命令，再将其转为计算机可以理解的机械码，Linux 内核才能执行该命令。图 1-6 描述了 Shell 在 Linux 系统中的位置。

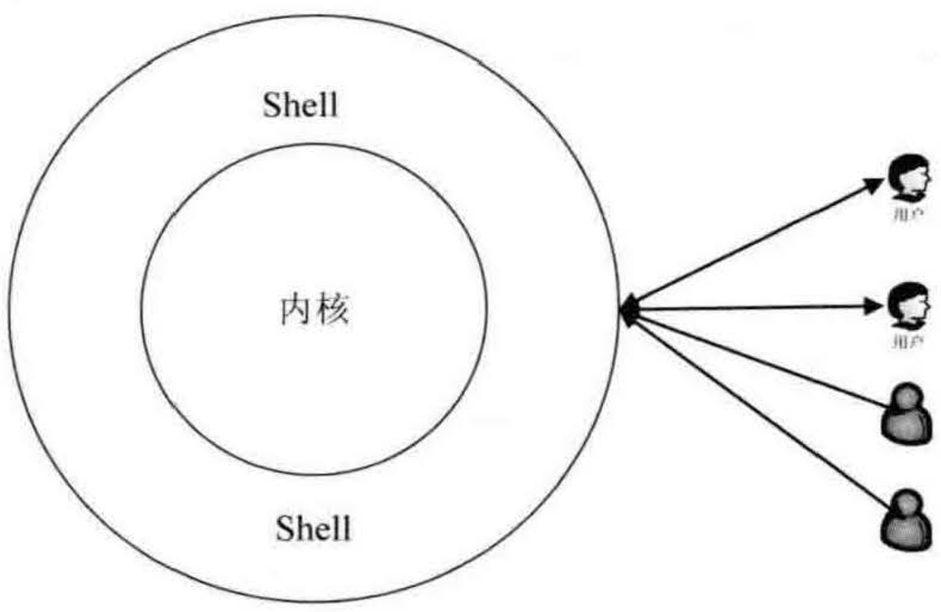


图 1-6　Shell 在 Linux 系统中的位置

混混藏书阁 http://book-life.blog.163.com

用户可以通过两种方式打开 Shell，第一种是在 Linux 系统图形用户界面（Ubuntu 中为 Unity）下的 Dash 中选择终端，并且可以将终端的快捷启动方式锁定在启动栏上，图 1-7 是在 Ubuntu 12.04 的 Unity 图形环境下打开 Shell 的示意。

图 1-7　Ubuntu 的 Shell 窗口

十分熟悉 Linux 系统的用户一般不通过图形用户界面来操纵 Linux 系统，而是直接通过 Shell 登录到 Linux 系统。因此，第二种打开 Shell 的方式就是利用一些软件工具以 SSH 的方式远程登录到 Linux 系统，目前比较流行的 Shell 软件工具是 Putty，该软件是非常小巧的工具，而且是绿色软件，无须安装，图 1-8 显示了 Putty 的登录设置界面，同样是输入登录主机的 IP 地址，然后单击“Open”按钮连接，在图 1-9 所示的界面上输入登录用户名及其密码，成功登录后出现命令提示符。

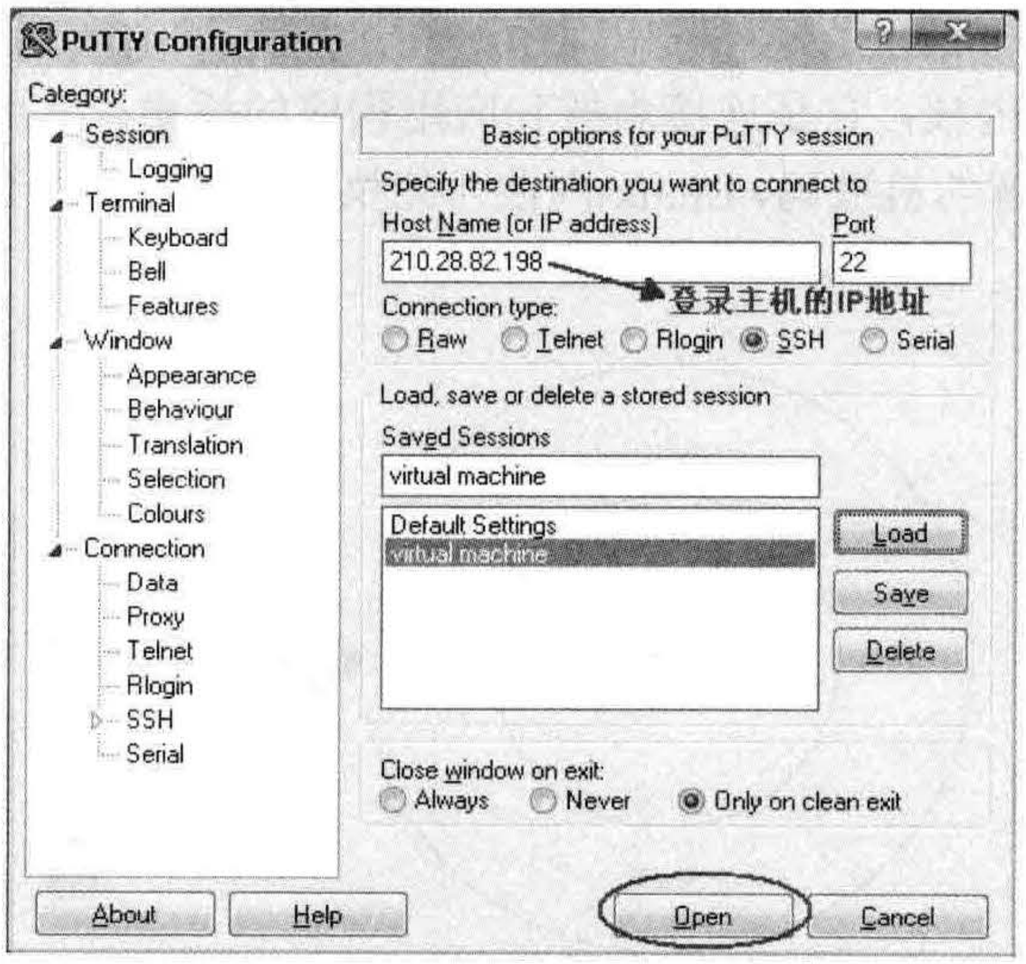


图 1-8　Putty 登录设置界面

图 1-9 Putty 登录成功后的界面

本节简单介绍了 Shell 的概念，对 Shell 概念的介绍并不完整，因为要介绍清楚 Shell，必然要涉及很多操作系统方面的知识，而本书的主旨是 Shell 编程；然后，本节给出三种启动 Shell 的方法，这是进行 Shell 编程的基础。

1.1.3 Shell 的种类和版本

Shell 作为 Linux 系统下的一个程序，有许多种类（本书将在第 11 章对常见的 Shell 进行介绍），其也会随着 Linux 系统的发展和升级而出现不同的版本，可以通过查看 Linux 下的相应文件来得知当前 Linux 中安装的 Shell 种类。

```
alloy@alloy:~$ cat /etc/shells
# /etc/shells: valid login shells
/bin/sh
/bin/dash
/bin/bash
/bin/rbash
```

通过对该文件的分析可以看到，在 Ubuntu 12.04 下安装了 dash、bash 和 rbash 三种不同的 Shell（sh 是执行快捷方式）。

还可以使用 bash -version 命令来查看当前 Shell 的版本号，Ubuntu 12.04 中安装的 Shell 版本号为 4.2.25，这也是本书中使用的 Shell 版本。

```
alloy@alloy:~$ bash -version
GNU bash, 版本 4.2.25(1)-release (x86_64-pc-linux-gnu)
Copyright (C) 2011 Free Software Foundation, Inc.
许可证 GPLv3+: GNU GPL 许可证版本 3 或者更高 <http://gnu.org/licenses/gpl.html>

这是自由软件，您可以自由地更改和重新发布。
在法律允许的范围内没有担保。
```

1.2 Shell 脚本编程的优势

脚本语言（Script Language）是相对于编译型语言而言的，它是为了缩短编译型语言编写—编译—链接—运行（Edit-Compile-Link-Run）过程而创建的计算机编程语言。由于脚本语言常常运行于底层，所处理的是字节、整数、浮点数或其他机器层的对象，因而脚本语言是低级程序设计语言。如 C\C++、Ada、Java、C#等都属于编译型语言，也可称为高级程序设计语言，这类语言所编写的程序需要经过编译，将源代码转化为目标代码才能运行。而脚本语言往往是解释运行而非编译，即由解释器（Interpreter）读入脚本程序代码，将其转换成

混混藏书阁 http://book-life.blog.163.com

内部的形式执行，而解释器本身则是编译型程序。

脚本语言的好处是简单、易学、易用，适合处理文件和目录之类的对象，以简单的方式快速完成某些复杂的事情通常是创造脚本语言的重要原则，脚本语言的特性可以总结为以下 4 个方面。

- 语法和结构通常比较简单。
- 学习和使用通常比较简单。
- 通常以容易修改程序的“解释”作为运行方式，而不需要“编译”。
- 程序的开发产能优于运行效能。

脚本语言的灵活性是以执行效率为代价的，脚本语言的执行效率通常不如编译型语言。当然，脚本语言一般不适用于大型的项目、计算复杂的工程或有高级需求的应用软件，它适用于系统管理、文本处理等方面完成特定功能的常用的小工具或小程序。

Shell 脚本语言是 Linux/UNIX 系统上一种重要的脚本语言，在 Linux/UNIX 领域应用极为广泛，熟练掌握 Shell 脚本语言是一个优秀的 Linux/UNIX 开发者和系统管理员的必经之路。利用 Shell 脚本语言可以简洁地实现复杂的操作，而且 Shell 脚本程序往往可以在不同版本的 Linux/UNIX 系统上通用。

尽管 Shell 脚本语言延续了脚本语言易学的特性，易学体现在 Shell 脚本语言门槛较低，易于上手，读者可以毫不费力就学会编写一个简单的 Shell 脚本程序，并很容易地学会执行它，但是，要深入透彻地学会 Shell 脚本语言是有难度的，因为 Shell 脚本语言涉及几乎所有 Linux 命令的灵活使用，而且 Linux 系统下的小工具（如：awk、sed）也较多，它们常常出现在 Shell 脚本之中。另外，Shell 脚本语言还提供了类似于高级程序设计语言的语法结构，如分支判断语句、变量和函数、循环结构、数组、算术和逻辑运算等。

综上所述，Shell 脚本语言是 Linux/UNIX 系统上应用广泛的实用程序设计语言，它是“易学难精”的，真正学会 Shell 脚本编程需要读者清晰地掌握 Linux 重要命令的语法，理解 Linux 命令重要选项的作业和区别，还需要掌握 Shell 脚本语言的语法结构，以及一些常用小工具。

1.3 第一个 Shell 脚本例子

1.3.1 Shell 脚本的基本元素

使用 Shell 脚本的最初动机可能在于省去手动输入命令的麻烦，Shell 脚本将一系列的 Linux 命令放在一个文件中，这样，我们就不必每次都手动输入同样的命令，比如：当一个用户登录系统后，其想查看当前有哪些用户在登录，那么，这个用户将会依次输入以下命令完成上述操作：

```
date
who
```

显然，用户每次输入重复的命令显得比较麻烦，我们能否把多个命令写入一个文件，然后通过执行这个文件来执行这些命令呢？这就是最原始的 Shell 脚本。上述的两条命令可以

写入下面的文件中，该文件取名为 whologged.sh：

```
#!/bin/bash
date                    #显示日期命令
who                     #显示当前的登录用户
```

whologged.sh 文件就是一个典型的 Shell 脚本，需要注意的是，whologged.sh 文件的第 1 行是“#!/bin/bash”，“#!”符号称为“Sha-bang”符号，是 Shell 脚本的起始符号，“#!”符号是指定一个文件类型的特殊标记，它告诉 Linux 系统这个文件的执行需要指定一个解释器。“#!”符号之后是一个路径名，这个路径名指明了解释器在系统中的位置，对于一般的 Shell 脚本而言，解释器是 bash，也可以是 sh，即用下面两种方式作为脚本的第 1 行都是正确的：

```
#!/bin/bash
#!/bin/sh
```

当然，Linux 还存在其他一些解释器，如 sed 和 awk 等，指定这些解释器就需要对第 1 行做相应的改动，本书第 4 章将会介绍 sed 和 awk，在此，请读者记住大部分脚本都是用 bash 解释器的，但是，其他解释器依然是存在的。

“#!/bin/bash”行之后，whologged.sh 文件按顺序写入需要执行的三条命令，每条命令后面有一段以“#”符号起始的中文，“#”符号是注释符，它后面直到本行结束的所有内容是注释，脚本执行时是不执行注释的，“#”符号类似于 C++和 Java 语言中的“//”符号，脚本注释可以是整行，也可以在某行的后面：

```
command                 #在行后面的注释
#整行的注释
```

注释能增加 Shell 脚本的可读性，便于人们理解该脚本。因此，读者在编写脚本时，应养成勤加注释的好习惯。

从 whologged.sh 脚本也可以看出，命令（command）是 Shell 脚本的最基本元素，命令通常由命令名称、选项和参数三部分组成，三部分之间用空格键或 Tab 键分隔。下面以例 1-1 来说明命令的三个组成部分。

```
#例 1-1：介绍 Linux 命令的组成部分
alloy@alloy:/$ ls -l /etc/sh*           #一条简单的 Linux 命令
-rw-r----- 1 root shadow 1094  3月 28 10:06 /etc/shadow
-rw------- 1 root root   1094  3月 28 10:06 /etc/shadow-
-rw-r--r-- 1 root root     73  2月  4 19:58 /etc/shells
alloy@alloy:~/linuxshell$
```

例 1-1 中的命令用于列出/etc 目录下以“sh”开头文件的详细信息，这一条简单的 Linux 命令由三个部分组成，“ls”是命令名称，“-l”是选项，“/etc/sh*”是参数。命令名称在命令中是不可或缺的，而选项和参数则可以不出现。选项的开头符号是一个减号（-），后面跟一个或多个字母，选项是对命令的补充说明，读者在学习 Linux 命令时需要在辨析和理解选项上花力气，读者在后续章节的学习中一定能够体会到选项的重要性。参数可以理解为命令的作用对象，“/etc/sh*”参数中“*”符号称为通配符，通配符经常在命令参数中出现。我们将在第 3 章详细讨论通配符的用法。

Linux 系统有成千上万条命令，我们很难全部掌握这些命令，而且即便是经常使用的命令，我们往往也会忘记这些命令的选项和用法，幸好 Linux 系统为我们提供了命令的手册页（manual page），我们可以用如下格式的命令打开某命令的手册页。

```
man [命令名称]
```


混混藏书阁 http://book-life.blog.163.com

man 命令可以打开其他 Linux 命令的手册页，手册页上能详细显示命令名称、基本格式、对选项的详细描述等信息，man 是 Linux 程序员和管理员用于查询命令用法的常用手段。

注意：对于同一个字符串命令，在手册页上可能存在不同的帮助信息，此时手册会用(1)、(2)等编号进行区分，在查询的时候也可以通过 man+数字+字符串的方式来进行区分，例如，man 2 open。

分号（;）可以用来隔开同一行内的多条命令，Shell 会依次执行用分号隔开的多条命令，例 1-2 演示了分号的用法。

```
#例 1-2：分号的用法
alloy@alloy:/$ ls -l /etc/sh*;date;who                          #同一行内有三条命令，用分号隔开
-rw-r----- 1 root shadow 1094  3 月 28 10:06 /etc/shadow        #ls 命令的结果
-rw------- 1 root root   1094  3 月 28 10:06 /etc/shadow-
-rw-r--r-- 1 root root     73  2 月  4 19:58 /etc/shells
2014 年 04 月 28 日 星期一 07:57:57 CST                          #date 命令的结果
alloy    tty7         2014-04-27 23:44                          #who 命令的结果
alloy    pts/1        2014-04-28 07:47 (192.168.0.102)
alloy@alloy:/$
```

例 1-2 的同一行内有三条命令，用分号隔开，依次显示了这三条命令的执行结果。

本节给出了第一个 Shell 脚本，Shell 脚本以“#!”符号开头，该符号后面跟解释器的路径；命令是 Shell 脚本的最基本元素，它定义了 Shell 脚本的动作，包含命令名称、选项和参数三个部分；Shell 脚本中也可以添加注释，以注释符“#”引出。除此之外，Shell 脚本还可以包含变量、各种控制结构，以及算术和逻辑运算符，这些内容将在后续章节中展开。

1.3.2 执行 Shell 脚本

在编写好一个 Shell 脚本后，如何执行这个 Shell 脚本呢？这与 Linux 系统对文件的管理有关，Linux 系统管理文件结合考虑了用户权限和文件权限，本书第 2 章将详细介绍 Linux 系统用户管理和文件管理的基础知识，简言之，要执行一个 Shell 脚本，只需要使当前用户具备执行该脚本文件的权限。一般来说，当我们用文本编辑器创建一个 Shell 脚本文件时，该文件是没有可执行权限的，即 x 权限。因此，我们需要先赋给 Shell 脚本可执行权限，再去执行它。下面的例 1-3 给出 whologgedsh 脚本的执行过程：

```
#例 1-3：执行 whologgedsh 脚本
alloy@alloy:~/linuxshell/CH01$ chmod u+x whologgedsh
#为 whologgedsh 脚本赋可执行权限
alloy@alloy:~/linuxshell/CH01$ ls -l whologgedsh          #查看 whologgedsh 的权限
-rwxrw-r-- 1 alloy alloy 115  4 月 28 08:11 whologgedsh   #具备可执行权限了
alloy@alloy:~/linuxshell/CH01$ ./whologgedsh              #执行 whologgedsh 脚本
2014 年 04 月 28 日 星期一 08:12:34 CST                    #执行结果
alloy    tty7         2014-04-27 23:44
alloy    pts/1        2014-04-28 07:47 (192.168.0.102)
alloy@alloy:~/linuxshell/CH01$
```

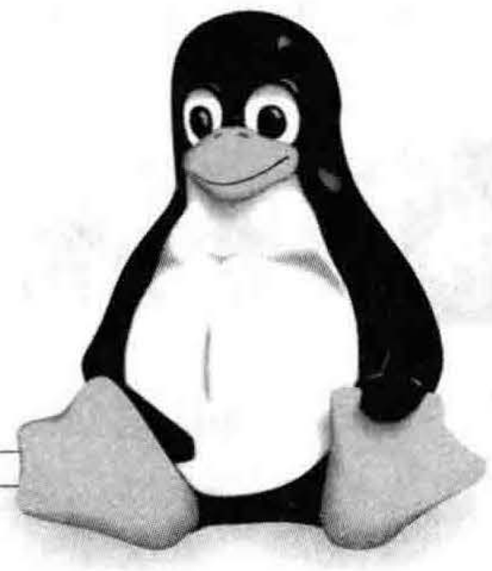
例 1-3 首先利用 chmod 命令为 whologgedsh 脚本赋可执行权限，chmod 命令将在第 2 章详细介绍，查看 whologgedsh 的权限时发现，whologgedsh 文件确实具备了 x 权限，即 whologgedsh 文件具备了可执行权限。最后，使用./[脚本名]格式的命令执行 whologgedsh 脚本，Shell 打印出 whologgedsh 脚本的执行结果。

缺乏 Linux 文件管理基础知识的读者可能并不理解 Shell 脚本执行的原因，这类读者可以通过阅读本书第 2 章掌握 Linux 用户管理、文件管理和文本编辑器，掌握第 2 章内容之后，一定能轻松地掌握 Shell 脚本的执行。

1.4 本章小结

本章首先简明扼要地介绍了 Linux 操作系统的起源与发展、Shell 的概念和作用、Shell 脚本编程的优势等理论性内容，试图让读者理解 Linux 和 Shell 什么、为什么需要学习 Shell 脚本编程。然后，本章给出第一个 Shell 脚本的例子，从该例子说明 Shell 脚本的基本元素，以及执行脚本的方法。

混混藏书阁 http://book-life.blog.163.com



第2章

Linux文件系统和文本编辑器

本章后续章节需要用到Linux基础知识。首先，介绍Linux用户和用户组的概念以及管理的常用命令；其次，重点介绍文件和目录操作，包含文件和目录复制、移动和删除等常用命令，文件和目录权限的概念以及管理的常用命令，查找文件命令——find；最后，介绍两种文本编辑器：命令行环境下的vim编辑器和Unity桌面环境的Gedit编辑器。熟悉Linux系统基础命令的读者可以忽略本章，直接进入下一章的阅读。

2.1 用户和用户组管理

Linux 是多用户多任务的操作系统，用户（user）和用户组（group）的管理是 Linux 使用者应该了解和掌握的基础之一，本节介绍 Linux 用户和用户组的管理，它也是 2.2 节文件操作的基础。

2.1.1 用户管理常用命令

用户在系统中是分角色的，在 Linux 系统中，由于角色不同，权限和所完成的任务也不同。值得注意的是，用户的角色是通过 UID 来识别的，用户的 UID 是全局唯一的。Linux 用户可以分为以下三类。

- root 用户（也称为超级用户）：系统唯一，是真实的，可以登录系统，可以操作系统中的任何文件和命令，拥有最高权限。
- 虚拟用户：这类用户也被称为伪用户或假用户，与真实用户区分开来，这类用户不具有登录系统的能力，但却是系统运行不可缺少的用户，比如 bin、daemon、adm、ftp、mail 等；这类用户是系统自身拥有的，而非后来添加的，当然，我们也可以添加虚拟用户。
- 普通真实用户：这类用户能登录系统，但只能操作其根目录的内容，权限受到限制，这类用户都是系统管理员自行添加的。

Linux 用户管理的常用命令主要有：用户账号添加命令 useradd 或 adduser、修改用户命令 usermod、删除用户命令 userdel 及用户口令管理命令 passwd 等，下面将讲解这些命令。

1. 用户账号添加命令——useradd 或 adduser

useradd 和 adduser 是完全等价的两条命令，都是用于创建新的用户账号，下面以 useradd 为代表介绍它们的用法，命令格式如下：

```
useradd [option] [username]
```

其中，[option]为 useradd 命令选项，而[username]是要创建的用户名。执行该命令，将在系统中做以下一些事情。

- 在/etc/passwd 文件中增添了一行记录。
- 在/home 目录下创建新用户的主目录，并将/etc/skel 目录中的文件复制到该目录中。

使用该命令后，新建的用户暂时还无法登录，因为还没有为该用户设置口令，需要再用 passwd 命令为其设置口令后，才能登录。用户的 UID 和 GID 是 useradd 自动选取的，它是将/etc/passwd 文件中的 UID 加 1，将 etc/group 文件中的 GID 加 1。

表 2-1 列出了 useradd 命令中各选项的含义。

表 2-1 useradd 或 adduser 命令的选项及其意义

选　项	意　义
-g [initial_grp]	用于添加用户账号时指定该用户的私有组。如不指定-g 参数，useradd 命令将自动建立与用户账号同名的组作为该账号的私有组

混混藏书阁 http://book-life.blog.163.com

续表

选　项	意　义
-G [grp...]	用于添加附属组
-D	用于显示或设置 useradd 命令所使用的默认值
-d [directory]	指定用户主目录，如果此目录不存在，则同时使用-m 选项来创建主目录
-m	使用的目录若不存在，则自动建立
-u　UID	指定用户的用户号，如果同时有-o 选项，则可以重复使用其他用户的标识号。注意，ID 值不能为负值，预设为最小不得小于 99 而逐次增加。0~99 传统上是保留给系统账号使用

使用 useradd 或 adduser 命令增加新用户时，系统将为用户创建一个与用户名相同的组，称为私有组，这一方法是为了能让新用户与其他用户隔离，确保安全性的措施。如果要改变私有组的名字，可以使用-g 选项来完成。

例 2-1 通过增加一个用户 wang，并查看其相关信息，来帮助用户理解该命令所执行的操作。可以通过 tail -l 命令查看文件/etc/passwd 来查看文件新创建的用户和新创建的用户主目录，而通过查看/etc/shadow 创建用户的密码。

```
#例 2-1：创建用户 wang 并查看其相关信息，该操作需要 root 权限
alloy@alloy:/$ sudo useradd wang
[sudo] password for alloy:
#查看 passwd 文件中添加的用户账号信息
alloy@alloy:/$ tail -1 /etc/passwd
usbmux:x:108:46:usbmux daemon,,,:/home/usbmux:/bin/false
kernoops:x:109:65534:Kernel Oops Tracking Daemon,,,:/:/bin/false
pulse:x:110:119:PulseAudio daemon,,,:/var/run/pulse:/bin/false
rtkit:x:111:122:RealtimeKit,,,:/proc:/bin/false
speech-dispatcher:x:112:29:Speech Dispatcher,,,:/var/run/speech-dispatcher:/bin/sh
hplip:x:113:7:HPLIP system user,,,:/var/run/hplip:/bin/false
saned:x:114:123::/home/saned:/bin/false
alloy:x:1000:1000:alloy,,,:/home/alloy:/bin/bash
wang:x:501:501:wang,,,:/home/wang:/bin/bash      #新创建的 wang 用户
sshd:x:115:65534::/var/run/sshd:/usr/sbin/nologin
xrdp:x:116:125::/var/run/xrdp:/bin/false

#查看加密后的用户账号及密码信息，该操作需要 root 权限
alloy@alloy:/$ sudo tail -1 /etc/shadow
[sudo] password for alloy:
usbmux:*:16105:0:99999:7:::
kernoops:*:16105:0:99999:7:::
pulse:*:16105:0:99999:7:::
rtkit:*:16105:0:99999:7:::
speech-dispatcher:!:16105:0:99999:7:::
hplip:*:16105:0:99999:7:::
saned:*:16105:0:99999:7:::
alloy:$6$F20Y/JSL$/ct4nAP23Nqj5T59h2n1rYG8ss8l9Z3CfAuoywxWMoifQj0siH3Zy3mCDDLHXyRN8
Kq679VvrVVVIuaGD4USp1:16144:0:99999:7:::
wang:!!:14671:0:99999:7:::
sshd:*:16144:0:99999:7:::
xrdp:!:16157:0:99999:7:::

#查看所建立账号的主目录
```

```
alloy@alloy:/$ ls /home
alloy  wang
alloy@alloy:/$
```

例 2-1 中，首先通过 useadd 命令创建了一个新的用户 wang，然后通过“tail -l /etc/passwd”命令查看文件/etc/passwd，可以看到 wang 用户的 UID 为 501，创建的新目录为/home/wang，接着通过“tail -l /etc/shadow”命令查看文件/etc/shadow，可以获得用户 wang 的密码，由于还没有为 wang 用户创建密码，可以看到 wang 后为“!!”，表示用户密码不可用，最后通过 ls 命令查看/home 目录，可以看到为 wang 用户创建的主目录已经存在。

2. 修改用户账号——usermod

usermod 命令可用来修改用户账号的各种属性，包括用户主目录、私有组、登录 Shell 等内容，usermod 的命令格式如下：

```
usermod [option] [username]
```

其中，[option]为 useradd 命令选项，而[username]是需修改的用户名。表 2-2 列出了命令 usermod 命令选项及其意义。

表 2-2　usermod 命令的选项及其含义

选　项	意　义
-d [directory]	修改用户登入时的目录
-e [days]	修改账号的有效期限，days 表示天数
-g [group]	修改用户所属的群组
-l [login_name]	变更用户登录时的名称为 login_name
-p [password]	修改用户密码
-s [shell]	指定用户登录的 Shell，如果不设置，则选用系统预设的 Shell

需要注意的是，在使用 usermod 命令时最好不要用其修改用户密码，因为如果用 usermod 命令，则显示在文件“/etc/shadow”中的密码是明密码，应该用 passwd 命令修改密码。下面的例 2-2 使用 usermod 来修改密码，可以看到，通过 tail 命令可以看到该用户的密码为明码，这样是很危险的，如果其他用户打开文件“/etc/shadow”，就可使用该用户名和密码登录到该账号上。

```
#例 2-2：使用 usermod 修改密码，同样需要 root 权限
alloy@alloy:/$ sudo usermod -p 123456 wang
[sudo] password for alloy:
#通过 tail -l 查看用户 wang 的密码
alloy@alloy:/$ sudo tail -l /etc/shadow
[sudo] password for alloy:
usbmux:*:16105:0:99999:7:::
kernoops:*:16105:0:99999:7:::
pulse:*:16105:0:99999:7:::
rtkit:*:16105:0:99999:7:::
speech-dispatcher:!:16105:0:99999:7:::
hplip:*:16105:0:99999:7:::
saned:*:16105:0:99999:7:::
alloy:$6$F20Y/JSL$/ct4nAP23Nqj5T59h2n1rYG8ss819Z3CfAuoywxWMoifQj0siH3Zy3mCDDLHXyRN8
Kq679VvrVVVIuaGD4USp1:16144:0:99999:7:::
#可以看到 wang 用户的密码为 123456
```


混混藏书阁 http://book-life.blog.163.com

```
wang:123456:14672:0:99999:7:::
sshd:*:16144:0:99999:7:::
xrdp:!:16157:0:99999:7:::
alloy@alloy:/$
```

还需注意，usermod 不允许改变已登录用户的用户账号名称，当用户修改 UID 时，也必须确认这个用户没有在电脑上执行任何程序。下面的例 2-3 演示了修改正在登录用户的账名称时所产生的错误。

```
#例 2-3：删除正在登录用户的用户账号所产生的错误
#alloy 是当前登录用户，用 usermod 修改时发生错误，该操作也需要 root 权限
alloy@alloy:/$ sudo usermod alloy
[sudo] password for alloy:
bash: /usr/sbin/usermod: Permission denied
alloy@alloy:/$
```

3. 删除用户账号命令——userdel

userdel 命令非常简单，其只有一个可选项-r，如果在 userdel 后加上-r 选项，则在删除用户的同时也一并删除存储在/home 目录下的该用户目录和文件。下面的例 2-4 说明了使用-r 选项和不使用-r 选项的区别。

```
#例 2-4：userdel 命令使用-r 选项和不使用-r 选项的区别，该操作也需要 root 权限
alloy@alloy:/$ ls /home
alloy wang  wang1  wang2
alloy@alloy:/$ sudo userdel -r wang1
[sudo] password for alloy:
alloy@alloy:/$ ls /home
wang  wang2  wyq
alloy@alloy:/$ sudo userdel wang2
[sudo] password for alloy:
alloy@alloy:/$ ls /home
wang  wang2  wyq
alloy@alloy:/$
```

从例 2-4 中可以看出，当使用-r 选项删除用户账号 wang1 时，目录/home 下的子目录 wang1 被删除了；删除 wang2 用户账号时若不使用-r 选项，则 wang2 子目录仍然存在。

4. 用户口令管理命令——passwd

用户管理一个重要的内容就是用户口令管理。用户账号刚建立时是没有口令的，但是会被系统锁定，必须为其指定口令才能使用，这时需使用 passwd 命令，下面是 passwd 命令的语法格式：

```
passwd [option] [username]
```

其中，[option]为 passwd 命令选项，[username]为用户名。表 2-3 列出了 passwd 命令选项及其意义。

表 2-3　passwd 命令的选项及其意义

选　项	意　义
-l	锁定用户口令，即禁止使用该用户账号
-u	口令解锁
-d	关闭使用者的密码确认功能，使用者在登录时可以不用输入密码，只有具备 root 用户的使用者才可使用
-f	强迫用户下次登录时修改密码

续表

选　项	意　义
-l [login_name]	变更用户登录时的名称为 login_name
-s	显示指定使用者的密码认证种类，同样只有具备 root 权限的用户才可使用

当用户作为普通用户修改自己的密码时，首先会提示用户的原密码，然后会要求用户重新输入两次来验证用户新口令，如果口令一致，则新口令设置成功。而超级用户 root 为用户设定密码时，则不需要原密码。

例 2-1 中创建的用户账号 wang 还无法使用，还需要使用 passwd 命令来创建账号密码。下面的例 2-5 使用 passwd 命令为 wang 用户创建账号密码。

```
#例 2-5：为 wang 用户创建账号密码，需要 root 权限
alloy@alloy:/$sudo passwd wang
[sudo] password for alloy:
Changing password for user wang.
New password:
Retype new password:
BAD PASSWORD: it is based on a dictionary word
BAD PASSWORD: is too simple
passwd: all authentication tokens updated successfully.

#查看加密后的用户账号及密码信息，可以看出该账号后无“!!”，表示可以使用该账号了
alloy@alloy:/$ tail -1 /etc/shadow
usbmux:*:16105:0:99999:7:::
kernoops:*:16105:0:99999:7:::
pulse:*:16105:0:99999:7:::
rtkit:*:16105:0:99999:7:::
speech-dispatcher:!:16105:0:99999:7:::
hplip:*:16105:0:99999:7:::
saned:*:16105:0:99999:7:::
alloy:$6$F20Y/JSL$/ct4nAP23Nqj5T59h2n1rYG8ss819Z3CfAuoywxWMoifQj0siH3Zy3mCDDLHXyRN8
Kq679VvrVVVIuaGD4USp1:16144:0:99999:7:::
wang:$6$s2PKmNwn$wjpv2JzAA5EPOGkU4Iir4/hqvTlYkTARuB6B23OwavXGeqUKumG8BPHXBRkDggxAlx
po2gXEt4ltFmHwbVy100:14671:0:99999:7:::
sshd:*:16144:0:99999:7:::
xrdp:!:16157:0:99999:7:::
alloy@alloy:/$
```

通过例 2-1 和例 2-5 对比可以看出，用户 wang 在文件“/etc/shadow”中已经发生了改变，凡是在文件“/etc/shadow”中的用户名含“!!”，则该用户账号不可使用，而为加密信息时是可以使用的。

2.1.2 用户组管理常用命令

用户组就是具有相同特征的用户的集合体，用户和用户组的关系是多对多的，一个用户可以属于多个用户组，同样，一个用户组可以包含多个用户。Linux 下每个文件都有一个用户组，当创建一个文件或目录时，系统会赋予其一个用户组关系。

用户组的管理命令包含用户组添加命令 groupadd、用户组修改命令 groupmod 和用户组删除命令 groupdel，下面将对这些命令进行详解。

混混藏书阁 http://book-life.blog.163.com

1. 用户组添加命令——groupadd

groupadd 可指定用户组名称来建立新的用户组，需要时可从系统中取得新用户组值。其语法格式为：

```
groupadd [option] [groupname]
```

其中[option]为 groupadd 命令选项，[groupname]是将要创建的用户组名，表 2-4 列出了 groupadd 命令的选项及其意义。

表 2-4　groupadd 命令的选项及其意义

选　　项	意　　义
-g GID	除非使用-o 参数，否则 GID 值必须是唯一且数值不可为负，预设值以/etc/login.defs 为准
-o GID	运行 GID 不唯一
-r	加入组 GID 号，且其 GID 号低于 499 系统账号
-f	新增一个已经存在的用户组账号，系统会出现错误信息，然后结束

groupadd 命令其实用起来非常简单，下面的例 2-6 添加了一个 GID 为 666 的用户组 wangyq，可以在文件/etc/group 的目录中查看到 GID 为 666 的用户组 wangyq。

```
#例 2-6：使用 groupadd 命令添加用户组 wangyq，需要 root 权限
alloy@alloy:/$ sudo groupadd -g 666 wangyq
[sudo] password for alloy:
alloy@alloy:/$ tail -1 /etc/group
avahi-autoipd:x:117:
avahi:x:118:
pulse:x:119:
pulse-access:x:120:
utempter:x:121:
rtkit:x:122:
saned:x:123:
alloy:x:1000:
wangyq:x:666:
sambashare:x:124:alloy
xrdp:x:125:
alloy@alloy:/$
```

如果调用 groupadd 命令时不设置 GID 号，如下面的命令：

```
groupadd group1
```

则在系统中增加一个新组 group1，新组的组标识 GID 是在当前最大组标识的基础上加 1。

2. 用户组修改命令——groupmod

groupmod 可指定用户组名称来修改新的用户组号或用户组名称，其语法格式为：

```
groupmod [option] [groupname]
```

其中，[option]为 groupmod 命令选项，[groupname]为用户组名，表 2-5 为 groupmod 命令选项及其意义。

表 2-5　groupmod 命令的选项及其意义

选　　项	意　　义
-g GID	用户指定新的 GID
-o GID	重复使用 GID
-n	为群组改名

例 2-7 是将用户组 wangyq 的群组号修改为 555，通过命令 tail 可以看出该用户组的 GID 修改成功。

```
#例 2-7：使用 groupmod 修改用户组号，需要 root 权限
alloy@alloy:/$sudo groupmod -g 555 wangyq
[sudo] password for alloy:
alloy@alloy:/$ tail -1 /etc/group
avahi-autoipd:x:117:
avahi:x:118:
pulse:x:119:
pulse-access:x:120:
utempter:x:121:
rtkit:x:122:
saned:x:123:
alloy:x:1000:
wangyq:x:555:
sambashare:x:124:alloy
xrdp:x:125:
alloy@alloy:/$
```

3. 用户组删除命令——groupdel

groupdel 可指定用户组名称来删除已有的用户组，其语法格式为：

```
groupdel [groupname]
```

该命令非常简单，但需注意的是，如果该用户组中包含某些用户，则必须先删除这些用户，然后才能删除该用户组。下面的例 2-8 是一个删除用户组 wangyq 的例子，通过 tail 命令可以看出该用户组已找不到了。

```
#例 2-8：用 groupdel 删除用户组，需要 root 权限
alloy@alloy:/$sudo groupdel wangyq
[sudo] password for alloy:
#查看文件/etc/group 可以看到用户组 wangyq 已被删除
alloy@alloy:/$ tail -1 /etc/group
avahi-autoipd:x:117:
avahi:x:118:
pulse:x:119:
pulse-access:x:120:
utempter:x:121:
rtkit:x:122:
saned:x:123:
alloy:x:1000:
sambashare:x:124:alloy
xrdp:x:125:
alloy@alloy:/$
```

2.2 文件和目录操作

在多数操作系统中都有文件的概念。文件是 Linux 用来存储信息的基本结构，它是被命名（称为文件名）的存储在某种介质（如磁盘、光盘和磁带等）上的一组信息的集合。Linux 文件均为无结构的字符流形式。文件名是文件的标识，它由字母、数字、下画线和圆点组成的字符串构成。Linux 要求文件名的长度限制在 255 个字符以内。用户应该选择有意义的文件名。

混混藏书阁 http://book-life.blog.163.com

为了便于识别和管理，用户可以把扩展名作为文件名的一部分，文件名与扩展名之间用圆点分开，扩展名对于文件分类是十分有用的。用户可能已经对某些大众已接纳的标准扩展名比较熟悉了，例如，C 语言编写的源代码文件总是具有 C 的扩展名。用户可以根据自己的需要，加入自己的文件扩展名。

在计算机系统中存有大量的文件，如何有效地组织与管理它们，并为用户提供一个使用方便的接口是文件系统的一大任务。Linux 系统以文件目录的方式来组织和管理系统中的所有文件。所谓文件目录就是将所有文件的说明信息采用树形结构组织起来，即我们常说的目录。也就是说，整个文件系统有一个“根”（root），然后在根上分“杈”（directory），任何一个分杈上都可以再分杈，杈上也可以长出“叶子”。“根”和“杈”在 Linux 中被称为“目录”或“文件夹”，而“叶子”则代表一个个的文件。实践证明，此种结构的文件系统效率比较高。

对文件进行访问时，需要用到“路径”（Path）的概念。何谓路径？顾名思义，路径是指从树形目录中的某个目录层次到某个文件的一条道路。路径的主要构成是目录名称，中间用“/”符号分开。任一文件在文件系统中的位置都是由相应的路径决定的。用户在对文件进行访问时，要给出文件所在的路径，这又分相对路径和绝对路径。相对路径是从用户工作目录开始的路径；绝对路径是指从“根”开始的路径，也称为完全路径。

应该注意到，在树形目录结构中到某一确定文件的绝对路径和相对路径均只有一条。绝对路径是确定不变的，而相对路径则随着用户工作目录的变化而不断变化。这一点对于我们以后使用某些命令如 cp 和 tar 等大有好处。

理解了文件、目录和路径的概念后，下面讲述文件和目录操作中经常用到的各种命令。

2.2.1 文件操作常用命令

文件操作常用命令包括文件清单命令 ls、文件复制命令 cp、文件移动命令 mv 和删除文件命令 rm，下面逐一讲述这些命令。

1. 文件清单命令——ls

ls 命令是英文单词 list 的简写，其功能是列出目录下的文件和子目录的相关信息。ls 命令是用户最常用的命令之一，因为用户需要不时地查看某个目录中的信息，该命令类似于 DOS 下的 dir 命令，其语法格式为：

```
ls [option] [file or directory]
```

其中，[option]为命令选项，[file or directory]对应于文件或目录。如果[file or directory]为目录，该命令将列出该目录下的所有子目录与文件；如果[file or directory]为文件，则 ls 命令将输出该文件名及相关信息。默认情况下，输出条目按字母顺序排序，当未给出目录名或是文件名时，就显示当前目录的信息。表 2-6 给出了 ls 命令选项及其意义。

表 2-6　ls 命令的选项及其意义

选　　项	对应含义
-a	显示指定目录下所有的子目录与文件，包括隐藏文件
-A	显示指定目录下所有的子目录与文件，包括隐藏文件，且不列出“.”和“..”
-b	对文件名中的不可显示字符用八进制逃逸字符显示
-c	按文件的修改时间排序

续表

选　　项	对应含义
-C	分成多列显示各项
-d	如果参数是目录，只显示其名称而不显示其下的各文件。往往与 l 选项一起使用，以得到目录的详细信息
-f	不排序，该选项将使 lts 选项失效，使用 aU 选项有效
-i	在输出的第一列显示文件的 i 节点号
-l	以长格式来显示文件的详细信息，每行列出的信息依次是： 文件类型与权限、链接数、文件属主、文件属组、文件大小、最近修改的时间、名字
-L	若指定的名称为一个符号链接文件，则显示链接所指向的文件
-m	输出按字符流格式，文件跨页显示，以逗号分开
-n	输出格式与 l 选项相同，只不过在输出文件属主和属组是用相应的 UID 号和 GID 号来表示，而不是实际的名称
-o	与 l 选项相同，只是不显示拥有者信息
-p	在目录后面加一个“/”
-q	将文件名中的不可显示字符用“?”代替
-r	按字母逆序或最早优先的顺序显示输出结果
-R	递归式地显示指定目录的各个子目录中的文件
-s	给出每个目录项所用的块数，包括间接块
-t	显示时按修改时间（最近优先）而不是按名字排序；若文件修改时间相同，则按字典顺序，修改时间取决于是否使用了 c 或 u 选项。默认的时间标记是最后一次修改时间。
-u	显示时按文件上次存取的时间（最近优先）而不是按名字排序。即将-t 的时间标记修改为最后一次访问的时间
-x	按行显示出各排序项的信息

下面的例 2-9 演示了 ls 命令-l 选项的用法，-l 是 ls 命令最常用的选项，用于列出文件和目录的详细信息。

```
#例 2-9：列出/home/alloy/linuxshell/CH04 目录下的所有文件和目录的详细信息
alloy@alloy:~/linuxshell/CH04$ ls -l
总用量 36
-rwxr-xr-x 1 alloy alloy  72 10 月 15  2009 argv.awk
-rwxr-xr-x 1 alloy alloy  79 10 月 14  2009 array.awk
-rwxr-xr-x 1 alloy alloy  62  9 月 22  2009 delete.sed
-rwxr-xr-x 1 alloy alloy 234 10 月 15  2009 findphone.awk
-rwxr-xr-x 1 alloy alloy  48 11 月 25  2009 insert.sed
-rwxr-xr-x 1 alloy alloy  47 11 月 25  2009 modify.sed
-rwxr-xr-x 1 alloy alloy  73 10 月 13  2009 pass.awk
-rwxr-xr-x 1 alloy alloy  72  7 月 20  2010 scr1.awk
-rwxr-xr-x 1 alloy alloy  78 10 月 22  2009 scr2.awkalloy@alloy:/$
```

从例 2-9 的结果可以看出，ls -l 命令列出的详细信息包含文件或目录的权限、所属用户和用户组、创建时间等。需要指出的是，Linux 系统将 ll 命令作为 ls -l 命令的别名，即 ll 命令也可以列出文件或目录的详细信息。

2. 文件复制命令——cp

cp 命令可以将给出的文件或目录复制到另一文件或目录中，就如同 DOS 下的 copy 命令

混混藏书阁 http://book-life.blog.163.com

一样，功能非常强大。cp 命令的一般格式如下所示：

```
cp [option] [source] [destination]
```

该命令把指定的源文件复制到目标文件或把多个源文件复制到目标目录中，其中，[option]为 cp 命令选项，[source]为源文件，而[destination]为目标目录或目标文件，表 2-7 给出了 cp 命令选项及其意义。

表 2-7　cp 命令的选项及其意义

选　项	意　义
-a	该选项通常在复制目录时使用，它保留链接、文件属性，并递归地复制目录
-d	复制时保留链接
-f	删除已经存在的目标文件而不提示
-i	在覆盖目标文件之前将给出提示要求用户确认。回答 y 时目标文件将被覆盖，是交互式复制
-p	此时 cp 除复制源文件的内容外，还将把其修改时间和访问权限也复制到新文件中
-r	若给出的源文件是目录文件，此时 cp 将递归复制该目录下所有的子目录和文件。此时的目标文件必须为一个目录名
-l	不进行复制操作，只是链接文件

需要说明的是，为防止用户在不经意的情况下用 cp 命令破坏另一个文件，如用户指定的目标文件名已存在，用 cp 命令复制文件后，这个文件就会被新源文件覆盖。因此，建议用户在使用 cp 命令复制文件时，最好使用 i 选项，在覆盖文件时进行最后确认。

下面的例 2-10 演示了将文件 chapter13/file.txt 复制到目录 testdir 中，可以看出，在不加任何选项的情况下，文件的权限改变了，同时文件的修改时间也改变了。

```
#例 2-10：将文件 CH04/argv.awk 复制到目录 testdir 中
alloy@alloy:~/linuxshell$ cp CH04/argv.awk testdir
alloy@alloy:~/linuxshell$ ls -l CH04/argv.awk testdir
-rwxr-xr-x 1 alloy alloy   72 10 月 15  2009 CH04/argv.awk

testdir:
总用量 4
-rwxr-xr-x 1 alloy alloy 72  4 月 28 08:56 argv.awk
alloy@alloy:~/linuxshell$
```

下面的例 2-11 在 cp 命令的基础上加了-a 选项，则两个文件权限和修改时间未发生改变。

```
#例 2-11：将文件 CH04/argv.awk 复制到目录 testdir 中，且不改变权限和修改日期

alloy@alloy:~/linuxshell$ cp -a CH04/argv.awk testdir
alloy@alloy:~/linuxshell$ ls -l CH04/argv.awk testdir
-rwxr-xr-x 1 alloy alloy   72 10 月 15  2009 CH04/argv.awk

testdir:
总用量 4
-rwxr-xr-x 1 alloy alloy 72 10 月 15  2009 argv.awk
alloy@alloy:~/linuxshell$
```

cp 命令的-r 选项可实现目录的复制，如果直接复制目录，将产生如下错误：

```
alloy@alloy:~/linuxshell$ cp CH04 testdir
cp: 略过目录"CH04"
alloy@alloy:~/linuxshell$
```

下面的例 2-12 演示了正确的目录复制过程，将目录 chapter13 中的全部内容复制到目录 testdir 中。

```
#例 2-12：将目录中的所有内容复制到另一个目录中
#查看 CH04 目录下的文件和子目录
alloy@alloy:~/linuxshell$ ls -l CH04
总用量 36
-rwxr-xr-x 1 alloy alloy  72 10 月 15  2009 argv.awk
-rwxr-xr-x 1 alloy alloy  79 10 月 14  2009 array.awk
-rwxr-xr-x 1 alloy alloy  62  9 月 22  2009 delete.sed
-rwxr-xr-x 1 alloy alloy 234 10 月 15  2009 findphone.awk
-rwxr-xr-x 1 alloy alloy  48 11 月 25  2009 insert.sed
-rwxr-xr-x 1 alloy alloy  47 11 月 25  2009 modify.sed
-rwxr-xr-x 1 alloy alloy  73 10 月 13  2009 pass.awk
-rwxr-xr-x 1 alloy alloy  72  7 月 20  2010 scr1.awk
-rwxr-xr-x 1 alloy alloy  78 10 月 22  2009 scr2.awk
#查看 testdir 目录下的内容
alloy@alloy:~/linuxshell$ ls -l testdir
总用量 4
-rwxr-xr-x 1 alloy alloy 72 10 月 15  2009 argv.awk
#将 CH04 目录及其内部的所有文件复制到 testdir 目录下
alloy@alloy:~/linuxshell$ cp -r CH04 testdir
#查看 testdir 目录内容
alloy@alloy:~/linuxshell$ ls -l testdir
总用量 8
-rwxr-xr-x 1 alloy alloy   72 10 月 15  2009 argv.awk
drwxr-xr-x 2 alloy alloy 4096  4 月 28 09:00 CH04
alloy@alloy:~/linuxshell$
```

可以看出，在复制前，testdir 目录只有一个文件 argv.awk，而使用“cp -r CH04 testdir”命令时，testdir 目录中存在目录 CH04，说明 CH04 目录复制成功。

3. 文件移动命令——mv

mv 命令可用于将文件或目录从一个位置移动到另一个位置，同时可移动多个文件。mv 命令还可用于重命名，其一般的格式为：

```
mv [option] [source] [destination]
```

其中，[option]为 mv 命令选项，[source]为源目录或文件，[destination]为目标目录或目标文件，如果[destination]类型是文件时，mv 命令将所给的源文件或目录重命名为给定的目标文件名，此时，源文件只能有一个（也可以是源目录）；如果[destination]是已存在的目录名称，源文件或目录参数可以有多个，mv 命令将各参数指定的源文件全部移至目标目录中。在跨文件系统移动时，mv 先复制，再将原有文件删除，从而导致该文件的链接丢失。表 2-8 给出了 mv 命令选项及其意义。

表 2-8　mv 命令选项及其含义

选　项	含　义
-i	交互方式操作。如果 mv 操作将导致对已存在的目标文件的覆盖，此时系统询问是否重写，要求用户回答 y 或 n，这样可以避免错误地覆盖文件
-f	禁止交互操作。在 mv 操作要覆盖某已有的目标文件时不给出任何指示，指定此选项后，i 选项将不再起作用
-p	移动时保持权限

混混藏书阁 http://book-life.blog.163.com

如果所给目标文件（不是目录）已存在，此时该文件的内容将被新文件覆盖。为防止用户用 mv 命令破坏另一个文件，使用 mv 命令移动文件时，最好使用 i 选项，例 2-13 是交互式 mv 命令的应用，该例使用 mv 命令将文件 file1.txt 重命名为 file2.txt。

```
#例 2-13：将文件 file1.txt 重命名为 file2.txt
alloy@alloy:~/linuxshell/CH13$ ls -il file*
  427 -rw-r--r--. 1 alloy alloy 0 2010-05-19 06:18 file1.sh
28476 -rw-r--r--. 1 alloy alloy 0 2010-05-19 06:19 file1.txt
28477 -rw-r--r--. 1 alloy alloy 0 2010-05-19 06:20 file2.txt
28478 -rw-r--r--. 1 alloy alloy 0 2010-05-19 06:21 file5
28479 -rwxr-xr-x. 1 alloy alloy 51 2010-05-18 09:22 file.txt
#使用 mv 交互式命令将 file1.txt 重命名为 file2.txt
alloy@alloy:~/linuxshell/CH13$mv -i file1.txt file2.txt
mv: overwrite `file2.txt'? y
#再次查看 chapter13 中含 file 的文件，可以看出 file1.txt 消失，file2.txt 的修改时间改变
alloy@alloy:~/linuxshell/CH13$ ls -il file*
  427 -rw-r--r--. 1 alloy alloy 0 2010-05-19 06:18 file1.sh
28476 -rw-r--r--. 1 alloy alloy 0 2010-05-19 06:20 file2.txt
28478 -rw-r--r--. 1 alloy alloy 0 2010-05-19 06:19 file5
28479 -rwxr-xr-x. 1 alloy alloy 51 2010-03-04 01:05 file.txt
alloy@alloy:~/linuxshell/CH13$
```

对于目录的重命名，同样可以使用 mv 命令实现，例 2-14 将目录 testdir 命名为 chapter2，而且目录中的内容也未发生改变。

```
#例 2-14：将目录 testdir 命名为 chapter2
alloy@alloy:~/linuxshell$ls
chapter13  chapter15  chapter16  functions.new  rmdir  testdir  tt.sh
alloy@alloy:~/linuxshell $ mv testdir chapter2
alloy@alloy:~/linuxshell$ ls
chapter13  chapter15  chapter16  chapter2  functions.new  rmdir  tt.sh
alloy@alloy:~/linuxshell$
```

4．删除文件命令——rm

rm 命令提供删除文件功能，该命令可以删除目录中的一个或多个文件或子目录，它也可以将某个目录及其下的所有文件及子目录均删除。删除单个文件不用带任何参数；如果是删除整个目录及目录下的所有文件，需要带-rf 参数。其一般格式为：

```
rm [option] [fileName or directoryName]
```

其中，[option]为 rm 命令选项，[fileName or directoryName]为文件名或子目录名。表 2-9 列出了 rm 命令选项及其意义。

表 2-9　rm 命令选项及其意义

选　项	意　义
-f	忽略不存在的文件，从不给出提示
-r	指示 rm 将参数中列出的全部目录和子目录均递归地删除
-i	进行交互式删除

使用 rm 命令要小心，因为一旦文件被删除，它是不能被恢复的。为了防止这种情况的发生，可以使用 i 选项来逐个确认要删除的文件。如果用户输入 y，文件将被删除，而如果输入任何其他字段，文件则不会被删除。

如果删除的是文件，则其格式为：

```
rm fileName
```

如果删除的是目录，则其格式为：

```
rm -rf directoryName
```

例 2-15 演示了 rm 命令的用法，该例删除目录 shell 下的文件 function.new 和目录 rmdir。可以看出，在删除文件时，Linux 会提示用户是否要删除该文件。

```
#例 2-15：使用 rm 删除一个目录及该目录下的文件和子目录
alloy@alloy:~/linuxshell$ ls
chapter13  file1.sh  file1.txt  file2.txt  file5  testvim
#使用交互式 rm 选项删除空目录 chapter13
alloy@alloy:~/linuxshell$ rm -ir chapter13
rm: remove directory `chapter13'? y
alloy@alloy:~/linuxshell$ ls
file1.sh  file1.txt  file2.txt  file5  testvim
alloy@alloy:~/linuxshell$
```

2.2.2 目录操作常用命令

目录操作常用命令包括创建目录命令 mkdir、删除目录命令 rmdir 和目录切换命令 cd，下面我们逐一讲述这些命令。

1. 创建目录命令——mkdir

mkdir 命令用于创建目录，为文件分门别类地放在这些创建的目录中，其执行要指定需要创建的目录即可，与 DOS 下的 md 命令类似，其语法格式如下所示：

```
mkdir [option] [directoryName]
```

其中，[option]为 mkdir 命令选项，[directoryName]是需要创建的目录名称。需要说明的是，该命令创建由[directoryName]命名的目录，要求创建目录的用户在当前目录中（directoryName 的父目录中）具有写权限，并且[directoryName]不能是当前目录中已有的目录或文件名称，表 2-10 列出了 mkdir 命令选项及其意义。

表 2-10 mkdir 命令选项及其意义

选　项	意　义
-m	对新建目录设置存取权限
-p	可以是一个路径名称。此时若路径中的某些目录尚不存在，加上此选项后，系统将自动建立好那些尚不存在的目录，即一次可以建立多个目录
-v	每次创建新目录都显示信息

在进行目录创建时可以使用-m 选项设置目录的权限，下面的例 2-16 创建了一个目录名为 tsk，让所有的用户都有 rwx（即读、写、执行的权限）。

```
#例 2-16：通过 mkdir 创建一个名为 tsk 的目录，所有的用户都可读和执行
alloy@alloy:~/linuxshell$ ls
chapter13  chapter15  chapter16  chapter2
alloy@alloy:~/linuxshell$ mkdir -m 777 tsk          #创建目录，并指定权限
alloy@alloy:~/linuxshell$ ls -l
total 20
drwxr-xr-x. 2 alloy alloy 4096 2010-03-04 00:08 chapter13
drwxr-xr-x. 2 alloy alloy 4096 2010-03-02 04:01 chapter15
drwxr-xr-x. 2 alloy alloy 4096 2010-03-03 19:51 chapter16
drwxr-xr-x. 3 alloy alloy 4096 2010-03-04 01:05 chapter2
```


混混藏书阁 http://book-life.blog.163.com

```
drwxrwxrwx. 2 alloy alloy 4096 2010-03-04 03:07 tsk
alloy@alloy:~/linuxshell$
```

上面的例 2-16 用-m 777 指定了目录的读、写、执行权限，2.2.3 节将介绍文件和目录的权限控制。

在创建目录的过程中，如果父目录不存在，则可使用-p 选项同时创建父目录和需要创建的子目录，下面的例 2-17 演示了 mkdir 命令-p 选项的用法：

```
#例 2-17：演示 mkdir 命令-p 选项的用法
#第 1 条命令：testdir 不存在，mkdir 不带-p 选项将出错
alloy@alloy:~/linuxshell$ mkdir testdir/test
mkdir: cannot create directory `testdir/test': No such file or directory
#第 2 条命令：加上-p 选项成功创建 testdir 目录及其子目录 test
alloy@alloy:~/linuxshell$ mkdir -p testdir/test
alloy@alloy:~/linuxshell$ ls
chapter13  chapter15  chapter16  chapter2  testdir  tsk
alloy@alloy:~/linuxshell$ cd testdir
alloy@alloy:~/linuxshell/ testdir$ ls
test
alloy@alloy:~/linuxshell/ testdir$
```

在例 2-17 中，由于 testdir 目录是不存在的，因而，mkdir 不带-p 选项创建 testdir/test 时产生语法错误；当带上-p 选项后，mkdir 能够成功创建 testdir 目录及其子目录 test。

2. 删除目录命令——rmdir

rmdir 命令可以删除一个或多个目录，在删除目录时，目录必须为空。rm 命令可以同时删除多个目录，在删除某一目录时，必须拥有该目录的父目录的写和执行的权限。其语法格式如下所示：

```
rmdir [option] [directoyName]
```

其中，[option]为 rmdir 命令选项，[directoryName]为目录名，表 2-11 列出了 rmdir 命令选项及其意义。

表 2-11　rmdir 命令的选项及其意义

选　项	意　义
-p	递归删除。当子目录删除后，其父目录为空时，也将会一同被删除。如果整个路径被删除或者由于某种原因保留部分路径，则系统会在标准输出上显示相应的信息
--ignore-fail-on-non-empty	忽略非空目录的错误信息

由于 rmdir 仅仅能删除父目录中只包含空子目录的情况，如果目录中存在文件，则使用 rmdir 和 rmdir -p 命令是无法删除该目录的，所以，需使用“rm -rf [directoryName]”命令代替 rmdir，删除某目录时，同样要求该用户拥有对父目录的写权限。

下面的例 2-18 演示了 rmdir -p 和 rm -rf 的用法。例子中首先使用 rmdir -p 命令删除只存在一个空子目录 test1 的目录 testdir1，可以看出该命令递归地删除了目录 testdir1 和其子目录 test1；然后再次使用 rmdir -p 命令删除含有文件 test.txt 和空的子目录 test2 的目录 testdir2，可以看出 testdir2 的空的子目录 test2 成功地被删除，但提示目录 testdir2 不为空，无法删除；最后使用 rm -rf 命令则成功地删除了不为空的目录 testdir2。

```
#例 2-18：使用 rmdir 删除存在子目录的目录
#目录 testdir1 中仅仅存在一个空子目录 test1，使用 rmdir -p 命令循环删除目录 testdir1 和 test1
alloy@alloy:~/linuxshell$ rmdir -p testdir1/test1
```

```
alloy@alloy:~/linuxshell$ls
chapter13  Chapter7  Chapter8  myfile  nusers  sn.txt  testdir2
#查看 testdir2 目录下的文件和子目录
alloy@alloy:~/linuxshell$ ls testdir2
test.txt test2
#由于 testdir2 中存在文件，使用 rmdir -p 命令仅仅删除了空目录 testdir2
alloy@alloy:~/linuxshell$ rmdir -p testdir2/test2
rmdir: failed to remove directory “testdir2”：目录非空
#使用 rm -rf 命令则成功地删除了含有文件的目录 testdir2
alloy@alloy:~/linuxshell$ rm -rf testdir2
[alloy@alloy:~/linuxshell$ ls
chapter13  Chapter7  Chapter8  myfile  nusers  sn.txt
alloy@alloy:~/linuxshell$
```

3. 目录切换命令——cd

在 Linux 操作系统的文本模式下，在目录之间进行切换都需要通过命令来完成。显然，这没有图形化界面下使用鼠标操作那么方便。但是，Shell 中提供了一个目录切换字符——cd，在该字符的帮助下，系统管理员可以轻松地在不同的目录之间进行切换。其语法格式如下：

```
cd [directoryName]
```

该命令用于变换工作目录，其中，[directoryName]可以为绝对路径，也可为相对路径，绝对路径从“/”（指代根）开始，然后切换到所需的目录；相对路径从当前目录开始，当前目录可以是文件系统的任何地方。需要注意的是想要访问目录或文件的相对路径之前，要确保知道当前工作目录的位置，如果标明的是到另一个目录或文件的绝对路径，则不必担心文件系统所在的位置，如果不能肯定，可输入 pwd 命令，就会显示当前的工作目录。表 2-12 是 cd 命令的具体使用说明。

表 2-12　cd 命令的具体使用说明

命　令	使用说明
cd	返回登录目录
cd ~	同样是返回登录目录
cd /	返回系统根目录
cd /root	返回到根用户或超级用户（在安装时创建的账号）的主目录，但必须是根用户才能访问该目录
cd /home	返回到 home 目录，home 目录通常为用户登录目录的上级目录
cd ..	向上移动一级目录
cd -	返回上次访问的目录

在目录切换的过程中，还有两个比较重要的特殊字符，分别为“.”与“..”符号，其中“.”表示当前目录。这个符号很重要，在很多地方都需要用到。如在定义 PATH 环境变量的时候，在路径的最后需要加上这个“.”号，这表示当前目录。如果系统管理员想运行当前目录下的一个脚本文件，如 setup.sh，则可以不采用绝对路径，而直接使用“./setup.sh”命令，其中这个“.”符号就代表当前目录。在 cd 命令中也可以使用“.”符号，如“cd ./setup”命令表示进入到当前目录的下一个子目录 setup 下面。

另外一个特殊的字符就是“..”（英文状态下的双点号），它在系统中表示的是上一级目录。如果管理员利用 cd 命令定义到一个目录后，又想回到上一级目录中，则可以使用命令“cd ..”

混混藏书阁 http://book-life.blog.163.com

命令来实现，注意，cd 命令与点号之间要有空格。下面举例说明 cd 命令的用法，例 2-19 通过使用 cd 命令返回到根目录中，并通过 ls 命令返回根目录下的文件和子目录，然后通过“cd –”命令回到上次访问的目录。

```
#例 2-19：一个使用 cd 的例子
alloy@alloy:~/linuxshell$ cd /    #查看根目录
alloy@alloy:/$ls
bin  cdrom  etc  initrd.img  lib  lost+found  mnt  proc  run  selinux  sys  usr  vmlinuz
boot  dev  home  initrd.img.old  lib64  media  opt  root  sbin  srv  tmp  var  vmlinuz.old
alloy@alloy:/$ cd -
/home/alloy/linuxshell/CH13
alloy@alloy:~/linuxshell/CH13$
```

“~”符号也是一个比较有用的符号，如系统管理员想从任何目录中回到用户的主目录下，除了按原路返回外，要回到用户的主目录下，有一个很便捷的方式，就是通过一个特殊的字符“~”来完成。通常情况下，当管理员创建某个用户后，在系统的/home 目录中会以这个用户的名字建立一个文件夹。这个文件夹所在的目录就是用户的主目录。当用户不知道自己位于哪个目录，而需要迅速回到自己的主目录时，可以使用命令“cd ~”来实现。下面的例 2-20 用于说明“~”及其用法，该例通过“cd ~”命令迅速回到了当前用户 alloy 的主目录。

```
#例 2-20：使用“~”快速到达用户主目录
alloy@alloy:~/linuxshell/CH13$ cd ~
alloy@alloy:~$
```

2.2.3 文件和目录权限管理

Linux 系统中的每个文件和目录都有访问许可权限，用它来确定用户能以何种方式对文件和目录进行访问和操作。

文件或目录的访问权限分为只读、只写和可执行三种。以文件为例，只读权限表示只允许读其内容，而禁止对其做任何的更改操作；可执行权限表示允许将该文件作为一个程序执行。文件被创建时，文件所有者自动拥有对该文件的读、写和可执行权限，以便于对文件的阅读和修改。用户也可根据需要把访问权限设置为任何组合。

有三种不同类型的用户可对文件或目录进行访问：文件所有者、同组用户和其他用户。文件所有者一般是文件的创建者，他可以允许同组用户访问文件，还可以将文件的访问权限赋予系统中的其他用户，从而使系统中每一位用户都能访问该所有者拥有的文件或目录。

每一文件或目录的访问权限都有三组，每组用三位表示，分别为文件属主的读、写和执行权限，与属主同组的用户的读、写和执行权限，以及系统中其他用户的读、写和执行权限。下面的例 2-21 列出一个文件和一个目录的详细信息。

```
#例 2-21：查看文件和目录的权限
alloy@alloy:~/linuxshell$ ls -l CARGO.db
-rw-r--r--. 1 root root 225 03-15 13:34 CARGO.db
alloy@alloy:~/linuxshell$ ls -l example/
总计 4
drwxr-xr-x. 2 root root 4096 2009-09-05 ch01
alloy@alloy:~/linuxshell$
```

例 2-21 中，CARGO.db 是一个文件，详细信息的第 1 个字符是横线（-）；而 example 是一个目录，详细信息的第 1 个字符是 d。权限字段 r 代表只读，w 代表写，x 代表可执行。

文件和目录都有三组权限，以 CARGO.db 为例，第 1 组是 rw，表示该文件的属主具有读写权限；第 2 组是 r，表示与文件属主同组的用户只有读权限；第 3 组是 r，其他用户也只有读权限；ch01 目录也是同样的，图 2-1 显示了 CARGO.db 文件和 ch01 目录的文件权限对应关系。

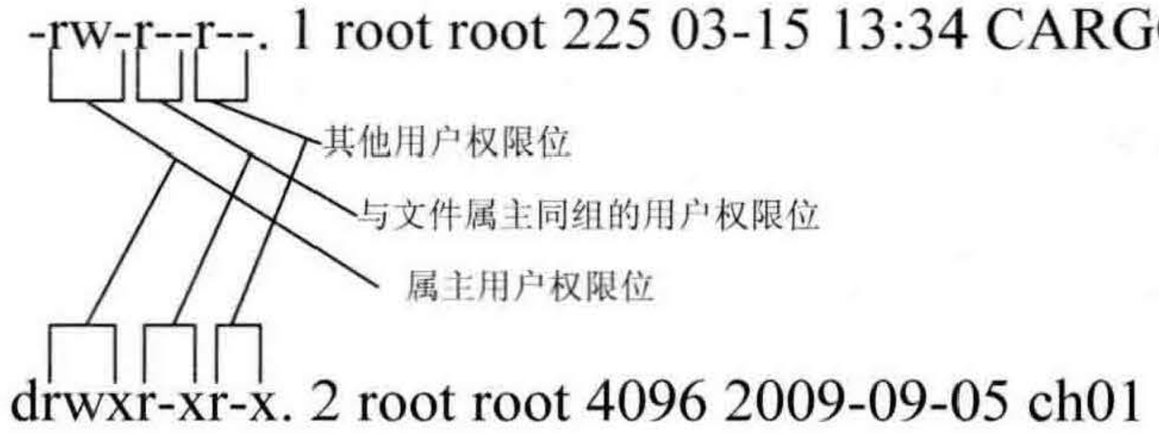


图 2-1　文件权限位示意图

文件和目录权限管理依赖于两个极其重要的命令：更改文件（目录）权限命令 chmod 和更改文件（目录）用户命令 chown，下面我们详细讲述这两个命令。

1．更改文件（目录）权限命令——chmod

chmod 命令用于更改文件或目录的访问权限，它有两种用法：一种是包含字母和操作符表达式的文字设定法；另一种是包含数字的数字设定法。文字设定法的格式为：

```
chmod [userType] [signal] [type] [filename]
```

chmod 命令的三种参数类型如表 2-13 所示。

表 2-13　chmod 的三种参数类型

用户类型（userType）	数学符号（signal）	文件类型（type）
u 表示用户（user），即文件或目录的所有者	+ 添加某个权限	r 可读
g 表示同组（group），即与文件属主同组的用户	- 取消某个权限	w 可写
o 表示其他（others）用户	= 赋予给定权限并取消其他所有的权限	x 可执行
a 表示所有（all）用户。它是系统默认值		

下面的例 2-22 利用 chmod 命令改变 testvim 文件的权限，将 testvim 文件的权限改成：属主可读、可写、可执行，同组用户可读、可写。

```
#例 2-22：修改文件 testvim 的属性
alloy@alloy:~/linuxshell/CH13$ ls -l testvim
-rw-r--r--. 1 root root 0 2010-03-04 04:11 testvim
alloy@alloy:~/linuxshell/CH13$  chmod u+x,g+w testvim
alloy@alloy:~/linuxshell/CH13$  ls -l testvim
-rwxrw-r--. 1 root root 0 2010-03-04 04:11 testvim
alloy@alloy:~/linuxshell/CH13$
```

注意：chmod 命令的不同参数之间需要用逗号隔开。

那么什么是 chmod 的数字设定法呢？首先需要了解用数字表示属性的含义：0 表示没有权限，1 表示可执行权限，2 表示可写权限，4 表示可读权限，然后将其相加。所以数字属性的格式应为 3 个从 0 到 7 的八进制数，其顺序是 u，g，o。比如例 2-22 中的

混混藏书阁 http://book-life.blog.163.com

```
chmod u+x,g+w testvim
```

命令等价于下面的命令：

```
chmod 764 testvim
```

下面的例 2-23 新创建了一个文件 testvim1，然后使用了 chmod 数字设定法对 testvim1 进行修改权限，可以看出，执行后的文件 testvim1 和上面 testvim 文件的权限相同，其中 touch 命令可以用于文件的创建。

```
#例 2-23：使用 chmod 数字设定法对文件 testvim1 进行权限控制
alloy@alloy:~/linuxshell/CH13$  touch testvim1       #创建文件 testvim1
alloy@alloy:~/linuxshell/CH13$  ls -l testvim1       #查询文件 testvim1 的权限
-rw-r--r--. 1 root root 0 2010-03-04 04:22 testvim1
alloy@alloy:~/linuxshell/CH13$  chmod 764 testvim1   #修改 testvim1 的权限
alloy@alloy:~/linuxshell/CH13$  ls -l testvim1       #查看修改后的 testvim1 的权限
-rwxrw-r--. 1 root root 0 2010-03-04 04:22 testvim1
alloy@alloy:~/linuxshell/CH13$
```

2. 更改文件（目录）属主命令——chown

利用 chown 命令可以改变文件或目录的属主。一般来说，这个指令只由系统管理者（root）使用，一般使用者没有权限改变别人的文件或目录属主，也没有权限将自己的文件属主更改为别人的，只有系统管理者（root）才有这样的权限。chown 命令的一般形式为：

```
chown [option] [owner] [filename]
```

其中，[option]为 chown 命令选项，[owner]为改变后的用户属主，而[filename]为需要改变属主的文件或目录。表 2-14 是 chown 命令的选项及其意义。

表 2-14　chown 命令的的选项及其意义

选　项	意　义
-c	若该文件或目录属主确实已经更改，才显示其更改动作
-h	改变符号链接文件的属主时不影响该链接所指向的目标文件
-f	若该文件或目录属主无法被更改也不要显示错误信息
-v	显示属主变更的详细资料
-R	对目前目录下的所有文件与子目录进行相同的拥有者变更（即以递归的方式逐个变更）

需要注意的是，当用户要改变一个文件的属主时，所使用的用户除了是 root 用户外，还可以是目标属主的成员。下面的例 2-24 是使用 chown 命令将属于 root 用户的文件 file.txt 改为 wyq 用户的。

```
#例 2-24：使用 chown 改变用户属主
alloy@alloy:~/linuxshell/CH2$ # ls -l
total 8
drwxr-xr-x. 2 root root 4096 2010-03-04 01:34 chapter13
-rwxr-xr-x. 1 root root   51 2010-03-03 23:36 file.txt
#更改 file.txt 文件的属主为 wyq 用户
alloy@alloy:~/linuxshell/CH2$ chown -h wyq file.txt
alloy@alloy:~/linuxshell/CH2$ ls -l
total 8
drwxr-xr-x. 2 root root 4096 2010-03-04 01:34 chapter13
-rwxr-xr-x. 1 wyq  root   51 2010-03-03 23:36 file.txt
alloy@alloy:~/linuxshell/CH2$
```

3. 特殊权限命令——SUID 与 SGID

除了上面提到的基本权限操作外，还有所谓的特殊权限存在。由于特殊权限会拥有一些“特权”，因而，用户若无特殊需要，不应该去打开这些权限，避免安全方面出现严重漏洞，甚至摧毁系统。但有时却需要没有被授权的用户完成某项任务，例如 passwd 程序，它允许用户改变口令，这就要求改变/etc/passwd 文件的口令域。然而系统管理员决不允许普通用户拥有直接改变这个文件的权利，为了解决这个问题，SUID/SGID 便应运而生，下面介绍了这两个特殊权限的说明。

- SUID：当一个设置了 SUID 位的可执行文件被执行时，该文件以所有者的身份运行，也就是说无论谁来执行这个文件，它都拥有文件所有者的特权，可以任意使用该文件拥有者能使用的全部系统资源。如果所有者是 root，那么执行人就有超级用户的特权了。
- SGID：当一个设置了 SGID 位的可执行文件被执行时，该文件将具有所属组的特权，任意存取整个组所能使用的系统资源；若一个目录设置了 SGID，则所有被复制到这个目录下的文件，其所属的组都会被重设为和这个目录一样，除非在复制文件时加上 -p 选项，才能保留原来所属的群组设置。还可以使用符号方式来设置 SUID/GUID。

SUID 和 SGID 占据了 ls -l 清单中 x 位相同的空间，如果开始设置了可执行权限 x 位，则其相应的位置用小写的 s 表示；如果没有设置可执行权限 x 位，则其相应的位置表示为大写的 S。设置和除去 SUID 与 SGID 很间接，设置 SUID 位和去除 SUID 位分别使用如下命令：

```
chmod u+s [filename]   #设置[filename]的 SUID 位
chmod u-s [filename]    #去除[filename]的 SUID 位
```

同样设置和去除 SGID 的命令分别为：

```
chmod g+s [filename]   #设置[filename]的 SGID 位
chmod g-s [filename]    #去除[filename]的 SGID 位
```

下面举一个例子讲解如何设置 SUID 位，例 2-25 中的 file.txt 开始的权限没有设置 SUID，使用 chmod 命令后就改变了 SUID。

```
#例 2-25：使用 chmod 命令实现 SUID/GUID
alloy@alloy:~/linuxshell/CH2$ ls -l
total 8
drwxr-xr-x. 2 root root 4096 2010-03-04 01:34 chapter13
-rwxr-xr-x. 1 wyq  root  435 2010-03-04 15:48 file.txt
alloy@alloy:~/linuxshell/CH2$ chmod u+s file.txt
alloy@alloy:~/linuxshell/CH2$ ls -l
total 8
drwxr-xr-x. 2 root root 4096 2010-03-04 01:34 chapter13
-rwsr-xr-x. 1 wyq  root  435 2010-03-04 15:48 file.txt
alloy@alloy:~/linuxshell/CH2$
```

SUID 的程序往往伴随着一定的安全问题。有时一个 SUID 程序与一个系统程序（或库函数）之间的交互作用会产生连程序的编制者也不知道的安全漏洞。一个典型的例子是/usr/lib/preserve 程序，它被 vim 和 ex 编辑器使用，当用户在写出对文件的改变前被意外地与系统中断时，它可以自动制作一个正被编辑的文件副本。这个保存的 preserve 程序将改变写在一个专门的目录内的临时文件中，然后利用/bin/mail 程序发送给用户一个“文件已经被存”的通知。

混混藏书阁 http://book-life.blog.163.com

2.2.4 查找文件命令——find

find 命令是 Linux 系统查找文件的命令，find 命令能帮助用户在使用、管理 Linux 的日常事务时方便地查找出用户所需要的文件。find 命令的基本格式是：

```
find [路径] [选项] [操作]
```

在上述 find 命令中，路径是 find 命令所查找的目录路径，例如，用“.”来表示当前目录，用“/”来表示系统根目录。选项用于指定查找条件，如：可以指定按照文件属主、更改时间、文件类型等条件来查找，表 2-15 列出了 find 命令的常用选项及其意义。

表 2-15 find 命令常用选项及其意义

选　项	意　义
name	根据文件名查找文件
perm	根据文件权限查找文件
prune	使用这一选项可以使 find 命令不在当前指定的目录中查找，如果同时使用-depth 选项，那么-prune 将被 find 命令忽略
user	根据文件属主查找文件
group	根据文件所属的用户组查找文件
mtime -n +n	根据文件的更改时间查找文件，-n 表示文件更改时间距今在 n 天之内，+n 表示文件更改时间距今在 n 天前
nogroup	查找无有效所属组的文件，即该文件所属的组在/etc/groups 中不存在
nouser	查找无有效属主的文件，即该文件的属主在/etc/passwd 中不存在
-newer file1 ! file2	查找更改时间比文件 file1 新但比文件 file2 旧的文件
type	查找某一类型的文件，type 后跟的子选项及其意义如下： b:块设备文件 d:目录 c:字符设备文件 p:管道文件 l:符号链接文件 f:普通文件
size n:[c]	查找文件长度为 n 块的文件，带有 c 时表示文件长度以字节计
depth	在查找文件时，首先查找当前目录中的文件，然后在其子目录中查找

find 命令的操作用于指定结果的输出方式，表 2-16 列出了 find 命令的操作名称及其意义。

表 2-16 find 命令的操作名称及其意义

操作名称	意　义
print	将匹配的文件输出到标准输出
exec	对匹配的文件执行该参数所给出的 Shell 命令。相应命令的形式为'command' {　} \;。注意{　　}和\;之间的空格
ok	和-exec 的作用相同，只不过以一种更安全的模式来执行该参数所给出的 Shell 命令，在执行每一个命令之前，都会给出提示，让用户来确定是否执行

下面举几个例子来说明 find 命令的用法，以及其路径、选项、操作的用法。首先，请看

下面的例 2-26。

```
#例 2-26：演示 find 命令的 print 操作
#第 1 条命令：查找当前目录下文件名以 t 开头的，且文件属主具有读、写、执行权限的文件
alloy@alloy:~/linuxshell/CH2$ find . -name 't*' -perm 744 -print
./DEBUG/trapdebug.sh
./test.sh
./traploop.sh
./testvar.sh
#第 2 条命令：查找更改时间距今 90 天内的文件
alloy@alloy:~/linuxshell/CH2$ find . -mtime -90 -print
.
./hfile
./loggg
./newfile
alloy@alloy:~/linuxshell/CH2$
```

例 2-26 中的两条命令变换了 find 的查找条件，但是，操作都是用 print，即打印出满足查找条件的所有结果，第 1 条命令查找当前目录下文件名以 t 开头的，且文件属主具有读、写、执行权限的文件，命令用“.”表示当前目录，-name 't*'表示以 t 开头的文件名，其中的“*”字符表示任意字符，这种用法称为通配，本书第 3 章将会介绍这一概念。-perm 744 表示文件属主具有读、写、执行权限，744 是表示文件权限的数字设定法。第 1 条命令结果的./表示当前目录下的文件，./DEBUG 则表示子目录 DEBUG 下的文件。第 2 条命令查找更改时间距今 90 天内的文件，使用了-mtime -n 用法。

find 命令 exec 操作将对匹配的文件执行该参数所给出的 Shell 命令，下面的例 2-27 演示了 exec 操作的用法。

```
#例 2-27：演示 find 命令的 exec 操作
alloy@alloy:~/linuxshell/CH2$ find /etc -type f -name "rc*" -exec ls -l {} \;
-rwxr-xr-x. 1 root root 220 2009-05-02 /etc/rc.d/rc.local
-rwxr-xr-x. 1 root root 2596 2009-05-02 /etc/rc.d/rc
-rwxr-xr-x. 1 root root 23756 2009-05-02 /etc/rc.d/rc.sysinit
-rw-r--r--. 1 root root 401 2009-05-02 /etc/event.d/rc4
-rw-r--r--. 1 root root 401 2009-05-02 /etc/event.d/rc3
-rw-r--r--. 1 root root 817 2009-05-02 /etc/event.d/rcS
-rw-r--r--. 1 root root 644 2009-05-02 /etc/event.d/rc1
-rw-r--r--. 1 root root 420 2009-05-02 /etc/event.d/rc6
-rw-r--r--. 1 root root 401 2009-05-02 /etc/event.d/rc2
-rw-r--r--. 1 root root 550 2009-05-02 /etc/event.d/rc0
-rw-r--r--. 1 root root 401 2009-05-02 /etc/event.d/rc5
-rw-r--r--. 1 root root 519 2009-05-02 /etc/event.d/rcS-sulogin
alloy@alloy:~/linuxshell/CH2$
```

例 2-27 中的命令表示查找/etc 目录下文件名以 rc 开头的普通文件，并对查找结果执行 ls -l 命令，即列出查找结果的详细文件信息。例 2-27 的结果表明该命令列出了/etc 目录及其子目录下所有 rc 开头的普通文件的详细信息，列出详细信息是 exec 操作进一步处理的结果。

ok 操作和 exec 的作用相同，只不过以一种更安全的模式来执行该参数所给出的 Shell 命令，在执行每一个命令之前，都会给出提示，让用户来确定是否执行。在执行一些危险操作时，建议使用 ok 操作，比如：下面的例 2-28 试图删除/var/log 目录下更改时间距今 3 天内的所有文件，由于删除文件的操作是不可以恢复的，因此，find 命令使用 ok 操作在删除文件之前给出提示，等待用户确定后再执行删除操作。

混混藏书阁 http://book-life.blog.163.com

```
#例 2-28：演示 find 命令的 ok 操作
#删除/var/log 目录下更改时间距今 3 天内的所有文件
alloy@alloy:~/linuxshell/CH2$ find /var/log -mtime -3 -ok rm {} \;
< rm ... /var/log > ? n
< rm ... /var/log/audit/audit.log > ? y
< rm ... /var/log/spooler > ? y
< rm ... /var/log/rpmpkgs > ? y
< rm ... /var/log/wtmp > ? y
< rm ... /var/log/secure-20100705 > ? y
.....
alloy@alloy:~/linuxshell/CH2$
```

由例 2-28 的结果可以看出，一旦 find 命令使用了 ok 操作，在对查找结果执行进一步操作前 Shell 出现提示信息，用户可输入 y 或 n 选项确认是否执行该项操作。

总之，find 命令的选项在指定查找条件时显得灵活多变，可以根据文件的各种属性来查找文件；对于查找到的结果，find 命令可以有三种处理方式，print 仅将结果输出到屏幕，而 exec 和 ok 可以对结果进一步处理，区别在于：exec 直接进行处理，而 ok 在处理之前出现提示信息，供用户最终确定是否执行该操作。

2.3 文本编辑器

编写 Shell 脚本需要使用文本编辑器，Linux 系统中的文本编辑器种类极多，如：Emacs、Kvim、vim、Gedit、Kate 等，本节介绍两种文本编辑器：第一种是 vim 编辑器，它适用于命令行界面，当我们使用 SSH Secure Shell 或 Putty 登录 Linux 系统时，vim 编辑器是不二选择；第二种是 Gedit 编辑器，它适用于图形化界面，当我们登录 GNOME 桌面环境时，一般使用 Gedit 编辑文件。

2.3.1 vim 编辑器

vim 是 UNIX 世界中最通用的全屏编辑器，vim 是“Vi IMproved”的简称，其是 vi 编辑器的加强版，提供了执行输入、输出、删除、查找、替换、块操作等众多文本操作，用户还可以根据自己的需要对其进行定制。

1. vim 的基础操作

vim 编辑器可以执行输出、删除、查找、替换、块操作等众多文本操作，而且用户可以根据自己的需要对其进行定制，这是其他编辑程序所没有的。vim 编辑器以命令行的方式处理文本，尽管不如图形化处理方式直观，但它具有操作速度快、功能全面等优点。另外，vim 不是一个排版程序，它不像 Word 或 WPS 那样可以对字体、格式、段落等其他属性进行编排，它只是一个文本编辑程序，其基本格式如下：

```
vim [option] [filename…]
```

或

```
vim [option] [filename…]
```

其中，[option]是选项，[filename]是需要编辑的一个或多个文件名。如果在启动 vim 时没有指定文件名，则 vim 命令会自动产生一个无名的空文件。如果指定的[filename]文件不存在，

则 vim 将创建一个名为[filename]的新文件。启动 vim 后，消息行会显示文件的名称、文件中的行数和字数。消息行显示的信息随着所运行的命令的不同而不同，如果文件中的任何一行上有一条波浪线（~），就说明没有足够的行来填满屏幕。注意，vim 并不锁住所编辑的文件，因此多个用户可能同时编辑一个文件，那么最后保存的文件版本将被保留。表 2-17 列出了 vim 命令的选项及其意义。

表 2-17　vim 命令的选项及其意义

选　　项	意　　义
-c command	在对文件进行编辑前，先执行 command 命令
-r filename	恢复文件 filename
-R	以只读方式编辑文件
+n file	编辑 file 文件，并将光标置于第 n 行
+ file	编辑 file 文件，并将光标置于最后一行
+/string file	编辑 file 文件，并将光标置于第一个保护 string 所表示的字符串的行

2. vim 的工作模式

输入 vim 命令打开 vim 编辑器后，其运行状态共有以下三种模式。

（1）一般模式（Normal mode）

输入 vim 命令进入 vim 文本编辑器的时候，就是一般模式，该模式将用户的输入看作命令，这个模式允许用户移动游标，且允许搜索文本功能。图 2-2 是用 vim 编辑器打开 stack.sh 文件时的一般模式，此时，vim 编辑器的最后一行是文件名、文件包含的字符数和字节数。退出 vim 编辑器、保存当前的修改也是在一般模式中进行的，在一般模式下按冒号按钮，将出现如图 2-3 所示的界面，vim 编辑器的最后一行显示冒号，冒号后面输入保存、退出等命令，这些命令共有四种，如表 2-18 所示。

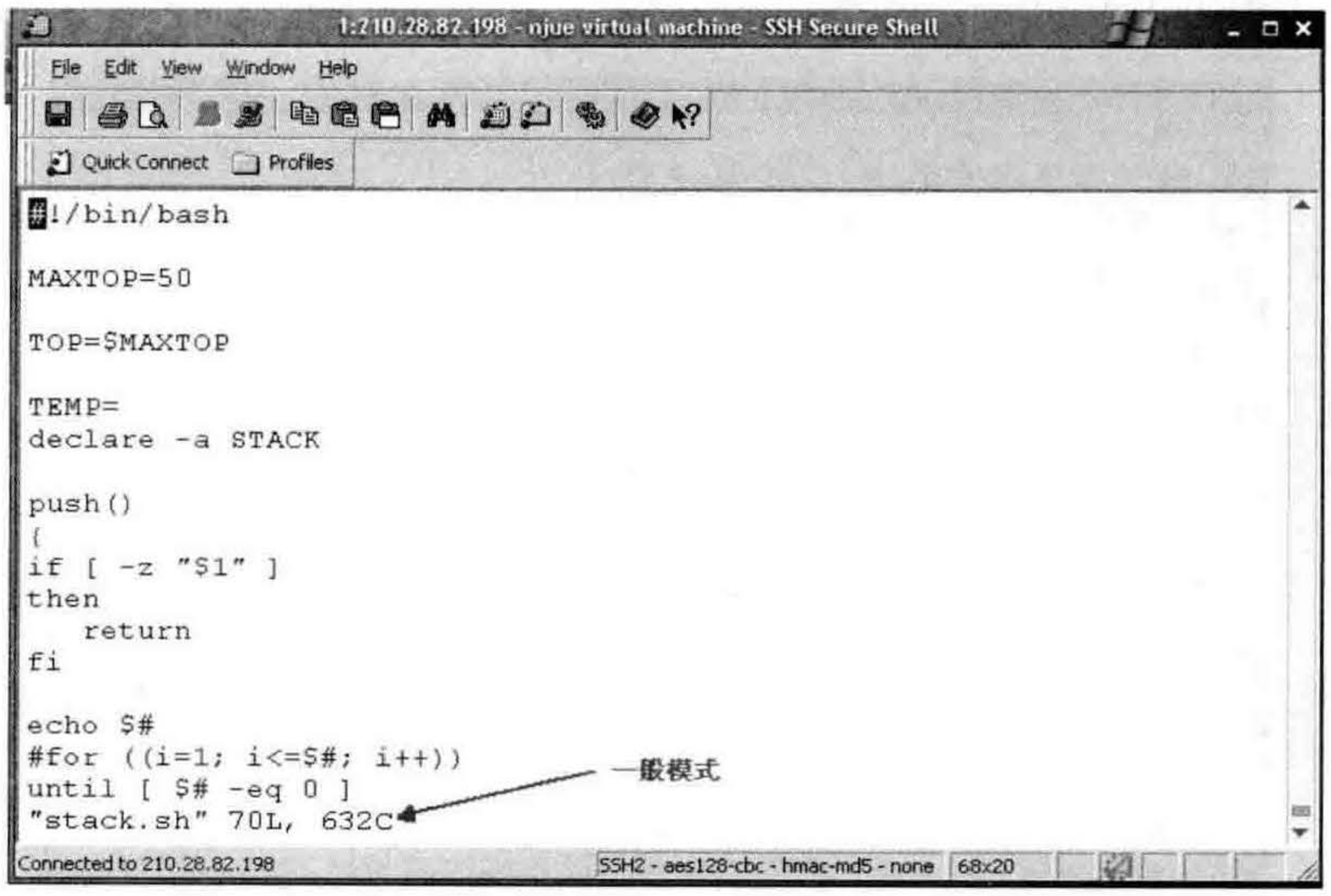


图 2-2　vim 编辑器的一般模式：打开文件时

混混藏书阁 http://book-life.blog.163.com

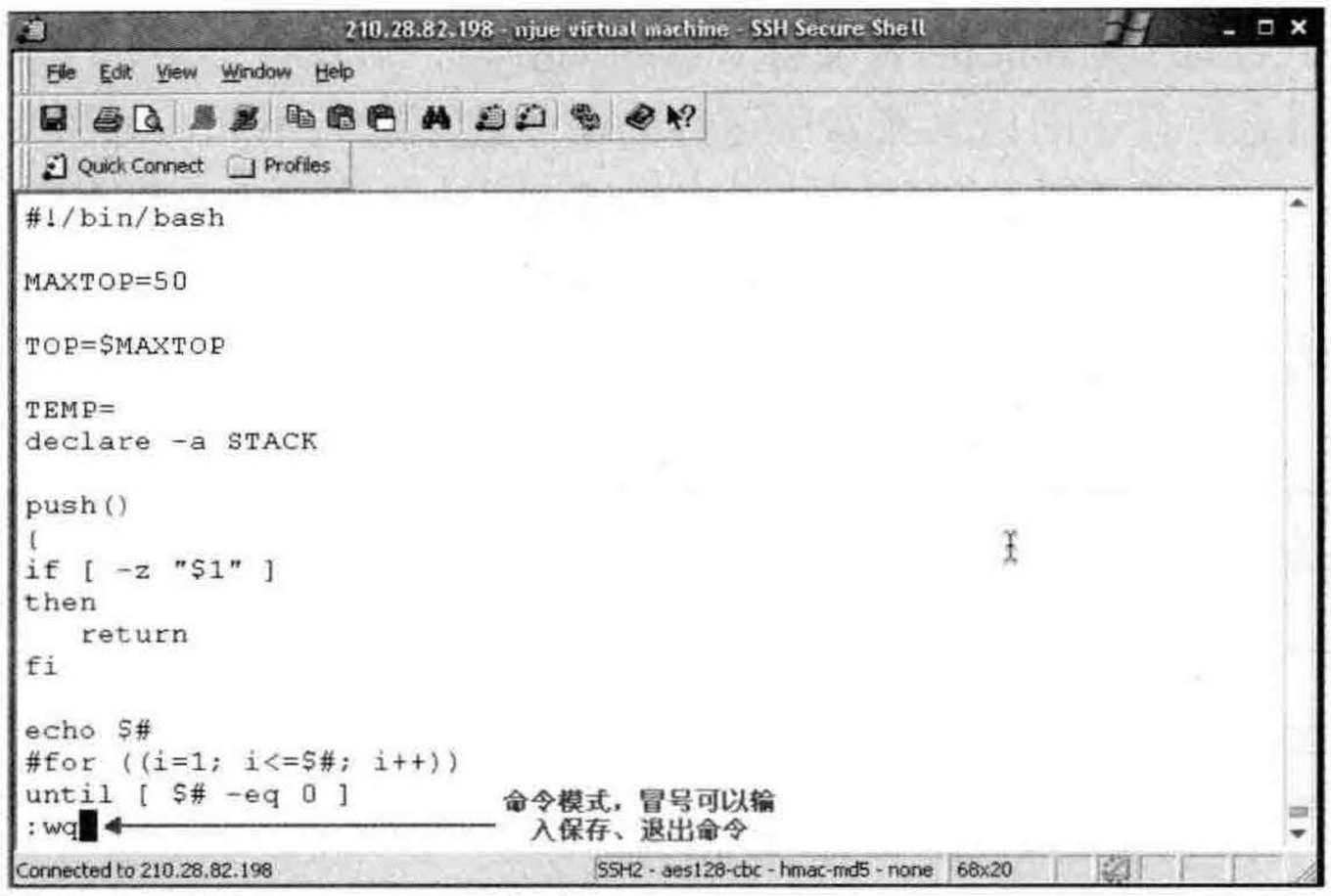


图 2-3　vim 编辑器的一般模式：保存和退出

表 2-18　vim 编辑器文本保存和退出命令

保存和退出命令	描　　述
W	将编辑的文本存储
Q	离开 vim 文本编辑器
q!	曾修改过文本，但是不想存储，使用该命令强制离开 vim 文本编辑器
Wq	存储文本并离开 vim 文本编辑器

（2）插入模式（Insert mode）

在一般模式按下 i、o、a 等字母都可以进入编辑模式，在此模式下，vim 将用户的输入插入到当前光标位置，修改暂时保存到缓冲区，按 Esc 键则从编辑模式退回到一般模式。图 2-4 显示了 vim 编辑器的编辑模式，vim 编辑器的最后一行显示 INSERT，这表示能插入新字符。

图 2-4　vim 编辑器的编辑模式

vim 编辑器在插入模式下编辑文件时有两个关键技巧：移动光标到适当的位置和编辑文本。vim 编辑器提供了丰富的移动光标命令，如表 2-19 所示，对于较短的文件，使用 h、j、k、l 进行上下左右的移动就可以满足需求。但是，对于较长的文件，经常需要根据段落、句子、行数来移动光标。

表 2-19　用于移动光标的 vim 命令

命　令	动　作
h	将光标向左移动
j、加号（+）、Enter	将光标向下移动
k、减号（-）	将光标向上移动
l	将光标向右移动
}	将光标移动到当前段落的末尾
{	将光标移动到当前段落的开头
)	将光标移动到当前句子的末尾
(	将光标移动到当前句子的开头
^	移动到当前行的第一个非空字符
$	移动到当前行末尾
:n	移动到行 n

编辑文件似乎不需要展开多解释，一般情况下，我们可以像使用 Windows 系统的记事本一样编辑文件，表 2-20 列出了 vim 编辑器经常用到的编辑命令，熟练使用这些命令能够提高编辑文件的效率，比如：dd 能够删除光标所在的整行文本，d$能够删除当前光标位置到该行结束的所有文本等。

表 2-20　常用的 vim 编辑命令

命　令	动　作
x	删除光标当前位置的字符
dd	删除光标所在的整行文本
d$	删除当前光标位置到该行结束的所有文本
dw	从当前光标位置向前删除单词
J	将下一行文本内容合并到本行行尾
a	在当前光标位置后附加内容
A	在当前光标所在行的后面附加内容

（3）底行工作模式

在该工作模式下，用户输入的任何字符串都会被当作命令，会在 vim 的最下面一行显示，按下回车键后便会执行该命令，如果该字符串并不是一个有效的命令，则会出现错误提示。

我们在编辑文件时常常需要搜索单词或替换单词，vim 编辑器也提供了搜索和替换功能。首先按下斜杠按钮（/）进入底行工作模式，此时光标会自动移到 vim 编辑器下方的命令行，用户输入待搜索的字符串，按下 Enter 键开始搜索，vim 编辑器可能用以下三种方式响应用户的搜索。

混混藏书阁 http://book-life.blog.163.com

- 当满足搜索条件的字符串出现在当前光标位置后面时，vim 编辑器将光标跳转到第一个满足搜索条件的位置。
- 当满足搜索条件的字符串出现在当前光标位置前面时，vim 编辑器将跳转到从头开始的第一个满足搜索条件的位置。
- 文件中不存在满足条件的字符串时，vim 编辑器提示错误信息。

图 2-5 显示了 vim 编辑器的搜索功能，它是在/etc/servimces 文件中搜索 ssh 字符串，vim 编辑器下方出现斜杠（/），在/后面输入待搜索的字符串。

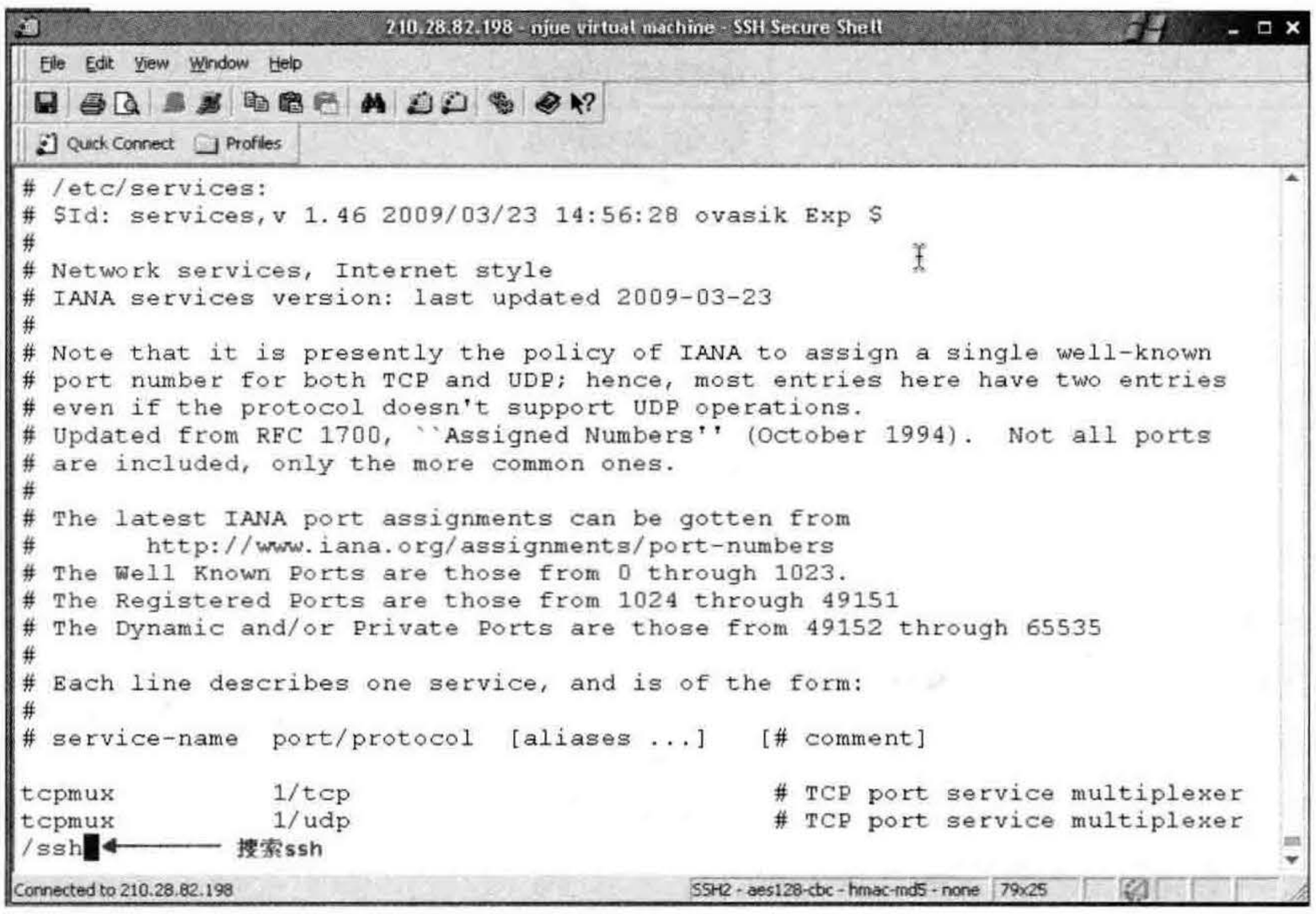


图 2-5　vim 编辑器的搜索功能

vim 编辑器的替换命令的基本格式为：

```
:s/old_string/new_string
```

上述替换命令表示将第一次出现的 old_string 替换成 new_string。当然，可以在上述替换命令后加上 g 选项，表示将所有的 old_string 替换成 new_string，命令为：

```
:s/old_string/new_string/g
```

图 2-6 显示了 vim 编辑器的替换功能，该命令将所有的 TOP 替换为 STACKTOP，表 2-21 归纳了 vim 编辑器的搜索和替换命令，我们可以看到，vim 编辑器在替换文本时还可以指定行号的范围。

表 2-21　vim 编辑器的搜索和替换命令

命令名称	意　义
/word	自当前光标位置向下搜索名字为 word 的字符串
?word	自当前光标位置向上搜索名字为 word 的字符串
:n1,n2s/word1/word2/g	在 n1 行与 n2 行之间搜索名字为 word1 的字符串，并将其替换为 word2
:1,$s/word1/word2/g	在第 1 行与最后一行之间搜索名字为 word1 的字符串，并将其替换为 word2

图 2-6　vim 编辑器的替换功能

图 2-7 是 vim 编辑器三种工作模式的总结。

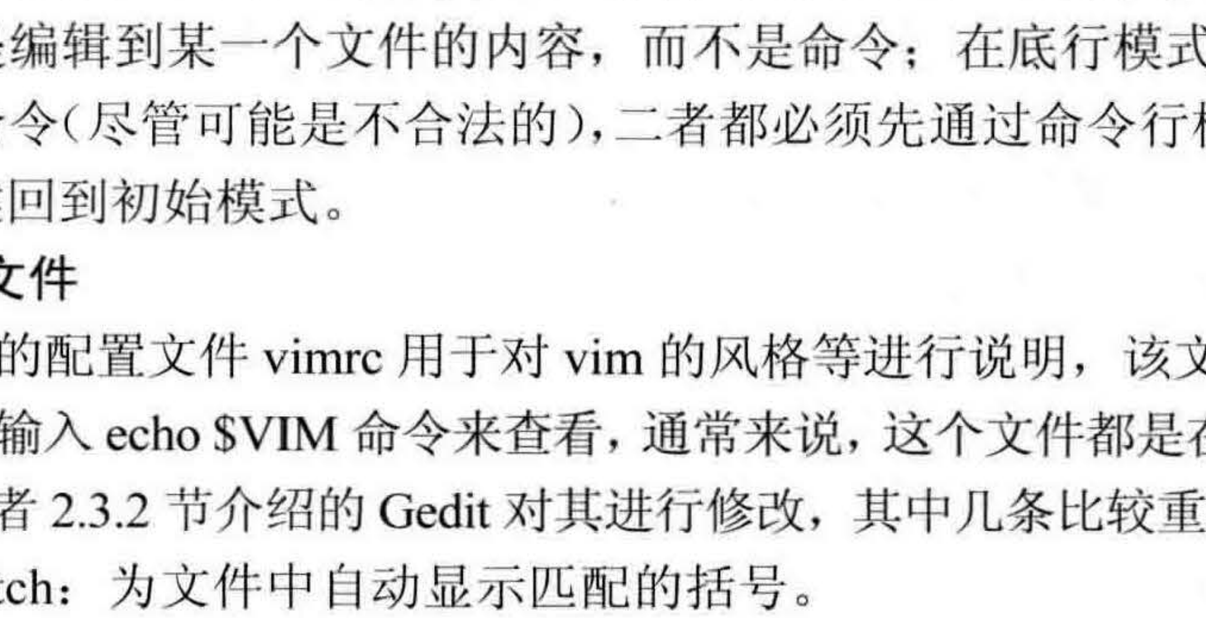


图 2-7　vim 三种工作模式间的切换方法

从图 2-7 中可以看到，命令行工作模式是 vim 编辑器的初始模式，从该模式下可以实现到任何模式的切换；而插入模式和底行模式之间不能相互切换，因为在插入模式下，任何输入的字符都被认为是编辑到某一个文件的内容，而不是命令；在底行模式下，任何输入的字符都被看作是底行命令(尽管可能是不合法的)，二者都必须先通过命令行模式才能进入对方，即需要先按下 Esc 键回到初始模式。

3. vim 的配置文件

vim 有一个对应的配置文件 vimrc 用于对 vim 的风格等进行说明，该文件的位置可以通过在 vim 的底行模式下输入 echo $VIM 命令来查看，通常来说，这个文件都是在/usr/share/vim 目录下，可以使用 vim 或者 2.3.2 节介绍的 Gedit 对其进行修改，其中几条比较重要的设置说明如下。

- set showmatch：为文件中自动显示匹配的括号。
- set nu：在文件中显示行号。

混混藏书阁 http://book-life.blog.163.com

- set autoindent：编辑时自动缩进。
- set cindent：编辑时按照 C 语言自动缩进，但是这个缩进很大。
- set mouse：打开对鼠标的支持，滚轮和单击均可。

2.3.2 Gedit 编辑器

Gedit 是一个 Ubuntu 桌面环境 Unity 下兼容 UTF-8 的文本编辑器。它简单易用，有良好的语法高亮显示功能，对中文支持很好，还支持包括 GB2313、GBK 在内的多种字符编码。Gedit 利用 GNOME VFS 库还可以编辑远程文件。它支持完整的恢复和重做系统，以及查找和替换；还支持包括多语言拼写检查和一个灵活的插件系统，可以动态地添加新特性。Gedit 还增加了一些小特性，包括行号显示、括号匹配、文本自动换行、当前行高亮以及自动文件备份。Gedit 需要在 GNOME 图形化环境下使用，Gedit 的启动命令为：

```
gedit file1 file2 … filen
```

gedit 是 Gedit 的启动命令，可以同时打开多个文件，每个文件将在 Gedit 编辑器上生成一个标签，单击相应的标签就可以看到相应的文件。Gedit 能将 Shell 脚本、C 语言等文件的关键字显示不同的颜色，即所谓的语法高亮功能，极大地方便了程序员编辑文件。图 2-8 显示了 Gedit 编辑器的界面，其界面与 Windows 的文本编辑器风格相似，读者一定不会感到陌生。

图 2-8 Gedit 编辑器的界面

Gedit 编辑器包含菜单栏、工具栏、显示区域、输出窗口和状态栏等部分，其菜单栏包括如下项目：

- File（文件）：用于处理新文件、保存现有文件和打印文件。
- Edit（编辑）：用于操作活动缓冲区中的文本，以及设置编辑器首选项。
- View（查看）：用于设置在窗口中显示的编辑器特性，已经设置文本高亮模式。
- Search（搜索）：用于查找和替换活动编辑器缓冲区中的文本。
- Tools（工具）：用于访问 Gedit 中安装的插件工具。
- Documents（文档）：用于管理在缓冲区中打开的文件。
- Help（帮助）：用于访问完整的 Gedit 手册。

File 菜单提供了 Open Location 选项，允许使用 WWW 世界中流行的标准统一资源标识符（URL）格式从网络上打开文件，该格式表示可以用于访问文件的协议（如 HTTP 或 FTP）、文件所在的服务器和该文件在服务器的完整访问路径。

Gedit 有若干种可用的插件，但并非所有的插件都是默认安装的。表 2-22 列出了当前可用的插件。

表 2-22　Gedit 插件列表

插　　件	描　　述
Change Case	更改所选文本的大小写
Document Tools	报告单词、行、字符和非空字符的数量
External Tools	在编辑器中提供一个 Shell 环境，用于执行命令和脚本
File Brower Pane	提供一个简单的文件浏览器，便于选择要编辑的文件
Indent Lines	提供高级缩进和不缩进功能
Insert Date/Time	在当前光标位置插入各种格式的当前日期和时间
Modelines	在编辑器窗口底部提供 Emacs 样式的消息行
Python Console	在编辑器窗口底部提供一个交互式控制台，用于使用 Python 编程语言输入命令
Snippets	允许保存经常使用的文本段，以便在文本的任何地方检索
Sort	对整个文件或所选文本快速排序
Spell Checker	为文本文件提供字典拼写检查
Tag List	提供经常使用的字符串列表，使用户可以方便地在文本中输入这些字符串
User name	在当前光标位置插入当前用户的登录名

2.4 本章小结

本章简明扼要地介绍了与 Shell 脚本编程相关的 Linux 基础知识，包含用户和用户组管理的常用命令、文件和目录操作的常用命令、文件和目录的权限管理、查找文件命令——find、两种文本编辑器：vim 和 Gedit。

由于篇幅所限，本章不可能全面、系统地介绍 Linux 基础知识，对此还需进一步学习的读者可以参考 Linux 基础类的参考书。

2.5 上机提议

1．假定目录下有如下文件：

```
alloy@alloy:~/linuxshell/CH16$ ls
bad_style_indent.sh          login1.sh
color_script1.sh             login2.sh
color_script2.sh             no_interaction_script1.sh
```


混混藏书阁 http://book-life.blog.163.com

```
file1                          no_interaction_script.sh
file11                         no_shift.sh
file2                          package_sed.sh
file_can_execute_or_not1.sh    print_ascii.sh
file_can_execute_or_not2.sh    root_or_not.sh
file_exist_or_not1.sh          shift_exam1.sh
file_exist_or_not2.sh          shift_exam1.sh2.sh
getopts_exam1.sh               shift_exam2.sh
getopts_exam2.sh               show_num1.sh
getopts_exam3.sh               test.txt
ip_login.sh                    use_shift.sh
alloy@alloy:~/linuxshell/CH16$
```

则下面的命令将输出什么内容？

（1）echo *

（2）echo f*

（3）echo get*

（4）echo *.*

（5）echo *.sh

2．使用文件操作命令创建一个文件，然后使用文件复制、移动和删除等命令对该文件进行操作。

3．创建一个目录，然后在该目录下再创建一个文件，并对该文件进行删除操作。

4．新建一个用户组 group1，并新建一个系统组 group2。

（1）更改组 group2 的 GID 为 103，更改组名为 grouptest。

（2）删除组 grouptest。

5．新建用户 user1，指定 UID 为 777，目录为/home/user1，初始组为 group1，有效组为 root，指定 shell 为/bin/bash。

（1）新建一个系统用户 user2。

（2）查看用户 user1 的组群，切换到 user1，在主目录下新建文件 test1，切换有效组为 root，再新建文件 test2。

（3）修改用户 user1 的个人说明为 This is a test（提示加-c 选项）。

（4）修改用户密码过期时间为 2010-09-01。

（5）更改用户 user1 的密码为 111111，加锁用户 user1 并查看/etc/shadow，用户 user1 通过 ssh 登录 127.0.0.1。

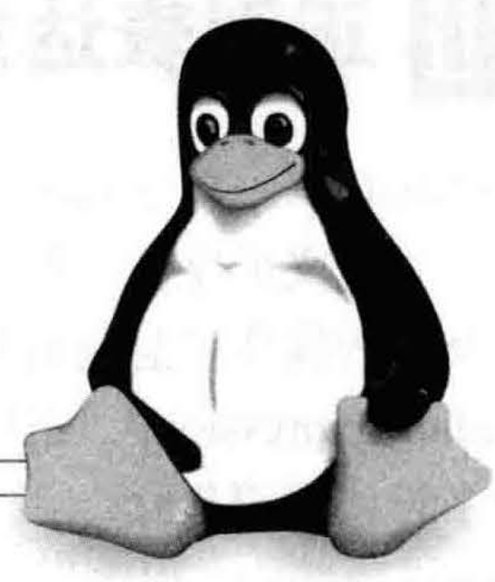
（6）更改用户主目录/home/user1 为/home/user11。

（7）列出用户 user1 的 UID、GID 等。

（8）增加用户 user3、user4，增加组 testgroup，给组 testgroup 设定密码，将组 testgroup 管理权授予 user1，并同时将 root、user1、user3 加入到 testgroup，检查结果，切换到 user1，将 user4 加入到 testgroup 组。

（9）使用 passwd 将 user1 用户密码冻结，用 passwd 查看 user1 的相关信息，最后用 passwd 将用户 user1 解冻。

（10）切换 user1 用户，用 su 加命令行直接查看 shadow 的尾。



第3章

正则表达式

由于很多 Linux Shell 编程的工具和命令普遍使用了正则表达式，如 grep、sed 和 awk 等，因此，不理解正则表达式就无法理解和熟练使用 Shell 编程的工具和命令。为此，本章深入介绍正则表达式的基础知识，首先，详细讨论基本正则表达式和扩展正则表达式中元字符的意义和用法。然后，介绍 Shell 在搜索匹配文件时经常使用的机制——通配，通配所用到的元字符与正则表达式存在细微差别，本章结合例子逐一讨论了通配的元字符。最后，介绍 Linux 系统中使用广泛的 grep 命令，着重讨论 grep 命令的基本用法，及如何与正则表达式相结合以更加灵活地进行文本搜索。此外，还将简单介绍 grep 命令族中的其他两个命令 egrep 和 fgrep。

混混藏书阁 http://book-life.blog.163.com

3.1 正则表达式基础

Linux Shell 以一串字符作为表达式向系统传达意思。元字符（Metacharacters）是用来阐释字符表达式意义的字符，简言之，元字符就是描述字符的字符，它用于对字符表达式的内容、转换及各种操作信息进行描述。正则表达式是由一串字符和元字符构成的字符串，简称 RE（Regular Expression）。正则表达式的主要功能是文本查询和字符串操作，它可以匹配文本的一个字符或字符集合。

在 Linux 系统中，程序设计语言 Java、Perl 和 Python 等，Shell 工具 sed、awk 和 grep 等，以及 MySQL 和 PostgreSQL 等数据库服务器都使用了正则表达式，图 3-1 描述了正则表达式用于数据流处理的过程。实际上，正则表达式完成了数据过滤，将不满足正则表达式定义的数据拒绝掉，剩下与正则表达式匹配的数据。

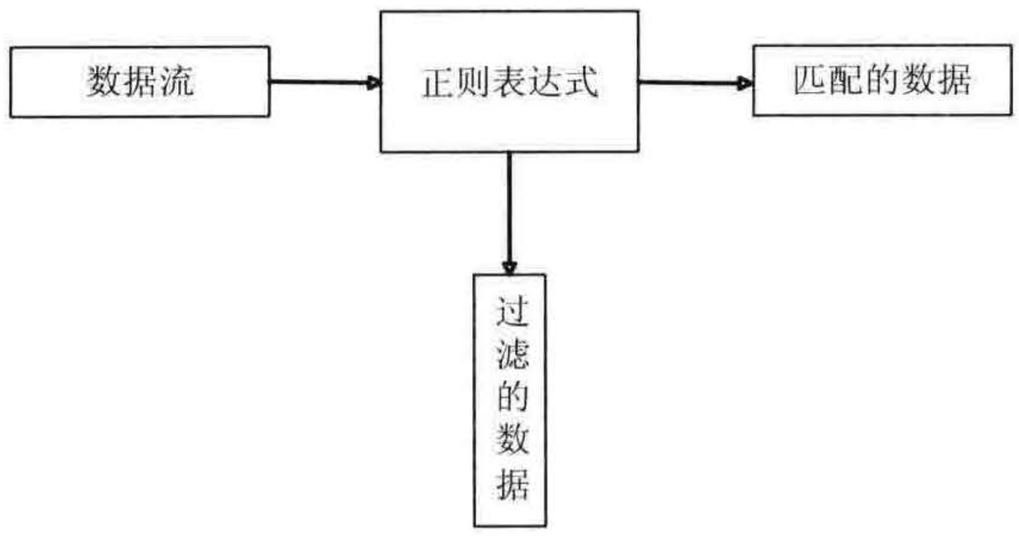


图 3-1　正则表达式处理数据过程

正则表达式的基本元素包括普通字符和元字符，例如，a、b、1、2 等字符属于普通字符，普通字符可以按照字面意思理解，如：a 只能理解为英文的小写字母 a，没有其他隐藏含义。而*、^、[]等元字符，Shell 赋予了它们超越字面意思的意义，如：*符号的字面意义只是一个符号，而实际上却表示了重复前面的字符 0 次或多次的隐藏含义。因此，掌握正则表达式基本元素主要是对正则表达式中元字符意义的掌握。

POSIX 标准将正则表达式分为两类：基本的正则表达式和扩展的正则表达式，大部分 Linux 应用和工具仅支持基本的正则表达式，因而，本节所述的正则表达式基础是掌握正则表达式的关键，表 3-1 列出了基本的正则表达式中元字符集合及其意义。

表 3-1　基本的正则表达式元字符集合及其意义

符　　号	意　　义
*	0 个或多个在*字符之前的那个普通字符
.	匹配任意字符
^	匹配行首，或后面字符的非
$	匹配行尾
[]	匹配字符集合

续表

符　号	意　义
\	转义符，屏蔽一个元字符的特殊意义
\<\>	精确匹配符号
\{n\}	匹配前面字符出现 n 次
\{n,\}	匹配前面字符至少出现 n 次
\{n,m\}	匹配前面字符出现 n～m 次

下面逐一介绍正则表达式元字符的意义和用法，并举一些例子结合使用元字符集合。

1. “*” 符号

“*” 符号用于匹配前面一个普通字符的 0 次或多次重复，如：

```
#例 3-1：*符号的意义
hel*o
```

“*”符号前面的普通字符是 l，*字符就表示匹配 l 字符 0 次或多次，如字符串 helo、hello、hellllllo 都可以由 hel*o 来表示。

2. “.” 符号

点号 “.” 用于匹配任意一个字符，如：

```
#例 3-2：.符号的意义
…73.
```

由于 “.” 符号只能匹配一个字符，因此，上述字符串表示前面三个字符为任意字符，第 4 和第 5 个字符是 7 和 3，最后一个字符为任意字符，如 xcb738、4J973U 都能匹配上述字符串。值得注意的是，“.” 符号可以匹配一个空格，因此，x b738、ui 73e 也能匹配上述字符串。

3. “^” 符号

“^” 符号用于匹配行首，表示行首的字符是 “^” 字符后面的那个字符，如：

```
#例 3-3：^符号的意义
^cloud
```

这表示匹配以 cloud 开头的行。结合上面介绍的 “*” 符号和 “.” 符号，再举一个例子：

```
#例 3-4：^符号、.符号和*符号结合使用
^…X86*
```

该字符串表示行首的三个字符为任意字符（可以是空格），第 4～6 个字符为 X86，第 7 个字符开始可以重复匹配 6，如：866X86666、8 6X86 都可以匹配上述字符串。

4. “$” 符号

$符号匹配行尾，$符号放在匹配字符之后，与 “^” 符号的功能和用法都相反，如：

```
#例 3-5：$符号的意义
micky$
```

该正则表达式表示匹配以 micky 结尾的所有行。一个特殊的正则表达式是匹配所有空行的表达式，为：

```
#例 3-6：空行的表示方法
^$
```

该正则表达式既匹配行首，又匹配行尾，中间没有任何字符，因此，为空行。读者需要牢记例 3-6 所示的空行表示方法，很多命令都用到这个正则表达式来表示空行。

如果需要匹配只包含一个字符的行，如下面的例 3-7 所示：

混混藏书阁 http://book-life.blog.163.com

```
#例 3-7：包含一个字符的行
^.$
```

5. “[]”符号

方括号[]匹配字符集合，该符号支持穷举方法列出字符集合的所有元素，也支持使用“-”符号表示字符集合范围，表明字符集合范围从“-”左边字符开始，到“-”右边字符结束。如果要匹配任意一个数字，可以使用如例 3-8 所示的两种方法，前一种穷举了阿拉伯数字，后一种用数字范围表示，显得比较简洁。

```
#例 3-8：匹配任意一个数字
[0123456789]
[0-9]
```

[]也可以用作字母匹配，例 3-9 给出了匹配字母的例子：

```
#例 3-9：匹配字母
[a-z]                                    #所有小写字母
[A-Z]                                   #所有大写字母
[b-p]                                    #小写字母 b 到 p
```

Linux 系统对大小写是敏感的，并且支持字母排序，因此，Linux 中有大写字母序列和小写字母序列，两者是分开的，例 3-9 中 a～z 表示所有小写字母，A～Z 表示所有大写字母，而 b～p 表示从 b 到 p 之间的所有小写字母。

我们知道，^符号表示匹配行首，但是，^符号放到[]符号中就不再表示匹配行首了，而是表示取反符号，请看下面的例 3-10。

```
#例 3-10：^表示取反
[^b-d]
```

例 3-10 的正则表达式匹配不在 b～d 范围之内的所有字符，此时，符号^不再表示匹配行首，而是取反符号，不在 b～d 范围内的字符实际上涵盖了除了小写字母 b、c 和 d 之外的所有字符（包括其他字母、数字、空格等）。再举一个[]符号和*符号结合的例子：

```
#例 3-11：匹配所有的英文单词
[A-Za-z] [A-Za-z]*
```

例 3-11 的正则表达式表示以任意一个字母开头，再以任意字母进行 0 次或任意次重复，实际上，这个正则表达式可以匹配任意英文单词。

6. “\”符号

“\”符号是转义符，用于屏蔽一个元字符的特殊意义，即以字面含义来解释“\”符号后面的元字符，如：

```
#例 3-12：转义符
\.
```

反斜杠后面的字符“.”是元字符，经过转义后，“.”不再表示任意一个字符，而是一个普通字符点号“.”。转义符“\”是引用符的一种，我们将在 6.2.3 节深入讨论它，在此不再展开论述。

7. “\<\>”符号

“\<\>”符号是精确匹配符号，该符号利用“\”符号屏蔽“<>”符号，如：

```
#例 3-13：精确匹配
\<the\>
```

该正则表达式精确匹配 the 这个单词，而不匹配包含 the 字符的单词，如 them、there、another 等。

8. “\{\}”系列符号

“\{\}”系列符号与“*”符号类似，都表示前一个字符的重复。但是，“*”符号表示重复 0 次或任意次，而“\{\}”系列符号可以指定重复次数，“\{\}”系列符号包括三种形式。

- \{n\}：匹配前面字符出现 n 次。
- \{n,\}：匹配前面字符至少出现 n 次。
- \{n,m\}：匹配前面字符出现 n～m 次。

请看下面的例 3-14。

```
#例 3-14：\{\}系列符号的用法
JO\{3\}B                                    #重复字符 O 3 次
JO\{3,\}B                                   #重复字符 O 至少 3 次
JO\{3,6\}B                                  #重复字符 O 3～6 次
```

JO\{3\}B 表示重复字符 O 3 次，匹配值为：JOOOB。

JO\{3,\}B 表示重复字符 O 至少 3 次，JOOOB、JOOOOB、JOOOOOB 等字符串都可由该正则表达式来匹配。

JO\{3,6\}B 表示重复字符 O 至少 3 次，至多 6 次，JOOOB、JOOOOOOB 等字符串都满足，但是 JOOB、JOOOOOOOB 等字符串就不满足。

再举一个例子：

```
#例 3-15：精确匹配 5 个小写字母
[a-z] \{5\}
```

例 3-15 中的表达式表示精确匹配 5 个小写英文字母，比如 hello、house 等。

3.2 正则表达式的扩展

除了表 3-1 列出的正则表达式的元字符之外，awk 和 Perl 等 Linux 工具还支持正则表达式扩展出来的一些元字符，这些元字符如表 3-2 所示。

表 3-2　扩展的正则表达式元字符及其意义

符　号	意　义
?	匹配 0 个或 1 个在其之前的那个普通字符
+	匹配 1 个或多个在其之前的那个普通字符
()	表示一个字符集合或用在 expr 中
|	表示“或”，匹配一组可选的字符

下面详细介绍扩展的正则表达式元字符及其用法。

1. “?”符号

匹配“?”符号之前的那个字符 0 次或 1 次，如：

```
#例 3-16：?符号的意义
JO?B
```

该表达式表示匹配 O 字符 0 次或 1 次，即匹配 JOB 或 JOOB。需要注意的是，“?”字符至多可以匹配 1 个字符。

混混藏书阁 http://book-life.blog.163.com

2. “+” 符号

与“*”符号类似，都是匹配其前面的那个字符多次，但是，“*”符号可以匹配 0 次，而“+”符号至少匹配 1 次，如：

```
#例 3-17：+符号的意义
S+EU
```

该表达式表示匹配 S 1 次或任意次，SSEU、SSSSEU 等字符串都可由该表达式进行匹配，而 SEU 却不能由 S+EU 来匹配。

3. “()” 符合和 “|” 符号

“()”符号通常与“|”符号结合使用，表示一组可选字符的集合，如：

```
#例 3-18：()符号和|符号的意义
re(a|e|o)d
```

该表达式中的(a|e|o)表示在字符 a、e 和 o 中选择任意一个字符，即 read、reed、reod 都可由该表达式进行匹配。

事实上，“()”符号很少被使用到，因为“[]”符号完全能够替代“()”符号表示一组可选字符的集合，re(a|e|o)d 就等价于：

```
re[aeo]d
```

“|”符号也可以表示多个正则表达式的“或”关系，基本格式为：

```
RE1 | RE2 | RE3 |…
```

上述格式中，RE1、RE2 和 RE3 表示正则表达式。

“|”符号在扩展的正则表达式中表示“或”，遗憾的是，“|”符号的这种用法却很少被人记住，“|”符号最著名的是其管道符用法，这将在第 10 章中详细介绍。

3.3 通配

bash Shell 本身不支持正则表达式，使用正则表达式的是 Shell 命令和工具，如 grep、sed、awk 等，这些命令将在本书后续章节详细介绍。但是，bash Shell 可以使用正则表达式中的一些元字符实现通配（Globbing）功能，通配是把一个包含通配符的非具体文件名扩展存储在计算机、服务器或者网络上的一批具体文件名的过程。最常用的通配符包括正则表达式元字符：?、*、[]、{}、^等。这些元字符在通配中的意义与正则表达式中的意义不完全一致，*符号不再表示其前面字符的重复，而是表示任意位的任意字符，?字符表示一个任意字符，^符号在通配中不代表行首，而是代表取反。

例如，如果一个用户不知道在一个扩展名为.rtf 的文件名中一个人的首名是如何拼写的，是 Philip 还是 Phillip。这个用户可以输入：

```
Phi*ip.rtf
```

下面举例说明通配的使用和通配元字符的意义，这些例子都用 ls 命令进行通配，ls 命令是 Linux 下最常用的命令之一，它用于列出目录下的文件，它可以有很多选项，ls -l 表示列出文件的详细信息，ll 命令等价于 ls -l 命令。/home/alloy/linuxshell/CH02 目录下的所有文件如下：

```
alloy@alloy:~/linuxshell/CH02$ ll   #目录下文件和目录的详细信息
```

```
总用量 436
drwxrwxr-x  3 alloy alloy   4096   4月 28 14:29 ./
drwxrwxr-x 18 alloy alloy   4096   4月 28 14:22 ../
-rw-rw-r--  1 alloy alloy     18   4月 28 14:22 00.pem
-rw-rw-r--  1 alloy alloy     73   4月 28 14:23 08.pem
-rw-rw-r--  1 alloy alloy     22   4月 28 14:25 11.pem
-rw-------  1 alloy alloy    372736   4月 28 14:24 .11.pem.swp
-rw-rw-r--  1 alloy alloy     14   4月 28 14:25 append.sed
-rw-rw-r--  1 alloy alloy     28   4月 28 14:26 argv.awk
-rw-rw-r--  1 alloy alloy     58   4月 28 14:26 array.awk
drwxrwxr-x  2 alloy alloy   4096   4月 28 14:26 BUILD/
-rw-rw-r--  1 alloy alloy     22   4月 28 14:27 delete.sed
-rw-rw-r--  1 alloy alloy     37   4月 28 14:27 environ.awk
-rw-rw-r--  1 alloy alloy      9   4月 28 14:27 findphone.awk
-rw-rw-r--  1 alloy alloy     34   4月 28 14:27 finephone.awk
-rw-rw-r--  1 alloy alloy     47   4月 28 14:28 null-undeclear
-rw-rw-r--  1 alloy alloy     53   4月 28 14:28 pass.awk
-rw-rw-r--  1 alloy alloy     22   4月 28 14:28 scr.awk
-rw-rw-r--  1 alloy alloy     51   4月 28 14:29 stephane
-rw-rw-r--  1 alloy alloy     61   4月 28 14:29 sturecord
alloy@alloy:~/linuxshell/CH02$
```

/home/alloy/linuxshell/CH02 目录下包含一个子目录 BUILD，子目录的详细信息以 d 开头，d 表示 directory 的意思，其他以横杠（-）开头的都是文件。

如果我们仅需要列出/home/alloy/linuxshell/CH02 目录下以.awk 结尾的文件，就可以使用*.awk 匹配所有以.awk 结尾的文件，如例 3-19 所示。

```
#例 3-19：列出以.awk 结尾文件的详细信息
alloy@alloy:~/linuxshell/CH02$ ls -l *.awk
-rw-rw-r-- 1 alloy alloy 28  4月 28 14:26 argv.awk
-rw-rw-r-- 1 alloy alloy 58  4月 28 14:26 array.awk
-rw-rw-r-- 1 alloy alloy 37  4月 28 14:27 environ.awk
-rw-rw-r-- 1 alloy alloy  9  4月 28 14:27 findphone.awk
-rw-rw-r-- 1 alloy alloy 34  4月 28 14:27 finephone.awk
-rw-rw-r-- 1 alloy alloy 53  4月 28 14:28 pass.awk
-rw-rw-r-- 1 alloy alloy 22  4月 28 14:28 scr.awk
alloy@alloy:~/linuxshell/CH02$
```

如果我们需列出以 0 开头、后面跟 1 个字符且以.pem 为后缀的文件，可以使用 0?.pem 来匹配这些文件，如例 3-20 所示。

```
#例 3-20：列出以 0 开头、后面跟 1 个字符且以.pem 为后缀的文件
alloy@alloy:~/linuxshell/CH02$ ls -l 0?.pem
-rw-rw-r-- 1 alloy alloy 18  4月 28 14:22 00.pem
-rw-rw-r-- 1 alloy alloy 73  4月 28 14:23 08.pem
alloy@alloy:~/linuxshell/CH02$
```

下面举一个用[]符号进行通配的例子，若我们需列出以 a～h 范围内的字母开头，并以.awk 结尾的文件，则可以用[a-h]*.awk 来匹配这些文件，如例 3-21 所示。

```
#例 3-21：列出以 a～h 范围内的字母开头，以.awk 结尾的文件
alloy@alloy:~/linuxshell/CH02$ ls -l [a-h]*.awk
-rw-rw-r-- 1 alloy alloy 28  4月 28 14:26 argv.awk
-rw-rw-r-- 1 alloy alloy 58  4月 28 14:26 array.awk
-rw-rw-r-- 1 alloy alloy 37  4月 28 14:27 environ.awk
-rw-rw-r-- 1 alloy alloy  9  4月 28 14:27 findphone.awk
-rw-rw-r-- 1 alloy alloy 34  4月 28 14:27 finephone.awk
```


混混藏书阁 http://book-life.blog.163.com

```
alloy@alloy:~/linuxshell/CH02$
```

例 3-21 的结果确实仅列出从 a 到 h 范围内的字母开头、.awk 结尾的文件，pass.awk 和 scr2.awk 并未列出。如果要列出从 a 到 h 范围内的字母开头、句点后不是以 awk 结尾的文件，可以使用[a-h]*.[^awk]*来匹配这些文件，句点后方括号内使用^符号表示取反，即除去 a、w 和 k 这三个字母，而且最后一个*符号必不可少，否则句点后仅匹配一个字符，例 3-22 给出[a-h]*.[^awk]*的匹配结果：

```
#例 3-22：列出 a～h 范围内的字母开头，不以.awk 结尾的文件
alloy@alloy:~/linuxshell/CH02$ ls -l [a-h]*.[^awk]*
-rw-rw-r-- 1 alloy alloy 14  4 月 28 14:25 append.sed
-rw-rw-r-- 1 alloy alloy 22  4 月 28 14:27 delete.sed
alloy@alloy:~/linuxshell/CH02$
```

由例 3-22 可知，[]符号的意义与正则表达式中[]符号的意义一样，那么通配中的花括号“{}”表示何种意义呢？正则表达式中只有在花括号前加上转义符的用法，即\{\}，用于限制匹配字符的个数。但是，通配中的“{}”符号表示一组表达式的集合，如：

```
{[a-h]*.awk ,0?.pem}
```

上述通配表示满足[a-h]*.awk 或 0?.pem 的所有文件，下面的例 3-23 给出这一通配的执行结果。

```
#例 3-23：列出匹配[a-h]*.awk 或 0?.pem 的所有文件
alloy@alloy:~/linuxshell/CH02$ ls -l {[a-h]*.awk,0?.pem}
-rw-rw-r-- 1 alloy alloy 18  4 月 28 14:22 00.pem
-rw-rw-r-- 1 alloy alloy 73  4 月 28 14:23 08.pem
-rw-rw-r-- 1 alloy alloy 28  4 月 28 14:26 argv.awk
-rw-rw-r-- 1 alloy alloy 58  4 月 28 14:26 array.awk
-rw-rw-r-- 1 alloy alloy 37  4 月 28 14:27 environ.awk
-rw-rw-r-- 1 alloy alloy  9  4 月 28 14:27 findphone.awk
-rw-rw-r-- 1 alloy alloy 34  4 月 28 14:27 finephone.awk
alloy@alloy:~/linuxshell/CH02$
```

上述例 3-23 的结果中既有满足 0?.pem 的 00.pem 和 08.pem 两个文件，也有满足[a-h]*.awk 的 argv.awk 等文件。注意，{}符号内的表达式是“或”的关系，即只要有{}符号内的一个表达式的文件，就能被列出。

通配的结果由计算机搜索大量的文件或者目录进行匹配而输出，通配对处理能力和内存资源有很高的需求。黑客输入包含通配符的文件名故意让服务器重复和连续不断地进行通配可能引起的拒绝服务攻击。因此，大型服务器经常通过限制服务器执行通配功能的次数、限制一个具体用户每次输入的通配符或者如果通配符太普通，则拒绝执行通配等方法来提高服务器的安全性。

内部变量 GLOBIGNORE 保存了通配时所忽略的文件名集合，9.1 节将详细介绍 GLOBIGNORE 变量的用法，应该说，?、*、[]、{}、^五个符号和 GLOBIGNORE 变量构成了 Shell 通配的所有内容。

3.4 grep 命令

grep 是 Global search Regular Expression and Print out the line 的简称，即全面搜索正则表

达式，并把行打印出来。grep 是一种强大的文本搜索工具，它能使用正则表达式搜索文本，并把匹配的行打印出来，grep 也是 Linux 中最广泛使用的命令之一。本节重点介绍 grep 命令，以及 grep 命令与正则表达式结合使用，并简略介绍 grep 命令族中的其他命令用法。

3.4.1 grep 命令基本用法

grep 命令是支持正则表达式的一个多用途文本搜索工具，grep 的一般格式为：

```
grep [选项][模式][文件...]
```

grep 命令由选项、模式和文件三部分组成，它在一个或多个文件中搜索满足模式的文本行，模板后的所有字符串被看作文件名，文件名可以有多个，搜索的结果被打印到屏幕，不影响原文件的内容。grep 命令的选项用于对搜索过程进行补充说明，grep 命令的选项及其意义如表 3-3 所示。

表 3-3 grep 命令选项及其意义

选　项	意　义
-c	只输出匹配行的数量
-i	搜索时忽略大小写
-h	查询多文件时不显示文件名
-l	只列出符合匹配的文件名，而不列出具体的匹配行
-n	列出所有的匹配行，并显示行号
-s	不显示不存在或无匹配文本的错误信息
-v	显示不包含匹配文本的所有行
-w	匹配整词
-x	匹配整行
-r	递归搜索，不仅搜索当前工作目录，而且搜索子目录
-q	禁止输出任何结果，以退出状态表示搜索是否成功
-b	打印匹配行距文件头部的偏移量，以字节为单位
-o	与-b 选项结合使用，打印匹配的词距文件头部的偏移量，以字节为单位
-E	支持扩展的正则表达式
-F	不支持正则表达式，按照字符串的字面意思进行匹配

grep 命令的模式十分灵活，可以是字符串，也可以是变量，还可以是正则表达式。需要说明的是，无论模式是何种形式，只要模式中包含空格，就需要使用双引号将模式引起来，如果不加双引号，空格后的单词容易被误认为是文件名，如普通字符串为“hello world”，grep hello world 命令就将 world 认为是文件名，因此，grep “hello world” filename 才是正确的写法。大部分情况下，使用单引号将模式引起来也是可以的，第 6 章将详细讨论双引号和单引号的区别，在此，我们只简单告诉读者：一旦模式中包含空格，就需要使用双引号或单引号将模式引起来。下面的例 3-24 可以充分验证这个观点。

```
#例 3-24：模式包含空格时，是否使用双引号的区别
#搜索 00.pem 文件中包含 certificate 字符串的行，不需要引号引起模式
```


混混藏书阁 http://book-life.blog.163.com

```
alloy@alloy:~/linuxshell/CH02$ grep certificate 00.pem
The above string is known as your user certificate subject, and it
To install this user certificate, please save this e-mail message
If you have any questions about the certificate contact

#若需要搜索 00.pem 文件中包含 user certificate 字符串的行
#我们看一看不用引号将 user certificate 引起来的结果
alloy@alloy:~/linuxshell/CH02$ grep user certificate 00.pem
grep: certificate: 没有那个文件或目录
#Shell 将 certificate 解析为文件名，提示没有此文件的错误
#下面给出 00.pem 文件中包含 user 字符串的行
00.pem:The above string is known as your user certificate subject, and it
00.pem:uniquely identifies this user.
00.pem:To install this user certificate, please save this e-mail message
00.pem: /home/alloy/linuxshell/CH02/usercert.pem

#用引号将 user certificate 引起来后得到正确的结果
alloy@alloy:~/linuxshell/CH02$ grep "user certificate" 00.pem
The above string is known as your user certificate subject, and it
To install this user certificate, please save this e-mail message
alloy@alloy:~/linuxshell/CH02$
```

例 3-24 首先搜索 00.pem 文件中包含 certificate 字符串的行，由于模式 certificate 中不包含空格，因此是否用引号引起模式对 grep 命令不产生影响。当我们要搜索 00.pem 文件中包含 user certificate 字符串的行时，不用双引号将 user certificate 引起来时，Shell 提示没有 certificate 这个文件或目录，然后，给出 00.pem 文件中包含 user 字符串的行，这说明 Shell 将 grep user certificate 00.pem 这条命令解释为在 certificate 和 00.pem 两个文件中搜索包含 user 字符串的行，这显然与我们的初衷不符。而当我们用双引号将 user certificate 引起来后，就得到了正确的结果。

grep 支持多文件查询，请看下面的例 3-25。

```
#例 3-25：演示 grep 的多文件查询
alloy@alloy:~/linuxshell/CH02$ grep Certificate 00.pem 08.pem
00.pem:This is a Certificate Request file:
00.pem:Certificate Subject:
08.pem:Certificate:
alloy@alloy:~/linuxshell/CH02$
```

上例搜索 00.pem 和 08.pem 两个文件中包含 Certificate 字符串的行，命令逐个给出待搜索的文件，结果打印出所有包含 Certificate 字符串的行，并以文件名开头。

grep 命令指定多个文件时可以使用通配，这样就不必逐个给出待搜索的文件了，例 3-25 的命令可以改成如例 3-26 所示的等价命令。

```
#例 3-26：用通配符表示多文件
alloy@alloy:~/linuxshell/CH02$ grep Certificate 0?.pem
00.pem:This is a Certificate Request file:
00.pem:Certificate Subject:
08.pem:Certificate:
alloy@alloy:~/linuxshell/CH02$
```

上例利用 0?.pem 代替了 00.pem 和 08.pem 两个文件，显得十分简洁。

表 3-3 已经列出了 grep 命令的选项，下面结合具体的例子逐个说明 grep 选项的含义和用法。

1. -c 选项

-c 选项表示输出匹配字符串行的数量，默认情况下，grep 命令打印出包含模式的所有行，一旦加上-c 选项，就只显示包含模式行的数量，下面给出一个使用-c 选项的例子。

```
#例 3-27: grep -c 的用法
alloy@alloy:~/linuxshell/CH02$ grep -c Certificate *.pem
00.pem:2                                    #00.pem 文件中有两行包含 Certificate
08.pem:1
11.pem:1
alloy@alloy:~/linuxshell/CH02$
```

例 3-27 对当前目录下所有.pem 的文件查找 Certificate 关键字，指定文件使用了通配，结果 00.pem:2 表示 00.pem 中有两行包含关键字 Certificate，08.pem 和 11.pem 各有一行包含关键字 Certificate。

2. -n 选项

-n 选项列出所有的匹配行，并显示行号。默认情况下，grep 搜索单个文件时，只显示每行的内容，搜索多个文件时，显示文件名及每行的内容，加上-n 选项后，将在行内容前附加显示行号，下面给出一个使用-n 选项的例子。

```
#例 3-28: grep -n 的用法
alloy@alloy:~/linuxshell/CH02$ grep -n Certificate *.pem
00.pem:1:This is a Certificate Request file:                #00.pem 文件的第 1 行
00.pem:7:Certificate Subject:
08.pem:1:Certificate:
11.pem:1:Certificate:
alloy@alloy:~/linuxshell/CH02$
```

例 3-28 仍然对当前目录下所有.pem 的文件查找 Certificate 关键字，结果 00.pem:1:表示 00.pem 文件的第 1 行包含 Certificate 关键字，并列出 00.pem 的第 1 行具体内容。对于 08.pem 和 11.pem 的搜索也同样显示了行号。

3. -v 选项

-v 选项显示不包含模式的所有行，下面给出一个使用-v 选项的例子。

```
#例 3-29: grep -v 的用法
alloy@alloy:~/linuxshell/CH02$ grep -vc Certificate *.pem       #同时使用-v 和-c 选项
00.pem:39                              #00.pem 文件中有 39 行不包含 Certificate 字符串
08.pem:50
11.pem:50
alloy@alloy:~/linuxshell/CH02$
```

例 3-29 结合使用-v 和-c 参数列出 00.pem 文件中不包含 Certificate 关键字的行数，结果 00.pem:39 表示 00.pem 文件中有 39 行不包含 Certificate 字符串。例 3-29 还说明 grep 命令可同时使用多个选项。

4. -i 选项

默认情况下，grep 命令对大小写是敏感的，如果加上-i 选项，则表示 grep 命令不区分大小写，下面给出一个使用-i 选项的例子。

```
#例 3-30: grep -i 的用法
alloy@alloy:~/linuxshell/CH02$ grep -i certificate 00.pem
This is a Certificate Request file:
Certificate Subject:
```


混混藏书阁 http://book-life.blog.163.com

```
The above string is known as your user certificate subject, and it
To install this user certificate, please save this e-mail message
If you have any questions about the certificate contact
-----BEGIN CERTIFICATE REQUEST-----
-----END CERTIFICATE REQUEST-----
alloy@alloy:~/linuxshell/CH02$
```

上例在 00.pem 文件中搜索不区分大小写的 certificate 字符串的所有行，可以看出，结果匹配到 Certificate、CERTIFICATE 等关键字。

5. -h 选项

-h 选项表示查询多文件时不显示文件名，默认情况下，用 grep 命令查询多个文件时，在匹配行之前显示文件名，加上-h 选项后，grep 命令将不再显示文件名。下面给出一个使用-h 选项的例子。

```
#例 3-31: grep -h 的用法
alloy@alloy:~/linuxshell/CH02$ grep -h Certificate *.pem
This is a Certificate Request file:                    #在匹配行前不再显示文件名了
Certificate Subject:
Certificate:
Certificate:
alloy@alloy:~/linuxshell/CH02$
```

例 3-31 仍然是在当前目录下所有.pem 的文件中查找 Certificate 字符串，加上-h 后，结果就只显示匹配行的内容，而不显示文件名。注意例 3-31 与例 3-28 中 grep 打印结果的区别。

6. -l 选项

-l 选项表示只列出符合匹配的文件名，而不列出具体匹配的行，下面给出一个使用-l 选项的例子。

```
#例 3-32: grep -l 的用法
alloy@alloy:~/linuxshell/CH02$ grep -l Certificate *
00.pem                                                 #只显示包含 Certificate 字符串的文件名
08.pem
11.pem
alloy@alloy:~/linuxshell/CH02$
```

例 3-32 的命令用于搜索当前目录下所有的文件中包含 Certificate 字符串的文本行，加上-l 选项后，在结果中不再显示具体的匹配行，只列出包含 Certificate 字符串的文件名。

7. -s 选项

-s 选项表示不显示不存在或无匹配文本的错误信息，默认情况下，grep 在待搜索文件不存在或搜索不到符合模式的文本行时将打印错误信息。下面给出一个使用-s 选项前后比较的例子。

```
#例 3-33: grep -s 的用法
alloy@alloy:~/linuxshell/CH02$ grep user certificate 00.pem        #未使用-s 选项
grep: certificate: 没有那个文件或目录                              #打印了错误信息
00.pem:The above string is known as your user certificate subject, and it
00.pem:uniquely identifies this user.
00.pem:To install this user certificate, please save this e-mail message
00.pem: /home/alloy/linuxshell/CH02/usercert.pem
alloy@alloy:~/linuxshell/CH02$ grep -s user certificate 00.pem
                                                       #使用-s 选项后，不打印错误信息
00.pem:The above string is known as your user certificate subject, and it
00.pem:uniquely identifies this user.
00.pem:To install this user certificate, please save this e-mail message
```

```
00.pem: /home/alloy/linuxshell/CH02/usercert.pem
alloy@alloy:~/linuxshell/CH02$
```

例 3-33 给出同样的命令使用-s 选项前后的结果，两条命令都是在 cerficate 和 00.pem 两个文件中搜索 user 字符串，当未使用-s 选项时，提示 certificate 文件不存在的错误信息，但是，在 grep 后加上-s 选项后，就不再打印错误信息了。

8. -r 选项

默认情况下，grep 命令只对当前目录下的文件进行搜索，而不对子目录中的文件进行搜索。-r 选项表示递归搜索，不仅搜索当前目录，而且搜索子目录。下面举一个-r 选项的例子。

```
#例 3-34: grep -r 的用法
alloy@alloy:~/linuxshell/CH02$ grep -r CERTIFICATE *
00.pem:-----BEGIN CERTIFICATE REQUEST-----
00.pem:-----END CERTIFICATE REQUEST-----
08.pem:-----BEGIN CERTIFICATE-----
08.pem:-----END CERTIFICATE-----
11.pem:-----BEGIN CERTIFICATE-----
11.pem:-----END CERTIFICATE-----
#以下是对子目录 BUILD 中文件的搜索结果
BUILD/globus_simple_ca_1c5c054a_setup-0.19/1c5c054a.0:-----BEGIN CERTIFICATE-----
BUILD/globus_simple_ca_1c5c054a_setup-0.19/1c5c054a.0:-----END CERTIFICATE-----
BUILD/66.pem:-----BEGIN CERTIFICATE REQUEST-----
BUILD/66.pem:-----END CERTIFICATE REQUEST-----
alloy@alloy:~/linuxshell/CH02$
```

例 3-34 对当前目录递归搜索 CERTIFICATE 字符串，不仅列出当前目录下文件的搜索结果，还列出了子目录 BUILD 下文件包含 CERTIFICATE 字符串的文本行。

9. -w 和-x 选项

grep 命令的模式是支持正则表达式的，正则表达式的元字符将被解释成特殊的含义，-w 选项表示匹配整词，即以模式的字面含义去解析它。因此，grep 命令使用-w 选项后，元字符不再被解释为特殊含义，下面的例子说明了-w 选项的功能。

```
#例 3-35: grep -w 的用法
alloy@alloy:~/linuxshell/CH02$ grep cer* 00.pem      #搜索包含以 cer 开头字符串的文本行
#有 4 行文本满足条件
The above string is known as your user certificate subject, and it
To install this user certificate, please save this e-mail message
/home/alloy/linuxshell/CH02/usercert.pem
If you have any questions about the certificate contact

#加上-w 选项后，表示搜索包含 cer*字符串的文本行
alloy@alloy:~/linuxshell/CH02$ grep -w cer* 00.pem
alloy@alloy:~/linuxshell/CH02$                       #没有满足该条件的文本行
```

上例中两条命令的模式都为 cer*，当未用-w 选项时，模式中的*被解析为任意字符，即搜索包含以 cer 开头字符串的文本行，00.pem 文件共有 4 行文本满足该条件；加上-w 选项后，*被解析为字面含义，表示搜索包含 cer*字符串的文本行，00.pem 文件不包含 cer*这一完整的字符串。因此，无任何结果输出。

-x 选项是匹配整行，即只有当文件中有整行内容与模式匹配时，grep 命令才输出该行结果，下面的例 3-36 说明 grep 命令的-w 和-x 选项的区别。

```
#例 3-36: 说明 grep 命令的-w 和-x 选项的区别
alloy@alloy:~/linuxshell/CH02$ cat world.txt          #第 1 条命令：查看 world.txt 内容
```


混混藏书阁 http://book-life.blog.163.com

```
Hello World
World
World Cup
African
One One World
#第 2 条命令：搜索包含单词'World'的文本行
alloy@alloy:~/linuxshell/CH02$ grep -w 'World' world.txt
Hello World                                    #所有包含单词'World'的文本行都被输出
World
World Cup
One One World
#第 3 条命令：搜索整行文本是单词'World'的行
alloy@alloy:~/linuxshell/CH02$ grep -x 'World' world.txt
World                                          #只有此行满足条件
alloy@alloy:~/linuxshell/CH02$
```

例 3-36 中，world.txt 有 5 行文本，第 2 条命令在 world.txt 文件中搜索包含单词'World'的文本行，结果列出 4 行满足条件的文本行。但是，第 3 条命令在 world.txt 文件中搜索整行文本是单词'World'的行时，只有一行满足条件。这可以看出，-w 选项搜索的是整词匹配，而 -x 选项搜索的是整行匹配。

10. -q 选项

从上面的讲解中知道，grep 命令默认情况下是输出结果的，但是，grep 命令后一旦加上 -q 选项，grep 将不再输出任何结果，而是以退出状态表示搜索是否成功，退出状态 0 表示搜索成功，退出状态 1 表示未搜索到满足模式的文本行，退出状态 2 表示命令或程序由于错误而未能执行。有关退出状态的内容将在第 7 章介绍，读者将 grep -q 选项与退出状态结合起来阅读，能更深入地理解-q 选项。下面举例说明 grep -q 选项的含义。

```
#例 3-37：演示 grep -q 选项
#第 1 条命令：grep 命令搜索成功
alloy@alloy:~/linuxshell/CH02$ grep -q -x 'World' world.txt
alloy@alloy:~/linuxshell/CH02$ echo $?
0                                              #退出状态是 0
#第 3 条命令：grep 命令未搜索到满足模式的文本行
alloy@alloy:~/linuxshell/CH02$ grep -q -x 'World African' world.txt
alloy@alloy:~/linuxshell/CH02$ echo $?
1                                              #退出状态是 1
#第 5 条命令：grep 命令执行失败
alloy@alloy:~/linuxshell/CH02$ grep -q -x 'World African' world
grep: world: 没有那个文件或目录
alloy@alloy:~/linuxshell/CH02$ echo $?
2                                              #退出状态是 2
alloy@alloy:~/linuxshell/CH02$
```

例 3-37 列举了三种场景，第 1 条命令 grep 在 world.txt 文件中搜索整行文本是单词'World'的行，由例 3-36 能找到满足模式的行，因而退出状态是 0（echo $?命令用于输出上条命令的退出状态，第 7 章将会讲述）；第 3 条命令 grep 在 world.txt 文件中搜索整行文本'World African'的行，由于此时 grep 命令未搜索到满足模式的文本行，因而退出状态是 1；第 5 条命令中的 world 文件不存在，grep 发生语法错误，因而退出状态是 2。

11. -b 和-o 选项

grep -b 选项打印匹配行距文件头部的偏移量，以字节为单位。如果在-b 选项后再加上-o

选项，grep 命令将打印匹配的词距文件头部的偏移量。下面的例 3-38 演示了 grep 的-b 和-o 选项。

```
#例 3-38：演示 grep 命令的-b 和-o 选项
#第 1 条命令：打印匹配行距文件头部的偏移量
alloy@alloy:~/linuxshell/CH02$ grep -b -w 'World' world.txt
0:Hello World
12:World
18:World Cup
36:One One World
#第 2 条命令：打印匹配词距文件头部的偏移量
alloy@alloy:~/linuxshell/CH02$ grep -b -o -w 'World' world.txt
6:World
12:World
18:World
44:World
alloy@alloy:~/linuxshell/CH02$
```

例 3-38 中的第 1 条命令打印匹配行距文件头部的偏移量，由于“Hello World”是 world.txt 第一行，因而该行距离文件头部的偏移量是 0 字节；然而，当第 2 条命令加上-o 选项打印匹配词距文件头部的偏移量时，第一行“Hello World”的词 World 距离文件头部是 6 字节。

grep 命令的-E 和-F 选项分别等价于 grep 命令族中的 egrep 和 fgrep 命令，我们将在 3.4.3 节介绍，有关 grep 命令选项、模式和文件的介绍到此为止，3.4.2 节将举例介绍 grep 和正则表达式的结合使用。

3.4.2 grep 和正则表达式结合使用的一组例子

将带元字符的正则表达式用于 grep 命令能够更灵活地匹配信息，使用时需要用单引号将正则表达式引起来，以免发生一些不可预知的错误。下面通过一组例子来介绍 grep 与正则表达式结合的用法，以加强读者对 grep 命令的认识，同时让读者有利于对正则表达式的巩固。

1. 匹配行首

元字符“^”表示行首，若需要匹配.pem 为后缀文件中以横杠（-）开头的行，可输入如下命令：

```
#例 3-39：grep 查找以-符号开头的行
alloy@alloy:~/linuxshell/CH02$ grep ^- *.pem
00.pem:-----BEGIN CERTIFICATE REQUEST-----
00.pem:-----END CERTIFICATE REQUEST-----
08.pem:-----BEGIN CERTIFICATE-----
08.pem:-----END CERTIFICATE-----
11.pem:-----BEGIN CERTIFICATE-----
11.pem:-----END CERTIFICATE-----
alloy@alloy:~/linuxshell/CH02$
```

下面的例 3-40 结合 grep 和正则表达式搜索空白行，第 1 个命令的意义为匹配 00.pem 文件中空白行的行数，显示结果说明 00.pem 文件中有 14 行空白行。第 2 个命令为匹配 00.pem 文件中非空白行的行数，此时使用[^$]符号表示空白行范围，前面加上“^”符号取反，显然，^^$表达式是错误的，因为 grep 将第 1 个“^”理解为行首，显示结果说明 00.pem 文件中有 27 行空白行。

```
#例 3-40：查找空白行
```


混混藏书阁 http://book-life.blog.163.com

```
#第 1 条命令：搜索 00.pem 中的空白行，只打印行数
alloy@alloy:~/linuxshell/CH02$ grep -c ^$ 00.pem
14
#第 2 条命令：搜索 00.pem 中的非空白行，只打印行数
alloy@alloy:~/linuxshell/CH02$ grep -c ^[^$] 00.pem
27
alloy@alloy:~/linuxshell/CH02$
```

2. 设置大小写

利用-i 符号可以使 grep 命令不区分大小写，当然也可利用[]符号来实现这一功能。下面给出用[]符号设置大小写的例子。

```
#例 3-41：用[]符号设置大小写
alloy@alloy:~/linuxshell/CH02$ grep -n [Cc]ertificate 00.pem
1:This is a Certificate Request file:
7:Certificate Subject:
11:The above string is known as your user certificate subject, and it
14:To install this user certificate, please save this e-mail message
26:If you have any questions about the certificate contact
alloy@alloy:~/linuxshell/CH02$
```

上例匹配 00.pem 文件中 certificate 和 Certificate 两个关键字的行，并显示匹配的行号。如果不区分大小写来查找 00.pem 中 certificate 关键字的行，则可以使用下面的两条等价命令。

```
grep "certificate" 00.pem
grep '[Cc][Ee][Rr][Tt][Ii][Ff][Ii][Cc][Aa][Tt][Ee]' 00.pem
```

3. 匹配重复字符

匹配重复字符通常可以利用“.”符号和“*”符号来实现，首先举一个“.”符号的例子。

```
#例 3-42：grep 和.符号
alloy@alloy:~/linuxshell/CH02$ grep ^/..../ 00.pem
/home/alloy/linuxshell/CH02/usercert.pem
alloy@alloy:~/linuxshell/CH02$
```

上例搜索 00.pem 文件中以/字符开始、中间 4 个任意字符、第 6 个字符仍为/的行，显示结果/home/满足匹配条件。

然后给出一个“*”符号的例子。

```
#例 3-43：grep 和*符号
alloy@alloy:~/linuxshell/CH02$ grep ^-*B 00.pem
-----BEGIN CERTIFICATE REQUEST-----
BAIwADANBgkqhkiG9w0BAQQFAAOBgQBoHRUaaB/Tyu+LuALwnT3Muw/0jDIYxc5a
alloy@alloy:~/linuxshell/CH02$
```

上例搜索以“-”开头，重复“-”符号任意次，然后是 B 字符的行，第 1 条结果-----B 表示“-”符号重复 5 次满足匹配条件，第 2 条结果 B 表示“-”符号重复 0 次，仍然符合“*”符号的语法，因此满足匹配条件。

4. 转义符

如果匹配的目标字符串中包含元字符，则需要利用转义符“\”屏蔽其意义。如果需要搜索包含 seu.edu.cn 字符串的行，由于点号“.”字符是元字符，所以需要在“.”符号之前加上“\”符号进行转义。如果将命令写成：

```
grep 'seu.edu.cn' 00.pem
```

则是匹配 seu 和 edu、edu 和 cn 之间存在任意单个字符的行，如 seuxeduxcn 能够满足条件。下面给出搜索包含 seu.edu.cn 字符串的行的例子。

```
#例 3-44: grep 和转义符
alloy@alloy:~/linuxshell/CH02$ grep seu\.edu\.cn 00.pem
It should be mailed to xd_ni@seu.edu.cn
    /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus
the Globus Simple CA at xd_ni@seu.edu.cn
alloy@alloy:~/linuxshell/CH02$
```

由上例的结果可以看出，转义符使得元字符“.”符号被解析为字面含义，打印出了包含seu.edu.cn 字符串的行。

横杠“-”字符较为特别，它虽然不属于正则表达式元字符，但是，由于“-”字符是引出 grep 命令选项的特殊字符，所以，当模式以“-”符号开头时，需要用转义符将其转义，请看下面的例 3-45。

```
#例 3-45: -字符在 grep 命令中的特殊性
[root@zawu globus]# grep -\{5\} 00.pem                    #模式以-符号开头
grep: 无效选项 -- {                                       #提示错误，grep 将模式解析为选项
Usage: grep [OPTION]... PATTERN [FILE]...
Try `grep --help' for more information.
[root@zawu globus]# grep '-\{5\}' 00.pem                  #将模式用引号引起也解决不了问题
grep: 无效选项 -- \
Usage: grep [OPTION]... PATTERN [FILE]...
Try `grep --help' for more information.
```

上例原本是要搜索“-”符号重复 5 次的文本行，模式当然以“-”符号开头，结果 grep 将模式解析为选项，Shell 提示无效选项错误。尽管我们将模式用引号引起来，但是仍然得到相同的错误。正确的用法应该如下：

```
alloy@alloy:~/linuxshell/CH02$ grep '\-\{5\}' 00.pem
-----BEGIN CERTIFICATE REQUEST-----
-----END CERTIFICATE REQUEST-----
alloy@alloy:~/linuxshell/CH02$
```

上面的命令在“-”符号前加上转义符，并用引号将模式引起来，得到了正确的结果。注意，这里模式上的引号十分重要，如果不加引号，仍然提示无效选项错误。

```
alloy@alloy:~/linuxshell/CH02$ grep \-\{5,\} 00.pem
grep: 无效选项 -- {
Usage: grep [OPTION]... PATTERN [FILE]...
Try `grep --help' for more information.
alloy@alloy:~/linuxshell/CH02$
```

本章上机提议第 8 题建议读者执行与此相关的几条有趣的命令，读者如果执行它们，并分析其中的原因，一定能对转义符、-符号和 grep 的执行过程有更深的理解。

5. POSIX 字符类

为了保持不同国家的字符编码的一致性，POSIX（Portable Operating System Interface）增加了特殊的字符类，以[:classname]的格式给出，grep 命令支持 POSIX 字符类，首先将 POSIX 类及其意义列于表 3-4 中。

表 3-4　POSIX 字符类

类　名	意　义
[:upper:]	表示大写字母[A～Z]
[:lower:]	表示小写字母[a～z]
[:digit:]	表示阿拉伯数字[0～9]

混混藏书阁 http://book-life.blog.163.com

续表

类　名	意　义
[:alnum:]	表示大小写字母和阿拉伯数字[0～9a～zA～Z]
[:space:]	表示空格或 Tab 键
[:alpha:]	表示大小写字母[a～zA～Z]
[:cntrl:]	表示 Ctrl 键
[:graph:]或[:print:]	表示 ASCII 码 33～126 之间的字符
[:xdigit:]	表示 16 进制数字[0～9A～Fa～f]

下面举例说明 POSIX 字符类的用法。首先，给出一个利用 POSIX 字符类搜索以大写字母开头的行。

```
#例 3-46：利用 POSIX 字符类搜索以大写字母开头的行
alloy@alloy:~/linuxshell/CH02$ grep ^[[:upper:]] 00.pem
This is a Certificate Request file:
It should be mailed to xd_ni@seu.edu.cn
Certificate Subject:
The above string is known as your user certificate subject, and it
To install this user certificate, please save this e-mail message
If you have any questions about the certificate contact
MIIB4zCCAUwCAQAwcTENMAsGA1UEChMER3JpZDETMBEGA1UECxMKR2xvYnVzVG
CxMKc2V1LmVkdS5jbjEPMA0GA1UEAxMGZ2xvYnVzMIGfMA0GCSqGSIb3DQEBAQUA
A4GNADCBiQKBgQCw50H88r7sAGjQGLZTmMxTiw9AgDqppBMhP6Fg3eJTqBvqBdha
YTRt1eSBT/AJUi3rTDRIABJPgu8cZkwb1AE8uEJSeCKwgk3J9QHK2NcZXwIDAQAB
BAIwADANBgkqhkiG9w0BAQQFAAOBgQBoHRUaaB/Tyu+LuALwnT3Muw/0jDIYxc5a
YaA4dWCB6/2yVYyfmyRCNox3rIsyUvqL9p81d/hpNiAB/0OazMBialq5Gcpaansd
alloy@alloy:~/linuxshell/CH02$
```

上例的命令使用 POSIX 字符类作为模式，[:upper:]表示大写字母集合，再用一层方括号将[:upper:]括起表示匹配字符集合，得到的结果确实是所有以大写字母开头的文本行。

再举一个搜索以空格开头文本行的例子。

```
#例 3-47：搜索以空格开头文本行
alloy@alloy:~/linuxshell/CH02$ ^[[:space:]] 00.pem
    /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus
     You need not edit this message in any way. Simply
     save this e-mail message to the file.
alloy@alloy:~/linuxshell/CH02$
```

上例的命令仍以 POSIX 字符类作为模式，[:space:]用法与[:upper:]类似。事实上，OSIX 字符类作为模式的用法都类似，使用时只要注意用方括号将 POSIX 字符类括起来即可。

6. 精确匹配

3.1 节讲述正则表达式时未曾讲述“\<\>”符号，该符号用于精确匹配，在此结合 grep 命令对此符号进行介绍。

首先给出一个说明性的例子，我们结合例子阐释何为精确匹配，以及\<\>符号的用法，例子如下。

```
#例 3-48：精确匹配
alloy@alloy:~/linuxshell/CH02$ cat re01                    #显示 re01 文件的内容
Line1:there are four lines in this file
```

```
Line2:this the line 2
Line3:this is another line
Line4:this is line4
alloy@alloy:~/linuxshell/CH02$ grep the re01                    #列出所有包含 the 字符串的行
Line1:there are four lines in this file                         #there 中包含了 the
Line2:this the line 2
Line3:this is another line                                      #another 中包含了 the
alloy@alloy:~/linuxshell/CH02$ grep "\<the\>" re01              #精确匹配 the 这个单词
Line2:this the line 2                                           #只要第 2 行有此单词
alloy@alloy:~/linuxshell/CH02$
```

上例创建名为 re01 的示例文件，该文件中有 4 行字符串，cat re01 命令列出了这 4 行字符串内容，关于 cat 命令的用法，可以参考 10.1.2 节，在此不介绍。未用“\<\>”符号时为模糊匹配，列出所有包含 the 字符串的行：Line 1、Line 2 和 Line 3，因为 Line 1 中包含 there，Line 2 中包含 the，Line 3 中包含 another。用“\<\>”符号精确匹配 the 这个单词，因而只列出了 Line 2，注意\<the\>上的引号必不可少。

事实上，grep 命令的-w 选项也可用于精确匹配，下面的命令等价于上例的 grep "\<the\>" re01 命令：

```
alloy@alloy:~/linuxshell/CH02$ grep -w the re01                 #利用-w 选项实现精确匹配
Line2:this the line 2                                           #结果仍是第 2 行
alloy@alloy:~/linuxshell/CH02$
```

7. 或字符

或字符“|”是扩展的正则表达式中定义的，grep 需要加上-E 选项才能支持它，下面给出 grep 命令使用“|”字符的例子。

```
#例 3-49：grep 命令与|字符
alloy@alloy:~/linuxshell/CH02$ grep -E "OU|seu" 00.pem    #带-E 选项的 grep 执行成功
It should be mailed to xd_ni@seu.edu.cn
    /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus
the Globus Simple CA at xd_ni@seu.edu.cn
alloy@alloy:~/linuxshell/CH02$ grep "OU|seu" 00.pem
alloy@alloy:~/linuxshell/CH02$
```

例 3-49 的两个命令用于匹配带有 OU 或 seu 字符串的行，grep 带上-E 选项后得到正确的结果。而 grep 没有带-E 选项时，返回结果为空，这是因为 grep 命令将“|”字符解析为字面意义。注意，OU|seu 两边的引号必不可少。

grep 命令与正则表达式结合使用极大地增加了命令应用的灵活性，同时也增大了使用命令的难度，读者只有在本节例子的基础上多上机练习，多思考分析，才有可能熟练而灵活地运用 grep 命令。

3.4.3 grep 命令族简介

Linux 系统支持三种形式的 grep 命令，通常将这三种形式的 grep 命令称为 grep 命令族，这三种形式具体如下。

- grep：标准 grep 命令，支持基本正则表达式，上面两小节已经对此命令进行了详细的讨论。
- egrep：扩展 grep 命令，支持基本和扩展正则表达式。
- fgrep：快速 grep 命令，不支持正则表达式，按照字符串的字面意思进行匹配。

混混藏书阁 http://book-life.blog.163.com

egrep 命令与 grep –E 等价，fgrep 命令与 grep –F 等价，在某些 Linux 发行版中，egrep 和 fgrep 都是 grep 命令的别名，分别将其符号链接到 grep –E 和 grep –F 命令。下面举两个例子来说明 egrep 和 fgrep 命令。首先，举一个 egrep 命令的例子。

```
#例 3-50：egrep 命令的用法
alloy@alloy:~/linuxshell/CH02$ egrep "seu.edu|certificate" 00.pem          #第 1 条命令
It should be mailed to xd_ni@seu.edu.cn
    /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus
The above string is known as your user certificate subject, and it
To install this user certificate, please save this e-mail message
If you have any questions about the certificate contact
the Globus Simple CA at xd_ni@seu.edu.cn
alloy@alloy:~/linuxshell/CH02$egrep "^-+B" 00.pem                          #第 2 条命令
-----BEGIN CERTIFICATE REQUEST-----
alloy@alloy:~/linuxshell/CH02$
```

上例第 1 条命令 egrep 使用了扩展的正则表达式元字符“|”符号，表示搜索包含 seu.edu 字符串或 certificate 字符串的文本行，egrep 成功地解析元字符“|”符号的含义，得到正确结果。第 2 条命令使用了“+”符号，意义为查找 00.pem 文件中以“-”符号开头、重复任意次，然后是 B 字符的行。请注意“^-+B”和“^-*B”的区别，“+”符号表示“-”符号至少重复一次，不匹配以 B 字符开头的行。

其次，举一个 fgrep 命令的例子。

```
#例 3-51：fgrep 命令的用法
alloy@alloy:~/linuxshell/CH02$ fgrep ^-*B 00.pem
#第 1 条命令，fgrep 命令不支持正则表达式
alloy@alloy:~/linuxshell/CH02$ fgrep certificate 00.pem
#第 2 条命令，fgrep 命令支持普通字符串
The above string is known as your user certificate subject, and it
To install this user certificate, please save this e-mail message
If you have any questions about the certificate contact
alloy@alloy:~/linuxshell/CH02$
```

上例的第 1 条命令是 fgrep 后加正则表达式，无任何返回结果，这说明 fgrep 不支持正则表达式，第 2 条命令 fgrep 后面加普通字符串，返回正确结果，这说明 fgrep 支持普通字符串。

egrep 和 fgrep 命令极少使用，因为 grep 命令的功能已十分强大，足可替代 egrep 和 fgrep 命令。因此，读者只需阅读本节对 egrep 和 fgrep 命令有所了解即可。

3.5 本章小结

本章介绍了 Shell 命令和工具所涉及的基本用法——正则表达式，重点介绍了基本正则表达式和扩展正则表达式中元字符的意义和用法。在此基础上，介绍了 Shell 的通配功能，结合例子逐个讨论了通配的元字符。本章还介绍了 Linux 系统中使用广泛的 grep 命令，着重讨论了 grep 命令的基本用法，及如何与正则表达式相结合以更加灵活地进行文本搜索。此外，还简单介绍了 grep 命令族中的其他两个命令 egrep 和 fgrep。

3.6 上机提议

1. 分析下面的正则表达式表达了什么含义。

```
(1)  kK*
(2)  k\{6,8\}
(3)  k\{6,\}
(4)  k\{10\}
(5)  ^NEW YEAR$
(6)  ^$
(7)  [0-9][0-9][a-z]
(8)  [A-H]\{1,3\},[0-9]\{5\}
(9)  ^\...
(10) [^p-z]*\.
```

2. 利用通配功能列出某目录下所有以数字开头、最后 3 位是句点和 2 个任意字母的文件名。

3. 利用通配功能列出 Windows 下文本格式文件，即以.doc、.txt、.ppt、.docx 和.pptx 等结尾的文件。

4. 在上一题的基础上，不区分大小写搜索 Windows 下文本格式文件包含 chapter 字符串的文本行内容及其行号，要求写出等价的两种形式的命令。

5. 下面的第（1）条命令是 3.4.1 节讲述 grep 命令的-r 选项时的示例命令，执行下面的第（2）条命令，观察该命令是否仍将对 BUILD 子目录进行搜索，分析其原因。

```
(1)  grep -r CERTIFICATE *
(2)  grep -r CERTIFICATE *.pem
```

6. 重做 3.4.2 节搜索空白行和非空白行的例子，并试验用^^$匹配非空白行这种错误用法，即执行下面的三条命令：

```
(1)  grep -c ^$ 00.pem
(2)  grep -c ^[^$] 00.pem
(3)  grep -c ^^$ 00.pem
```

7. 统计当前目录及其子目录下的所有文件所包含空白行的行数。再写一个命令统计当前目录及其子目录下的所有文件包含非空白行的行数。

8. 结合 3.4.2 节对“-”符号的阐述，执行下面几条命令，观察是否仍然提示无效选项错误，分析其中的原因：

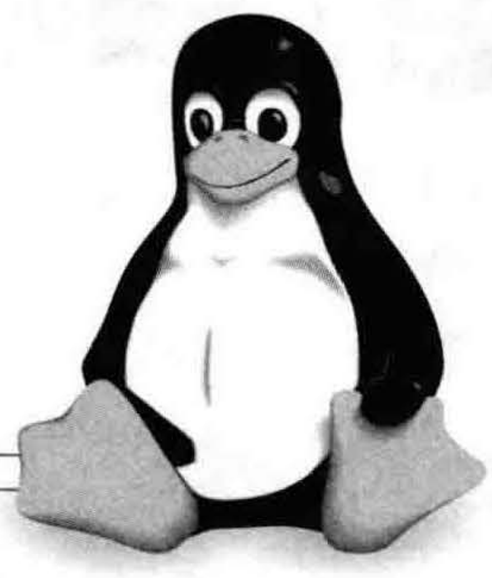
```
grep -n -\{5,\} 00.pem
grep -n '-\{5,\}' 00.pem
```

9. 对 3.4.2 节精确匹配示例中的 re01 文件，依次执行下面的 4 条命令，前 3 条命令是 3.4.2 节曾讲述过的，第 4 条命令是错误的命令，分析第 4 条命令得不到正确结果的原因。

```
(1)  grep the re01
(2)  grep "\<the\>" re01
(3)  grep -w the re01
(4)  grep \<the\> re01
```

10. 分别用 grep -E 和 egrep 两个命令，结合扩展的正则表达式实现在 00.pem 中查询以冒号（:）结尾，或以非英文字母结尾的文本行。

混混藏书阁 http://book-life.blog.163.com



第4章

sed 命令和 awk 编程

本章介绍 Linux/UNIX 系统中两种功能强大的文本处理工具：sed 和 awk。由于 sed 是流编辑器，因此才有了 sed 这个名字（stream editor），它是一个将一系列编辑命令作用于一批文本文件的理想工具。awk 因其三位缔造者的名字而命名（Aho、Weinberger 和 Kernighan），是一种能够对结构化数据进行操作，并产生格式化报表的编程语言。sed 和 awk 有很多共同之处，如使用正则表达式进行模式匹配等，而且掌握 grep 命令将有助于学习 sed 和 awk。因此，第 3 章的内容是本章学习的基础。

4.1 sed 命令基本用法

sed 是一个非交互式文本编辑器，它可对文本文件和标准输入进行编辑，标准输入可以是来自键盘输入、文件重定向、字符串、变量，甚至来自管道的文本。sed 从文本的一个文本行或标准输入中读取数据，将其复制到缓冲区，然后读取命令行或脚本的第一个命令，对此命令要求的行号进行编辑，重复此过程，直到命令行或脚本中的所有命令都执行完毕。相对于如 vim 等其他文本编辑器，sed 可以一次性处理所有的编辑任务，显得非常高效，为用户节省了大量的时间，sed 适用于以下三种场合。

- 编辑相对交互式文本编辑器而言太大的文件。
- 编辑命令太复杂，在交互式文本编辑器中难以输入的情况。
- 对文件扫描一遍，但是需要执行多个编辑函数的情况。

sed 只是对缓冲区中原始文件的副本进行编辑，并不编辑原始的文件。因此，如果需要保存改动的内容，需要将输出重定向到另一个文件，可以使用下面格式的命令。

```
sed 'sed命令' input-file > result-file
```

该命令将 sed 命令对 input-file 的更改保存到 result-file 中，“>”符号是重定向符号，关于重定向的内容，将在第 10 章详细介绍，读者在此只需记住 sed 保存更改的命令即可。sed 编辑命令中的 w 选项也可将结果保存到某个文件中，这一用法将在 4.2 节介绍。

调用 sed 有三种方式，一种为 Shell 命令行方式，另外两种是将 sed 命令写入脚本文件，然后执行该脚本文件。三种方式的命令格式归纳如下。

① 在 Shell 命令行输入命令调用 sed，格式为：

```
sed [选项] 'sed命令' 输入文件
```

注意，需要用单引号将 sed 命令引起来。

② 将 sed 命令插入脚本文件后，然后通过 sed 命令调用它，格式为：

```
sed [选项] -f sed脚本文件 输入文件
```

③ 将 sed 命令插入脚本文件后，最常用的方法是设置该脚本文件为可执行，然后直接执行该脚本文件，格式为：

```
./sed脚本文件 输入文件
```

第③种方式的 sed 脚本文件与第②种有所不同，其 sed 脚本文件需要以 sha-bang（#!）符号开头，sha-bang 后面是解析这个脚本的程序名。不管是哪一种调用方式，如果没有指定输入文件，sed 将从标准输入中接受输入。sed 的常用选项有三个，如表 4-1 所示。

表 4-1　sed 命令选项及其意义

选　项	意　义
-n	不打印所有的行到标准输出
-e	表示将下一个字符串解析为 sed 编辑命令，如果只传递一个编辑命令给 sed，-e 选项可以省略
-f	表示正在调用 sed 脚本文件

sed 命令通常由定位文本行和 sed 编辑命令两部分组成，sed 编辑命令对定位文本行进行

混混藏书阁 http://book-life.blog.163.com

各种处理，sed 提供以下两种方式定位文本。

- 使用行号，指定一行，或指定行号范围。
- 使用正则表达式。

表 4-2 给出了 sed 定位文本的方法。

表 4-2　sed 命令定位文本的方法

选　项	意　义
x	x 为指定行号
x,y	指定从 x 到 y 的行号范围
/pattern/	查询包含模式的行
/pattern/pattern/	查询包含两个模式的行
/pattern/,x	从与 pattern 的匹配行到 x 号行之间的行
x,/pattern/	从 x 号行到与 pattern 的匹配行之间的行
x,y!	查询不包括 x 和 y 行号的行

sed 提供了丰富的编辑命令，用于标识对文本进行何种处理，如打印、删除、追加、插入、替换等。本章详细介绍常用的 sed 编辑命令，读者掌握了这些常用的编辑命令就能够较好地利用 sed 进行文本处理。对于使用频率较低、较生僻的编辑命令，本章用一节专门做介绍，供有兴趣的读者参考。表 4-3 列出了本章所述的所有的 sed 编辑命令。

表 4-3　sed 编辑命令

选　项	意　义
p	打印匹配行
=	打印文件行号
a\	在定位行号之后追加文本信息
i\	在定位行号之前插入文本信息
d	删除定位行
c\	用新文本替换定位文本
s	使用替换模式替换相应模式
r	从另一个文件中读文本
w	将文本写入到一个文件
y	变换字符
q	第一个模式匹配完成后退出
l	显示与八进制 ASCII 码等价的控制字符
{}	在定位行执行的命令组
n	读取下一个输入行，用下一个命令处理新的行
h	将模式缓冲区的文本复制到保持缓冲区
H	将模式缓冲区的文本追加到保持缓冲区
x	互换模式缓冲区和保持缓冲区内容
g	将保持缓冲区的内容复制到模式缓冲区
G	将保持缓冲区的内容追加到模式缓冲区

4.2 sed 命令的一组示例

本节通过一组例子来说明 sed 选项、sed 文本定位、sed 基本编辑命令的常见用法，另外，sed 编程还经常与正则表达式相结合。因此，第 3 章是本节 sed 编程的基础知识，有必要随时参考。

新建一个名为 input 的文件，作为例子的输入文件，内容如下：

```
    This is a Certificate Request file:

    It should be mailed to zawu@seu.edu.cn

    ================================================================
    Certificate Subject:

        /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

    The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
    To install this user certificate, please save this e-mail message into the following
file.

    /home/alloy/linuxshell/CH02/usercert.pem
```

4.2.1 sed 命令选项的一组例子

本节通过一组例子说明三个 sed 命令选项的意义，选项需要与一些编辑命令结合才能体现出它的作用。因此，本节的例子也会涉及几个编辑命令的介绍，但是重点不在于此。

1. sed 命令的-n 选项

sed 编辑命令 p 实现打印匹配行功能。下面的例 4-1 演示了 sed 命令的-n 选项，第 1 条命令表示打印 input 文件的第 1 行，-n 表示不打印 input 文件的所有行，因此，该命令的运行结果只是 input 文件的第 1 行。第 2 条命令去掉-n 参数，可以看到结果首先显示 input 文件的第 1 行，然后将 input 文件（sed 的编辑对象）的全部内容打印到标准输出。从这个例子的两条命令，我们可以清晰地理解-n 选项表示“不打印”功能是指：不打印 sed 编辑对象的全部内容。

```
    #例 4-1: sed -n 的用法
    #第 1 条命令，带-n 选项，只打印第 1 行

    alloy@alloy:~/linuxshell/CH04$ sed -n '1p' input
    This is a Certificate Request file:
    #第 2 条命令，不带-n 选项，不仅打印第 1 行，还打印输入文件的全部内容
    alloy@alloy:~/linuxshell/CH04$ sed '1p' input
    This is a Certificate Request file:
    This is a Certificate Request file:

    It should be mailed to zawu@seu.edu.cn

    ================================================================
```


混混藏书阁 http://book-life.blog.163.com

```
    Certificate Subject:

        /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

    The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
    To install this user certificate, please save this e-mail message into the following
file.

    /home/alloy/linuxshell/CH02/usercert.pem

    alloy@alloy:~/linuxshell/CH04$
```

下面的例 4-2 利用 sed 命令打印范围行，例 4-2 中的命令打印 input 文件的第 3～6 行，其中第 4 行为空白行。

```
    #例 4-2：sed 命令打印范围行
    alloy@alloy:~/linuxshell/CH04$ sed -n '3,6p' input
    It should be mailed to zawu@seu.edu.cn

    ============================================================
    Certificate Subject:
    alloy@alloy:~/linuxshell/CH04$
```

下面的例 4-3 利用 sed 命令打印匹配模式行，命令使用表 4-2 中的/pattern/方法进行模式匹配，表示打印匹配 certificate 关键字的行。从例 4-3 的结果可以看出，第 1 行包含 Certificate 关键字未被打印，这说明 sed 匹配关键字也是大小写敏感的。

```
    #例 4-3：sed 打印匹配模式行
    alloy@alloy:~/linuxshell/CH04$ sed -n '/certificate/p' input
    The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
    To install this user certificate, please save this e-mail message into the following
file.
    alloy@alloy:~/linuxshell/CH04$
```

2. sed 命令的-e 选项

-e 选项表示将下一个字符串解析为 sed 编辑命令，如果只传递一个编辑命令给 sed，-e 选项可以省略，换句话说，只有向 sed 命令传递多个编辑命令时，-e 选项才有用武之地，我们通过一个打印行号的例子来说明-e 选项的意义和用法，如例 4-4 所示。例 4-4 中的第 1 条命令利用 sed 编辑命令“=”号打印匹配 Certificate 关键字的行号，结果为 1 和 6。如果需要将与匹配 Certificate 关键字行的内容和行号都打印出来，就要向 sed 传递“p”和“=”两个编辑命令，此时就需要使用-e 选项，-e 选项指定其后面紧跟着的字符串为 sed 编辑命令，打印匹配行内容及其行号的命令如例 4-4 第 2 条命令所示。

```
    #例 4-4：sed -e 的用法
    #第 1 条命令：简单的打印行号命令
    alloy@alloy:~/linuxshell/CH04$ sed -n '/Certificate/=' input
    1
    6
    #第 2 条命令：打印行内容及行号，传递两个编辑命令给 sed
    alloy@alloy:~/linuxshell/CH04$ sed -n -e '/Certificate/p' -e '/Certificate/=' input
    This is a Certificate Request file:
    1
    Certificate Subject:
```

```
6
alloy@alloy:~/linuxshell/CH04$
```

需要注意的是，sed 不支持同时带多个编辑命令的用法，如：

```
sed -n '/Certificate/p=' input
```

带多个编辑命令 sed 的一般格式为：

```
sed [选项] -e 编辑命令1 -e 编辑命令2 … -e 编辑命令n 输入文件
```

3. sed 命令的-f 选项

-f 选项只有调用 sed 脚本文件时才起作用，追加文本、插入文本、修改文本、删除文本和替换文本等功能往往需要几条 sed 命令来完成，所以往往将这些命令写入 sed 脚本，然后调用 sed 脚本来完成。我们以 sed 追加文本功能为例说明 sed 脚本的编写和调用方法，从中说明-f 选项的作用。

sed 编辑命令 a\符号用于追加文本，它可以将指定文本的一行或多行追加到指定行后面。如果不指定文本追加位置，sed 默认放置到每一行后面，追加文本的格式为：

```
sed "指定地址 a\text" 输入文件
```

指定地址以匹配模式/pattern/或行号的形式给出，用于定位新文本的追加位置，sed 对\a 后的文本进行追加操作。

例 4-5 利用 sed 命令追加文本，例 4-5 中的命令表示在与关键字“file:”相匹配的行后追加文本内容“We append a new line.”，结果显示第 1 行之后已经追加了上述文本。值得注意的是，sed 完成追加文本功能后，只是将结果输出到标准输出上（Shell），原始文件 input 并未做任何改变。

```
#例 4-5: sed追加文本
alloy@alloy:~/linuxshell/CH04$ sed '/file:/a\We append a new line.' input
This is a Certificate Request file:
We append a new line.

It should be mailed to zawu@seu.edu.cn

==========================================================
Certificate Subject:

    /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
To install this user certificate, please save this e-mail message into the following
file.

/home/alloy/linuxshell/CH02/usercert.pem

alloy@alloy:~/linuxshell/CH04$
```

接下来，我们通过将例 4-5“sed 追加文本”的例子转换为 sed 脚本来说明如何编写和调用 sed 脚本文件。利用 vi 编辑器创建名为 append.sed 的文件，见例 4-6 输入如下的内容：

```
#例 4-6: append.sed演示sed追加文本的用法
#!/bin/sed -f
/file:/a\                              #a\表示此处换行添加文本

#所添加的文本内容
```


混混藏书阁 http://book-life.blog.163.com

```
We append a new line.\          #“\”符号表示换行
We append another line.
```

sed 脚本的第 1 行与 bash Shell 脚本一样，以 sha-bang（#!）符号开头，sha-bang 后面解释器的路径，sed 脚本是 sed 命令的路径，一般在/bin 目录下，当然，如果读者不知道 sed 在哪个目录下，可以用下面的命令获得：

```
alloy@alloy:~/linuxshell/CH04$ which sed                    #获取 sed 路径
/bin/sed
alloy@alloy:~/linuxshell/CH04$
```

sed 选项使用-f 表示正在调用脚本文件，-f 选项在脚本中必不可少，若无此选项，执行脚本时将报错。如果追加上去的文本有多行，需要用反斜杠符号“\”换行，如 append.sed 脚本所示。将 append.sed 文件赋予可执行权限，然后执行该脚本文件即可得到结果，sed 脚本的执行方法与 bash 脚本是一样的。

在执行脚本时，仍然需要加上输入文件的名称，下面给出 append.sed 脚本的执行命令及其执行结果，由例 4-6 中的结果可见，附加的文本分成了两行，这就是“\”符号所起的作用。

```
#例 4-6 append.sed 脚本的执行结果
alloy@alloy:~/linuxshell/CH04$ chmod u+x append.sed
#为 append.sed 赋可执行权限
alloy@alloy:~/linuxshell/CH04$ ./append.sed input
 #执行 append.sed 脚本，带上输入文件 input
This is a Certificate Request file:
We append a new line.                                       #追加的两行文本
We append another line.

    It should be mailed to zawu@seu.edu.cn

    ==========================================================
    Certificate Subject:

        /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

    The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
    To install this user certificate, please save this e-mail message into the following
file.

    /home/alloy/linuxshell/CH02/usercert.pem
    alloy@alloy:~/linuxshell/CH04$
```

4.2.2 sed 文本定位的一组例子

本节通过一组例子说明 sed 文本定位的方法，由于 sed 命令选项和 sed 编辑命令的例子中都会涉及 sed 文本定位，因此，本节只是介绍几个特殊的 sed 文本定位例子。

1. 匹配元字符

如果 sed 命令所要匹配的目标字符串中包含元字符，需要使用转义符“\”屏蔽其特殊意义，下面的例 4-7 利用 sed 命令匹配元字符，例 4-7 分别给出匹配句点“.”元字符和“$”元字符的命令。

```
#例 4-7：sed 匹配元字符
alloy@alloy:~/linuxshell/CH04$ sed -n '/\./p' input       #匹配.符号
```

```
    It should be mailed to zawu@seu.edu.cn
        /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus
    The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
    To install this user certificate, please save this e-mail message into the following
file.
    /home/alloy/linuxshell/CH02/usercert.pem
    alloy@alloy:~/linuxshell/CH04$
```

2. 使用元字符进行匹配

sed 命令可以灵活使用正则表达式的元字符进行匹配，$在正则表达式中表示行尾，但是在 sed 命令中却表示最后一行，例 4-8 利用 sed 命令打印最后一行，在此附带说明一下 p 参数的位置，sed 基本编辑命令可以放在单引号内，也可放在单引号外，如例 4-8 的两条命令是等价的，本书统一将 sed 编辑命令放在单引号内。

```
#例 4-8：sed 结合$符号匹配最后一行

alloy@alloy:~/linuxshell/CH04$ sed -n '$p' input
/home/alloy/linuxshell/CH02/usercert.pem
alloy@alloy:~/linuxshell/CH04$ sed -n '$'p input
/home/alloy/linuxshell/CH02/usercert.pem
alloy@alloy:~/linuxshell/CH04$
```

再举一个使用元字符进行任意字符匹配的例子，如例 4-9 所示，/.*bus/表示匹配包含以 bus 结尾字符串的行。

```
#例 4-9：用.和*符号匹配任意字符
alloy@alloy:~/linuxshell/CH04$ sed -n '/.*bus/p' input
    /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus
alloy@alloy:~/linuxshell/CH04$
```

3. !符号

!符号表示取反，x,y!表示匹配不在 x 和 y 行号范围内的行，例 4-10 利用 sed 命令用于打印不在 2～10 之间的行。

```
#例 4-10：匹配不在指定行号范围内的行
alloy@alloy:~/linuxshell/CH04$ sed -n '2,10!p' input
This is a Certificate Request file:
To install this user certificate, please save this e-mail message into the following
file.

/home/alloy/linuxshell/CH02/usercert.pem
alloy@alloy:~/linuxshell/CH04$
```

x!表示匹配除了 x 行号外的所有行，但是，!符号不能用于关键字匹配，如无法表示不与/pattern/匹配的行。

4. 使用行号与关键字匹配限定行范围

表 4-2 列出了/pattern/,x 和 x,/pattern/两种形式限定行号与关键字匹配行之间的范围，实际上，这两种形式与 x、y 是一样的，只是将 x 或 y 用/pattern/代替而已。

下面的例 4-11 使用行号与关键字匹配限定行范围，例 4-11 的第 1 条命令是打印与 seugrid 的匹配行到最后一行，第 2 条命令是打印第 3 行到与 Certificate 的匹配行。

```
#例 4-11：sed 命令使用行号与关键字匹配限定行范围
#第 1 条命令：打印与 seugrid 的匹配行到最后一行
alloy@alloy:~/linuxshell/CH04$ sed -n '/seugrid/,$p' input
```


混混藏书阁 http://book-life.blog.163.com

```
        /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

    The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
    To install this user certificate, please save this e-mail message into the following
file.

    /home/alloy/linuxshell/CH02/usercert.pem
    #第 2 条命令：打印第 3 行到与 Certificate 匹配的行
    alloy@alloy:~/linuxshell/CH04$ sed -n '3,/seugrid/p' input
    It should be mailed to zawu@seu.edu.cn

    ==========================================================
    Certificate Subject:

        /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus
    alloy@alloy:~/linuxshell/CH04$
```

4.2.3 sed 基本编辑命令的一组例子

本节通过一组例子讲述 sed 的基本编辑命令，这些基本编辑命令是 sed 编程中最常见的命令，读者只有深刻理解了这些命令的用法，才能自如地利用 sed 进行 Shell 编程。我们在 4.2.1 节中已经介绍了追加文本命令，因此，本节从插入文本开始介绍 sed 基本编辑命令。

1. 插入文本

插入文本和追加文本类似，区别仅在于追加文本是在匹配行的后面插入，而插入文本是在匹配行的前面插入，sed 编辑命令的插入文本符号为 i\，插入文本的格式为：

```
sed '指定地址 i\text' 输入文件
```

接下来，举例看一看插入文本的用法，新建名为 insert.sed 的脚本，内容如下：

```
#例 4-12：insert.sed 脚本演示 sed 插入文本的用法
#!/bin/sed -f
/file:/i\                                    #i\表示此处换行插入文本
We insert a new line.                        #插入的文本内容
```

执行 append.sed 脚本，并加上 input 文件名作为参数，下面的例 4-12 给出了 append.sed 脚本的执行结果，可以看到，在与 file:关键字的匹配行上方插入了新文本。

```
#例 4-12 append.sed 脚本的执行结果
alloy@alloy:~/linuxshell/CH04$ chmod u+x insert.sed
alloy@alloy:~/linuxshell/CH04$ ./insert.sed input
We insert a new line.                        #插入的新文本行
This is a Certificate Request file:

It should be mailed to zawu@seu.edu.cn

==========================================================
Certificate Subject:

    /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
To install this user certificate, please save this e-mail message into the following
file.
```

```
/home/alloy/linuxshell/CH02/usercert.pem
alloy@alloy:~/linuxshell/CH04$
```

2. 修改文本

修改文本是指将所匹配的文本行利用新文本替代，sed 编辑命令的修改文本符号为 c\，修改文本的格式为：

```
sed '指定地址 c\text' 输入文件
```

接下来，举例子看一看修改文本的用法，新建名为 modify.sed 的脚本，内容如下：

```
#例 4-13: modify.sed 脚本演示修改文本的用法
#!/bin/sed -f

/file:/c\                                    #c\表示此处换行修改文本
We modify this line.                         #修改的文本内容
```

下面的例 4-13 给出了 modify.sed 脚本的执行结果，可以看到，与 file:关键字的匹配行被修改为了“We modify this line.”。

```
#例 4-13 modify.sed 脚本的执行结果
alloy@alloy:~/linuxshell/CH04$ chmod u+x modify.sed
alloy@alloy:~/linuxshell/CH04$ ./modify.sed input
We modify this line.                         #整个匹配行被修改为新文本行

It should be mailed to zawu@seu.edu.cn

================================================================
Certificate Subject:

    /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

    The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
    To install this user certificate, please save this e-mail message into the following
file.

    /home/alloy/linuxshell/CH02/usercert.pem
    alloy@alloy:~/linuxshell/CH04$
```

3. 删除文本

sed 删除文本命令可以将指定行或指定行的范围进行删除，sed 编辑命令的删除文本符号为 d，删除文本的格式为：

```
指定地址 d
```

注意，sed 编辑命令中的删除操作符号是 d，后面不带“\”符号，与附加、插入、修改等命令有所区别。删除文本可以灵活地制定删除地址，例 4-14 演示删除文件的第一行和最后一行的方式。

```
#例 4-14: sed 命令删除第一行和最后一行
alloy@alloy:~/linuxshell/CH04$# sed '1d' input
#第 1 条命令: 删除第一行文本

It should be mailed to zawu@seu.edu.cn

================================================================
Certificate Subject:
```


混混藏书阁 http://book-life.blog.163.com

```
        /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

    The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
    To install this user certificate, please save this e-mail message into the following
file.

    /home/alloy/linuxshell/CH02/usercert.pem
    alloy@alloy:~/linuxshell/CH04$ sed '$d' input
    #第 2 条命令：删除最后一行文本
    This is a Certificate Request file:

    It should be mailed to zawu@seu.edu.cn

    ================================================================
    Certificate Subject:

        /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

    The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
    To install this user certificate, please save this e-mail message into the following
file.
    alloy@alloy:~/linuxshell/CH04$
```

例 4-14 中，第 1 条命令表示删除第 1 行，用数字'1d'指定第一行；而第 2 条命令指定最后一行时，到了正则表达式元字符$表示最后 1 行。

我们也可以指定删除行的范围，下面的例 4-15 用 sed 命令删除指定范围内所有的行。

```
    #例 4-15：删除指定范围内所有的行
    alloy@alloy:~/linuxshell/CH04$ sed '1,10d' input
    #第 1 条命令：删除第 1 行到第 10 行
    To install this user certificate, please save this e-mail message into the following
file.

    /home/alloy/linuxshell/CH02/usercert.pem
    alloy@alloy:~/linuxshell/CH04$ sed '5,$d' input
    #第 2 条命令：删除第 5 行到最后一行
    This is a Certificate Request file:

    It should be mailed to zawu@seu.edu.cn

    alloy@alloy:~/linuxshell/CH04$
```

例 4-15 中的第 1 条命令用'1,10d'表示删除第 1～10 行，第 2 条命令用'5,$d'表示删除第 2 行到最后一行。

当然，也可以删除与关键字匹配的行，新建 delete.sed 脚本，内容如下：

```
#例 4-16：delete.sed 脚本演示删除不区分大小写与 certificate 匹配的行的用法
#!/bin/sed -f
/[Cc][Ee][Rr][Tt][Ii][Ff][Ii][Cc][Aa][Tt][Ee]/d
```

delete.sed 脚本表示删除匹配 certificate 关键字的所有行，用正则表达式[]符号表示在匹配 certificate 关键字时不区分大小写，下面的例 4-16 给出 delete.sed 脚本的执行结果。

```
#例 4-16 delete.sed 脚本的执行结果
```

```
alloy@alloy:~/linuxshell/CH04$ chmod u+x delete.sed
alloy@alloy:~/linuxshell/CH04$ ./delete.sed input

It should be mailed to zawu@seu.edu.cn

================================================================

    /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus


/home/alloy/linuxshell/CH02/usercert.pem
alloy@alloy:~/linuxshell/CH04$
```

4. 替换文本

sed 替换文本操作将所匹配的文本行利用新文本替换，替换文本与修改文本功能有相似之处，它们之间的区别在于：替换文本可以替换一个字符串，而修改文本是对整行进行修改。另外，替换文本通过替换选项使得文本替换更为灵活，功能更为强大，sed 编辑命令的替换文本符号为 s，替换文本的格式为：

```
s/被替换的字符串/新字符串/[替换选项]
```

s 表示 sed 执行替换文本操作，sed 命令首先匹配被替换的字符串，匹配成功后用新字符串替换它。替换选项对 sed 替换操作做进一步的细化，具体符号和含义如表 4-4 所示。

表 4-4　sed 替换选项及其意义

选　项	意　义
g	表示替换文本中所有出现被替换字符串之处
p	与-n 选项结合，只打印替换行
w 文件名	表示将输出定向到一个文件

sed 替换文本命令是一个相对复杂的命令，读者需要正确理解 sed 替换选项所表示的意义，下面结合具体例子详细解释表 4-4 所示的 sed 替换选项。

从最简单的 p 选项谈起，默认情况下，sed s 命令将替换后的全部文本都输出，如果要求只打印替换行，需要结合使用-n 和 p 选项，命令格式如下：

```
sed -n 's/被替换的字符串/新字符串/p' 输入文件
```

上述命令如果缺少 p 选项，将不打印任何内容。p 选项的官方文档解释是：使-n 选项无效。为清晰起见，本书将 p 选项解释为：与-n 选项结合，只打印替换行。例 4-17 说明了 sed 命令替换文本时 p 选项的用法，例中的三条命令都是将 Certificate 替换成 CERTIFICATE，第 1 条命令不加任何参数，结果为显示替换后文本的所有内容，第 2 条命令将-n 和 p 结合使用，结果仅为替换行，第 3 条命令用了-n 参数，但是未加 p 选项，结果中不打印任何内容。

```
#例 4-17：sed 命令替换文本时 p 选项的用法
#第 1 条命令：默认情况下，打印全部输入文件内容
alloy@alloy:~/linuxshell/CH04$ sed 's/Certificate/CERTIFICATE/' input
This is a CERTIFICATE Request file:

It should be mailed to zawu@seu.edu.cn

================================================================
CERTIFICATE Subject:
```


混混藏书阁 http://book-life.blog.163.com

```
    /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

    The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
    To install this user certificate, please save this e-mail message into the following
file.

    /home/alloy/linuxshell/CH02/usercert.pem
    #第 2 条命令：-n 和-p 选项结合使用，只打印替换行
    alloy@alloy:~/linuxshell/CH04$ sed -n 's/Certificate/CERTIFICATE/p' input
    This is a CERTIFICATE Request file:
    CERTIFICATE Subject:
    #第 3 条命令：少了 p 选项，不打印任何内容
    alloy@alloy:~/linuxshell/CH04$ sed -n 's/Certificate/CERTIFICATE/' input
    alloy@alloy:~/linuxshell/CH04$
```

再回到例 4-17，第 1 条命令将 input 中仅有的两处 Certificate 都替换成 CERTIFICATE，而 sed 并没有加上 g 选项，但是，g 选项的解释是：替换文本中所有出现被替换字符串之处。为了透彻地说明 g 选项的意义，请看下面的例 4-18。

```
    #例 4-18：sed 替换选项 g 的用法
    #第 1 条命令：不带 g 选项的结果
    alloy@alloy:~/linuxshell/CH04$ sed -n 's/seu/njue/p' input
    It should be mailed to zawu@njue.edu.cn
        /O=Grid/OU=GlobusTest/OU=simpleCA-njuegrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus
    #第 2 条命令：带 g 选项的结果
    alloy@alloy:~/linuxshell/CH04$ sed -n 's/seu/njue/pg' input
    It should be mailed to zawu@njue.edu.cn
        /O=Grid/OU=GlobusTest/OU=simpleCA-njuegrid1.njue.edu.cn/OU=njue.edu.cn/CN=globus
    alloy@alloy:~/linuxshell/CH04$
```

例 4-18 中的两条命令都是将 input 文件中的 seu 字符串替换为 njue 字符串，seu 在第 3 行出现 1 次、在第 8 行出现 2 次，例 4-18 中的第 1 条命令不带 g 选项，它替换了第 3 行出现的 seu 和第 8 行出现的第 1 个 seu；第 2 条命令带有 g 选项，它对 input 中出现的 3 个 seu 都做了替换。因此，sed 替换命令在默认情况下，即不带 g 选项时，对某行的第 1 处匹配关键字进行替换后，就跳转到下面匹配行。而 g 选项使得 sed 替换命令对某行的所有关键字都进行替换。换句话说，当被替换字符串在文本中每行至多出现 1 次时，是否带有 g 选项的结果是一样的；只有当被替换字符串在某行出现 2 次以上时，g 选项才发挥作用。

sed 替换文本命令还可指定替换第几次匹配的关键字，只需在替换选项加上相应的数字即可，数字范围需要在 1～512 之间。例 4-19 演示 sed 指定替换第几次匹配，例 4-19 中的两条命令分别是替换第 2 次和第 3 次所匹配的 seu 关键字。

```
    #例 4-19：sed 替换第 n 次匹配
    alloy@alloy:~/linuxshell/CH04$ sed -n 's/seu/njue/2p' input
        /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.njue.edu.cn/OU=seu.edu.cn/CN=globus
    alloy@alloy:~/linuxshell/CH04$ sed -n 's/seu/njue/3p' input
        /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=njue.edu.cn/CN=globus
    alloy@alloy:~/linuxshell/CH04$
```

w 选项较为简单，w 选项后加文件名表示将输出定向到这个文件，如果输出文件未曾建立，sed 命令自动建立输出文件，默认目录是当前工作目录。例 4-20 演示了 w 选项的用法，

第 1 条命令将 seu 字符串改为 njue，并将结果写入 output 文件，第 2 条命令显示 output 文件内容，里面包含替换过的两行文本。

```
#例 4-20：sed 命令 w 选项的用法
#第 1 条命令：将 seu 字符串改为 njue，并将结果写入 output 文件
alloy@alloy:~/linuxshell/CH04$ sed -n 's/seu/njue/w output' input
#第 2 条命令：查看 output 文件内容
alloy@alloy:~/linuxshell/CH04$ cat output
It should be mailed to zawu@njue.edu.cn
    /O=Grid/OU=GlobusTest/OU=simpleCA-njuegrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus
alloy@alloy:~/linuxshell/CH04$
```

介绍完 sed 的三个替换选项后，我们讲 sed 替换文本中经常使用到的&符号，&符号可用来保存被替换的字符串以供调用，下面用一个具体例子来讲述&符号的用法，如果我们需将 seu 用圆括号括起来，可用以下两条等价的命令：

```
sed -n 's/seu/(&)/pg' input
sed -n 's/seu/(seu)/pg' input
```

以上两条命令都是用(seu)字符串替换 seu，第 1 条命令中的(&)就代表了(seu)，因为&符号保存了被替换字符串 seu 的值；第 2 条命令显得比较直观，两条命令得到同样的执行结果。下面的例 4-21 演示上述两条命令的执行结果：

```
#例 4-21：ed 替换文本中&符号的用法
alloy@alloy:~/linuxshell/CH04$ sed -n 's/seu/(&)/pg' input          #&表示了 seu
It should be mailed to zawu@(seu).edu.cn

/O=Grid/OU=GlobusTest/OU=simpleCA-(seu)grid1.(seu).edu.cn/OU=(seu).edu.cn/CN=globus
    alloy@alloy:~/linuxshell/CH04$ sed -n 's/seu/(seu)/pg' input
    It should be mailed to zawu@(seu).edu.cn

/O=Grid/OU=GlobusTest/OU=simpleCA-(seu)grid1.(seu).edu.cn/OU=(seu).edu.cn/CN=globus
    alloy@alloy:~/linuxshell/CH04$
```

5. 写入一个新文件

4.1 节中就提到 sed 命令只是对缓冲区中输入文件的复制内容进行编辑，如果要保存编辑结果，需要将编辑后的文本重定向到另一个文件，sed 写入文件的符号为 w，基本格式为：

```
指定地址 w 文件名
```

w 的用法与 sed 替换文本中 w 选项相似。下面举两个简单的例子加以说明，例 4-22 给出将 1～5 行写入 output 文件的例子，命令通过指定行号范围来实现。

```
#例 4-22：将 1～5 行写入 output 文件
alloy@alloy:~/linuxshell/CH04$ sed -n '1,5 w output' input
alloy@alloy:~/linuxshell/CH04$ cat output
This is a Certificate Request file:

It should be mailed to zawu@seu.edu.cn

==========================================================================
alloy@alloy:~/linuxshell/CH04$
```

例 4-23 将与 globus 关键字相匹配的行写入 output 文件。

```
#例 4-23：将匹配关键字的行写入文件
alloy@alloy:~/linuxshell/CH04$ sed -n '/globus/w output' input
alloy@alloy:~/linuxshell/CH04$ cat output
    /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus
```


混混藏书阁 http://book-life.blog.163.com

```
/home/alloy/linuxshell/CH02/usercert.pem
alloy@alloy:~/linuxshell/CH04$
```

6. 从文件中读入文本

sed 命令还可将其他文件中的文本读入，并附加在指定地址之后，sed 读入文件的符号为 r，基本格式为：

```
指定地址 r 文件名
```

r 通知 sed 从另一个文件源中读入文本，下面举一个例子来说明 r 的用法。首先，新建一个名字为 otherfile 的文件，内容如下：

```
This is the first line of the otherfile.
This is the second line of the otherfile.
```

例 4-24 利用 sed 的 r 选项读取文本，它将在匹配 Certificate 关键字的行后面加上 otherfile 中的内容，结果显示在两处匹配行后都加上了 otherfile 的两行内容。

```
#例 4-24：sed 利用 r 选项读文件
alloy@alloy:~/linuxshell/CH04$ cat otherfile                    #查看 otherfile 文件的内容
This is the first line of the otherfile.
This is the second line of the otherfile.
#在与 Certificate 匹配的行后读入 otherfile 文件
alloy@alloy:~/linuxshell/CH04$ sed '/Certificate/r otherfile' input
This is a Certificate Request file:
This is the first line of the otherfile.                        #读到的 otherfile 文件内容
This is the second line of the otherfile.

It should be mailed to zawu@seu.edu.cn

==========================================================
Certificate Subject:
This is the first line of the otherfile.                        #读到的 otherfile 文件内容
This is the second line of the otherfile.

   /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
To install this user certificate, please save this e-mail message into the following
file.

/home/alloy/linuxshell/CH02/usercert.pem
alloy@alloy:~/linuxshell/CH04$
```

7. 退出命令

sed 命令的 q 选项表示完成指定地址的匹配后立即退出，基本格式为：

```
指定地址 q
```

比如，我们需要打印前 5 行，然后退出，可以用下面的命令：

```
sed '5 q' input
```

假设需要查找任意字符后跟 r 字符，再跟 0 个或多个任意字符的字符串，若要找出所有这样的字符串，只需用下面的命令实现：

```
sed -n '/.r.*/p' input
```

若加上 q 选项，则表示匹配到第 1 个满足要求的字符串后就退出，两条命令的执行结果如例 4-25 所示。

```
    #例 4-25: sed 退出命令
    alloy@alloy:~/linuxshell/CH04$ sed -n '/.r.*/p' input     #匹配全部字符串
    This is a Certificate Request file:
    Certificate Subject:
       /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus
    The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
    To install this user certificate, please save this e-mail message into the following
file.
    /home/alloy/linuxshell/CH02/usercert.pem
    alloy@alloy:~/linuxshell/CH04$ sed '/.r.*/q' input        #匹配第 1 个字符串后立即退出
    This is a Certificate Request file:
    alloy@alloy:~/linuxshell/CH04$
```

8. 变换命令

sed 命令的 y 选项表示字符变换，它将一系列的字符变换为相应的字符，sed y 命令是对字符的逐个处理，它的基本格式为：

```
sed 'y/被变换的字符序列/变换的字符序列/' 输入文件
```

sed y 命令将被变换字符序列中的字符逐个用变换字符序列中的字符替代，比如以下命令将 input 文件中 1 变换为 A、2 变换为 B、3 变换为 C、4 变换为 D、5 变换为 E。

```
sed 'y/12345/ABCDE/' input
```

sed y 命令要求被变换的字符序列和变换的字符序列等长，否则 sed y 命令将报错，下面的例 4-26 中的第 1 条命令就是 sed y 的报错信息，例 4-26 中的第 2 条命令将 input 文件中的 f、m 和 j 字符都变换成大写字母。

```
#例 4-26: 演示 sed 变换命令的用法
#第 1 条命令: 变换字符序列不等长, sed 命令报错
alloy@alloy:~/linuxshell/CH04$ sed 'y/fmj/FMJF/' input
sed: -e 表达式 #1, 字符 11: strings for `y' command are different lengths
#第 2 条命令: 将 fmj 三个字符变换为大写
alloy@alloy:~/linuxshell/CH04$ sed 'y/fmj/FMJ/' input
This is a CertiFicate Request File:

It should be Mailed to zawu@seu.edu.cn

============================================================
CertiFicate SubJect:

   /O=Grid/OU=GlobusTest/OU=siMpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

   The above string is known as your user certiFicate subJect, and it uniquely identiFies
this user. $88
   To install this user certiFicate, please save this e-Mail Message into the Following
File.

   /home/alloy/linuxshell/CH02/usercert.peM
   alloy@alloy:~/linuxshell/CH04$
```

9. 显示控制字符

控制字符就是非打印字符，如退格键、F1 键、Shift 键等，有些文件中会包含这些字符，sed l 命令可以显示出文件中的控制字符，方便用户对控制字符进行处理。

一个名为 control 的文件中包含有控制字符，我们用 sed l 命令显示该文件中所有的控制

混混藏书阁 http://book-life.blog.163.com

字符，如下面的例 4-27。

```
#例 4-27：sed 显示控制字符
alloy@alloy:~/linuxshell/CH04$ sed -n '1,$l' control
\033[11~ <F7> <F10>\r$
alloy@alloy:~/linuxshell/CH04$
```

例 4-27 中命令用正则表达式“'1,$'”表示从第一行到最后一行，各系统控制字符的显示值可能不同。

10. 在定位行执行命令组

sed 编辑命令中的“{}”符号可以指定在定位行上所执行的命令组，它的作用与 sed 的-e 选项类似，都是为了在定位行执行多个编辑命令。

例 4-28 说明了“{}”符号的用法，其中，第 1 条命令是打印与 Certificate 关键字匹配行的内容及其行号，与例 4-4 的例子是等价的，即以下两条命令是等价的：

```
sed -n -e '/Certificate/p' -e '/Certificate/=' input
sed -n 'Certificate/{p;=} ' input
```

例 4-28 中的第 2 条命令是在与 certificate 关键字匹配行将全部的 i 替换为 I、将第 1 个 le 替换为 99。

```
#例 4-28：sed {}符号的用法
#第 1 条命令：打印与 Certificate 匹配的行及其行号
alloy@alloy:~/linuxshell/CH04$ sed -n '/Certificate/{p;=}' input
This is a Certificate Request file:
1
Certificate Subject:
6
#第 2 条命令：在与 certificate 关键字匹配行将全部的 i 替换为 I、将第 1 个 le 替换为 99
alloy@alloy:~/linuxshell/CH04$ sed '/Certificate/{s/i/I/g;s/le/99/;}' input
ThIs Is a CertIfIcate Request fI99:

It should be mailed to zawu@seu.edu.cn

==========================================================
CertIfIcate Subject:

   /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

   The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
   To install this user certificate, please save this e-mail message into the following
file.

   /home/alloy/linuxshell/CH02/usercert.pem
   alloy@alloy:~/linuxshell/CH04$
```

4.2.4 sed 高级编辑命令的一组例子

相对于 4.2.3 节介绍的 sed 基本编辑命令，本节介绍的 sed 的一些编辑命令较为生僻，不太常用，本节举一组例子介绍 sed 高级编辑命令，供需要深入掌握 sed 编程的读者参考。

1. 处理匹配行的下一行

sed 编辑命令 n 的意义是读取下一个输入行，用 n 后面的一个命令处理该行，由于此时

通常有多个编辑命令，所以，编辑命令 n 需要与{}符号结合使用。

例 4-29 演示了编辑命令 n 的用法，例中命令的意义是找出 certificate 关键字的匹配行（例 4-29 结果中 certificate 用下画线标注处），然后在匹配行的下一行执行 s/ll/99 命令，即在下一行将 ll 字符串替换为 99。结果显示为第 1 个 certificate 的匹配行是第 10 行，然后将第 11 行的 install 替换为了 insta99（例 4-29 结果中 insta99 用下画线标注处）。

```
#例 4-29：sed n 编辑命令的用法
alloy@alloy:~/linuxshell/CH04$ sed '/certificate/{n;s/ll/99/;}' input
This is a Certificate Request file:

It should be mailed to zawu@seu.edu.cn

==========================================================
Certificate Subject:

    /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
To insta99 this user certificate, please save this e-mail message into the following
file.

/home/alloy/linuxshell/CH02/usercert.pem
alloy@alloy:~/linuxshell/CH04$
```

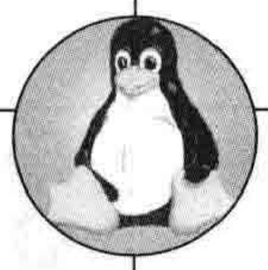
2. sed 缓冲区的处理

4.1 节提及 sed 命令将输入文件复制到缓冲区，对缓冲区中的副本处理后，将其写入输出文件。实际上，sed 有两种缓冲区：模式缓冲区（Pattern Buffer）和保持缓冲区（Hold Buffer）。前面提及的是模式缓冲区，而保持缓冲区是另一块内存空间，sed 的一些编辑命令可以对保持缓冲区进行处理，并与模式缓冲区的内容互换。

下面举几个例子来说明 sed 缓冲区的处理，例 4-30 演示了 sed 命令的 h、x 和 G 选项的含义。

```
#例 4-30：sed h、x 和 G 命令的用法
alloy@alloy:~/linuxshell/CH04$ sed -e '/Subject/h' -e '/seugrid/x' -e '$G' input
This is a Certificate Request file:

It should be mailed to zawu@seu.edu.cn

==========================================================
Certificate Subject:                                    #匹配 Subject，将此行写入保持缓冲区

Certificate Subject:#匹配 seugrid，将此行写入保持缓冲区，并将原来保持缓冲区的内容输出

The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
To install this user certificate, please save this e-mail message into the following
file.

/home/alloy/linuxshell/CH02/usercert.pem
   /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus
#到最后一行时，输出保持缓冲区的内容
alloy@alloy:~/linuxshell/CH04$
```


混混藏书阁 http://book-life.blog.163.com



例 4-30 中的命令由三个-e 选项带上三个编辑命令组成，第 1 个-e 选项后的编辑命令将 Subject 关键字的匹配行写入保持缓冲区（h 命令指将模式缓冲区内容复制到保持缓冲区）；第 2 个-e 选项后的编辑命令是遇到 seugrid 关键字的匹配行时，将保持缓冲区中的内容输出，并将 seugrid 关键字的匹配行写入保持缓冲区，因此，例 4-30 结果中，原本 seugrid 行的位置变为了 Subject 行（x 命令是互换模式缓冲区和保持缓冲区的文本内容）；第 3 个-e 选项后的编辑命令表示到最后一行时（用$符号匹配），将保持缓冲区的内容追加到模式缓冲区中（G 命令的意义），因此，图 4-30 的结果中，最后一行是 seugrid 行。

h 和 H、g 和 G 是两组对应的命令，h 和 H 命令是模式缓冲区的内容替换保持缓冲区的内容，不过 h 是副本，即将保持缓冲区的旧内容覆盖掉，而 H 是追加，即在保持缓冲区旧的内容上增加新的内容；g 和 G 命令是保持缓冲区的内容替换模式缓冲区的内容，同样，g 是副本，G 是追加。下面的例 4-31 说明了 sed 命令的 H 选项，该例子中的命令仅将例 4-30 命令中的 x 改成 H，即第 2 个-e 选项后的编辑命令将 seugrid 关键字的匹配行追加到保持缓冲区中，由此，保持缓冲区中的文本应该是两行：Subject 的匹配行和 seugrid 的匹配行；第 2 个-e 选项后的编辑命令将保持缓冲区的内容在模式缓冲区最后输出，结果显示为上述两行。

```
#例 4-31：sed H 命令的用法
alloy@alloy:~/linuxshell/CH04$ sed -e '/Subject/h' -e '/seugrid/H' -e '$G' input
This is a Certificate Request file:

It should be mailed to zawu@seu.edu.cn

==========================================================================
Certificate Subject:

    /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus

The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
To install this user certificate, please save this e-mail message into the following
file.

/home/alloy/linuxshell/CH02/usercert.pem
Certificate Subject:
    /O=Grid/OU=GlobusTest/OU=simpleCA-seugrid1.seu.edu.cn/OU=seu.edu.cn/CN=globus
#以上画线的两行来自保持缓冲区
alloy@alloy:~/linuxshell/CH04$
```

3. 利用分号分隔多个编辑命令

除了使用-e 和{}符号可以实现 sed 的多编辑命令之外，利用分号“;”也可实现类似的功能，基本格式为：

```
sed '编辑命令 1;编辑命令 2;.....' 输入文件
```

例 4-32 演示了利用分号“;”分隔多个编辑命令，例中的命令用分号分隔两个替换操作。

```
#例 4-32：利用分号分隔多个编辑命令
alloy@alloy:~/linuxshell/CH04$ sed 's/globus/GLOBUS/; s/seugrid/SEUGRID/' input
This is a Certificate Request file:

It should be mailed to zawu@seu.edu.cn

==========================================================================
```

```
    Certificate Subject:

        /O=Grid/OU=GlobusTest/OU=simpleCA-SEUGRID1.seu.edu.cn/OU=seu.edu.cn/CN=GLOBUS

    The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
    To install this user certificate, please save this e-mail message into the following
file.

    /home/alloy/linuxshell/CH02/usercert.pem
    alloy@alloy:~/linuxshell/CH04$
```

利用分号分隔多条编辑命令等价于用-e 选项引出多个编辑命令，如下面的两条命令，但是，用分号显得更为简洁。

```
sed 's/globus/GLOBUS/; s/seugrid/SEUGRID/' input
sed -e 's/globus/GLOBUS/' -e 's/seugrid/SEUGRID/' input
```

在 Bourne Shell 中，还有一种 sed 多编辑命令的用法，输入“sed'”，按 Enter 键，Shell 将出现“>”二级命令提示符，在此可以输入多条编辑命令，最后一条编辑命令之后加上“'”符号，然后跟输入文件结束 sed 命令。例 4-33 是说明这一用法的例子，在二级命令提示符下连续输入三条编辑命令。

```
#例 4-33：在二级命令提示符下输入多个编辑命令
alloy@alloy:~/linuxshell/CH04$ sed '
> s/globus/GLOBUS/
> s/seugrid/SEUGRID/
> $d' input
    This is a Certificate Request file:

    It should be mailed to zawu@seu.edu.cn

    ==========================================================================
    Certificate Subject:

        /O=Grid/OU=GlobusTest/OU=simpleCA-SEUGRID1.seu.edu.cn/OU=seu.edu.cn/CN=GLOBUS

    The above string is known as your user certificate subject, and it uniquely identifies
this user. $88
    To install this user certificate, please save this e-mail message into the following
file.

    alloy@alloy:~/linuxshell/CH04$
```

值得注意的是，C Shell 不支持 sed 二级命令提示符，bash Shell 是 Bourne Shell 的扩展。因此，bash Shell 兼容 Bourne Shell 的特征，Bourne Shell、C Shell 和 bash Shell 的起源与关系可参见本书第 11 章的论述。

4.3 awk 编程

awk 是由 Alfred V.Aho、Peter J.Weinberger 和 Brian W.Kernighan 于 1977 年开发的编程语言，awk 是上述三位创建者姓的首字母。awk 的基本语法与 C 语言类似，读者如果对 C 语言

混混藏书阁 http://book-life.blog.163.com

很熟悉，那么学习 awk 编程将事半功倍。

awk 功能与 sed 相似，都是用来进行文本处理的，awk 语言可以从文件或字符串中基于指定规则浏览和抽取信息，在抽取信息的基础上，才能进行其他的文本操作。

awk 自诞生以来，存在很多个版本，最初的 awk、nawk、POSIX awk，以及最新的 gawk 等。nawk 是 1985 年开发的，*The awk Programming Language* 对其进行了详细描述。1993 年，ANSI 正式发布 POSIX awk。目前，使用的是 gawk，Linux 系统中/bin 目录下有 awk 和 gawk 两个命令，awk 实际上是/bin/gawk 的链接，gawk 是一种功能很强且很实用的语言，利用 gawk 语言可以实现数据查找、抽取文件中的数据、创建管道流命令等功能。

awk 是一种编程语言，gawk 是目前最新的版本，当前的 Linux 版本用的都是 gawk，本书对 awk 和 gawk 不再区分，统称为 awk 编程。

4.3.1 awk 编程模型

awk 为程序员提供了完善的编程模型，理解 awk 编程模型非常重要，这有利于读者从宏观上理解 awk，为学习具体的 awk 编程细节打下基础。

awk 程序由一个主输入循环（Main Input Loop）维持，主输入循环反复执行，直到终止条件被触发。当然，主输入循环无须由程序员去写，awk 已经搭好主输入循环的框架，程序员写的代码被嵌到主输入循环框架中执行。主输入循环自动依次读取输入文件行，以供处理，而处理文件行的动作是由程序员添加的。其他编程语言，如 C、C++和 Java 等，程序员需要写一个 main 函数，然后打开文件、读取文件行、进行相应处理、关闭文件，但是，awk 自动完成了上述步骤，使程序员可以更加便捷地书写程序，这正是 awk 和其他编程语言的本质区别。

awk 还定义了两个特殊的字段：BEGIN 和 END，BEGIN 用于在主输入循环之前执行，即在未读取输入文件行之前执行，END 则相反，用于在主输入循环之后执行，即在读取输入文件行完毕后执行，awk 程序的执行过程如图 4-1 所示，我们也可以简单地将 awk 编程模型分为三个阶段：读输入文件之前的执行代码段（由 BEGIN 关键字标识）、读取输入文件时的执行代码段和读输入文件完毕之后的执行代码段（由 END 关键字标识）。

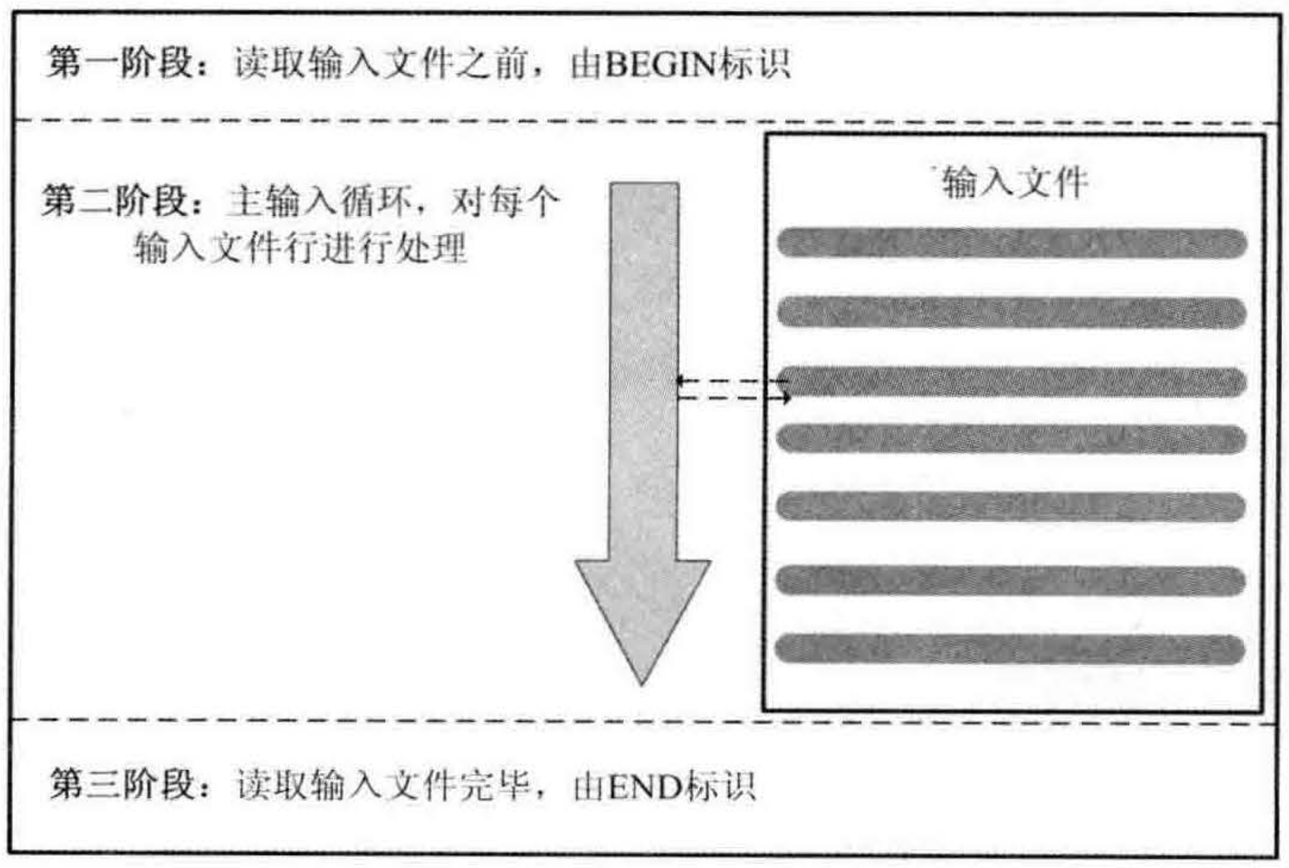


图 4-1　awk 编程模型

4.3.2 awk 调用方法

调用 awk 的方法与调用 sed 类似，也有三种方式，一种为 Shell 命令行方式，另外两种是将 awk 程序写入脚本文件，然后执行该脚本文件。三种方式的命令格式归纳如下。

① 在 Shell 命令行输入命令调用 awk，格式为：

```
awk [-F 域分隔符] 'awk 程序段' 输入文件
```

同样要注意，需要用单引号将 awk 程序段引起来。

② 将 awk 程序段插入脚本文件，然后通过 awk 命令调用它，格式为：

```
awk -f awk 脚本文件 输入文件
```

与 sed 命令类似，awk 也用-f 选项表示调用 awk 脚本文件。

③ 将 sed 命令插入脚本文件后，最常用的方法是设置该脚本文件为可执行，然后直接执行该脚本文件，格式为：

```
./awk 脚本文件 输入文件
```

如用第③种方式调用 awk，awk 脚本文件仍以 sha-bang（#!）符号开头，但是，sha-bang 符号后面加上了 awk 或 gawk 的路径。

4.4 awk 编程的一组例子

awk 是最难掌握的一种 Shell 文本处理工具，因为 awk 语法复杂，并包含了太多的编程细节。本节通过 10 个主题介绍 awk 编程，这 10 个主题虽然远不能涵盖 awk 的所有内容，但是足以让读者掌握 awk 语言的基本语法，并在 Shell 编程中熟练使用 awk。

4.4.1 awk 模式匹配

任何 awk 语句都由模式（pattern）和动作（action）组成。模式是一组用于测试输入行是否需要执行动作的规则，动作是包含语句、函数和表达式的执行过程。简言之，模式决定动作何时触发和触发事件，动作执行对输入行的处理。本节通过一组例子讲述 awk 模式匹配，awk 模式匹配经常需要用到正则表达式，awk 支持表 3-1 和表 3-2 所示的所有正则表达式元字符，awk 支持“?”和“+”两个扩展元字符，而 grep 和 sed 并不支持。

首先给出一个简单的模式匹配例子，以进一步解释模式和动作，同时给出本例的三种调用方法，以便读者掌握 4.3.3 节所述的 awk 调用方法。请看下面的例 4-34：

```
#例 4-34：第一种方法调用 awk 打印空白行
alloy@alloy:~/linuxshell/CH04$ awk '/^$/{print "This is a blank line."}' input
This is a blank line.
This is a blank line.
This is a blank line.
This is a blank line.
This is a blank line.
alloy@alloy:~/linuxshell/CH04$
```

例 4-34 中的命令是 awk 的第一种调用方式，单引号中间是 awk 命令，该 awk 命令由两部分组成，以“/”符号分隔，^$部分是模式，花括号部分是动作，该 awk 命令表示一旦读入的输入文件行是空行，就打印“This is a blank line.”。^$是正则表达式，表示空白行，print

混混藏书阁 http://book-life.blog.163.com

表示该动作是打印操作，input 是输入文件名称。

下面的例 4-35 演示了 awk 的第二种调用方式，scr.awk 文件中只有一行 awk 单引号中的内容，再用 awk -f 调用 scr1.awk 文件。

```
#例 4-35：第二种方法调用 awk 打印空白行
alloy@alloy:~/linuxshell/CH04$ cat scr.awk                    #查看 scr.awk 文件内容
/^$/{print "This is a blank line."}
alloy@alloy:~/linuxshell/CH04$ awk -f scr.awk input           #调用 scr.awk 文件
This is a blank line.
This is a blank line.
This is a blank line.
This is a blank line.
This is a blank line.
alloy@alloy:~/linuxshell/CH04$
```

如果我们要用直接执行 awk 脚本文件的方法来调用，与 sed 脚本一样，在 awk 脚本的第 1 行加上 sha-bang 符号，脚本内容为：

```
#scr1.awk：打印空白行
#!/bin/awk -f
/^$/{print "This is a blank line."}
```

然后为 scr1.awk 附加可执行权限，并执行它，第三种方法调用 awk 如例 4-36 所示。

```
#例 4-36：第三种方法调用 awk 打印空白行
alloy@alloy:~/linuxshell/CH04$ chmod u+x scr1.awk
alloy@alloy:~/linuxshell/CH04$ ./scr1.awk input
This is a blank line.
This is a blank line.
This is a blank line.
This is a blank line.
This is a blank line.
alloy@alloy:~/linuxshell/CH04$
```

awk 模式匹配远不止这些简单的内容，在进一步讲述 awk 模式匹配前，我们先讲述 awk 的记录（Records）和域（Fields）。

4.4.2 记录和域

awk 认为输入文件是结构化的，awk 将每个输入文件行定义为记录，行中的每个字符串定义为域，域之间用空格、Tab 键或其他符号进行分隔，分隔域的符号就叫分隔符。图 4-2 描述了文本中的一条记录，该条记录由三个域组成（图中用下画线标出），前两个域 Li 和 Hao 之间用空格符分隔，后两个域 Hao 和 025-83481010 之间用 Tab 键分隔，两个或多个连续的空格或 Tab 键当作一个分隔符来处理。对文本文件分域处理是 Linux 系统中很多命令都使用的方法。第 5 章将要介绍的 sort、uniq、join、cut 等命令都用到了域和域分隔符的概念，希望读者在此能对记录的域及其域分隔符有准确的理解。

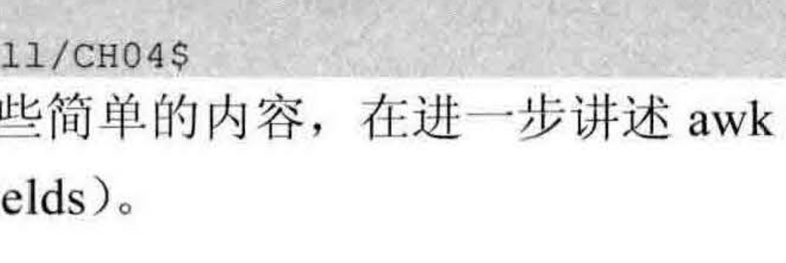


图 4-2 包含三个域的一条记录

awk 定义域操作符$来指定执行动作的域，域操作符$后面跟数字或变量来标识域的位置，每条记录的域从 1 开始编号，如$1 表示第 1 个域、$2 表示第 1 个域、$0 表示所有的域。

接下来，举几个例子说明 awk 记录和域的用法，新建名为 sturecord 的文件作为输入文件，这个文件的每个记录是学生名字、所在学校和电话号码，sturecord 文件的记录、域及域分隔符如图 4-3 所示。

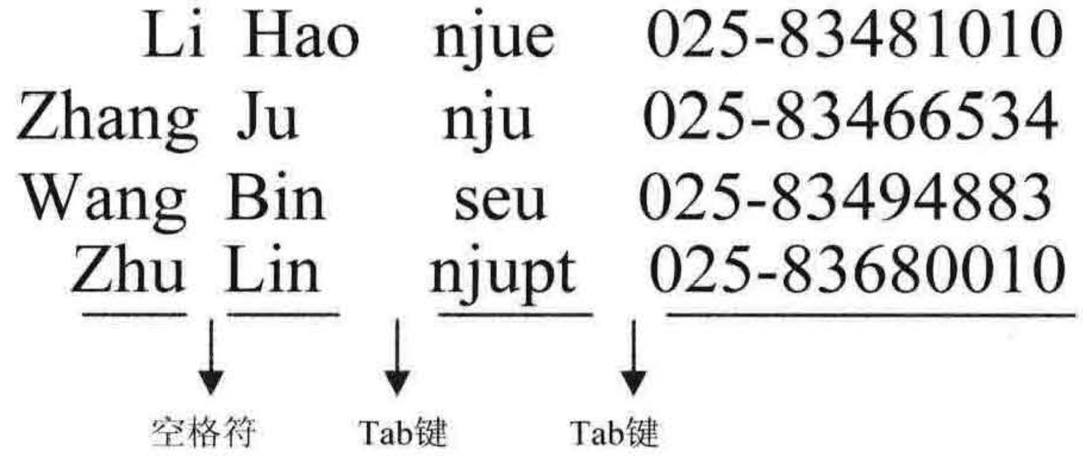


图 4-3 sturecord 文件的记录

sturecord 中的记录分为 4 个域，以第一条记录为例，第 1 个域为 Li，第 2 个域为 Hao，第 3 个域为 njue，第 4 个域为 025-83481010。例 4-37 利用 awk 打印域，例中第 1 条命令按照 2、1、4、3 的次序打印 sturecord 的域，第 2 条命令利用$0 打印 sturecord 的全部域。

```
#例 4-37：awk 命令打印域
#第 1 条命令：打印指定域
alloy@alloy:~/linuxshell/CH04$ awk '{print $2,$1,$4,$3}' sturecord
Hao Li 025-83481010 njue
Ju Zhang 025-83466534 nju
Bin Wang 025-83494883 seu
Lin Zhu 025-83680010 njupt
#第 2 条命令：打印全部域
alloy@alloy:~/linuxshell/CH04$ awk '{print $0}' sturecord
Li Hao          njue    025-83481010
Zhang Ju        nju     025-83466534
Wang Bin        seu     025-83494883
Zhu Lin         njupt   025-83680010
alloy@alloy:~/linuxshell/CH04$
```

域操作符$后面还可以跟变量，或者变量运算表达式，例 4-38 中在$后跟变量指定域号：

```
#例 4-38：$后跟变量指定域号
alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {one=1;two=2} {print $(one+two)}' sturecord
njue
nju
seu
njupt
alloy@alloy:~/linuxshell/CH04$
```

BEGIN 字段中定义 one 和 two 两个变量并赋值，4.3.2 节介绍过 BEGIN 字段中的语句是在遍历输入文件文本之前执行的，print 语句后跟$(one+two)变量运算表达式，one+two=3，因此，该命令打印 sturecord 的第 3 个域。

例 4-37 和例 4-38 的分隔符都是空格键，这是 awk 的默认设置，Tab 键被看作连续的空格键来处理，我们可以使用 awk 的-F 选项来改变分隔符，例 4-39 使用-F 选项将分隔符改为 Tab 键，并打印第 3 个域，可以看到，分隔符为 Tab 键时的第 3 个域是电话号码，因为 sturecord

混混藏书阁 http://book-life.blog.163.com

文件中姓和名之间是用空格分隔的，此时被认为是一个域。

```
#例 4-39：-F 选项改变分隔符
alloy@alloy:~/linuxshell/CH04$ awk -F"\t" '{print $3}' sturecord
025-83481010
025-83466534
025-83494883
025-83680010
alloy@alloy:~/linuxshell/CH04$
```

值得注意的是，大写 F 选项是改变分隔符的，小写-f 选项则表示调用 awk 脚本。

尽管-F 选项可以改变分隔符，但是，awk 还提供了另一种更方便、实用的方法来改变分隔符，这就是使用 awk 环境变量 FS，我们可以通过在 BEGIN 字段中设置 FS 的值来改变分隔符。下面举个例子说明 FS 的用法，我们将 sturecord 改为用逗号来分隔，内容如下：

```
Li Hao,njue,025-83481010
Zhang Ju,nju,025-83466534
Wang Bin,seu,025-83494883
Zhu Lin,njupt,025-83680010
```

sturecord 文件中的记录包含三个域：姓名、学校和电话号码，例 4-40 演示了如何使用 FS 变量改变分隔符。

```
#例 4-40：使用 FS 变量改变分隔符
#第 1 条命令：打印全部域
alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=","} {print $0}' sturecord
Li Hao,njue,025-83481010
Zhang Ju,nju,025-83466534
Wang Bin,seu,025-83494883
Zhu Lin,njupt,025-83680010
#第 2 条命令：打印第 1 域和第 3 域
alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=","} {print $1,$3}' sturecord
Li Hao 025-83481010
Zhang Ju 025-83466534
Wang Bin 025-83494883
Zhu Lin 025-83680010
alloy@alloy:~/linuxshell/CH04$
```

BEGIN 语句中将 FS 赋值为逗号，表示以逗号为分隔符来处理文件，print 语句设置需要打印的域号，第 1 条命令打印全部域，第 2 条命令打印 1 和 3 号域，从结果中可以看出，姓名被看作是一个域，域号是 1，电话号码的域号是 3。

对于不同的输入文件，需要根据实际情况设置相应的分隔符，如 Linux 系统/etc/passwd 文件是以冒号为分隔符的。当然，我们也可以使用正则表达式将分隔符设置为多个字符，请看下面的例 4-41：

```
#例 4-41：不同的域分隔符对同一条记录解析出不同结果
#下面是对 FS 变量的两种赋值
(1) FS="\t"
(2) FS="\t+"
#下面是一条示例性记录
wza\t\tcq
```

例 4-41 中，第（1）种 FS 赋值将一个 Tab 键作为分隔符，第（2）种以一个或多个 Tab 键作为分隔符，当解析下面这条示例性的记录时，两种赋值将产生不同的结果。第（1）种将其解析为三个域：wza、空域、cq，第（2）种将其解析为两个域：wza 和 cq。

4.4.3 关系和布尔运算符

awk 定义了一组关系运算符用于 awk 模式匹配，表 4-5 列出关系运算符及其意义。

表 4-5 awk 关系运算符及其意义

运 算 符	意 义
<	小于
>	大于
<=	小于或等于
>=	大于或等于
==	等于
!=	不等于
~	匹配正则表达式
!~	不匹配正则表达式

下面举一组例子说明 awk 关系运算符的用法，很多例子使用 Linux 系统/etc/passwd 文件作为输入文件，/etc/passwd 文件记录了 Linux 系统用户的关键信息，系统的每一个合法用户账号对应于该文件中的一行记录。这行记录定义了每个用户账号的属性。下面是一个 passwd 文件的部分摘录。

```
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
......
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
```

在该文件中，每一行用户记录的各个域用冒号分隔，分别定义了用户的各方面属性。各字段的顺序和含义如下：注册名：口令：用户标识号：组标识号：用户名：用户主目录：命令解释程序。

例 4-42 使用匹配正则表达式~符号，例中第 1 条命令打印/etc/passwd 文件中第 1 个域匹配 root 关键字的记录，结果为 root 用户的记录。第 2 条命令打印/etc/passwd 文件中全部域匹配 root 关键字的记录，结果中 operator 用户的第 6 域匹配 root 而被打印。第 3 条命令打印/etc/passwd 文件中所有域不匹配 nologin 关键字的记录。值得注意的是，例中的三条命令都在 BEGIN 字段中设置分隔符为冒号，并且由于输入文件不在当前工作目录，因此，采用绝对路径的方法指定输入文件的位置。

```
#例 4-42：使用匹配正则表达式~符号
#第 1 条命令：第 1 域匹配 root
alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=":"} $1~/root/' /etc/passwd
root:x:0:0:root:/root:/bin/bash
#第 2 条命令：全部域匹配 root
alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=":"} $0~/root/' /etc/passwd
root:x:0:0:root:/root:/bin/bash
operator:x:11:0:operator:/root:/sbin/nologin
#第 3 条命令：全部域不匹配 nologin
alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=":"} $0!~/nologin/' /etc/passwd
root:x:0:0:root:/root:/bin/bash
```


混混藏书阁 http://book-life.blog.163.com

```
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
globus:x:500:500:globus:/home/globus:/bin/bash
alloy@alloy:~/linuxshell/CH04$
```

awk 在进行模式匹配时，常常使用到条件语句，awk 条件语句与 C 语言类似，有 if 语句、if/else 语句和 if/else else 语句三种，下面的例 4-43 的 awk 命令利用 if 语句匹配第 3 域小于第 4 域的记录。

```
#例 4-43：awk 中的 if 语句
alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=":"} {if($3<$4) print $0}' /etc/passwd
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
uucp:x:10:14:uucp:/var/spool/uucp:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin
gopher:x:13:30:gopher:/var/gopher:/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
alloy@alloy:~/linuxshell/CH04$
```

例 4-43 中的命令是打印满足 if 条件的所有记录，if 后面条件为$3<$4，表示第 3 域值小于第 4 域值，请注意，if 条件需要用圆括号括起来。<=、>、>=、==等几个关系运算符较为简单，用法类似于例 4-43 中的“<”运算符，在此不再一一举例说明。

为进行多条件模式匹配，awk 定义了三个布尔运算符表示多条件之间的关系，表 4-6 列出了布尔运算符及其意义，可以看出，awk 布尔运算符与 C 语言的布尔运算符是一样的。

表 4-6　awk 布尔运算符及其意义

运　算　符	意　　义
||	逻辑或
&&	逻辑与
! ^	逻辑非

下面的例 4-44 说明了 if 语句的多条件匹配，例中第 1 条语句在/etc/passwd 文件中查找满足条件$3==10 或$4==10 的记录，即第 3 域等于 10 或第 4 域等于 10，中间用逻辑或符号连接。利用==符号进行的匹配可称为精确匹配，它匹配是域的值等于 10 的记录，结果显示了第 3 域为 10 的一条记录。第 2 条命令仅将第 1 条命令的==符号改为了~符号，表示模糊匹配，它表示查找域的值包含 10 个字符的记录，结果显示了三条记录，第 2、3 条记录的第 4 域值分别为 100 和 510，它们都包含 10 个字符，因而满足匹配条件。

```
#例 4-44：if 语句的多条件匹配
#第 1 条命令：多条件精确匹配
alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=":"} {if($3==10||$4==10) print $0}'
/etc/passwd
uucp:x:10:14:uucp:/var/spool/uucp:/sbin/nologin
#第 2 条命令：多条件模糊匹配
alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=":"} {if($3~10||$4~10) print $0}'
/etc/passwd
uucp:x:10:14:uucp:/var/spool/uucp:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin
alloy@alloy:~/linuxshell/CH04$
```

“逻辑与”和“逻辑非”的用法与“逻辑或”相似，在此不再专门举例说明，在下面的一些例子中将会涉及这两个符号的用法。

4.4.4 表达式

与其他编程语言一样，awk 表达式用于存储、操作和获取数据，一个 awk 表达式可由数值、字符常量、变量、操作符、函数和正则表达式自由组合而成。

变量是一个值的标识符，定义 awk 变量非常方便，只需定义一个变量名并将值赋给它即可。变量名只能包含字母、数字和下画线，而且不能以数字开头，如 3xyz 这样的变量名是非法的。awk 变量名是区分大小写的，xyz 和 XYZ 表示了两个不同的变量。定义 awk 变量无须声明变量类型，每个变量有两种类型的值：字符串值和数值，awk 根据表达式上下文来确定使用哪个值，变量的默认数值为 0、默认字符串值为空。

```
x=1
y="Good"
z="Very" "Good"
```

awk 变量的定义和赋值如上面三条语句所示，=是赋值符，第 1 条语句将变量 x 赋为 1，第 2 条语句将 y 赋为字符串 Good，第 3 条语句用空格分隔两个字符串，可将 z 赋为 Very Good。

表达式可进行变量和数字之间的算术操作，如加、减、乘、除、乘方等，awk 算术运算符如表 4-7 所示。

表 4-7　awk 算术运算符及其意义

运　算　符	意　　义
+	加
-	减
*	乘
/	除
%	模
^或**	乘方
++x	在返回 x 值之前，x 变量加 1
x++	在返回 x 值之后，x 变量加 1

接下来，我们举一个统计 input 文件空白行的例子说明表达式的用法，如例 4-45 所示，例中的命令表示一旦匹配就执行表达式 x+=1，即 x=x+1，然后打印 x+=1 的返回值，结果中空白行标号从 1 开始到 5 结束。

```
#例 4-45：统计 input 文件中的空白行
alloy@alloy:~/linuxshell/CH04$ awk '/^$/{print x+=1}' input          #^$匹配空白行
1
2
3
4
5
alloy@alloy:~/linuxshell/CH04$
```

熟悉 C 语言的读者应该已经了解++x 和 x++的区别（--x 和 x--与之类似）。下面举一个简

混混藏书阁 http://book-life.blog.163.com

单例子说明++x 和 x++的区别，为尚不了解此用法的读者提供参考。例子如例 4-46 所示，两条命令仍然是统计 input 文件的空白行，第 1 条命令使用 x++，即返回 x 值之后，x 变量增加 1，x 变量初值默认为 0，结果中空白行标号从 0 开始到 4 结束；第 1 条命令使用++x，即 x 变量增加 1，再返回 x 值。因此，结果中空白行标号从 1 开始到 5 结束。

```
#例 4-46：++x 和 x++的区别
alloy@alloy:~/linuxshell/CH04$ awk '/^$/{print x++}' input
0
1
2
3
4
alloy@alloy:~/linuxshell/CH04$ awk '/^$/{print ++x}' input
1
2
3
4
5
alloy@alloy:~/linuxshell/CH04$
```

再举一个计算平均值的例子，我们在 sturecord 文件中每个学生记录后加入 5 个域，表示学生 5 门学科的考试成绩，sturecord 内容如下：

```
Li Hao,njue,025-83481010,85,92,78,94,88
Zhang Ju,nju,025-83466534,89,90,75,90,86
Wang Bin,seu,025-83494883,84,88,80,92,84
Zhu Lin,njupt,025-83680010,98,78,81,87,76
```

接着，我们写一个名为 scr2.awk 的脚本用于计算 sturecord 文件中每个学生的平均成绩，并打印出来，脚本内容如下：

```
#例 4-47：scr2.awk 脚本演示如何计算 sturecord 文件中的平均值
#!/bin/awk -f
BEGIN {FS=","}
{total=$4+$5+$6+$7+$8
avg=total/5
print $1,avg}
```

脚本定义了两个变量 total 和 avg，total 是每条记录 5 门成绩的和、avg 是 5 门成绩的平均值，例 4-47 给出了 scr2.awk 脚本的执行结果，输出学生姓名及其平均成绩。

```
#例 4-47 scr2.awk 脚本的执行结果
alloy@alloy:~/linuxshell/CH04$ chmod u+x scr2.awk                    #给 scr2.awk 赋可执行权限
alloy@alloy:~/linuxshell/CH04$ ./scr2.awk sturecord                  #执行 scr2.awk 脚本
Li Hao 87.4                                                          #输出学生姓名及其平均成绩
Zhang Ju 86
Wang Bin 85.6
Zhu Lin 84
alloy@alloy:~/linuxshell/CH04$
```

4.4.5 系统变量

awk 定义了很多内建变量用于设置环境信息，我们称它们为系统变量，这些系统变量可分为两种：第 1 种用于改变 awk 的默认值，如域分隔符；第 2 种用于定义系统值，在处理文本时可以读取这些系统值，如记录中的域数量、当前记录数、当前文件名等，awk 动态改变第 2 种系统变量的值。表 4-8 列出了常见的 awk 环境变量及其意义。

表 4-8　awk 环境变量及其意义

变 量 名	意　义
$n	当前记录的第 n 个域，域间由 FS 分隔
$0	记录的所有域
ARGC	命令行参数的数量
ARGIND	命令行中当前文件的位置（以 0 开始标号）
ARGV	命令行参数的数组
CONVFMT	数字转换格式
ENVIRON	环境变量关联数组
ERRNO	最后一个系统错误的描述
FIELDWIDTHS	字段宽度列表，以空格键分隔
FILENAME	当前文件名
FNR	浏览文件的记录数
FS	字段分隔符，默认是空格键
IGNORECASE	布尔变量，如果为真，则进行忽略大小写的匹配
NF	当前记录中的域数量
NR	当前记录数
OFMT	数字的输出格式
OFS	输出域分隔符，默认是空格键
ORS	输出记录分隔符，默认是换行符
RLENGTH	由 match 函数所匹配的字符串长度
RS	记录分隔符，默认是空格键
RSTART	由 match 函数所匹配的字符串的第 1 个位置
SUBSEP	数组下标分隔符，默认值是\034

首先看到下面的例 4-48，例中的命令涉及 FS、NF、NR 和 FILENAME 四个系统变量，BEGIN 字段利用 FS 预设域分隔符为“,”，中间字段为一条 print，依次打印 NF、NR 和$0，NF 为记录的域数量，结果显示为 8，说明 sturecord 的每条记录都有 8 个域，NR 显示当前的记录数，该值根据读取输入文件的进度而变化，读取第 1 条记录时，NR=1，读到文件末尾时，NR 为该文件所包含的记录数。因此，结果中 NR 字段依次为 1～4，$0 表示打印记录的所有域。END 字段打印 FILENAME，FILENAME 保存了当前的输入文件名，显然为 sturecord。

```
    #例 4-48: FS、NF、NR 和 FILENAME 的用法
    alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=","} {print NF,NR,$0} END {print
FILENAME}' sturecord
    8 1 Li Hao,njue,025-83481010,85,92,78,94,88
    8 2 Zhang Ju,nju,025-83466534,89,90,75,90,86
    8 3 Wang Bin,seu,025-83494883,84,88,80,92,84
    8 4 Zhu Lin,njupt,025-83680010,98,78,81,87,76
    sturecord
    alloy@alloy:~/linuxshell/CH04$
```


混混藏书阁 http://book-life.blog.163.com

本节不准备一一举例说明每个系统变量的用法，因为下面的例子会涉及表 4-8 中大部分系统变量的用法，我们将随具体例子对系统变量加以解释。

4.4.6 格式化输出

前面几节的例子只涉及 awk 如何对输入文件进行处理，对于输出的格式并未规定，原因在于 print 语句只提供了最基本的输出格式。而 awk 的一大主要功能是产生报表，报表就要求按照预定的格式输出，awk 借鉴 C 语言的语法，定义了 printf 输出语句，它可以规定输出的格式。printf 的基本语法如下：

```
printf (格式控制符,参数)
```

printf 语句包含两部分：第一部分是格式控制符，都以%符号开始，用以描述格式规范；第二部分是参数列表，比如变量名列表，与格式控制符相对应，是输出的对象。我们将格式控制符分为 awk 修饰符和格式符两种，分别如表 4-9 和表 4-10 所示。

表 4-9　printf 修饰符及其意义

修　饰　符	意　　义
-	左对齐
width	域的步长
.prec	小数点右边的位数

表 4-10　printf 格式符及其意义

运　算　符	意　　义
%c	ASCII 字符
%d	整型数
%e	浮点数，科学记数法
%f	浮点数
%o	八进制数
%s	字符串
%x	十六进制数

例 4-49 说明了 printf 的基本用法，其中包含四种 printf 格式符。例 4-49 中的第 1 条命令是从域号获取值，$2 号域与%s 对应，为字符串；$8 号域与%d 对应，为整数值，两个域之间用 Tab 键隔开（\t 表示 Tab 键），每输出两个域换行（\n 表示换行）。第 2 条命令表示输出 ASCII 字符表中标号为 65 的值，%c 完成数值到 ASCII 字符的转换。第 3 条命令表示以浮点数的格式输出 2009，结果精确到小数点后六位。

```
    #例 4-49：printf 的基本用法
    #第 1 条命令：参数是变量列表
    alloy@alloy:~/linuxshell/CH04$  awk  'BEGIN  {FS=","}  {printf("%s\t%d\n",$2,$8)}'
sturecord
    njue    88
    nju     86
    seu     84
    njupt   76
```

```
#第 2 条命令：转换为 ASCII 字符
alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {printf("%c\n",65)}'
A
#第 3 条命令：转换为浮点数
alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {printf("%f\n",2010)}'
2010.000000
alloy@alloy:~/linuxshell/CH04$
```

接着，给出 printf 修饰符的例子，如例 4-50 所示，例中第 1 条命令表示以字符串的格式输出 sturecord 的第 1 和 3 号域，并对第 1 个%s 进行了修饰，-15 表示该字符串长度控制为 15 位，并且左对齐，若字符串不足 15 位，则用空格补全；如果我们要在输出的域上加解释语言，可以在 BEGIN 字段中添加相应的输出注释，如例 4-50 的第 2 条命令所示。

```
    #例 4-50：printf 修饰符-和 width 的用法
    alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=","} {printf("%-15s\t%s\n",$1,$3)}'
sturecord
    Li Hao          025-83481010
    Zhang Ju        025-83466534
    Wang Bin        025-83494883
    Zhu Lin         025-83680010
    alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=",";print "Name\t\tPhonenumber"}
{printf("%-15s\t%s\n",$1,$3)}' sturecord                    #与上一行是一条命令
    Name            Phonenumber
    Li Hao          025-83481010
    Zhang Ju        025-83466534
    Wang Bin        025-83494883
    Zhu Lin         025-83680010
    alloy@alloy:~/linuxshell/CH04$
```

printf 修饰符.prec 表示输出小数点后的位数，下面的例 4-51 说明了其用法，例 4-51 中第 1 条命令的%10.3 表示该浮点数长度控制在 10 位、小数点后保留 3 位，结果显示将 2009.1012 输出为 2009.101，且为右对齐（printf 的默认对齐方式）。当然，.prec 也可单独使用，如例 4-51 的第 2 条命令所示。

```
#例 4-51：printf 修饰符.prec 的用法
alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {printf("%10.3f\n",2009.1012)}'
  2009.101
alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {printf("%.3f\n",2009.1012)}'
2009.101
alloy@alloy:~/linuxshell/CH04$
```

由例 4-50 和例 4-51，我们可以总结出 printf 修饰符的一般形式：

```
%-width.prec 格式控制符
```

4.4.7 内置字符串函数

awk 提供了强大的内置字符串函数，用于实现文本的字符串替换、查找以及分隔等功能，表 4-11 列出了 awk 的内置字符串函数。

表 4-11 awk 字符串函数及其意义

函 数 名	意 义
gsub(r,s)	在输入文件中用 s 替换 r
gsub(r,s,t)	在 t 中用 s 替换 r

混混藏书阁 http://book-life.blog.163.com

续表

函 数 名	意 义
index(s,t)	返回 s 中字符串第一个 t 的位置
length(s)	返回 s 的长度
match(s,t)	测试 s 是否包含匹配 t 的字符串
split(r,s,t)	在 t 上将 r 分成序列 s
sub(r,s,t)	将 t 中第 1 次出现的 r 替换为 s
substr(r,s)	返回字符串 r 中从 s 开始的后缀部分
substr(r,s,t)	返回字符串 r 中从 s 开始长度为 t 的后缀部分

gsub 函数执行字符串替换功能，它将第一个字符串替换为第二个字符串。gsub 函数有两种形式，第一种形式作用于全部域，即$0，第二种形式作用于域 t。例 4-52 演示了 gsub 函数的用法，例中第 1 条命令将/etc/passwd 文件的第 1 域上的 root 字符串替换为 gridsphere 字符串，BEGIN 字段指定域分隔符和输出的域分隔符；例中第 2 条命令将/etc/passwd 文件全部域上的 root 字符串替换为 gridsphere 字符串，结果显示替换了两行，第 2 个结果替换的是第 6 域上的 root 字符串。

```
    #例 4-52：gsub 函数的用法
    #第 1 条命令：替换第 1 域上的 root 字符串
    alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=":";OFS=":"} gsub(/root/,"gridsphere",
$1) {print $0}' /etc/passwd
    gridsphere:x:0:0:root:/root:/bin/bash
    #第 2 条命令：替换全部域上的 root 字符串
    alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=":";OFS=":"} gsub(/root/,"gridsphere")
{print $0}' /etc/passwd
    gridsphere:x:0:0:gridsphere:/gridsphere:/bin/bash
    operator:x:11:0:operator:/gridsphere:/sbin/nologin
    alloy@alloy:~/linuxshell/CH04$
```

例 4-53 演示了 index 和 length 函数的用法，index 返回第二个字符串在第一个字符串出现的首位置，例 4-53 中 ph 在 gridsphere 中的第 6 位开始出现。length 返回字符串的长度，如 gridsphere 占了 10 位。

```
    #例 4-53：index 和 length 函数的用法
    alloy@alloy:~/linuxshell/CH04$   awk   'BEGIN   {print   index("gridsphere","ph")}'
#index 函数
    6
    alloy@alloy:~/linuxshell/CH04$   awk   'BEGIN   {print   length("gridsphere")}'
#length 函数
    10
    alloy@alloy:~/linuxshell/CH04$
```

match(s,t)测试 s 是否包含匹配 t 的字符串，t 可以是一个正则表达式，若匹配成功，返回匹配 t 的首位置；若不成功，则返回 0。例 4-54 演示了 match 函数的用法，例中第 1 条命令在 gridsphere 字符串中匹配 D，awk 默认状态是区分大小写的。因此，匹配不成功，返回 0；例中第 2 条命令将系统变量 IGNORECASE 设为 1，表示 awk 不分大小写匹配，匹配成功，返回值为 d 在 gridsphere 中的位置，为 4。

```
    #例 4-54：match 函数的用法
```

```
    #第 1 条命令：区分大小写匹配
    alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {print match("gridsphere",/D/)}'
    0
    #第 2 条命令：IGNORECASE 变量为 1 使匹配忽略大小写
    alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {IGNORECASE=1;print match("gridsphere",
/D/)}'
    4
    alloy@alloy:~/linuxshell/CH04$
```

sub(r,s,t) 将 t 中第 1 次出现的 r 替换为 s，r 可以为正则表达式，注意 sub 函数只替换模式出现的第 1 个位置。例 4-55 演示了 sub 函数的用法，例中第 1 条命令先定义 str 变量，赋值为 multiprocessor programming，然后用 sub 函数将 pro 改成大写的 PRO，str 有两个 pro，结果显示只有第 1 个 pro 改成了大写 PRO；第 2 条命令将 sturecord 文件中第 1 号域与 Li 匹配记录中的 10 字符改成 99 字符，结果显示第 1 条记录电话号码 83481010 的第 1 个 10 被改为了 99。

```
    #例 4-55：sub 函数的用法
    alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {str="multiprocessor programming";sub(/pro/,
"PRO",str);printf("%s\n",str)}'                    #与上一行是一条命令
    multiPROcessor programming
    alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=","} {$1~Li sub(/10/,"99",$0);print $0}'
sturecord
    Li Hao,njue,025-83489910,85,92,78,94,88
    Zhang Ju,nju,025-83466534,89,90,75,90,86
    Wang Bin,seu,025-83494883,84,88,80,92,84
    Zhu Lin,njupt,025-83680099,98,78,81,87,76
    alloy@alloy:~/linuxshell/CH04$
```

substr 返回字符串的指定后缀，提供了两种形式，例 4-56 给出这两种形式 substr 的用法，例中命令同样先定义 str 变量，第 1 条命令返回 str 从第 6 个字符开始的后缀部分，第 2 条命令返回 str 从第 6 个字符开始长度为 9 的后缀部分。

```
    #例 4-56：substr 函数的用法
    #第 1 条命令：返回 str 从第 6 个字符开始的后缀部分
    alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {str="multiprocessor programming";print
substr(str,6)}'
    processor programming
    #第 2 条命令：返回 str 从第 6 个字符开始长度为 9 的后缀部分
    alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {str="multiprocessor programming";print
substr(str,6,9)}'
    processor
    alloy@alloy:~/linuxshell/CH04$
```

split 函数在此不作介绍，我们将在 4.4.10 节结合 awk 数组进行介绍。

4.4.8 向 awk 脚本传递参数

awk 脚本内的变量可以在命令行中进行赋值，实现向 awk 脚本传递参数，变量赋值放在脚本之后、输入文件之前，格式为：

```
awk 脚本 parameter=value 输入文件
```

awk 所传递的参数可以是自定义的变量，也可以是系统变量，例 4-57 给出了一个向 awk 脚本传递参数的例子，pass.awk 脚本利用一个判断条件 NF!=MAX，表示记录的域数量是否等于 MAX 变量，若不等于，则输出包含 NR 和 MAX 两个变量的文本行，pass.awk 脚本内容

混混藏书阁 http://book-life.blog.163.com

如下：

```
#例 4-57：pass.awk 脚本判断 NF 是否等于 MAX 变量
#!/bin/awk -f
NF!=MAX
{print("The line "NR" does not have "MAX" filds")}
```

例 4-57 给出 pass.awk 脚本的执行结果，并在输入文件之前加上两条赋值语句，分别对 MAX 和 FS 变量赋值，语句之间用空格分隔开，需要注意的是，“=”符号两端不能有空格，由于 sturecord 有 8 个域，因此，在每条记录下方都输出 pass.awk 脚本中的语句。

```
#例 4-57 pass.awk 脚本的执行结果
alloy@alloy:~/linuxshell/CH04$ chmod u+x pass.awk
alloy@alloy:~/linuxshell/CH04$ ./pass.awk MAX=3 FS="," sturecord
Li Hao,njue,025-83481010,85,92,78,94,88
The line 1 does not have 3 filds
Zhang Ju,nju,025-83466534,89,90,75,90,86
The line 2 does not have 3 filds
Wang Bin,seu,025-83494883,84,88,80,92,84
The line 3 does not have 3 filds
Zhu Lin,njupt,025-83680010,98,78,81,87,76
The line 4 does not have 3 filds
alloy@alloy:~/linuxshell/CH04$
```

再看下面的例 4-58，例中命令输出 sturecord 的所有记录，每条记录前加上了其行号（输出 NR 变量值），然后重新定义 OFS 的值，改变输出域的分隔符，即 NR 和$0 之间用 OFS 定义的值分隔。

```
#例 4-58：重定义输出分隔符
alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {FS=","} {print NR,$0}' OFS="." sturecord
1.Li Hao,njue,025-83481010,85,92,78,94,88
2.Zhang Ju,nju,025-83466534,89,90,75,90,86
3.Wang Bin,seu,025-83494883,84,88,80,92,84
4.Zhu Lin,njupt,025-83680010,98,78,81,87,76
alloy@alloy:~/linuxshell/CH04$
```

最后讨论一个 awk 命令行参数中尤其需要注意之处：命令行参数不能被 BEGIN 字段语句访问。换句话说，直到输入文件的第 1 行被读取时，命令行参数方才生效。原因在于，awk 读到命令行参数的赋值语句时，并不知道这是一个命令行参数的赋值语句，而认为这是一个文件名，当然这个文件名是无效的，awk 继续读取后面的参数，直到一个正确的输入文件名被解析的时候，awk 才判定前面的语句是命令行参数的赋值语句。请看下面的例 4-59：

```
#例 4-59：命令行参数对 BEGIN 字段无效
alloy@alloy:~/linuxshell/CH04$ awk '
> BEGIN {print n}
> {if (n==1) print "Reading the first file!"
> }' n=1 sturecord

Reading the first file!
Reading the first file!
Reading the first file!
Reading the first file!
Reading the first file!
alloy@alloy:~/linuxshell/CH04$
```

例中命令 BEGIN 字段打印 n 变量值，awk 主输入循环中判断 n 的值，分别输出不同的语句，通过命令行将 n 赋为 1。该命令在读到 n=1 这条赋值语句时，它将 n=1 当作是输入文件

名，该命令在读取 n=1 这个输入文件前执行 BEGIN 字段，此时 n 为空，因此，打印一行空白行；接着，该命令发现 n=1 并非有效的文件名，继续读到 sturecord 参数，发现是一个有效的文件名，进而将 n=1 解析为命令行参数赋值语句，打印满足 n==1 时的语句。上面简单论述了命令行参数对 BEGIN 字段无效的原因，对于一些并不想知道其中详细原因的读者来说，记住结论就足够了。

4.4.9 条件语句和循环语句

awk 条件语句和循环语句与 C 语言的语法完全一样，因此，本节对 awk 条件语句和循环语句的语法不做深入介绍，只列出它们的基本形式，并举几个简单的例子加以说明。

条件语句 if 的语法如下：

```
if (条件表达式)
    动作 1
[else
    动作 2]
```

若条件表达式为真，执行动作 1，否则执行动作 2，else 语句是可选的。条件表达式可以包含算术、关系和布尔操作符。

比如，下面的语句表示当 x 等于 y 时，打印 x。值得注意的是，“==”是关系运算符，用于判断是否等值，而“=”是赋值符，x=y 将 y 的值赋给 x，这个表达式永真，这与 x==y 表示的意思完全不一样。

```
if (x==y) print x
```

当然，也可以使用“~”匹配符和正则表达式作为 if 语句的条件，如下面的语句：

```
if (x ~ /[Hh]el?o/) print x
```

循环语句有三种：while、do while 和 for。while 循环语法如下：

```
while (条件表达式)
    动作
```

若条件表达式为真，重复执行动作，直到条件表达式为假。

do while 语法如下：

```
do
    动作
while (条件表达式)
```

do while 与 while 的区别在于动作的位置不一样，do while 将动作前置，先执行动作，再判断条件表达式。因此，do while 中的动作至少执行一次，而 while 中的动作有可能一次都不执行。

for 循环语法如下：

```
for (设置计数器初值;测试计数器;计数器变化)
    动作
```

它首先为计数器设置初值，然后测试计数器是否满足一定的条件，若满足，则执行动作，并改变计数器值，进行下一轮的测试，直到计数器不满足条件为止。

4.4.10 数组

数组是用于存储一系列值的变量，这些值之间通常是有联系的，可通过索引来访问数组的值，索引需要用中括号括起，数组的基本格式为：

混混藏书阁 http://book-life.blog.163.com

```
array[index]=value
```

可以看到，awk 数组的形式与 C 语言一样，但是，awk 数组无须定义数组类型和大小，而可以直接赋值后使用。下面分别介绍 awk 数组的相关内容。

1. 关联数组

关联数组是指数组的索引可以是字符串，也可以是数字。在大部分编程语言中，数组的索引只能是数字，数组表示了存储值的一系列地址，数组索引是由存储地址的顺序来决定的，如 C 语言中 array[0]表示数组的起始地址。而关联数组在索引和数组元素值之间建立起关联，对每一个数组元素，awk 自动维护了一对值：索引和数组元素值。关联数组的值无须以连续的地址进行存储，因此，关联数组即便可以使用数字作为索引，但是该数字索引并不表示数组存储地址的信息。awk 的所有数组都是关联数组，这是 awk 数组和其他大部分编程语言数组的本质区别。

字符串和数字之间的差别是明显的，如，我们使用 array[09]指定一个数组值，如果换成 array[9]就不能指定到与 array[09]相同的值，例 4-60 论证了这种差别，例中命令首先定义 data[10.15]="1200"，然后利用 CONVFMT 系统变量将 10.15 转换为整数，10.15 就变成了 10，data[10.15]就等价于 data[10]，因此，打印结果 data[10.15]为空值。

```
#例 4-60：论证字符串和数字的区别
alloy@alloy:~/linuxshell/CH04$ awk
BEGIN{data[10.15]="1200";CONVFMT="%d";printf ("<%s>\n",data[10.15])}'
<>
alloy@alloy:~/linuxshell/CH04$
```

基于上述关联数组的含义，awk 特别定义了一种 for 循环用来访问关联数组，语法如下：

```
for (variable in array)
    do something with array[variable]
```

array 是已定义的数组名，variable 是任意指定的变量，可以看作是 for 循环中定义的临时变量。在此不专门举例说明这种 for 循环的用法，我们在下面讲述 split 函数时会涉及 for 循环的用法。

关键字 in 也可用在条件表达式中判断元素是否在数组中，条件表达式格式为：

```
index in array
```

如果 array[index]存在，则返回 1，否则返回 0，如例 4-61，例中命令先定义 data[10.15]，然后使用上述语法判断 data[10.15]是否存在。

```
#例 4-61：判断数组元素是否存在
alloy@alloy:~/linuxshell/CH04$ awk '
> BEGIN {data[10.15]="1200";
> if ("10.15" in data)
> print "Found element!"}'
Found element!
alloy@alloy:~/linuxshell/CH04$
```

2. split 函数

在 4.4.7 节中，我们对 split 函数未作介绍，因为 split(r,s,t)函数将字符串以 t 为分隔符，将 r 字符串拆分为字符串数组，并存放在 t 中。例 4-62 给出了 split 函数的用法，例中第 1 条命令以“/”为分隔符，将字符串 abc/def/xyz 分开，并存在 str 数组中，split 函数的返回值是数组的大小，第 1 条命令的执行结果为 3，表示将 abc/def/xyz 分成了 3 个元素存储在数组中。第 2 条命令将 sturecord 文件的第 1 号域以空格为分隔符分开，存储到 name 数组中，每行返

回一个结果，都为 2。

```
    #例 4-62：split 函数的用法
    #第 1 条命令：将 abc/def/xyz 分成了 3 个元素
    alloy@alloy:~/linuxshell/CH04$ awk 'BEGIN {print split("abc/def/xyz",str,"/")}'
    3
    #第 2 条命令：将 sturecord 文件的第 1 域划分为 2 个元素
    alloy@alloy:~/linuxshell/CH04$  awk  'BEGIN  {FS=","}  {print  split($1,name,"  ")}'
sturecord
    2
    2
    2
    2
    alloy@alloy:~/linuxshell/CH04$
```

接下来，我们举例看一下 split 函数所生成的数组内容，新建名为 array.awk 的脚本文件，内容如下：

```
    #例 4-63：array.awk 脚本演示 split 函数所生成的数组内容
    #!/bin/awk -f
    BEGIN {FS=","}
    {split($1,name," ");
    for(i in name) print name[i]}
```

array.awk 脚本的 split 函数如例 4-62 的第 2 条命令，在 split 函数分隔第 1 号域之后，利用 for 循环将 name 数组的内容打印出来。例 4-63 给出了 array.awk 的执行结果，可以看到，name 数组中确实存储了学生的姓名。同时，请注意 array.awk 中的 for 循环，name 数组是在 split 函数中已定义的，i 变量是临时变量。

```
    #例 4-63 array.awk 脚本的执行结果
    alloy@alloy:~/linuxshell/CH04$ chmod u+x array.awk
    alloy@alloy:~/linuxshell/CH04$ ./array.awk sturecord
    Li
    Hao
    Zhang
    Ju
    Wang
    Bin
    Zhu
    Lin
    alloy@alloy:~/linuxshell/CH04$
```

3. 数组形式的系统变量

awk 系统变量中有两个变量是以数组形式提供的：ARGV 和 ENVIRON。ARGC 是 ARGV 数组中元素的个数，与 C 语言一样，从 ARGV[0]开始，到 ARGV[ARGC-1]结束。首先，通过一个例子看一下 ARGV 中到底存储了哪些命令行参数，新建名为 argv.awk 的脚本，内容如下：

```
    #例 4-64：argv.awk 演示 ARGV 中存储了哪些命令行参数
    BEGIN {  for(x=0;x<ARGC;x++)
             print ARGV[x]
             print ARGC
    }
```

argv.awk 只有 BEGIN 字段，其中的语句利用 for 循环打印 ARGC 所有的元素，for 循环结束后，打印 ARGC。例 4-64 给出了 argv.awk 的执行结果，由于 argv.awk 未加#!符号，因此需要用 awk -f 进行调用，后面加上 xyz、n=99 和“Hello World”三个参数，从结果可以看到，

混混藏书阁 http://book-life.blog.163.com

ARGV[0]中存储的是 awk，即执行该脚本的程序名，ARGV[1]～ARGV[3]是上述的三个参数，ARGC=4。注意，-f 选项不在 ARGV 中。因此，一般来说，ARGC=2，ARGV[0]为 awk、ARGV[1]为输入文件名，如果没有输入文件，则 ARGC=1。

```
 alloy@alloy:~/linuxshell/CH04$ awk -f argv.awk xyz n=99 "Hello World"
awk
xyz
n=99
Hello World
4
alloy@alloy:~/linuxshell/CH04$
```

接下来，举一个稍微复杂的例子以说明 ARGV 的应用，我们需要对 sturecord 文件中的电话号码进行检索，即输入学生姓名，系统响应输出其电话号码。新建名为 findphone 的脚本文件，内容如下：

```
#例 4-65: findphone.awk 脚本演示如何应用 ARGV 查找姓名所对应的电话号码
#!/bin/awk -f
BEGIN {FS=",";
      #判断是否已经输入姓名
      if(ARGC>2) {
                  name=ARGV[1];
                  delete ARGV[1] }
      else {
            #若没有输入姓名，提示输入姓名
            while(!name){print "Pls. Enter a name";
                        getline name< "-"}
           }
     }
$1~name {print $1,$3}
```

findphone.awk 脚本的 BEGIN 字段中，用 if 判断 ARGC 是否大于 2，如果大于 2，表示用户已经输入需要查找的姓名，将 ARGV[1]赋给 name 变量（ARGV[0]="awk"，所以，ARGV[1]中存储了姓名），若 ARGC 不大于 2，则说明此时未输入姓名，则利用一个循环提示输入姓名，利用 getline 函数将输入赋给 name 变量。主输入循环变量判断第 1 号域是否与 name 变量模糊匹配，若是，则输出第 1 域和第 3 域的值。例 4-65 显示了 findphone.awk 脚本的执行情况，第 1 条命令带有姓名字符串，ARGC=3，直接返回结果；第 2 条命令 ARGC=2，提示输入姓名字符串，然后返回查询结果。

```
#例 4-65 findphone.awk 脚本的执行结果
alloy@alloy:~/linuxshell/CH04$ chmod u+x findphone.awk
#第 1 条命令：ARGC=3 的情况
alloy@alloy:~/linuxshell/CH04$ ./findphone.awk Zhu sturecord
Zhu Lin 025-83680010
#第 2 条命令：ARGC=2 的情况
alloy@alloy:~/linuxshell/CH04$ ./findphone.awk sturecord
Pls. Enter a name
Li
Li Hao 025-83481010
Zhu Lin 025-83680010
alloy@alloy:~/linuxshell/CH04$
```

ENVIRON 变量存储了 Linux 操作系统的环境变量，例 4-66 打印出 ENVIRON 数组的所有内容，其结果类似于 Linux 的 set 命令，它显示了当前系统所定义的所有环境变量。

```
#例 4-66: 打印 ENVIRON 数组
alloy@alloy:~/linuxshell/CH04$ awk '
> BEGIN { for (i in ENVIRON)
> print i "=" ENVIRON[i]}'
AWKPATH=.:/usr/share/awk
SSH_ASKPASS=/usr/libexec/openssh/gnome-ssh-askpass
SELINUX_LEVEL_REQUESTED=
OLDPWD=/root
SELINUX_ROLE_REQUESTED=
LANG=zh_CN.GB18030
HISTSIZE=1000
CVS_RSH=ssh
USER=root
_=/bin/awk
QTLIB=/usr/lib/qt-3.3/lib
SELINUX_USE_CURRENT_RANGE=
TERM=vt100
……
alloy@alloy:~/linuxshell/CH04$
```

从例 4-66 也可以看出，ENVIRON 的索引是环境变量名，因此，我们也可以通过环境变量名直接得到其值，如下面的用法：

```
ENVIRON["AWKPATH"]
ENVIRON["GRIDSIM"]
```

注意，环境变量名需要用双引号引起。当然，也可以通过 ENVIRON 数组改变环境变量的值，如：

```
ENVIRON["AWKPATH"]=/bin/gawk
```

4.5 本章小结

本章介绍了 sed 命令和 awk 编程，sed 用于流编辑，它将一系列的编辑命令作用于缓冲区中输入文件的副本，从而实现对输入文件的各种编辑操作。而 awk 的一大显著特点是处理结构化文件。所谓结构化文件，是指划分为记录和域的文件，并且 awk 提供 printf 语句能生成格式化报表。

有关 sed 和 awk 处理管道流的用法将在第 10 章介绍。读者如果需要进一步学习 sed 和 awk，建议参考 O'Reilly & Associates 于 2000 年出版的，Arnold Robbins 编著的 *sed & awk Pocket Reference* 一书。

4.6 上机提议

1．利用 sed 命令将 input 文件中的\OU 字符串修改为(ou)，在此基础上，将该 sed 命令改写为两种 sed 脚本，利用 sed 的第 2、3 种调用方式实现同样的目的。

2．利用 sed 命令打印 input 文件中除第 3～8 行之外的所有行，在以下三种不同选项的组

混混藏书阁 http://book-life.blog.163.com

合下运行该命令：（1）不带任何选项；（2）只带-n 选项；（3）同时带-n 和-p 选项，并分析以上三种不同选项组合的区别。

3.用两个不同的命令实现如下功能：将 input 文件中的\OU 字符串修改为(ou)，并在与\OU 的匹配行后追加“We find \OU!”字符串。

4．利用 sed 命令在/etc/passwd 中分别查找满足以下条件的行：（1）o 字符重复任意次；（2）o 字符重复一次以上；（3）o 字符重复两次以上。

5．将 input 文件中的 abcde 字符分别用 EDCBA 替换，并将替换后的文件覆盖原文件。

6．将/etc/passwd 文件中与 root 相匹配的行写入保存缓冲区，并与 globus 相匹配的行进行互换，最后输出保存缓冲区的内容。

7．利用 awk 的三种调用方式输出 sturecord 的所有域，输出格式如下：

```
行号: Li Hao:njue:025-83481010:85:92:78:94:88:该行包含的域数量
```

8．利用 awk 命令在/etc/passwd 中分别查找满足以下条件的行：（1）o 字符重复任意次；（2）o 字符重复一次以上；（3）o 字符重复两次以上。

9．利用 awk 命令将 2010 分别转换为八进制和十六进制后输出。

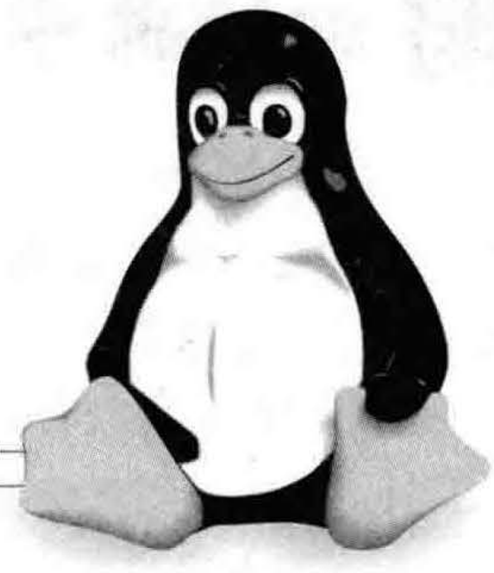
10．利用 awk 命令输出 sturecord 文件中学生成绩总和小于 450 的学生姓名、学校。

11．定义数组 data 为以下一列数：4，90，6.9，8，10，1，60，7.8，9.3。利用 awk 将 data 非递减进行排序，排序算法可选择任意一种，下面给出选择排序的 C++代码供读者参考：

```
void selectsort (int a[],const int n) {
   for (int i=0;i<n-1;i++) {
      int k=i;    //在a[i]到a[n-1]中，查找最小的数，放在a[k]中
      for(int j=i+1;j<n;j++)
        if(a[j]<a[k]) k=j;          //k为当前的最小数
      int temp=a[i]; a[i]=a[k]; a[k]=temp;          //交换a[i]和a[k]
   }
}
```

12．使用 sed 和 awk 两种方式，不区分大小写匹配 certificate，并将 certificate 替换为 lisence。

13．编写 awk 脚本，针对/etc/passwd 文件，实现用户名到行号、用户根目录之间的查找功能，即脚本的输入为表示用户名的字符串，输出为该用户名所对应的行号及其根目录路径。



第5章

文件的排序、合并和分割

Linux 文本处理命令是 Shell 编程中的常用命令，文本处理包含对文件记录的排序、文件的合并和分割等。本章介绍一些 Linux 文本处理命令，包含 sort 命令、uniq 命令、join 命令、cut 命令、paste 命令、split 命令、tr 命令和 tar 命令，这些命令能实现对文件记录排序、统计、合并、提取、粘贴、分割、过滤、压缩和解压等功能，它们与第 4 章介绍的 sed 和 awk 构成了 Linux 文本处理的所有命令和工具。

混混藏书阁 http://book-life.blog.163.com

5.1 sort 命令

计算机系统中存储的很多文本文件记录了大量的数据记录，如字典、图书馆书目、电话簿等文件，这些文件中的数据记录如果不按顺序排列，将使用户难以阅读，更难以查找到所需的信息。反之，如果将这些数据记录排序，则更易于程序化并提高处理的效率。Linux 的 sort 命令就是一种对文件排序的工具，sort 命令的功能十分强大，是 Shell 脚本编程时常用的文件排序工具。

sort 命令将输入文件看作由多条记录组成的数据流，而记录由可变宽度的字段组成，以换行符作为定界符。sort 命令与 awk 一样，可将记录分成多个域进行处理，默认的域分隔符是空格符，当然，域分隔符也可由用户指定其他符号。sort 命令的基本格式为：

```
sort [选项] [输入文件]
```

sort 命令的选项有很多，我们先将这些选项及其意义列在表 5-1 中，然后展开论述 sort 命令选项。

表 5-1　sort 命令选项及其意义

选　项	意　义
-c	测试文件是否已经被排序
-k	指定排序的域
-m	合并两个已排序的文件
-n	根据数字大小进行排序
-o [输出文件]	将输出写到指定的文件，相当于将输出重定向到指定文件
-r	将排序结果逆向显示
-t	改变域分隔符
-u	去除结果中的重复行

5.1.1　sort 命令的基本用法

sort 命令包含很多选项，比较复杂，下面分几个方面来介绍 sort 命令的基本用法。

1．-t 选项

sort 命令是分域对文件进行排序的，默认的域分隔符是空格符，-t 选项可用于设置分隔符。下面先看一个最简单的 sort 命令的例子，我们新建一个名为 CARGO.db 的文件，用于记录笔记本品牌、产地、价格、年代、型号等信息，各域间用冒号分隔。

```
#例 5-1：sort 命令用-t 选项设置分隔符
alloy@alloy:~/linuxshell/CH05$ cat CARGO.db                    #查看 CARGO.db 文件的内容
ThinkPad:USA:14000:2009:X301
ThinkPad:HongKong:10000:2008:T400
ThinkPad:USA:8000:2007:X60
HP:China:5600:2010:DM3
HP:China:12000:2010:NE808
SumSung:Korea:5400:2009:Q308
```

```
IdeaPad:China:8000:2007:U450
Acer:Taiwan:8000:2010:PT210
alloy@alloy:~/linuxshell/CH05$ sort -t: CARGO.db     #以默认方式对 CARGO.db 文件排序
Acer:Taiwan:8000:2010:PT210
HP:China:12000:2010:NE808
HP:China:5600:2010:DM3
IdeaPad:China:8000:2007:U450
SumSung:Korea:5400:2009:Q308
ThinkPad:HongKong:10000:2008:T400
ThinkPad:USA:14000:2009:X301
ThinkPad:USA:8000:2007:X60
alloy@alloy:~/linuxshell/CH05$
```

上述例 5-1 中 CARGO.db 文件是用冒号分隔域的，上例中使用 sort 命令对 CARGO.db 文件进行排序，用-t 选项指定域分隔符为冒号，注意，-t 与“:”之间可以有空格，也可以没有空格。sort 命令默认根据第 1 域对数据记录进行排序，如果第 1 域相同，再根据第 2 域排序，以此类推，如第 1 域为 ThinkPad 的 3 条记录，第 2 域 HongKong 的记录排在最前面（HongKong 开头字母“H”在 USA 开头字母“U”之前），第 2 域为 USA 的 2 条记录又按照第 3 域进行了排序。

当未指定-t 时，分隔符是空格符，这时记录内开头与结尾的空格都将被忽略；当用-t 选项改变分隔符时，空格符变得有意义，例如，对于下面的这条数据记录，若不指定-t 选项，这条记录只有一个域，为:root:，记录前后的空格符都被忽略了；若用-t 选项指定冒号，这条记录就包含了三个域，第 1 和 3 域是空格符，第 2 域是:root:。

```
空格符:root:空格符
```

2. -k 选项

从上例可以看出，sort 命令默认情况下是按第 1 域进行排序的，也可以指定某个域进行排序，-k 选项就是用于指定域的。sort 命令以 1 表示第 1 域、以 2 表示第 2 域，以此类推。CARGO.db 文件记录的第 3 域是价格，下面的例 5-2 给出按第 3 域对 CARGO.db 文件记录排序的命令：

```
#例 5-2：sort 命令用-k 选项指定排序的域号
alloy@alloy:~/linuxshell/CH05$ sort -t: -k3 CARGO.db      #根据第 3 域对 CARGO.db 排序
ThinkPad:HongKong:10000:2008:T400
HP:China:12000:2010:NE808
ThinkPad:USA:14000:2009:X301
SumSung:Korea:5400:2009:Q308
HP:China:5600:2010:DM3
IdeaPad:China:8000:2007:U450
ThinkPad:USA:8000:2007:X60
Acer:Taiwan:8000:2010:PT210
alloy@alloy:~/linuxshell/CH05$
```

例 5-2 的命令利用-k3 指定了第 3 域，注意，-k 和 3 之间也没有空格，从命令的执行结果可以看出，尽管第 3 域是数字，但是 sort 命令并未以数字大小来排序，而是仍然以字符串方式进行排序的，如第 3 域以 1 开始的所有记录排在最前面。如果几条记录的第 3 域相同，sort 命令再依次以第 4 域、第 5 域……进行排序。

3. -n 选项

如果需要根据笔记本价格从小到大的方式对 CARGO.db 文件进行排序，即以第 3 域的数

混混藏书阁 http://book-life.blog.163.com

字大小对 CARGO.db 文件排序，这时就需要使用-n 选项，-n 选项可以指定根据数字大小进行排序。下面给出的例 5-3 演示了-n 选项的用法：

```
#例 5-3：sort 命令用-n 选项设置根据数值大小排序
alloy@alloy:~/linuxshell/CH05$ sort -t: -k3n CARGO.db     #根据第 3 域的数字大小排序
SumSung:Korea:5400:2009:Q308
HP:China:5600:2010:DM3
Acer:Taiwan:8000:2010:PT210
IdeaPad:China:8000:2007:U450
ThinkPad:USA:8000:2007:X60
ThinkPad:HongKong:10000:2008:T400
HP:China:12000:2010:NE808
ThinkPad:USA:14000:2009:X301
alloy@alloy:~/linuxshell/CH05$
```

-n 选项不是单独使用的，一般放在域号之后，例 5-3 命令中的-k3n 就指定了以第 3 域排序，且不是按照第 3 域的字符串顺序排序，而是按第 3 域的数字大小进行排序的。从命令的执行结果可以看出，第 3 域最小的 5400 排在最前面，最大的 14000 排在最后，如果按字母排序，14000 是排在 5400 前面的。

4．-r 选项

-r 选项用于将排序结果逆向显示。如果需要根据笔记本价格从大到小的方式来对 CARGO.db 文件进行排序，就可以利用-k3n 选项将 CARGO.db 文件的第 3 域从小到大排序，然后用-r 选项将结果逆向显示，下面的例 5-4 给出这条命令及其结果：

```
#例 5-4：sort 命令用-r 选项将排序结果逆向
alloy@alloy:~/linuxshell/CH05$ sort -t: -k3nr CARGO.db
ThinkPad:USA:14000:2009:X301
HP:China:12000:2010:NE808
ThinkPad:HongKong:10000:2008:T400
Acer:Taiwan:8000:2010:PT210
IdeaPad:China:8000:2007:U450
ThinkPad:USA:8000:2007:X60
HP:China:5600:2010:DM3
SumSung:Korea:5400:2009:Q308
alloy@alloy:~/linuxshell/CH05$
```

例 5-4 命令使用-k3nr 实现了第 3 域从大到小的排序，其结果恰好是-k3n 排序结果的倒置。

5．-u 选项

-u 选项用于去除排序结果中的重复行，如果在 CARGO.db 文件中输入几条重复记录，然后利用 sort-u 对该文件进行排序。下面的例 5-5 通过比较使用-u 选项前后的结果来说明-u 选项的意义：

```
#例 5-5：sort 命令-u 选项的用法
alloy@alloy:~/linuxshell/CH05$ sort -t: CARGO.db      #根据第 1 域对 CARGO.db 排序
Acer:Taiwan:8000:2010:PT210
HP:China:12000:2010:NE808
HP:China:5600:2010:DM3
IdeaPad:China:8000:2007:U450
SumSung:Korea:5400:2009:Q308                          #重复记录
SumSung:Korea:5400:2009:Q308
ThinkPad:HongKong:10000:2008:T400
ThinkPad:USA:14000:2009:X301                          #重复记录
ThinkPad:USA:14000:2009:X301
```

```
ThinkPad:USA:14000:2009:X301
ThinkPad:USA:8000:2007:X60
alloy@alloy:~/linuxshell/CH05$ sort -t: -u CARGO.db        #利用-u 选项对 CARGO.db 排序
Acer:Taiwan:8000:2010:PT210
HP:China:12000:2010:NE808
HP:China:5600:2010:DM3
IdeaPad:China:8000:2007:U450
SumSung:Korea:5400:2009:Q308
ThinkPad:HongKong:10000:2008:T400
ThinkPad:USA:14000:2009:X301
ThinkPad:USA:8000:2007:X60                                 #重复记录已经去除
alloy@alloy:~/linuxshell/CH05$
```

例 5-5 中未加-u 选项时，结果中存在 2 条“SumSung:Korea:5400:2009:Q308”和 3 条“ThinkPad:USA:14000:2009:X301”重复记录，命令加上-u 选项之后，结果中的重复记录已经去除。

6. -o 选项

sort 命令默认将排序后的结果输出到屏幕上，如果需要将结果保存到另一个文件中，可以使用-o 选项加文件名来完成，下面给出的例 5-6 演示了-o 选项的用法：

```
#例 5-6: sort 命令-o 选项的用法
#将 CARGO.db 按第 3 域的数值大小排序，并将排序结果存储到 SORT_CARGO.db 文件中
alloy@alloy:~/linuxshell/CH05$ sort -t: -k3n -o SORT_CARGO.db CARGO.db
alloy@alloy:~/linuxshell/CH05$ cat SORT_CARGO.db           #查看 SORT_CARGO.db 文件内容
SumSung:Korea:5400:2009:Q308
HP:China:5600:2010:DM3
Acer:Taiwan:8000:2010:PT210
IdeaPad:China:8000:2007:U450
ThinkPad:USA:8000:2007:X60
ThinkPad:HongKong:10000:2008:T400
HP:China:12000:2010:NE808
ThinkPad:USA:14000:2009:X301
alloy@alloy:~/linuxshell/CH05$
```

例 5-6 根据第 3 域的数值大小对 CARGO.db 文件排序，并利用-o SORT_CARGO.db 将排序后的结果存储到 SORT_CARGO.db 文件，此时，sort 命令执行后就不在屏幕上显示任何信息，但是，查看 SORT_CARGO.db 文件内容后发现，该文件确实存储了 CARGO.db 排序后的记录。

实际上，-o 选项的功能与 Shell 提供的 I/O 重定向功能一样。第 10 章将详细介绍 I/O 重定向的用法，在此，读者仅需了解若需要保存 sort 命令排序后的结果，则使用-o 选项。

7. -c 选项

-c 选项用于测试文件是否已经排好序，下面给出的例 5-7 演示了-c 选项的用法：

```
#例 5-7: sort 命令-c 选项的用法
#第 1 条命令：测试 CARGO.db 是否按默认方式排序
alloy@alloy:~/linuxshell/CH05$ sort -t: -c CARGO.db
sort: CARGO.db:2: disorder: ThinkPad:HongKong:10000:2008:T400

#第 2 条命令：测试 SORT_CARGO.db 是否按默认方式排序
alloy@alloy:~/linuxshell/CH05$ sort -t: -c SORT_CARGO.db
sort: SORT_CARGO.db:2: disorder: HP:China:5600:2010:DM3
```


混混藏书阁 http://book-life.blog.163.com

```
#第 3 条命令：测试 SORT_CARGO.db 是否按第 3 域的数值大小排序
alloy@alloy:~/linuxshell/CH05$ sort -t: -k3n  -c SORT_CARGO.db
alloy@alloy:~/linuxshell/CH05$
```

例 5-7 的第 1 条命令测试 CARGO.db 是否按默认方式排序，Shell 提示“sort: CARGO.db:2: disorder: ThinkPad:HongKong:10000:2008:T400”，这说明 CARGO.db 未排序，Shell 提示信息中包含了该文件中第 1 条未排序的记录；第 2 条命令测试 SORT_CARGO.db 是否按默认方式排序，当然该文件也未排序；第 3 条命令测试 SORT_CARGO.db 是否按第 3 域的数值大小排序，显然，按这种方式 SORT_CARGO.db 已经排好序，Shell 不提示任何信息。

8. -m 选项

-m 选项用于将两个排好序的文件合并成一个排好序的文件，在文件合并前，它们必须已经排好序。-m 选项对未排序的文件合并是没有任何意义的，下面通过一个例子来说明-m 选项的效果：

```
#例 5-8：sort 命令-m 选项的用法
alloy@alloy:~/linuxshell/CH05$ cat CARGO2.db                    #查看 CARGO2.db 文件
DELL:USA:6700:2009:XPS
MACBOOK:USA:10198:2010:MB991ZP/A
alloy@alloy:~/linuxshell/CH05$ cat SORT_CARGO.db                #查看 SORT_CARGO.db 文件
Acer:Taiwan:8000:2010:PT210
HP:China:12000:2010:NE808
HP:China:5600:2010:DM3
IdeaPad:China:8000:2007:U450
SumSung:Korea:5400:2009:Q308
ThinkPad:HongKong:10000:2008:T400
ThinkPad:USA:14000:2009:X301
ThinkPad:USA:8000:2007:X60
alloy@alloy:~/linuxshell/CH05$ sort -t: -m CARGO2.db SORT_CARGO.db    #合并两个文件
Acer:Taiwan:8000:2010:PT210
DELL:USA:6700:2009:XPS                                          #来自于 CARGO2.db 文件
HP:China:12000:2010:NE808
HP:China:5600:2010:DM3
IdeaPad:China:8000:2007:U450
MACBOOK:USA:10198:2010:MB991ZP/A                                #来自于 CARGO2.db 文件
SumSung:Korea:5400:2009:Q308
ThinkPad:HongKong:10000:2008:T400
ThinkPad:USA:14000:2009:X301
ThinkPad:USA:8000:2007:X60
alloy@alloy:~/linuxshell/CH05$
```

例 5-8 首先创建了两个文件：CARGO2.db 和 SORT_CARGO.db，这两个文件都按第 1 域排好序，然后，使用-m 选项将这两个文件合并，得到两个文件的记录合并到一起的结果，但是，结果中的记录仍按第 1 域排好序，CARGO2.db 文件的两条记录被有序地插到了 SORT_CARGO.db 文件的记录中。

sort 命令的选项远不止本节所介绍的内容，但是，由于新版本的 sort 命令删掉了很多选项的功能，而且这八个选项是最常用的选项。因此，本节不再介绍 sort 命令的其他选项。

5.1.2 sort 和 awk 的联合用法

sort 和 awk 都是分域处理文件的工具，两者结合起来可以有效地对文本块进行排序。文

本块是指由多行记录组合而成的数据块，如下面的文件：

```
alloy@alloy:~/linuxshell/CH05$ cat PROFESSOR.db
J Luo                                                    #每个文件块由姓名、学校名和地址组成
Southeast University
Nanjing,China

Y Zhang
Victory University
Melbourne, Australia

D Hou
Beijing University
Beijing,China

B Liu
Shanghai Jiaotong University
Shanghai,China

C Lin
University of Toronto
Toronto,Canada
alloy@alloy:~/linuxshell/CH05$
```

PROFESSOR.db 中每个文件块记录了一位教授的信息，由三行组成：第 1 行是姓名，第 2 行是学校名，第 3 行是学校所处的城市和国家，如果需要根据姓名对文件块进行排序，仅使用 sort 命令是难以实现的，我们通过结合使用 sort 和 awk 来实现这一功能，如例 5-9 所示：

```
#例 5-9：利用 sort 和 awk 实现文件块的排序
alloy@alloy:~/linuxshell/CH05$ cat PROFESSOR.db |
awk -v RS="" '{gsub("\n","@");print}' |          #将每个文件块合并到一行
sort |                                           #对每行的记录排序
awk -v ORS="\n\n" '{gsub("@","\n");print}'       #将排序后的行分块打印

B Liu
Shanghai Jiaotong University
Shanghai,China

C Lin
University of Toronto
Toronto,Canada

D Hou
Beijing University
Beijing,China

J Luo
Southeast University
Nanjing,China

Y Zhang
Victory University
Melbourne, Australia

alloy@alloy:~/linuxshell/CH05$
```


混混藏书阁 http://book-life.blog.163.com

例 5-9 的命令看起来很复杂，由 4 条命令和 3 个管道符组成，cat 命令将 PROFESSOR.db 文件的内容作为第 1 条 awk 命令的输入数据，该 awk 命令将每个文件块合并到一行，并用 gsub 函数将换行符替换成@符号，例如，PROFESSOR.db 的第 1 个文件块将变为“J Luo@ Southeast University@Nanjing,China@”，sort 命令对这种格式的记录进行排序，默认以第 1 域排序，即姓名的字母顺序，并将排序后的行作为第 2 条 awk 命令的输入数据，第 2 条 awk 命令执行与第 1 条 awk 相反的功能，它将并为一行的数据划分为类似于 PROFESSOR.db 中的文件块输出，实现方法是用 gusb 函数将@符号替换成换行符。上述命令得到根据姓名的字母顺序排好序的文件块。

sort 命令是 Linux 中经常使用的命令之一，从上面两节的介绍来看，sort 的功能强大，且用法十分灵活，因篇幅所限，我们对 sort 命令的介绍就到此为止。读者若需了解 sort 的更多用法，不可在 Shell 中输入 man sort，参考 Linux 系统中 sort 命令的 manual page。

5.2 uniq 命令

uniq 命令用于去除文本文件中的重复行，这类似于 sort 命令的-u 选项，但是，uniq 命令和 sort -u 是存在一些区别的，请看下面的示例：

```
#例 5-10：uniq 命令的基本用法
alloy@alloy:~/linuxshell/CH05$ cat CARGO3.db          #查看 CARGO.db 文件的内容
ThinkPad:USA:14000:2009:X301                          #重复三行
ThinkPad:USA:14000:2009:X301
ThinkPad:USA:14000:2009:X301
HP:China:5600:2010:DM3
SumSung:Korea:5400:2009:Q308
ThinkPad:USA:14000:2009:X301                          #隔几行，再重复一次
IdeaPad:China:8000:2007:U450
Acer:Taiwan:8000:2010:PT210
Acer:Taiwan:8000:2010:PT210
alloy@alloy:~/linuxshell/CH05$ uniq CARGO3.db         #用 uniq 命令去除重复行
ThinkPad:USA:14000:2009:X301                          #已经去除重复行
HP:China:5600:2010:DM3
SumSung:Korea:5400:2009:Q308
ThinkPad:USA:14000:2009:X301                          #该行没有被去除
IdeaPad:China:8000:2007:U450
Acer:Taiwan:8000:2010:PT210
alloy@alloy:~/linuxshell/CH05$ sort -u CARGO3.db      #用 sort -u 去除重复行
Acer:Taiwan:8000:2010:PT210
HP:China:5600:2010:DM3
IdeaPad:China:8000:2007:U450
SumSung:Korea:5400:2009:Q308
ThinkPad:USA:14000:2009:X301                          #所有的重复行都被去除
alloy@alloy:~/linuxshell/CH05$
```

CARGO3.db 文件中的“ThinkPad:USA:14000:2009:X301”记录先重复三次，隔开两行后，该行重复出现一次，用 uniq 命令去除 CARGO3.db 文件重复行后发现，连续出现的三条重复“ThinkPad:USA:14000:2009:X301”记录仅剩下一条，而隔开两行出现的该记录却未被去除，

而用 sort –u 命令时，所有的重复记录都被去掉。因此，uniq 命令去除的重复行必须是连续重复出现的行，中间不能夹杂任何其他文本行。

uniq 命令有 3 个选项，我们将 uniq 命令选项及其意义列于表 5-2 中。

表 5-2　uniq 命令选项及其意义

选　项	意　义
-c	打印每行在文本中重复出现的次数
-d	只显示有重复的记录，每个重复记录只出现一次
-u	只显示没有重复的记录

uniq 命令的-c 选项打印每行在文本中重复出现的次数，下面举一个例子来说明其用法：

```
#例 5-11: uniq 命令-c 选项的用法
alloy@alloy:~/linuxshell/CH05$ uniq -c CARGO3.db
      3 ThinkPad:USA:14000:2009:X301
      1 HP:China:5600:2010:DM3
      1 SumSung:Korea:5400:2009:Q308
      1 ThinkPad:USA:14000:2009:X301
      1 IdeaPad:China:8000:2007:U450
      2 Acer:Taiwan:8000:2010:PT210
alloy@alloy:~/linuxshell/CH05$
```

例 5-10 的命令列出了 CARGO3.db 文件中每个记录所出现的次数，记录“ThinkPad:USA:14000:2009:X301”先出现 3 次，隔两行再出现 1 次，而记录“Acer:Taiwan:8000:2010:PT210”出现 2 次，其他记录都出现 1 次。

uniq 命令的-d 和-u 选项正好相反，-d 选项用于显示有重复的记录，而-u 选项显示没有重复的记录，下面的例 5-12 说明了这两个选项的意义：

```
#例 5-12: uniq 命令-m 选项的用法
alloy@alloy:~/linuxshell/CH05$ uniq -d CARGO3.db          #显示有重复的记录
ThinkPad:USA:14000:2009:X301
Acer:Taiwan:8000:2010:PT210
alloy@alloy:~/linuxshell/CH05$ uniq -u CARGO3.db          #显示没有重复的记录
HP:China:5600:2010:DM3
SumSung:Korea:5400:2009:Q308
ThinkPad:USA:14000:2009:X301
IdeaPad:China:8000:2007:U450
alloy@alloy:~/linuxshell/CH05$
```

uniq -d 显示了 CARGO3.db 文件的重复记录，共有两条，每一条显示一次；而 uniq –u 显示了 CARGO3.db 文件没有重复的行，共有四条记录，其中的“ThinkPad:USA:14000:2009:X301”记录是指隔开两行又出现的那条记录。

下面给出一个脚本，这个脚本结合使用 sort 和 uniq 命令，可以统计一个文件内每种单词出现的次数，这个脚本的名字是 count_word.sh，内容如下：

```
#count_word.sh: 该脚本统计文件中单词出现的次数
#!/bin/bash

ARGS=1
E_BADARGS=55
E_NOFILE=56
```


混混藏书阁 http://book-life.blog.163.com

```
#以下的 if/then 结构用于判断执行脚本时是否带了输入参数（即需要统计的文件名）
#如果未带输入参数，则返回 55 错误码
if [ $# -ne "$ARGS" ]
then
  echo "Usage: `basename $0` filename"
  exit $E_BADARGS
fi

#以下的 if/then 结构用于判断在当前目录下，输入的文件名是否存在
#若该文件不存在，则返回 56 错误码
if [ ! -f "$1" ]
then
  echo "File \"$1\" does not exits."
  exit $E_NOFILE
fi

#以下是统计文件单词数的核心命令
# sed 命令用于过滤句号、逗号、分号，当然可继续加上需要过滤的符号
#sed 命令的第 4 个-e 选项将单词间的空格转化为换行符
#sort 对 sed 过滤后的结果排序，每行一个单词
#uniq -c 输出重复行出现的次数，sort -nr 按照出现频率从大到小排序
sed -e 's/\.//g' -e 's/\,//g' -e 's/\://g' -e 's/ /\n/g' "$1" | sort | uniq -c | sort
-nr

exit 0
```

count_word.sh 脚本使用了多种 Shell 编程技巧，未读后续章节的读者可能很难全部理解该脚本，但是，并不影响理解统计文件单词数的核心命令。我们对两个 if/then 结构的功能做简单介绍，但不介绍其原理。count_word.sh 脚本运行时需要加上一个参数，该参数表示待统计单词出现频率的文件名，因此，第 1 个 if/then 结构用于判断执行脚本时是否带了输入参数，若未带输入参数，则返回 55 错误码；第 2 个 if/then 结构用于判断在当前目录下，输入的文件名是否存在，若该文件不存在，则返回 56 错误码。这两个 if/then 结构只是对脚本执行的异常情况进行处理，真正用于统计文件单词出现频率的命令只有一条，它由 sed、sort、uniq -c 和 sort -nr 四个命令组成，前面命令的输出作为后面命令的输入，sed 命令的前 3 个-e 选项用于过滤掉句号、逗号、分号，当然可继续加上需要过滤的符号（只需多加一个类似的-e 选项即可），sed 命令的第 4 个-e 选项将单词间的空格转化为换行符，这就将每个单词单独一行显示，便于 sort 命令排序。经过 sed 命令处理后，sort 对行进行排序，同一个单词必定出现在连续行，uniq -c 输出重复行的次数，即该单词重复出现的次数。由于 uniq -c 的输出结果是“次数 单词”格式，sort -nr 对第 1 域（即单词出现的次数）进行排序，并倒置，使得出现次数多的单词排在最前面。下面给出 count_word.sh 脚本的执行结果：

```
#count_word.sh 脚本的执行结果
# count_word.sh 不带输入参数
alloy@alloy:~/linuxshell/CH05$ ./count_word.sh
Usage: count_word.sh filename

# count_word.sh 带的文件名不存在
alloy@alloy:~/linuxshell/CH05$ ./count_word.sh hh
File "hh" does not exits.
alloy@alloy:~/linuxshell/CH05$ cat WORDLIST                    #查看 WORDLIST 内容
```

```
hello, caicai. world: watch, world caicai hello message
message world watch hello into the he she last into.
last save hello caicai, world: message.

#统计出 WORDLIST 中所有单词出现的次数
alloy@alloy:~/linuxshell/CH05$ ./count_word.sh WORDLIST
      4 world                          #world 单词出现 4 次，以下行的意义类似此行
      4 hello
      3 message
      3 caicai
      2 watch
      2 last
      2 into
      1 the
      1 she
      1 save
      1 he
alloy@alloy:~/linuxshell/CH05$
```

上面首先给出 count_word.sh 异常情况的执行结果。再给出用 count_word.sh 统计 WORDLIST 中所有单词的出现次数，WORDLIST 包含若干单词，以及句号、逗号和分号等标点符号。从结果可以看到，这些标点符号被过滤掉，单词及其出现频率按从大到小顺序输出。

uniq 命令和带-u 参数的 sort 的区别说明如下：

- uniq 命令必须针对完全相同的行进行判断，而带-u 的 sort 命令可以针对域进行判断。
- uniq 命令只会处理连续的行，对不连续的行不会进行处理，而带-u 的 sort 命令针对全部行进行处理。

5.3 join 命令

join 命令用于实现两个文件中记录的连接操作，连接操作是关系数据库中的概念，在此我们不做正式的定义，简言之，连接操作将两个文件中具有相同域的记录选择出来，再将这些记录所有的域放到一行（包含来自两个文件的所有域）。首先，我们通过一个例子来观察 join 命令的效果：

```
#例 5-13：join 命令的基本用法
alloy@alloy:~/linuxshell/CH05$ cat TEACHER.db
B Liu:Shanghai Jiaotong University:Shanghai:China
C Lin:University of Toronto:Toronto:Canada
D Hou:Beijing University:Beijing:China
J Luo:Southeast University:Nanjing:China
Y Zhang:Victory University:Melbourne:Australia
alloy@alloy:~/linuxshell/CH05$ cat TEACHER_HOBBY.db
B Liu:Tea
C Lin:Song
J Cao:Pingpong
Q Cai:Shopping
Y Zhang:Photograhy
Z Wu:Chess
```


混混藏书阁 http://book-life.blog.163.com

```
#下面执行 join 操作，将域分隔符改成冒号
alloy@alloy:~/linuxshell/CH05$ join -t: TEACHER.db TEACHER_HOBBY.db
B Liu:Shanghai Jiaotong University:Shanghai:China:Tea
C Lin:University of Toronto:Toronto:Canada:Song
Y Zhang:Victory University:Melbourne:Australia:Photograhy
alloy@alloy:~/linuxshell/CH05$
```

例 5-13 中有两个文件，TEACHER.db 文件的记录包含 4 个域，分别是姓名、学校、城市和国家，TEACHER_HOBBY.db 文件的记录只有 2 个域，是姓名和爱好。这两个文件是已排序的，join 命令只能对已排序的文件进行操作，对这两个文件执行 join 命令后，出现 3 条记录，如：“B Liu:Shanghai Jiaotong University:Shanghai:China:Tea”记录，包含 5 个域，第 1 个域是两个文件的共同域，第 2～4 个域来自 TEACHER.db 文件，第 5 个域来自 TEACHER_HOBBY.db 文件，join 命令使得具有共同域 B Liu 的两条记录连接到了一起。上例中 join 命令后跟-t 选项，与 sort 命令一样，-t 选项用于改变分隔符，除了-t 选项，join 命令还有很多其他选项，如表 5-3 所示。

表 5-3　join 命令选项及其意义

选　　项	意　　义
-a1 或-a2	除了显示以共同域进行连接的结果外，-a1 表示还显示第 1 个文件中没有共同域的记录，-a2 则表示显示第 2 个文件中没有共同域的记录
-i	比较域内容时，忽略大小写差异
-o	设置结果显示的格式
-t	改变域分隔符
-v1 或-v2	与-a 选项类似，但是，不显示以共同域进行连接的结果
-1 和-2	-1 用于设置文件 1 用于连接的域，-2 用于设置文件 2 用于连接的域

join 命令的基本语法为：

```
join [选项] 文件 1 文件 2
```

1．-a 和-v 选项

当两个文件进行连接时，文件 1 中的记录可能在文件 2 中找不到共同域，反过来，文件 2 中也可能存在这样的记录，join 命令的结果默认是不显示这些未进行连接的记录的。-a 和-v 选项就是用于显示这些未进行连接的记录，-a1 和-v1 指显示文件 1 中未连接的记录，而-a2 和-v2 指显示文件 2 中的未连接记录。-a 和-v 选项的区别在于：-a 选项显示以共同域进行连接的结果，而-v 选项则不显示这些记录。

首先，举例说明-a 选项，例子仍会用到 TEACHER.db 和 TEACHER_HOBBY.db 两个文件。

```
#例 5-14：join 命令-a 选项的用法
alloy@alloy:~/linuxshell/CH05$ join -t: -a1 TEACHER.db TEACHER_HOBBY.db
B Liu:Shanghai Jiaotong University:Shanghai:China:Tea
C Lin:University of Toronto:Toronto:Canada:Song
D Hou:Beijing University:Beijing:China                    #未进行连接的文件 1 中的记录
J Luo:Southeast University:Nanjing:China                  #未进行连接的文件 1 中的记录
Y Zhang:Victory University:Melbourne:Australia:Photograhy
alloy@alloy:~/linuxshell/CH05$ join -t: -a2 TEACHER.db TEACHER_HOBBY.db
B Liu:Shanghai Jiaotong University:Shanghai:China:Tea
```

```
C Lin:University of Toronto:Toronto:Canada:Song
J Cao:Pingpong                                          #未进行连接的文件 2 中的记录
Q Cai:Shopping                                          #未进行连接的文件 2 中的记录
Y Zhang:Victory University:Melbourne:Australia:Photograhy
Z Wu:Chess                                              #未进行连接的文件 2 中的记录
alloy@alloy:~/linuxshell/CH05$
```

例 5-14 的两条命令分别用-a1 和-a2 对 TEACHER.db 和 TEACHER_HOBBY.db 进行连接，结果除了显示成功连接的记录外，还分别显示 TEACHER.db 和 TEACHER_HOBBY.db 中未进行连接的记录。如果要同时显示两个文件中未进行连接的记录，join 命令可以同时带上-a1 和-a2 两个选项。

然后，举一个-v 选项的例子，从而得出-a 选项和-v 选项的区别：

```
#例 5-15：join 命令-v 选项的用法
alloy@alloy:~/linuxshell/CH05$ join -t: -v1 TEACHER.db TEACHER_HOBBY.db
D Hou:Beijing University:Beijing:China              #只显示未进行连接的文件 1 中的记录
J Luo:Southeast University:Nanjing:China
alloy@alloy:~/linuxshell/CH05$ join -t: -v2 TEACHER.db TEACHER_HOBBY.db
J Cao:Pingpong                                      #只显示未进行连接的文件 2 中的记录
Q Cai:Shopping
Z Wu:Chess
alloy@alloy:~/linuxshell/CH05$
```

比较例 5-14 和例 5-15，-a 选项和-v 选项的区别显而易见，-v 选项只显示两个文件中未进行连接的记录。

2. -o 选项

join 命令默认显示连接记录在两个文件中的所有域，而且是按顺序来显示的。-o 选项用于改变结果显示的格式，我们可以指定显示哪几个域、按什么顺序显示这些域。下面举一个-o 选项的例子：

```
#例 5-16：join 命令-o 选项的用法
alloy@alloy:~/linuxshell/CH05$ join -t: -o1.1 2.2 1.2 TEACHER.db TEACHER_HOBBY.db
B Liu:Tea:Shanghai Jiaotong University
C Lin:Song:University of Toronto
Y Zhang:Photograhy:Victory University
alloy@alloy:~/linuxshell/CH05$
```

例 5-16 命令中使用了选项“-o1.1 2.2 1.2”，这表示显示格式依次显示第 1 个文件中的第 1 个域、第 2 个文件的第 2 个域、第 1 个文件的第 2 个域，结果确实显示了三个域，即姓名、爱好和学校。

3. -1 和-2 选项

join 命令默认比较文件 1 和文件 2 的第 1 域，如果我们需要通过其他域进行连接，就需要使用-1 和-2 选项，-1 用于设置文件 1 用于连接的域，-2 用于设置文件 2 用于连接的域。下面举例说明这两个选项的用法：

```
#例 5-17：join 命令-1 和-2 选项的用法
alloy@alloy:~/linuxshell/CH05$ cat AREACODE.db   #查看 AREACODE.db 文件内容
BEIJING:86010
HONGKONG:852
SHANGHAI:86021
TORONTO:001416

#对 TEACHER.db 按第 3 域排序，并将结果存储到 TEACHER1.db 文件
```


混混藏书阁 http://book-life.blog.163.com

```
alloy@alloy:~/linuxshell/CH05$ sort -t: -k3 -o TEACHER1.db TEACHER.db
alloy@alloy:~/linuxshell/CH05$ cat TEACHER1.db        #查看 TEACHER1.db 文件内容
D Hou:Beijing University:Beijing:China
Y Zhang:Victory University:Melbourne:Australia
J Luo:Southeast University:Nanjing:China
B Liu:Shanghai Jiaotong University:Shanghai:China
C Lin:University of Toronto:Toronto:Canada

#以文件 1 的第 3 域和文件 2 的第 1 域进行连接，并忽略大小写
alloy@alloy:~/linuxshell/CH05$ join -t: -i -1 3 -2 1 TEACHER1.db AREACODE.db
Beijing:D Hou:Beijing University:China:86010
Shanghai:B Liu:Shanghai Jiaotong University:China:86021
Toronto:C Lin:University of Toronto:Canada:001416
alloy@alloy:~/linuxshell/CH05$
```

在例 5-17 中，新建了 AREACODE.db 文件，其中存放了城市及其地区码。TEACHER.db 的第 3 域是城市名，为了将 TEACHER.db 的城市名与 AREACODE.db 的城市名进行连接，首先要将 TEACHER.db 按城市名排序，并将排序结果保存到 TEACHER1.db 文件。然后，join 命令使用“-1 3 -2 1”选项指定用于连接的域，“-1 3”表示文件 1 的第 3 域，同理，“-2 1”表示文件 2 的第 1 域。由于 AREACODE.db 的城市名是大写字母，因此，join 命令带上-i 选项表示在比较域时忽略大小写，join 的结果显示出城市名、姓名、学校、国家和地区码。

如果我们不将 TEACHER.db 按第 3 域排序，直接进行 join 操作，会出现什么结果呢？请看下面的例 5-18：

```
#例 5-18：TEACHER.db 未排序，join 命令的出错信息
alloy@alloy:~/linuxshell/CH05$ join -t: -i -1 3 -2 1 TEACHER.db AREACODE.db
Shanghai:B Liu:Shanghai Jiaotong University:China:86021
join: file 1 is not in sorted order                    #出错，文件 1 未排序
Toronto:C Lin:University of Toronto:Canada:001416
```

例 5-18 直接将 TEACHER.db 和 AREACODE.db 连接，join 命令出错，提示文件 1 未排序。因此，join 命令在对两个文件进行连接时，两个文件必须都是按照连接域排好序的，按其他域排序是无效的。

5.4 cut 命令

cut 命令用于从标准输入或文本文件中按域或行提取文本，cut 命令的基本格式为：

```
cut [选项] 文件
```

cut 命令的选项及其意义列于表 5-4 中。

表 5-4 cut 命令选项及其意义

选　项	意　义
-c	指定提取的字符数，或字符范围
-f	指定提取的域数，或域范围
-d	改变域分隔符

cut 命令的选项仅有三个，-c 用于按字符提取文本，-f 用于按域提取文本，-d 类似于 sort

和 join 命令的-t 选项，用于改变域分隔符。下面通过几个例子来介绍 cut 命令的用法。首先给出例 5-19：

```
#例 5-19：cut 命令-c 选项的用法
#TEACHER.db 文件同例 5-13
[root@jselab shell-book]# cut -c3 TEACHER.db          #提取 TEACHER.db 的第 3 个字符
L
L
H
L
Z
[root@jselab shell-book]# cut -c1-5 TEACHER.db        #提取 TEACHER.db 的第 1～5 个字符
B Liu
C Lin
D Hou
J Luo
Y Zha
[root@jselab shell-book]#
```

例 5-19 演示了 cut 命令的-c 选项，-c3 表示提取 TEACHER.db 的第 3 个字符，-c1-5 表示提取 TEACHER.db 的第 1～5 个字符。-c 后跟数字表示字符数或字符范围，共有三种表示方式：① -cn 表示第 n 个字符；② -cn,m 表示第 n 个字符和第 m 个字符；③ -cn-m 表示第 n 个字符到第 m 个字符。由于-c 选项是按字符提取文本的，因此，无须使用-d 改变域分隔符，但是，当使用-f 按域提取文本时，就需要使用-d 根据实际文本来设置域分隔符了，请看下面的例 5-20：

```
#例 5-20：cut 命令-d 和-f 选项的用法
[root@jselab shell-book]# cut -d: -f1,4 TEACHER.db   #提取 TEACHER.db 的第 1 域和第 4 域
B Liu:China
C Lin:Canada
D Hou:China
J Luo:China
Y Zhang:Australia
[root@jselab shell-book]# cut -d: -f1-3 TEACHER.db   #提取 TEACHER.db 的第 1～3 域
B Liu:Shanghai Jiaotong University:Shanghai
C Lin:University of Toronto:Toronto
D Hou:Beijing University:Beijing
J Luo:Southeast University:Nanjing
Y Zhang:Victory University:Melbourne
[root@jselab shell-book]#
```

例 5-20 通过 cut 命令的-f 选项按域提取 TEACHER.db 的文本，-f 与-c 选项一样，同样可以用三种方式指定域数或域范围，例 5-19 中-f1,4 表示提取 TEACHER.db 的第 1 域和第 4 域，-f1-3 表示提取 TEACHER.db 的第 1～3 域。同时，例 5-20 利用-d 选项改变域分隔符。

cut 命令可以灵活地提取文本文件中的内容，它默认将提取的内容放到标准输出上，如果要将提取的内容保存到文件，可以使用文件重定向来实现。

5.5 paste 命令

paste 命令用于将文本文件或标准输出中的内容粘贴到新的文件，它可以将来自不同文件

混混藏书阁 http://book-life.blog.163.com

的数据粘贴到一起，形成新的文件。paste 命令的基本格式如下：

```
paste [选项] 文件 1 文件 2
```

paste 命令有三个选项，其含义如表 5-5 所示。

表 5-5 paste 命令选项及其意义

选　项	意　义
-d	默认域分隔符是空格或 Tab 键，设置新的域分隔符
-s	将每个文件粘贴成一行
-	从标准输入中读取数据

下面通过例子说明 paste 命令的用法，首先给出例 5-21，它演示了 paste 命令最简单的用法。

```
#例 5-21: paste 命令的基本用法
[root@jselab shell-book]# cat FILE1                    #FILE1 的内容
Shanghai Jiaotong University
University of Toronto
Beijing University
Southeast University
Victory University
[root@jselab shell-book]# cat FILE2                    #FILE2 的内容
Shanghai
Toronto
Beijing
Nanjing
Melbourne
[root@jselab shell-book]# paste FILE1 FILE2            #粘贴 FILE1 和 FILE2，FILE1 在前
Shanghai Jiaotong University    Shanghai
University of Toronto   Toronto
Beijing University      Beijing
Southeast University    Nanjing
Victory University      Melbourne
[root@jselab shell-book]# paste FILE2 FILE1            #粘贴 FILE2 和 FILE1，FILE2 在前
Shanghai        Shanghai Jiaotong University
Toronto University of Toronto
Beijing Beijing University
Nanjing Southeast University
Melbourne       Victory University
[root@jselab shell-book]#
```

例 5-21 新建两个文件：FILE1 和 FILE2，FILE1 中的每行是一个大学名称，FILE2 中的每行是一个城市名称。paste FILE1 FILE2 就将 FILE1 的内容作为每行记录的第 1 域，FILE2 的内容作为第 2 域，中间用 Tab 键作为分隔符；而 paste FILE2 FILE1 则相反，FILE2 的内容为第 1 域，FILE1 为第 2 域，此时，域分隔符是 Tab 键，如需改变域分隔符，则可以使用 paste 命令的-d 选项，请看下面的例 5-22：

```
#例 5-22: paste 命令-d 选项的用法
[root@jselab shell-book]# paste -d: FILE1 FILE2        #以:为域分隔符
Shanghai Jiaotong University:Shanghai
University of Toronto:Toronto
Beijing University:Beijing
Southeast University:Nanjing
```

```
Victory University:Melbourne
[root@jselab shell-book]# paste -d@ FILE1 FILE2          #以@为域分隔符
Shanghai Jiaotong University@Shanghai
University of Toronto@Toronto
Beijing University@Beijing
Southeast University@Nanjing
Victory University@Melbourne
[root@jselab shell-book]#
```

例 5-22 演示了 paste 命令-d 选项的用法，-d:将粘贴后的文件以“:”为域分隔符，而-d@则以@为域分隔符，sort、join 和 cut 命令改变域分隔符都是为了按域读取文件内容，而 paste 命令则不同，改变域分隔符是用于设置输出文件的格式。

从例 5-21 和例 5-22 都可以看出，paste 命令默认是将一个文件按列粘贴的，-s 选项可以实现将一个文件按行粘贴，例 5-23 给出了-s 选项的例子：

```
#例 5-23: paste 命令-s 选项的用法
[root@jselab shell-book]# paste -d: -s FILE1 FILE2        #按行粘贴文件
Shanghai Jiaotong University:University of Toronto:Beijing University:Southeast
University:Victory University                             #与上一行是同一行
Shanghai:Toronto:Beijing:Nanjing:Melbourne
[root@jselab shell-book]#
```

例 5-23 在将 FILE1 和 FILE2 粘贴成新文件时，加上了-s 选项，FILE1 内容被放到同一行，域之间以“:”分隔，FILE2 内容被粘贴到第 2 行，同样以“:”分隔。因此，简言之，paste 不加-s 选项时，将文件内容“竖着放”，加上-s 选项后，将文件内容“横着放”。

paste 命令的“-”选项比较特殊，当 paste 命令从标准输入中读取数据时，“-”选项才起作用，下面举一个例子来说明 paste 命令的“-”选项。

```
#例 5-24: paste 命令-选项的用法
[root@jselab shell-book]# ls | paste -d" " - - - - -          #从标准输入读取数据
anotherres.sh array_eval2.sh colon.sh example execerr.sh      #每行显示 5 个文件名
execin.sh exec.sh FILE1 FILE2 forever.sh
hfile loggg loggg1 loopalias.sh matrix.sh
newfile nokillme.sh part1 part2 part3
parttotal refor.sh reif.sh selfkill.sh sleep10.sh
sleep55.sh stack.sh subsenv.sh subsep.sh subsig.sh
subsparallel.sh subspipe.sh subsvar.sh TEACHER.db test.sh
testvar.sh traploop.sh
[root@jselab shell-book]#
```

例 5-24 中的 paste 命令没有输入文件，它是通过读取 ls 命令的输出结果，再进行粘贴。paste 命令后的-d" "将分隔符设置为空格符，在原本应出现“文件 1 文件 2”的位置上加上“-”选项，例 5-24 加了 5 个“-”选项。从结果可以看到，粘贴后每行显示 5 个文件名，每个“-”选项表示读取 1 次标准输入数据，即读取到标准输入数据中的一个域。

join 命令和 paste 命令在使用过程中需要注意以下几点：

- join 命令只能对排序之后的文件进行操作。
- 使用 join 命令时必须使用-t 参数。
- join 命令对两个文件进行操作时，如果两个文件的域不相同，则只会连接具有相同域的部分，可以用-a1 和-a2 参数分别显示第一个文件和第二个文件中未被连接的部分。
- 如果需要将 join 命令的输出写到文件，可以用“>”重定向操作，其没有输出命令参数。

混混藏书阁 http://book-life.blog.163.com

- join 命令默认的连接域是 1，如果需要使用其他域，可以使用-1 和-2 参数来指定对应的域。
- paste 命令的连接顺序和文件名称相关，默认使用域 1 作为连接选项，没有被连接的项会被写到最后，如果想把 paste 的结果写到文件中，同样可以使用重定向。

5.6 split 命令

split 命令用于将大文件切割成小文件，split 命令可以按照文件的行数、字节数切割文件，并能在输出的多个小文件中自动加上编号。split 命令的基本格式如下：

```
split [选项] 待切割的大文件 输出的小文件
```

split 命令的选项用于指定切割的依据，我们将 split 命令的选项及其意义列于表 5-6 中。

表 5-6 split 命令选项及其意义

选　项	意　义
-或-l	这两个选项等价，都用于指定切割成小文件的行数
-b	指定切割成小文件的字节
-C	与-b 选项类似，但是，切割时尽量维持每行的完整性

下面通过例子说明 split 命令的用法，首先给出一个按行数切割文件的例子，如例 5-25 所示：

```
#例 5-25：split 命令按行切割文件
#将 TEACHER.db 每 2 行进行切割，输出文件以 PEO.db 开头命名
[root@jselab shell-book]# split -2 TEACHER.db PEO.db
[root@jselab shell-book]# ls PEO*                          #得到如下 3 个小文件
PEO.dbaa  PEO.dbab  PEO.dbac
[root@jselab shell-book]# cat PEO.dbaa                     #包含两条记录
B Liu:Shanghai Jiaotong University:Shanghai:China
C Lin:University of Toronto:Toronto:Canada
[root@jselab shell-book]# cat PEO.dbab                     #包含两条记录
D Hou:Beijing University:Beijing:China
J Luo:Southeast University:Nanjing:China
[root@jselab shell-book]# cat PEO.dbac                     #包含一条记录
Y Zhang:Victory University:Melbourne:Australia
[root@jselab shell-book]#
```

例 5-25 中 split 命令利用-2 指定按 2 行对 TEACHER.db 进行切割，即每 2 行记录切割成 1 个文件，PEO.db 指定输出小文件名，由于小文件将多个，split 命令在 PEO.db 后面自动加上编号以区分不同的小文件，编号为 aazz，即第 1 个小文件是 PEO.dbaa、第 2 个是 PEO.dbab、第 3 个是 PEO.dbac，我们逐个查看小文件的记录，前面两个小文件包含 2 条记录，最后 1 个小文件只有 1 条记录。split 命令所切割生成的小文件最多包含 1000 行记录。

-b 和-C 选项都是用于指定小文件的字节数，即两个选项都是按照文件大小来切割文件的，两个选项略有区别，我们通过下面的两个例子来说明这两个选项。例 5-26 给出了-b 选项的用法：

```
#例 5-26: split 命令-b 选项的用法
[root@jselab shell-book]# ll T*                          #列出 TEACHER.db 的详细信息，大小为 220B
-rw-r--r--. 1 root root 220 03-09 11:32 TEACHER.db

#将 TEACHER.db 按每 100B 切割成小文件，此处未指定小文件的名字
[root@jselab shell-book]# split -b100 TEACHER.db
 [root@jselab shell-book]# ll x*                         #得到三个小文件，以 x 加上编号命名
-rw-r--r--. 1 root root 100 03-09 17:04 xaa
-rw-r--r--. 1 root root 100 03-09 17:04 xab
-rw-r--r--. 1 root root  20 03-09 17:04 xac

#下面逐个查看三个小文件的内容
[root@jselab shell-book]# cat xaa
B Liu:Shanghai Jiaotong University:Shanghai:China
C Lin:University of Toronto:Toronto:Canada
D Hou:B[root@jselab shell-book]# cat xab
eijing University:Beijing:China
J Luo:Southeast University:Nanjing:China
Y Zhang:Victory University:[root@jselab shell-book]# cat xac
Melbourne:Australia
[root@jselab shell-book]#
```

例 5-26 中的 split 命令利用-b 选项指定 100B 切割 TEACHER.db 文件，当 split 命令不指定小文件的名字时，将自动以 x 开头、aa-zz 为编号对这些小文件进行命名，用 ll 命令查看这三个小文件发现，xaa 和 xab 是 100B，xac 是 20B，这说明确实按照 100B 的大小切割了 TEACHER.db 文件。但是，当用 cat 命令查看这三个文件时，发现每个文件内容比较凌乱，甚至存放了不完整的单词。因此，split 命令-b 选项在切割文件时仅考虑了文件大小，并未考虑记录的完整性。

那么，如果我们用-C 选项指定 100B 切割 TEACHER.db 文件会得到什么结果呢？请看如下的例 5-27：

```
#例 5-27: split 命令-C 选项的用法
#-C 选项指定 100B 将 TEACHER.db 切割成小文件
[root@jselab shell-book]# split -C100 TEACHER.db

#三个小文件并不严格按照 100B 的大小进行切割
[root@jselab shell-book]# ll x*
-rw-r--r--. 1 root root 93 03-09 17:18 xaa
-rw-r--r--. 1 root root 80 03-09 17:18 xab
-rw-r--r--. 1 root root 47 03-09 17:18 xac

#逐个查看三个小文件
[root@jselab shell-book]# cat xaa
B Liu:Shanghai Jiaotong University:Shanghai:China
C Lin:University of Toronto:Toronto:Canada
[root@jselab shell-book]# cat xab
D Hou:Beijing University:Beijing:China
J Luo:Southeast University:Nanjing:China
[root@jselab shell-book]# cat xac
Y Zhang:Victory University:Melbourne:Australia
[root@jselab shell-book]#
```

例 5-27 利用-C 选项指定 100B 切割 TEACHER.db 文件，同样得到 xaa、xab 和 xac 三个

混混藏书阁 http://book-life.blog.163.com

小文件，但是，xaa 是 93B、xab 是 80B、xac 是 47B，xaa、xab 和 xac 这三个文件存放了完整的记录，由此可见，-C 选项并不严格按照 100B 的大小进行切割，而是在切割时尽量维持每行的完整性。比较例 5-26 和例 5-27，就可以清晰地看出-b 和-C 选项的异同。

5.7 tr 命令

tr 命令实现字符转换功能，其功能类似于 sed 命令，但是，tr 命令比 sed 命令简单。也就是说，tr 命令能实现的功能，sed 命令都可以实现。尽管如此，tr 命令依然是 Linux 系统下处理文本的常用命令，本节简单介绍 tr 命令，tr 命令的基本格式如下：

```
tr [选项] 字符串 1 字符串 2 <输入文件
```

tr 命令有三个选项，我们将 tr 命令的选项及其意义列于表 5-7 中。tr 命令可以跟两个字符串，很多情况下，只能跟一个字符串。“<输入文件”表示将输入文件重定向到标准输入，实际上，tr 命令与 sort、uniq、join 等命令不同，它只能从标准输入读取数据。因此，tr 命令要么将输入文件重定向到标准输入，要么从管道读入数据。读者在学习完本书第 10 章之后将会更清晰地理解这一点，在此，读者只需要记住 tr 命令的输入文件之前需要加上“<”符号即可。

表 5-7　tr 命令选项及其意义

选　项	意　义
-c	选定字符串 1 中字符集的补集，即反选字符串 1 中的字符集
-d	删除字符串 1 中出现的所有字符
-s	删除所有重复出现的字符序列，只保留一个

tr 命令的-d 选项最简单，因此，首先通过一个例子介绍 tr 命令的-d 选项。tr 命令的-d 选项只需跟一个字符串，它表示删除字符串中出现的所有字符，下面的例 5-28 演示了 tr -d 的基本用法：

```
#例 5-28：tr 命令-d 选项的用法
alloy@alloy:~/linuxshell/CH05$ cat AREACODE.db
BEIJING:86010
HONGKONG:852
SHANGHAI:86021
TORONTO:001416

#第 1 条命令：删除 AREACODE.db 文件中所有的大写字母
alloy@alloy:~/linuxshell/CH05$ tr -d A-Z <AREACODE.db
:86010
:852
:86021
:001416
#第 2 条命令：删除 AREACODE.db 文件中所有的数字
alloy@alloy:~/linuxshell/CH05$ tr -d 0-9 <AREACODE.db
BEIJING:
HONGKONG:
```

```
SHANGHAI:
TORONTO:
#第 3 条命令：删除 AREACODE.db 文件中所有的换行符
alloy@alloy:~/linuxshell/CH05$ tr -d "[\n]" <AREACODE.db
BEIJING:86010HONGKONG:852SHANGHAI:86021TORONTO:001416
alloy@alloy:~/linuxshell/CH05$
```

例 5-28 中的 tr 命令利用-d 选项删除 AREACODE.db 文件中的一些字符，第 1 条命令用 A～Z 表示所有的大写字母，第 2 条命令用 0～9 表示所有的数字，这与正则表达式非常相似，事实上，tr 命令支持正则表达式的一部分，如 a～z、A～Z、0～9、A*3 等。第 3 条命令用"[\n]"表示换行符，tr 命令支持 POSIX 字符类和一些控制字符，POSIX 字符类在第 3 章已经介绍过，在此不再重复列举，我们仅将 tr 命令支持的控制字符列在表 5-8 中。

表 5-8　tr 命令支持的控制字符

选　　项	意　　义	八进制方式
\a	Ctrl+G 铃声	\007
\b	Ctrl+H 退格符	\010
\f	Ctrl+L 走行换页	\014
\n	Ctrl+J 换行符	\012
\r	Ctrl+M 回车键	\015
\t	Ctrl+I Tab 键	\011

tr 命令的-s 选项用于删除所有重复出现的字符序列，只保留一个，即将重复出现的字符串压缩为一个字符。下面给出的例 5-29 利用-s 选项删除文件中的空白行。

```
#例 5-29：tr 命令-s 选项删除空白行
alloy@alloy:~/linuxshell/CH05$ cat AREACODE.db                    #查看 AREACODE.db 的内容
BEIJING:86010
                                                                  #两行空白行


HONGKONG:852
                                                                  #一行空白行
SHANGHAI:86021
                                                                  #三行空白行



TORONTO:001416
                                                                  #一行空白行
alloy@alloy:~/linuxshell/CH05$ tr -s "[\n]" <AREACODE.db #将重复出现的换行符压缩成一个
BEIJING:86010
HONGKONG:852
SHANGHAI:86021
TORONTO:001416
alloy@alloy:~/linuxshell/CH05$ tr -s "[\012]" <AREACODE.db#\012 是\n 的八进制表示方式
BEIJING:86010
HONGKONG:852
SHANGHAI:86021
TORONTO:001416
alloy@alloy:~/linuxshell/CH05$
```

例 5-29 中 AREACODE.db 文件包含若干空白行，空白行可以看作是该行只有一个换行

混混藏书阁 http://book-life.blog.163.com

符而无其他任何字符。因此，我们只要将重复出现的换行符压缩成一个就可以消除空白行了。例 5-29 中的两条命令利用 tr -s 将重复出现的\n 字符压缩成一个，\012 是\n 的八进制表示方式，同样可以用来表示换行符。下面再举一个例子说明 tr 命令的-s 选项：

```
#例 5-30：tr 命令-s 选项压缩重复字符
alloy@alloy:~/linuxshell/CH05$ cat REPEAT                          #REPEAT 文件有很多重复字母
Wooooomennnn
TTTTheyyyyyyy
alloy@alloy:~/linuxshell/CH05$ tr -s "[a-z],[A-Z]" <REPEAT    #将所有重复的字母压缩成一个
Women
They
alloy@alloy:~/linuxshell/CH05$
```

例 5-30 中的 REPEAT 文件有很多重复出现的字母，我们可以利用 tr -s 命令将重复出现的大小写字母都压缩成一个。

tr 命令也可以加上字符串 1 和字符串 2，将字符串 1 用字符串 2 来替换。下面给出的例 5-31 利用 tr 命令实现大小写字母的互换。

```
#例 5-31：tr 命令实现大小写字母的互换
alloy@alloy:~/linuxshell/CH05$ tr "[a-z]" "[A-Z]" <REPEAT     #将小写字母替换为大写字母
WOOOOOMENNNN
TTTTHEYYYYYY
alloy@alloy:~/linuxshell/CH05$ tr "[A-Z]" "[a-z]" <REPEAT     #将大写字母替换为小写字母
woooooomennnn
ttttheyyyyyyy
alloy@alloy:~/linuxshell/CH05$
```

例 5-31 中的两条命令分别将 REPEAT 文件的小写字母替换成大写字母，再将大写字母替换为小写字母，以[a-z]表示小写字母集合、[A-Z]表示大写字母集合。当然，我们也可以使用 POSIX 字符类来表示大小写字母集合，如[:lower:]表示小写字母集合、[:upper:]表示大写字母集合。因此，例 5-32 给出的命令实现了与例 5-31 完全一样的功能。

```
#例 5-32：用 POSIX 字符类表示大小写
alloy@alloy:~/linuxshell/CH05$ tr "[:lower:]" "[:upper:]" < REPEAT
WOOOOOMENNNN
TTTTHEYYYYYY
alloy@alloy:~/linuxshell/CH05$ tr "[:upper:]" "[:lower:]" < REPEAT
woooooomennnn
ttttheyyyyyyy
alloy@alloy:~/linuxshell/CH05$
```

tr 命令的-c 选项用于选定字符串 1 中字符集的补集，即反选字符串 1 中的字符集。下面给出的例 5-33 结合使用-c 和-s 选项删除 AREACODE.db 文件中的冒号、数字和空白行：

```
#例 5-33：tr 命令-c 选项的用法
#将不在"[a-z][A-Z]"字符集内的字符替换为换行符
#然后-s 选项将重复出现的换行符压缩成一个
alloy@alloy:~/linuxshell/CH05$ tr -cs "[a-z][A-Z]" "[\012*]" <AREACODE.db
BEIJING
HONGKONG
SHANGHAI
TORONTO
alloy@alloy:~/linuxshell/CH05$
```

例 5-29 曾显示过 AREACODE.db 文件的内容，它是由字母、冒号、数字和空白行组成的文本文件，例 5-33 结合使用-c 和-s 选项删除 AREACODE.db 文件中的冒号、数字和空白